

DLTSONHALLPROBESETUP

DLTS control software

Control and analysis software for deep-level transient spectroscopy on the Hall-probe stage: Zurich Instruments MFIA and Instec mK2000B, driven from one Tkinter GUI. Guides, worked examples and a reference page for every module.

Documentation build of 28 September 2026

Contents

Overview

1. DLTS control software: documentation

- Guides

- How the parts fit together

- Reference: one page per module

- Checking the software

Guide

1. Getting started

- What the software does

- Requirements

- Start the GUI

- Repository layout

2. Running a DLTS experiment

- 1. Before you start

- 2. Set the parameters (Input Parameters tab)

- 3. Run (Live Tools tab)

- 4. Look at the transients (Qualitative Analysis, lower part of Live Tools)

- 5. Rate windows and Arrhenius (Quick Analysis tab)

- 6. Many windows and error bars (Detailed Analysis tab)

- 7. Close

- Data files

3. Quick run examples

- A. Analyze a finished run (no instruments)

- B. A short HDF5 scan on the instruments

- C. One acquisition from a script

- D. Convert and inspect data files

4. Standalone instruments

- Temperature controller: `instecTempStage_Control.mk2000B`

- Impedance analyzer: `zurichInstruments_Control.ziDevice`

- Both together

- Analysis without instruments

Reference

1. DLTS GUI_MainWindow

2. runParamsTab

3. liveDataTab

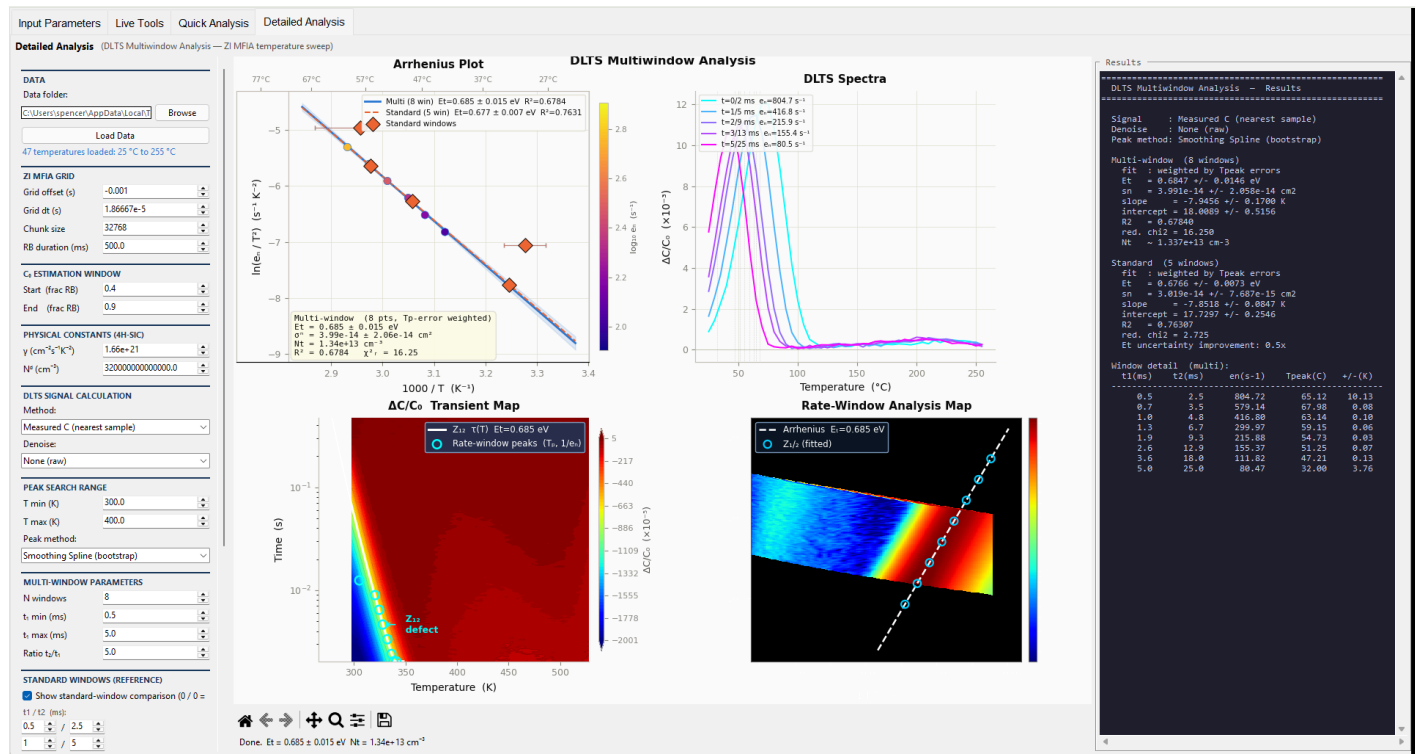
4. dataAnalysisTab

5. detailedAnalysisTab

6. [runDlts_Tools](#)
7. [zurichInstruments_Control](#)
8. [instecTempStage_Control](#)
9. [impedanceAnalysis_Tools](#)
10. [dltsConfig](#)
11. [convert_json_to_h5](#)
12. [convert_h5_to_text](#)
13. [Data formats](#)
14. [Other scripts](#)

DLTS control software: documentation

Control and analysis software for deep-level transient spectroscopy on the Hall-probe stage: a Zurich Instruments MFA impedance analyzer and an Instec mK2000B temperature controller, driven from one Tkinter GUI.



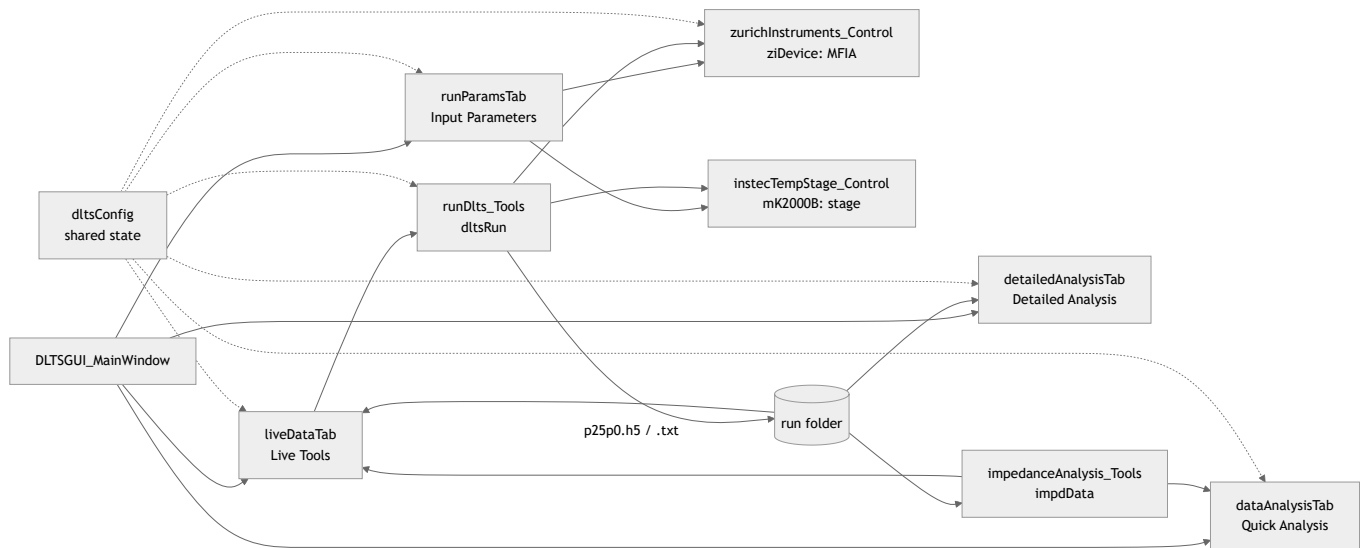
Detailed Analysis on a 47-temperature run

Guides

1. [Getting started](#): requirements, installation, starting the GUI, repository layout.
2. [Running a DLTS experiment](#): every step in the GUI, from parameters to the Arrhenius result.
3. [Quick run examples](#): four worked cases on real data, with the settings to use and the results to expect.
4. [Standalone instruments](#): the stage and the MFA from Python scripts, without the GUI.

All of these pages are also in one PDF, [docs/DLTS_Software_Documentation.pdf](#). After editing the docs, rebuild it with `python docs/build_docs_pdf.py` (needs Microsoft Edge; it downloads mermaid.js for the diagram below).

How the parts fit together



Reference: one page per module

Each page lists the module's constants and globals, every class, method and function with its parameters and return value, how to call it, and known limitations.

Module	What it is
DLTSGUI_MainWindow	GUI entry point, tab construction, close and room-temperature return
runParamsTab	Input Parameters tab
liveDataTab	Live Tools tab: run control, live/offline plots, Qualitative Analysis extractors
dataAnalysisTab	Quick Analysis tab: rate windows and Arrhenius
detailedAnalysisTab	Detailed Analysis tab: multi-window analysis, maps, weighted fits
runDlts_Tools	<code>dltsRun</code> : the run sequence, Redo/Retake, file naming
zurichInstruments_Control	<code>ziDevice</code> : MFIA parameters, acquisition, file writers
instecTempStage_Control	<code>mK2000B</code> : stage ramps, stabilization, room return
impedanceAnalysis_Tools	<code>impdData</code> : reading, cleanup, emission selection, denoising, DLTS signals
dltsConfig	Shared globals, logging, GUI-label conversion
convert_json_to_h5	JSON run folders to HDF5
convert_h5_to_text	One HDF5 step to a readable table or JSON
data_formats	Run folders, step files, runParams.txt, HDF5 layout, ZI CSV exports
other_scripts	Legacy apps, tests, benchmarks

Checking the software

```
python -m pytest tests -q
```

163 tests, all offline, about 40 s. The real-hardware benchmark and its stored report are in `benchmarks/hardware/` (latest: `reports/HDF5_Hardware_Benchmark_2026-09-28.pdf`).

Getting started

What the software does

The DLTS control GUI runs a deep-level transient spectroscopy temperature scan on two instruments and analyzes the result:

- **Zurich Instruments MFIA** impedance analyzer (device `dev32271`). It applies the fill / reverse-bias pulse train through its Aux output, measures the sample's impedance, and averages many triggered repetitions per temperature.
- **Instec mK2000B** temperature controller (serial port `COM7`). It ramps the stage to each setpoint and holds it.

For each setpoint the software ramps, waits until the stage is stable, acquires one averaged record and writes it as one file (`p25p0.h5`, `p30p0.h5`, ...). The analysis tabs turn those files into averaged capacitance transients, DLTS rate-window spectra and an Arrhenius plot (activation energy, capture cross-section, trap density).

Requirements

- Windows PC with the instruments connected. This PC: Python 3.13.9 (Anaconda at `C:\Users\spencer\anaconda3`).
- **Zurich Instruments LabOne** installed and its data server running. The MFIA is found by `zhinst.core.ziDiscovery` (on this PC: data server `192.168.176.30:8004`).
- **Instec controller** on a USB-serial port. The code expects `COM7` (`instecTempStage_Control.mK2000B(port='COM7')`); change it there if Windows assigns another port.
- Python packages: `pip install -r requirements.txt` from the repository folder. Tested versions are in that file (zhinst-core 26.01, zhinst-toolkit 1.3, h5py 3.12 with HDF5 1.14, numpy 2.1).

Check the installation without any instrument attached:

```
python -m pytest tests -q
```

All tests run offline (fake devices, synthetic data). They take about 30 s.

Start the GUI

```
python DLTSGUI_MainWindow.py
```

The window opens maximized with four tabs:

Tab	What it is for	Reference
Input Parameters	MFIA, temperature and output settings; connect and push them to the instruments; export an HDF5 step to text	runParamsTab
Live Tools	Run DLTS, pause/resume/redo/retake steps, watch the data come in, load a finished run, Qualitative Analysis (extract and average transients)	liveDataTab

Tab	What it is for	Reference
Quick Analysis	Five rate windows, DLTS spectra and an Arrhenius fit	dataAnalysisTab
Detailed Analysis	Many rate windows, bootstrap peak errors, weighted Arrhenius fit, transient and rate-window maps	detailedAnalysisTab

Closing the window always returns the stage to the Room Temperature set on the Input Parameters tab first (a dialog shows the stage temperature until it arrives). See [DLTSGUI_MainWindow](#).

Repository layout

Path	Contents
<code>DLTSGUI_MainWindow.py</code>	GUI entry point
<code>runParamsTab.py</code> , <code>liveDataTab.py</code> , <code>dataAnalysisTab.py</code> , <code>detailedAnalysisTab.py</code>	The four tabs
<code>zurichInstruments_Control.py</code>	MFIA driver (<code>ziDevice</code>), data file writers
<code>instecTempStage_Control.py</code>	Temperature controller driver (<code>mk2000B</code>)
<code>runDlts_Tools.py</code>	The run sequence (<code>dltsRun</code>): init, steps, redo/retake, finish, room return
<code>impedanceAnalysis_Tools.py</code>	Analysis library (<code>impdData</code>): reading, cleanup, emission selection, denoising
<code>dltsConfig.py</code>	Shared state between the tabs, logging, GUI-label to value conversion
<code>convert_json_to_h5.py</code> , <code>convert_h5_to_text.py</code>	Data converters
<code>examples/</code>	Standalone scripts: stage, MFIA, a minimal scan, offline analysis
<code>benchmarks/</code>	Synthetic and real-hardware HDF5 benchmarks, stored reports
<code>tests/</code>	pytest suite
<code>DLTS_APP.py</code> , <code>DrKayisScript.py</code>	Older standalone PyQt6 analysis apps the tabs were ported from

Next: [Running a DLTS experiment](#), or the [Quick run examples](#).

Running a DLTS experiment

This is the normal way to measure: everything through the GUI. It takes you from a mounted sample to a folder of step files and an Arrhenius result. For scripts without the GUI see [Standalone instruments](#).

Safety. A run heats and cools the stage and applies the pulse bias to the mounted sample. Check the sample, the wiring and the temperature range first.

1. Before you start

- 1. Mount and wire the sample. Switch on the MFIA and the Instec controller.
- 2. Start LabOne's data server (it normally runs as a service) so the MFIA can be found.
- 3. Start the GUI: `python DLTSGUI_MainWindow.py`.

2. Set the parameters (Input Parameters tab)

Input ParametersLive ToolsQuick AnalysisDetailed Analysis

Oscillation Amplitude

Oscillation Frequency

Oscillation ON/OFF

Max bandwidth

Input Control

Current Range

Voltage Range

Omega Suppression

Filter Harmonic

Filter Bandwidth

Data Transfer Rate

Equivalent Circuit Mode

Threshold Input Signal

State Enable Time

State Disable Time

Logic Unit Not

Aux Output Signal

Aux Output Scale

Aux Output Offset

Aux Output Lower Limit

Aux Output Upper Limit

Signal Output Add

Trigger Source Signal

0.300

501000

1 - On

10000

0 - Manual

0.010

3

80

1

2

60000

0 - 4-Terminal

59 - TU Output Value

0.006

0.003

1 - On

19 - TU Output Value

-1

-0.5

-10

0

1 - True

36 - Threshold 1

Initial Temperature (C)

Final Temperature (C)

Temperature Step (C)

Temperature Ramp (C/min)

Stability Delay (s)

Room Temperature (C)

Room Ramp (C/min)

Connect + Get Params

Apply + Push Params

Number of Points (power of 2)

Number of Repts

Data File Format

Data Root Folder

Browse...

Apply + Push Params

Export HDF5 File...

Parameter History

Save Current

Load Selected

Input Parameters tab

The tab has three groups. Every field starts at the default shown.

Impedance analyzer (blue, left)

These go to the MFIA. The ones that shape the DLTS pulse:

Field	Default	What it does in a DLTS run
State Enable Time	0.006 s	Reverse-bias duration. The transient is recorded during this time.
State Disable Time	0.003 s	Fill-pulse duration.
Aux Output Scale	-1 V	Pulse height. The Aux output is <code>Offset</code> during the fill pulse and <code>Offset + Scale</code> during reverse bias.
Aux Output Offset	-0.5 V	Fill-pulse level. Defaults give fill -0.5 V, reverse bias -1.5 V.
Aux Output Lower / Upper Limit	-10 / 0 V	Clamp on the Aux output.
Oscillation Frequency / Amplitude	501000 Hz / 0.300 V	Test signal for the impedance measurement.
Trigger Source Signal	36 - Threshold 1	Each pulse triggers one acquisition row; keep the default.

A typical DLTS setting from earlier runs on this setup: State Enable Time 0.5, State Disable Time 0.001, Aux Output Scale -5, Offset 0, Upper Limit 5 (1 ms fill at 0 V, 500 ms reverse bias at -5 V). Every parameter, its device node and its options are in [zurichInstruments_Control](#) and [runParamsTab](#).

Press **Connect + Get Params** (blue) to connect to the MFIA, then **Apply + Push Params** (blue) to send the values. After changing a value later, press **Apply + Push Params** again. A run uses the last pushed values.

Temperature controller (red, top right)

Field	Default	Meaning
Initial Temperature (C)	25	First setpoint
Final Temperature (C)	25	Last setpoint (always included, even if the span is not a multiple of the step)
Temperature Step (C)	5	Setpoint spacing
Temperature Ramp (C/min)	5	Ramp rate between setpoints
Stability Delay (s)	0	Extra wait after the stage is stable (whole seconds)
Room Temperature (C)	25	Where the stage goes after every run, on errors, and when the GUI closes
Room Ramp (C/min)	10	Ramp rate for that return

Press **Connect + Get Params** (red), then **Apply + Push Params** (red).

A step counts as stable when the stage reading is within a tolerance of the setpoint (`mK2000B.expected_de1_t`), confirmed again 10 s later:

Setpoint (°C)	25 or below	30	50	100	150	200	250	300
Tolerance (°C)	0.10	0.027	0.08	0.21	0.34	0.48	0.61	0.74

Below 0 °C the tolerance grows again (0.29 °C at 0 °C, 1.0 °C at -50 °C). There is no time limit on stabilizing: a stage that cannot reach the tolerance waits indefinitely (close the GUI to abort; it then returns to room temperature). A controller that stops answering is different: each reading times out after 2 s and is asked once more, then the step fails with `temperature controller on COM7 did not answer the temperature query` in the log, and the run stops as described under step 3.

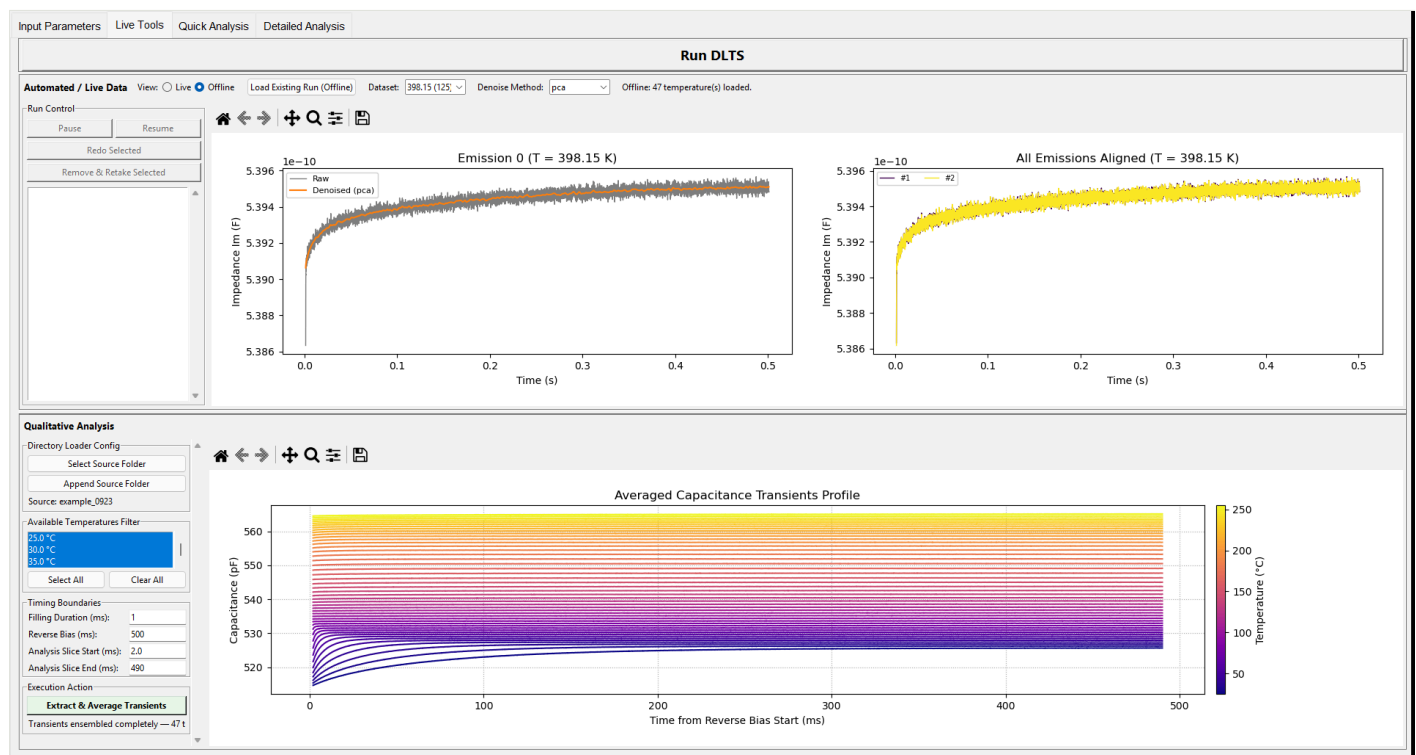
Output (green, middle right)

Field	Default	Meaning
Number of Points (power of 2)	16	Samples per step = 2^N . At the MFIA's 18.67 μ s sample period: 16 $\rightarrow$ 1.22 s, 18 $\rightarrow$ 4.89 s of data.
Number of Reps	500	Triggered repetitions averaged on the MFIA per step
Data File Format	TXT	TXT (JSON text, .txt) or HDF5 (.h5, about 7 $\times$ smaller, much faster to analyze)
Data Root Folder	none	Browse... to pick it (required: Run DLTS refuses to start without it). Each run creates Root\MMDDYY\HHMMSS\.

Press **Apply + Push Params** (green). **Export HDF5 File...** converts one .h5 step into a readable .txt table or .json for inspection (see [convert_h5_to_text](#)).

Parameter History keeps the last five parameter sets (**Save Current / Load Selected**).

3. Run (Live Tools tab)



Live Tools tab

Press **Run DLTS**. For each setpoint the run:

1. ramps to the setpoint and waits until stable,
2. re-pushes any MFIA parameter that no longer matches (after a factory reset from the second step on),
3. reads the stage temperature, acquires 2^N points averaged over the repetitions, reads the stage again,
4. writes the step file (p25p0.h5; negative setpoints n10p0.h5). HDF5 files also store the setpoint, the mean measured stage temperature, the acquisition time and all run parameters.

When the last step is done it writes `runParams.txt` and returns the stage to Room Temperature. Typical times on this setup with 5 °C steps at 5 °C/min: 3–5 min per step (ramp plus stabilization) and about 33 s per acquisition of 2^{18} points $\times$ 100 reps.

While it runs:

- **Automated / Live Data** plots each new file as it appears: Emission 0 (raw and denoised) and All Emissions Aligned, for the temperature chosen in **Dataset**.
- **Run Control** lists every step with its status (pending, running, done, failed). **Pause** stops after the current step; **Resume** continues. After a pause or at the end, select steps and press **Redo Selected** (overwrites those files) or **Remove & Retake Selected** (deletes them first). Steps are always re-measured in ascending temperature order.
- The log box on the Input Parameters tab shows every action and warning.

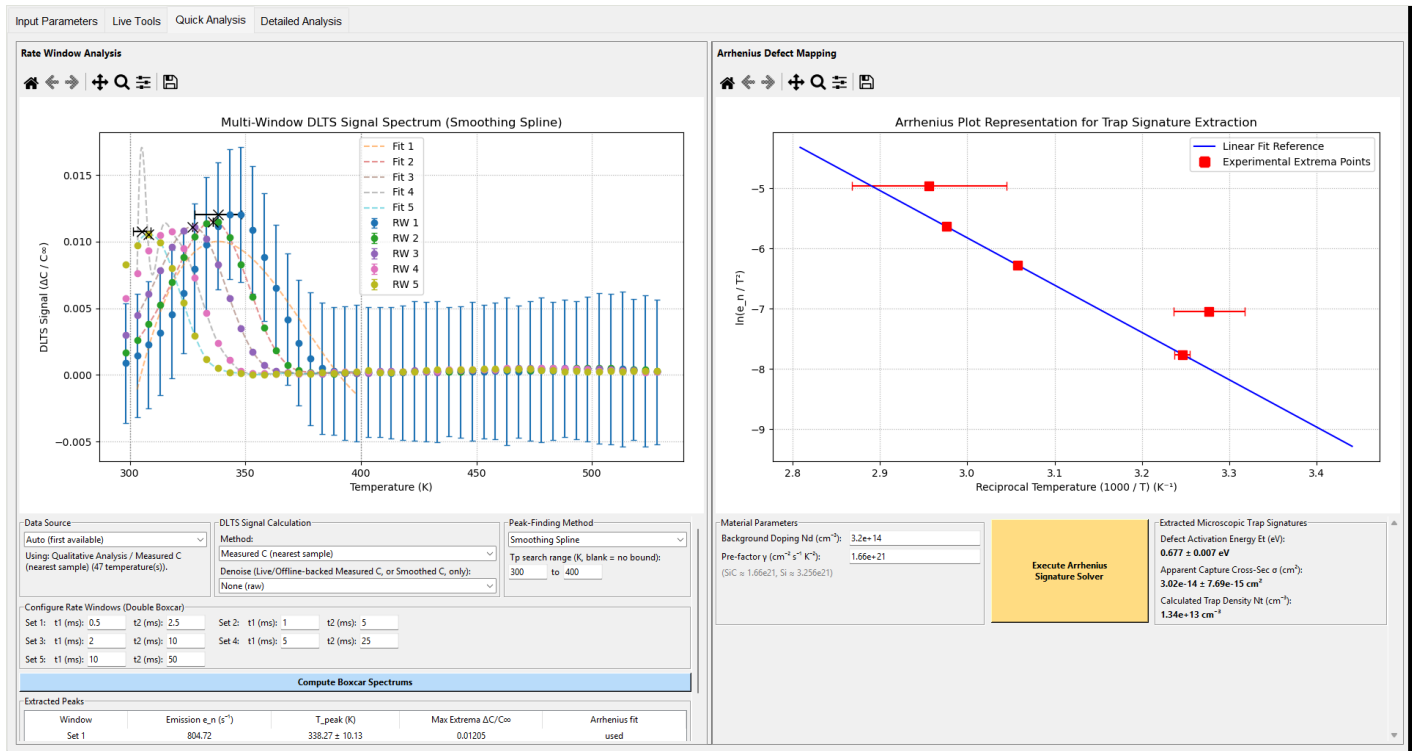
If a step fails (an instrument error, or a file that cannot be written), the run stops, the step is marked failed, earlier files are kept, and the stage returns to room temperature. Resume/Redo/Retake stay available.

4. Look at the transients (Qualitative Analysis, lower part of Live Tools)

1. **Select Source Folder** and pick the run folder (or **Append Source Folder** to add another one). It recognizes three layouts: this software's per-temperature `.txt` / `.h5` files, a single ZI MFIA CSV export, and ZI subfolder-per-temperature exports.
2. **Timing Boundaries** fill in from the folder's `runParams.txt` (reverse bias = State Enable Time, fill = State Disable Time). Check **Reverse Bias (ms)**: it sets the length of each averaged transient.
3. Choose temperatures in **Available Temperatures Filter**, then **Extract & Average Transients**.

Every reverse-bias window in each file is found from the excitation channel (the threshold is midway between the file's own two Aux levels) and averaged. The plot shows the Analysis Slice (default 2 ms to 98 % of the reverse bias). Temperatures that cannot be extracted are listed in the log with the reason (no pulses found, or the reverse bias is longer than the recorded data).

5. Rate windows and Arrhenius (Quick Analysis tab)

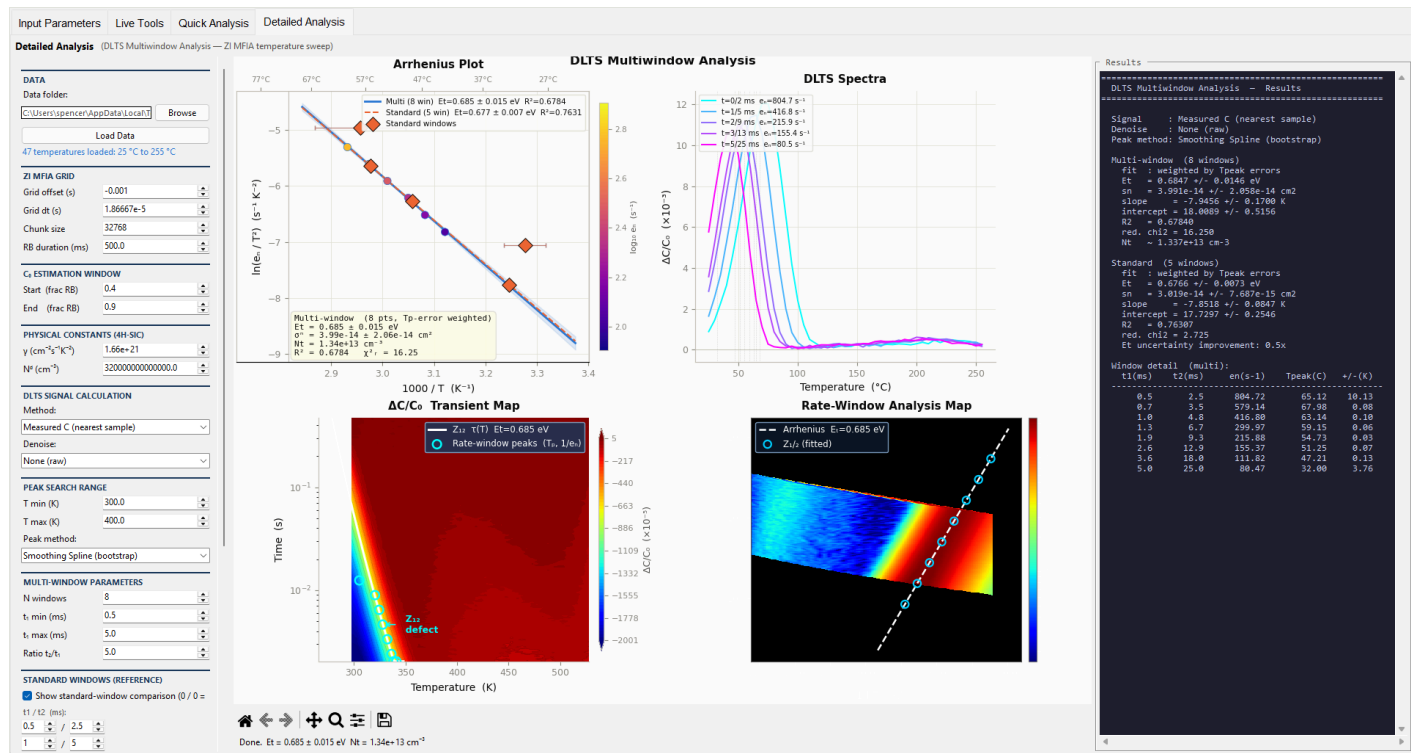


Quick Analysis tab

- Data Source:** Auto uses the Qualitative Analysis transients first, then Live or Offline data.
- Configure Rate Windows:** five t_1/t_2 pairs (ms), by default the same as Detailed Analysis' standard windows. The emission rate of each window is $e_n = \ln(t_2/t_1) / (t_2 - t_1)$ in s^{-1} , with t_1 and t_2 converted from ms to s. Choose windows so the peaks fall inside the scanned temperatures; faster windows (shorter t_1) move the peak to higher temperature.
- Tp search range:** the temperature range (K) searched for each peak.
- Compute Boxcar Spectrums**, then check **Extracted Peaks**. A window with no peak in the range is skipped; a peak on the edge of the range has no T_{peak} error.
- Enter **Background Doping Nd** and **Pre-factor γ** (4H-SiC $1.66e21$, Si $3.256e21$) and press **Execute Arrhenius Signature Solver** for E_t , σ and N_t . Edge peaks are left out of the fit (gray x) and marked **excluded** in the table.

Quick Analysis runs the same functions as Detailed Analysis. With the same transients, windows, search range, reverse bias and the smoothing spline, its result equals the **Standard** block of Detailed Analysis ([details](#)).

6. Many windows and error bars (Detailed Analysis tab)



Detailed Analysis tab

1. **Browse** to the run folder and set **RB duration (ms)**, then **Load Data**.
2. Set **N windows**, **t₁ min/max (ms)** and **Ratio t₂/t₁** (log-spaced windows), the **Peak Search Range** in K and the peak method.
3. **Run Analysis**. Results: Arrhenius plot (multi-window and the standard windows; a 0 / 0 standard row is unused), DLTS spectra, $\Delta C/C$ transient map, rate-window map, and a Results text panel. **Save Figure...** and **Export Results to TXT** save them.

The formulas and every option are in [detailedAnalysisTab](#) and [dataAnalysisTab](#). The [Quick run examples](#) walk through a real 47-temperature run with the settings that work for it.

7. Close

Close the window. It aborts any run, ramps the stage to Room Temperature and shows a dialog until it arrives; then it stops the controller and exits. **Close now** exits at once and leaves the controller ramping on its own.

Data files

Each run folder contains one file per setpoint plus `runParams.txt`. The formats (JSON `.txt`, HDF5 `.h5`, ZI CSV), the channels and their units are in [data_formats](#). Older JSON runs can be converted to HDF5 with `python convert_json_to_h5.py RUN_FOLDER (convert_json_to_h5)`.

Quick run examples

Four worked cases on real data from this setup. Each one lists exactly what to set and what you should see. The figures are screenshots of the GUI, or output of the example scripts, made from these runs.

Case	Needs instruments	Time
A. Analyze a finished run	No	5 min
B. A short HDF5 scan	Yes	about 65 min
C. One acquisition from a script	MFIA only	1 min
D. Convert and inspect data files	No	1 min

A. Analyze a finished run (no instruments)

Data: the 23 Sep 2026 run, C:\Users\spencer\Desktop\DATA\DLTS\092326\101901. It has 47 setpoints from 25 °C to 255 °C in 5 °C steps, 2¹⁶ points × 100 reps, 1 ms fill at 0 V and 500 ms reverse bias at −5 V. It was written as JSON (.json, 651 MB).

1. Convert it to HDF5 (optional, recommended). The Qualitative and Detailed tabs read .txt, .h5 and .csv step files, not .json. Converting a copy gives .h5 files that every tab reads and that load about 25× faster:

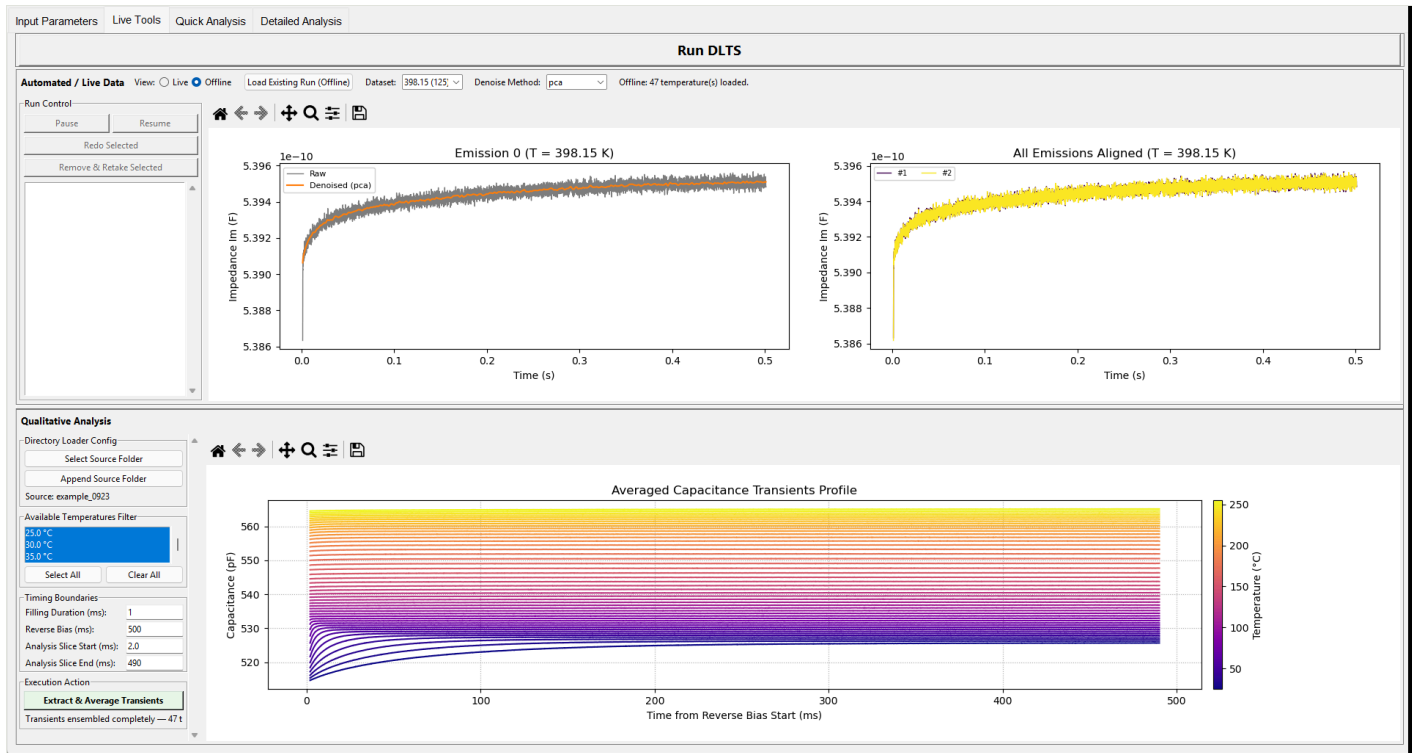
```
python convert_json_to_h5.py "D:\copy\101901" --keep-json
```

```
D:\copy\101901: 47 step file(s)
Converted 47 file(s): 651.5 MB of JSON -> 92.6 MB of HDF5. Skipped 0, failed 0.
```

It took 10 s. Every file is read back and compared value by value before the JSON is touched; --keep-json leaves the originals in place.

2. Live Tools → Load Existing Run (Offline). Select all 47 .h5 files. After loading, pick **Dataset** 398.15 (125): the upper plots show emission 0 raw and PCA denoised, and all emission blocks aligned.

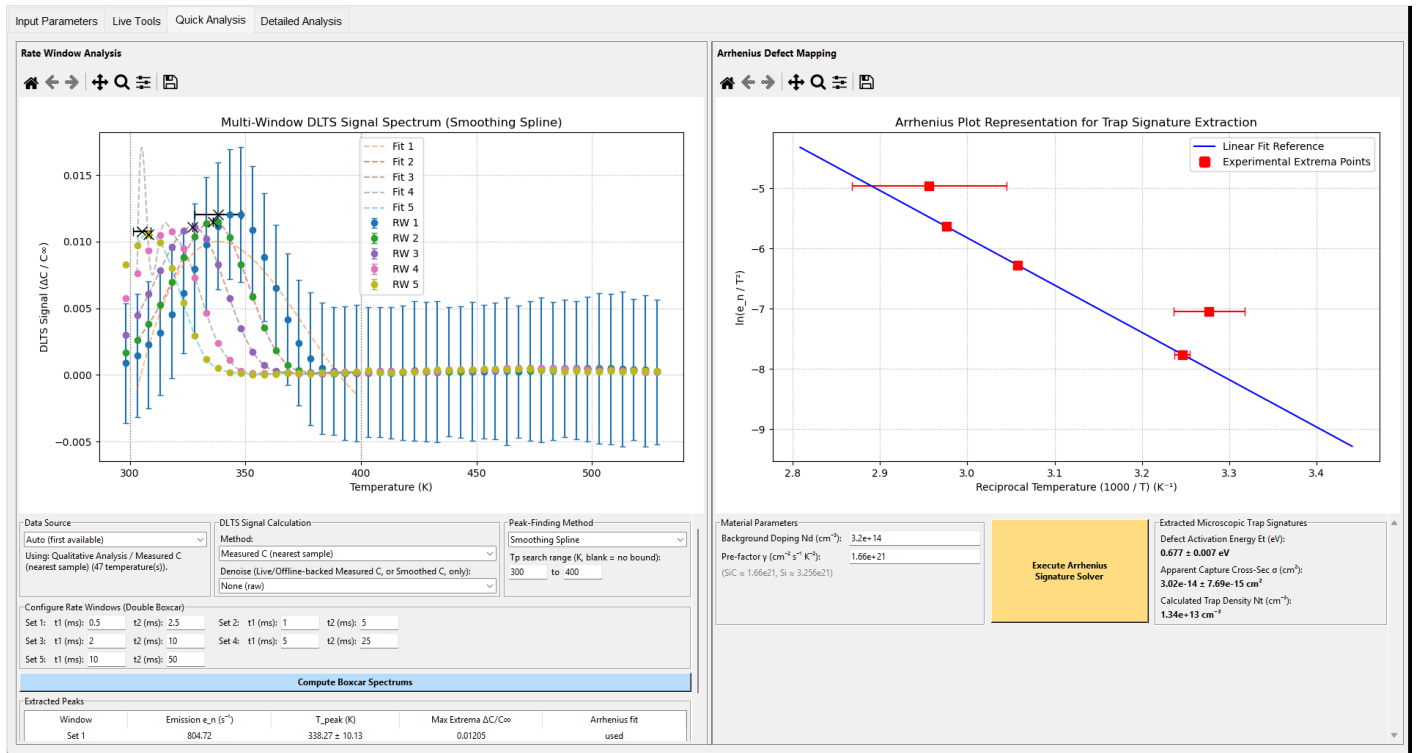
3. Qualitative Analysis → Select Source Folder → the converted folder. The log shows Timing Boundaries set from runParams.txt (fill 1 ms, reverse bias 500 ms). Press **Extract & Average Transients**.



Live Tools with the 23 Sep run loaded

The lower plot shows one averaged transient per temperature, colored by temperature. The fast recovery at the low temperatures (dark curves) is the trap emission. It speeds up with temperature and leaves the window above about 90 °C.

4. Quick Analysis. Set the five rate windows to $t_1/t_2 = 0.5/2.5, 1/5, 2/10, 5/25, 10/50$ ms and the **Tp search range** to 300 – 400 K. Press **Compute Boxcar Spectrums**, then **Execute Arrhenius Signature Solver** (defaults $N_d = 3.2 \times 10^{14} \text{ cm}^{-3}$, $\gamma = 1.66 \times 10^{21}$).

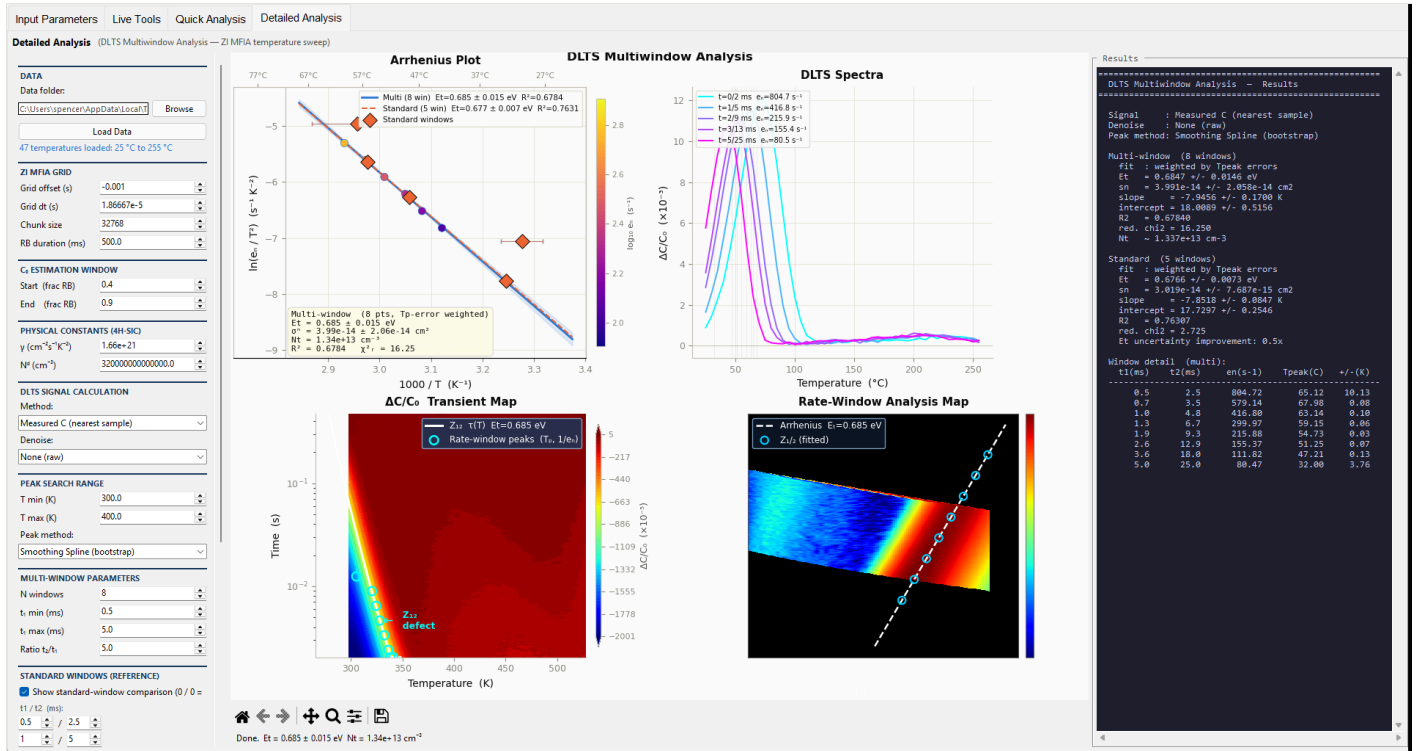


Quick Analysis on the 23 Sep run

Result: **Et = 0.677 $\pm$ 0.007 eV**, $\sigma = 3.0e-14 \pm 7.7e-15$ cm 2 , $N_t = 1.3e13$ cm $^{-3}$, all 5 windows used.

With the default windows (5/25 up to 100/490 ms) the two slowest windows peak at the very first setpoint (298 K), on the edge of the search range. Those peaks get no T_{peak} error, the **Arrhenius fit** column marks them **excluded (edge peak)**, and the fit uses the other three: Et = 0.708 $\pm$ 0.009 eV. Faster windows move the peaks to 305–338 K, inside the scan, so all of them count. When the peaks crowd one end of the scan, change the windows first.

5. Detailed Analysis. Browse to the folder, keep **RB duration** 500 ms, **Load Data**. Set **N windows** 8, **t₁ min** 0.5, **t₁ max** 5, ratio 5, **T min/max** 300/400 K, standard windows 0.5/2.5, 1/5, 2/10, 5/25, 10/50. **Run Analysis**.



Detailed Analysis on the 23 Sep run

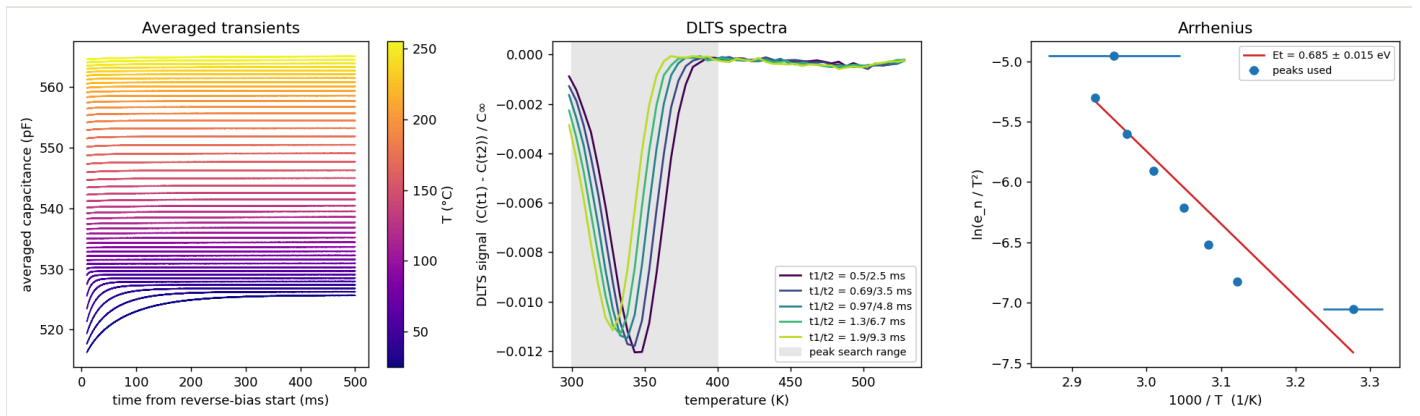
Result: $E_t = 0.685 \pm 0.015$ eV (8 windows, weighted by the peak-temperature errors), $\sigma = 4.0e-14$ cm², $N_t = 1.3e13$ cm⁻³. The **Standard** block (the same 5 windows as step 4) gives 0.6766 ± 0.0073 eV, $\sigma = 3.02e-14 \pm 7.69e-15$ cm² and $N_t = 1.34e13$ cm⁻³: the same numbers as Quick Analysis, because both tabs run the same functions on the same transients (see [Matching Detailed Analysis](#)).

The same analysis as a script (no GUI):

```
python examples/analyze_run.py "D:\copy\101901" --t1-min 0.5 --t1-max 5 --n-windows 8 --tp 300 400 --png summary.png
```

47 temperatures, 25 to 255 C, reverse bias 500 ms

$E_t = 0.685 \pm 0.015$ eV $\sigma = 3.99e-14$ cm² $N_t = 1.34e13$ cm⁻³ (8 windows used, $R^2 = 0.678$)



analyze_run.py output

It uses the Detailed Analysis tab's own functions (`_load_detailed_data`, `_compute_detailed_analysis`), so the numbers are the same.

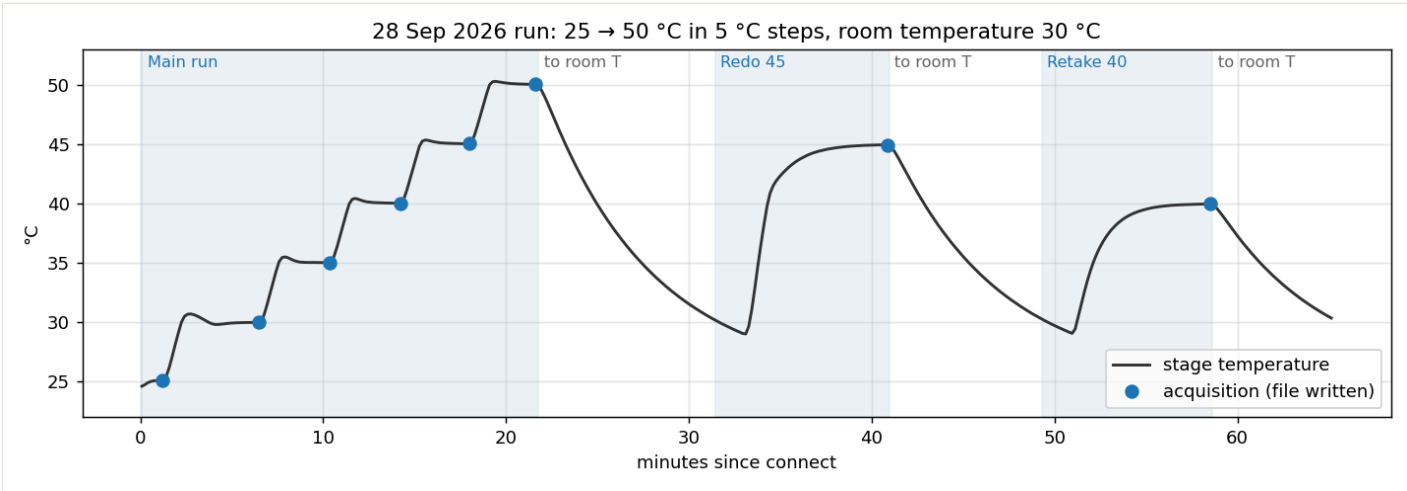
B. A short HDF5 scan on the instruments

This is the 28 Sep 2026 validation run. The stage is heated to 50 °C and the sample is biased; check both are safe.

Input Parameters:

Group	Settings
Impedance analyzer	all defaults (6 ms reverse bias, 3 ms fill, Aux -0.5 V / -1.5 V) → Connect + Get Params, Apply + Push Params
Temperature	Initial 25, Final 50, Step 5, Ramp 5, Stability Delay 0, Room Temperature 30, Room Ramp 10 → Connect + Get Params, Apply + Push Params
Output	Points 18, Reps 100, Data File Format HDF5 , Data Root Folder <code>C:\Users\spencer\Desktop\DATA\DLTS</code> → Apply + Push Params

Live Tools → Run DLTS. Afterwards, from room temperature: select 45 °C → **Redo Selected**; select 40 °C → **Remove & Retake Selected**.

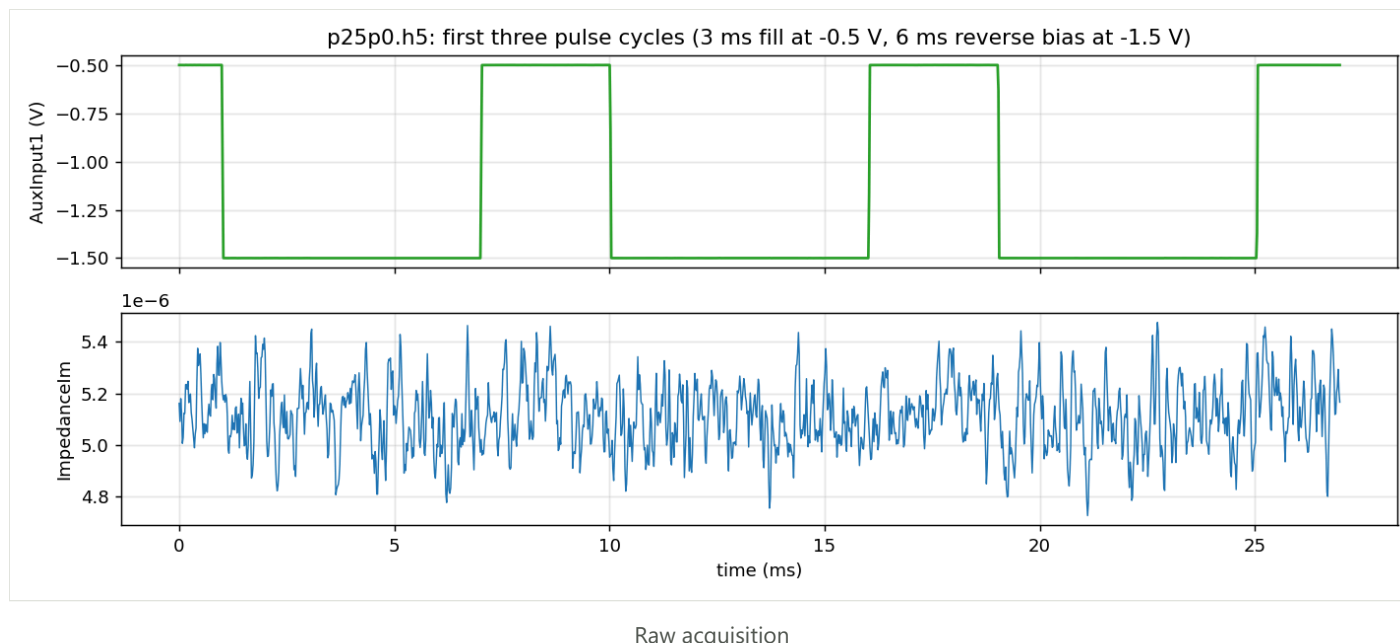


Stage temperature during the run

What to expect:

- six steps in about 22 minutes; the 30 °C step takes longest because its stability tolerance is the tightest (0.027 °C);
- each acquisition about 33 s, each HDF5 file 8.7 MB, written in 0.15 s;
- after every run and after each Redo/Retake, the stage returns to 30 °C (8–10 min passive cooling from 45–50 °C);
- the folder `...\DLTS\092826\114655\` with `p25p0.h5` ... `p50p0.h5` and `runParams.txt`.

One step's raw data (`p25p0.h5`):



With these default MFIA settings there is no clear emission in this sample; the run checks the software, not the sample. The same scan as a script, including the checks and a report, is `benchmarks/hardware/hw_run.py`. Its stored report is `benchmarks/hardware/reports/HDF5_Hardware_Benchmark_2026-09-28.pdf`.

C. One acquisition from a script

At the current stage temperature, with no ramp. The MFIA biases the sample.

```
python examples/standalone_impedance.py --out C:\temp\test_acq.h5 --points 18 --reps 100
```

It connects, calls `ziDevice.configure()` (GUI defaults plus any `--enable`, `--disable`, `--scale`, `--offset` overrides), takes one triggered acquisition, writes it with `writeDataH5()` and plots the Aux input and ImpedanceIm (as in the figure in case B). One call takes about 13–15 s: about 5 s of data for 2^{18} points (the 100 repetitions overlap, 9 ms apart) plus about 7.5 s of fixed waits in `pull_data()`.

Stage only (read, or ramp and wait):

```
python examples/standalone_temperature.py
python examples/standalone_temperature.py --goto 30 --ramp 5
```

Both instruments, a bare scan without the GUI's pause/redo:

```
python examples/standalone_scan.py --folder C:\temp\scan1 --start 25 --stop 50 --step 5 --room 30
```

See [Standalone instruments](#).

D. Convert and inspect data files

Old JSON runs to HDF5, whole data tree, originals moved into `json_originals\`:

```
python convert_json_to_h5.py --recursive "C:\Users\spencer\Desktop\DATA\DLTS" --dry-run
python convert_json_to_h5.py --recursive "C:\Users\spencer\Desktop\DATA\DLTS"
```

One HDF5 step to readable text: Input Parameters → **Export HDF5 File...**, or

```
python convert_h5_to_text.py "C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5"
```

It writes `p25p0_export.txt` (35.9 MB, about 3 s for 2^{18} points):

```
# DLTS step exported from p25p0.h5
# acquired_at: 2026-09-28T11:48:02
# format_version: 1
# run_params: {"Oscillation Amplitude": 0.3, "Oscillation Frequency": 501000.0, ...}
# setpoint_C: 25.0
# stage_temperature_C: 25.06325
# 262144 rows; columns are tab-separated, empty cells mean a shorter channel
timeStampImps   timeStampDemods  tickStampImps   tickStampDemods  ImpedanceRe      ImpedanceIm      AbsZ
AuxInput1
0.0      0.0      0.0      0.0      0.08808358165588324      5.1615895110309485e-06  0.08808358180711465
-0.4995596262122852
```

The table opens in Excel or Origin. Use `--format json` (or save as `.json` in the GUI) for a JSON file with the same attributes and channels. Names like `p25p0.txt` are refused, because the analysis tabs would read such a file as a step of the run.

Standalone instruments

Both instrument classes work without the GUI. Use them to test the hardware, take single measurements, or write your own measurement scripts. Ready-made scripts are in `examples/`.

Safety. Ramping heats or cools the sample. The MFIA applies the pulse bias (Aux output added to the signal output) as soon as its parameters are pushed.

Temperature controller: `instecTempStage_Control.mK2000B`

```
import instecTempStage_Control as tsC

stage = tsC.mK2000B()                # port 'COM7'; mK2000B(port='COM5') for another
stage.configure({'Temperature Ramp (C/min)': 5, 'Room Temperature (C)': 25})
stage.connect_temp_controller()      # opens the serial port; stage.state is True on success
print(stage.read_temp())              # current stage temperature, C

stage.go_to_temp(30, stage.tRamp, stage.tStableDelay) # ramp and wait until stable (no overall time limit)
stage.go_to_room_temp()               # ramp to Room Temperature and wait (gives up after the ramp time +
15 min)
stage.disconnect_temp_controller(stopControl=True)    # True: send STOP first; False: leave it holding/ramping
```

- `configure(values)` sets every parameter from `DEFAULT_PARAMS` plus your overrides, including the temperature grid (`stage.tempGrid`), with no connection needed.
- `go_to_temp()` blocks until the reading is within `expected_del_t(T)` of the setpoint (0.1 °C at 25 °C, 0.027 °C at 30 °C, 0.21 °C at 100 °C) and stays there 10 s later. From another thread, `stage.abort()` makes it return `-1`.
- Every serial exchange holds `stage._ioLock`, so a monitoring thread can call `read_temp()` while another thread ramps.
- Serial reads and writes time out after `tsC.SERIAL_TIMEOUT_S` (2 s). `read_temp()` asks once more, then raises `TimeoutError`, so a controller that stops answering fails your script within about 4 s instead of hanging it.

Full reference: [instecTempStage_Control](#). Script: `python examples/standalone_temperature.py --goto 30`.

Impedance analyzer: `zurichInstruments_Control.ziDevice`

```
import zurichInstruments_Control as ziC

dev = ziC.ziDevice()                 # device 'dev32271'
dev.connect_device()                  # dev.device is None if the connection failed
dev.configure({'State Enable Time': 0.5, 'State Disable Time': 0.001,
              'Aux Output Scale': -5.0, 'Aux Output Offset': 0.0, 'Aux Output Upper Limit': 5.0})
data = dev.pull_data(plot=False, trigger=True, numPoints=2**16, numReps=100)
dev.writeDataH5(data, r'C:\temp\p25p0.h5', setpoint_C=25.0, runParams=dev.params)
dev.disconnect_device()
```

- `configure(values, push=True)` sets all 23 parameters from `DEFAULT_PARAMS` (the GUI defaults) plus your overrides and pushes them. `push=False` only fills `dev.params`. Unknown names raise `KeyError`. Later `reload_params()` calls fall back to these values, so a script never hits the console prompts in `set_param_value()`.

- `pull_data(trigger=True, ...)` is what a run uses: the DAQ module records `numPoints` samples after each pulse trigger and averages `numReps` repetitions. It returns a dict of 8 NumPy arrays: `AuxInput1` (excitation, V), `ImpedanceIm`, `ImpedanceRe`, `AbsZ`, and four time channels in seconds. It takes about 7.5 s of fixed waits plus the data time, and gives up waiting after 60 s (a warning is printed).
- `writeDataH5()` writes the GUI's HDF5 step format; `writeDataJson()` the TXT format.
- All device node paths are hard-coded to `/dev32271/`. For another MFIA, change them in `push_param_to_device()`, `check_param()` and `pull_data()`.

Full reference: [zurichInstruments_Control](#). Script: `python examples/standalone_impedance.py --out C:\temp\test.h5`.

Both together

`examples/standalone_scan.py` is a complete minimal scan: configure both instruments, write `runParams.txt`, then for every setpoint ramp, acquire, and write `p25p0.h5` etc., and finally return to room temperature. Its files load in every analysis tab.

```
python examples/standalone_scan.py --folder C:\temp\scan1 --start 25 --stop 50 --step 5 --room 30 --points 16 --reps 100
```

It does not provide Pause, Redo or Remove & Retake. For those without the GUI, drive `runDltsTools.dltsRun` itself; `benchmarks/hardware/hw_run.py` shows the complete pattern (see [runDltsTools](#)).

Analysis without instruments

```
python examples/analyze_run.py RUN_FOLDER --t1-min 0.5 --t1-max 5 --tp 300 400 --png summary.png
```

It runs the Detailed Analysis computation on a run folder and saves a figure of the transients, DLTS spectra and Arrhenius fit ([Quick run case A](#)). For your own analysis code, start from `impedanceAnalysisTools.impdData` ([reference](#)).

DLTSGUI_MainWindow

What it's for

`DLTSGUI_MainWindow.py` is the entry point of the DLTS Control GUI. It creates the Tk root window, builds the notebook tabs by calling each tab module's `construct_*` function, and installs the close handler. Closing the window always tries to send the temperature stage back to the room temperature set on the **Input Parameters** tab before the program exits. It also registers backstop handlers that command the same ramp when the program ends without going through the window's close button.

Start the GUI from the repository folder:

```
python DLTSGUI_MainWindow.py
```

What the user sees

Main window

- Title: **DLTS Control GUI**.
- The window opens maximized (`root.state('zoomed')`). If that fails, the code sets the geometry to the full screen size.
- Minimum size: 860 px wide. The minimum height is `min(780, max(760, screen_height - 180))`, so it is always between 760 and 780 px.
- Notebook tab font: Arial 11, padding 6.

Tabs, in order

Tab label	Frame (<code>dltsConfig global</code>)	Built by
Input Parameters	<code>dlts.runParamsTab</code>	<code>runParamsTab.construct_runParamsTab()</code>
Live Tools	<code>dlts.livePlotTab</code>	<code>liveDataTab.construct_livePlotTab()</code>
Quick Analysis	<code>dlts.dataAnalysisTab</code>	<code>dataAnalysisTab.construct_dataAnalysisTab()</code>
Detailed Analysis	<code>dlts.detailedAnalysisTab</code>	<code>detailedAnalysisTab.construct_detailedAnalysisTab()</code>

Each tab module adds its own frame to `dlts.tabControl` and sets its label.

A fifth frame, `dlts.postprocessingTab`, is created but never added to the notebook. The call to `ppT.construct_postprocessingTab()` is commented out, and `postprocessingTab.py` is an empty file.

The only log in the application is the text box at the bottom of the **Input Parameters** tab. This module sets `dlts.maxTextLineCount = 10`, so the log keeps only the 10 most recent lines (see `dltsConfig.log_to_textbox`).

Closing the window

Clicking the window's close button calls `on_closing()`:

1. Every run-control button on the **Live Tools** tab is disabled (`ldt._set_run_control_buttons('returning')`).
2. `dlts.run_abortRequested` is set to `True`. The run loop in `runDlts_Tools.dltsRun.run_experiment()` checks this flag before each step, and `_run_single_step()` checks it again after the stage move and after the acquisition. Data acquired while the flag is set is discarded.
3. If a temperature controller is connected (`dlts.tempDev.state` is true), the code calls `tempDev.abort()`. It then commands a ramp to `tempDev.Troom` at `tempDev.roomRamp` °C/min. These two values come from **Room Temperature (C)** and **Room Ramp (C/min)** on the Input Parameters tab. They reach the device only when you click **Connect + Get Params** or **Apply + Push Params** in the temperature group. Until then the constructor defaults apply: 25 °C and 10 °C/min.
4. A small dialog opens, titled **Closing: returning to room temperature**. It shows:
 - "Returning the stage to room temperature (*Tr* °C at *ramp* °C/min). The program will close automatically when it arrives."
 - A bold status line, updated about every 2 s: Stage at *X* °C → room temperature *Y* °C (*R* °C/min), or Waiting for stage reading... (target *Y* °C) if the controller cannot be read.
 - A button, **Close now (controller keeps ramping to *Tr* °C)**.
5. A background thread polls the stage temperature every 2 s:
 - **Arrived** (within `ROOM_TOLERANCE_C` = 0.5 °C of the target): the controller is disconnected with `:TEMPerature:STOP`, the impedance analyzer is disconnected, the process pools are shut down, and the window is destroyed.
 - **Timed out**: the program exits the same way, but **without** STOP, so the controller keeps ramping on its own. The time limit is $|T_r - T| / \text{ramp} \times 60 \text{ s} + \text{ROOM_EXTRA_WAIT_S}$ (900 s), computed from the first successful reading. If no reading ever succeeds, the limit is 900 s from the start of the wait.
 - **Close now** (the button, or closing the dialog window): the program exits at once without STOP. The controller keeps ramping.
6. If no temperature controller is connected, or commanding the ramp raises an exception, the program skips the dialog. It disconnects the devices and closes at once, with `stopControl=True`. On an exception, the log first shows `Warning: could not command the room-temperature ramp on close: <error>`.

Clicking the main window's close button again while the dialog is open only raises the dialog.

Exits that bypass the close button

- `atexit.register(_emergency_room_return)` runs on a normal interpreter exit. This includes Ctrl+C in the console and an unhandled exception that ends `mainloop()`.
- SIGBREAK and SIGTERM are mapped to `_on_console_close` when the platform defines them. On Windows, closing the console window or pressing Ctrl+Break delivers SIGBREAK.

Both paths command the room-temperature ramp once, if the close handler has not already done so (`dlts.app_roomReturnSent`). They do not wait and do not send STOP.

Import

The module is an entry point. The codebase does not import it and has no alias for it. Importing it is safe because all GUI construction sits behind `if __name__ == '__main__':`. Importing it still runs `matplotlib.use('TkAgg')` and imports every tab module.

```
import DLTSGUI_MainWindow          # no GUI is built on import
```

The module imports the other parts of the application with these aliases, which the other reference pages also use:

```
import dltsConfig as dltsC
import runParamsTab as rpT
import liveDataTab as ldT
import dataAnalysisTab as daT
import detailedAnalysisTab as deT
```

Module constants and globals

Name	Value / type	Meaning
ROOM_TOLERANCE_C	0.5 (float, °C)	The close dialog treats the stage as arrived when <code>abs(Troom - T) <= 0.5</code> .
ROOM_EXTRA_WAIT_S	900 (int, s)	Extra time allowed beyond the expected ramp time before the close dialog gives up waiting.

`dltsConfig` globals this module reads or writes:

Name	Access	Meaning
<code>root</code> , <code>tabControl</code>	write	Tk root and the <code>ttk.Notebook</code> .
<code>runParamsTab</code> , <code>livePlotTab</code> , <code>dataAnalysisTab</code> , <code>detailedAnalysisTab</code> , <code>postprocessingTab</code>	write	One <code>ttk.Frame</code> per tab.
<code>maxTextLineCount</code> , <code>textlinecount</code> , <code>textboxes</code>	write	Log setup: 10 lines, counter 0, empty list of text boxes.
<code>tempDev</code> , <code>impDev</code>	read	Connected devices, created on the Input Parameters tab.
<code>manual_transientExecutor</code> , <code>detailed_executor</code>	read	Process pools that are shut down on close.
<code>run_abortRequested</code>	write	Set to <code>True</code> when the room-temperature ramp is commanded.
<code>app_closing</code>	read/ write	<code>True</code> once <code>on_closing()</code> has run. It prevents a second close sequence.
<code>app_roomReturnSent</code>	read/ write	<code>True</code> once the room ramp has been sent. The emergency handlers skip their ramp when it is set.
<code>app_closeNowRequested</code>	read/ write	<code>True</code> after Close now . It stops the wait thread.
<code>app_closeDialog</code> , <code>app_closeStatusLabel</code>	write	The close dialog and its status label.

`dltsConfig.init()` is never called, so these flags start as `None`. The code only tests them for truthiness, so `None` works the same as `False`.

Functions

`on_closing()`

The window's `WM_DELETE_WINDOW` handler. It starts the close sequence described under "Closing the window".

Returns `None`.

Side effects

- Sets `dltsc.app_closing = True`. A repeat call only lifts `dltsc.app_closeDialog`, if it exists.
- Disables all Live Tools run buttons through `ldt._set_run_control_buttons('returning')`.
- Calls `_send_room_ramp(lockTimeout=5.0)`, which sets `dltsc.run_abortRequested` and sends a serial command to the temperature controller.
- Writes to the log.
- Either calls `_finish_close(stopControl=True)` at once (no controller connected), or opens the close dialog and starts a daemon thread running `_wait_for_room_temp`.

Called by the main window's close button (`dltsc.root.protocol("WM_DELETE_WINDOW", on_closing)`).

Internal helpers

`_send_room_ramp(lockTimeout)`

Sets `dltsc.run_abortRequested = True`. If `dltsc.tempDev` exists and its `state` is true, it calls `tempDev.abort()` and `tempDev.start_ramp(tempDev.Troom, tempDev.roomRamp, lockTimeout=lockTimeout)`, sets `dltsc.app_roomReturnSent = True`, and returns `(Troom, roomRamp)`. It returns `None` when no controller is connected. `lockTimeout` is how long, in seconds, to wait for the controller's serial lock. If the wait expires, `start_ramp` sends the command anyway.

emergency_room_return(*)

The backstop registered with `atexit`, also called from `_on_console_close`. It does nothing if `dltsc.app_roomReturnSent` is already set. Otherwise it calls `_send_room_ramp(lockTimeout=2.0)` and ignores any exception. It does not wait for the ramp or send STOP.

`_on_console_close(signum, frame)`

Signal handler for `SIGBREAK` and `SIGTERM`. It calls `_emergency_room_return()` and then `os._exit(1)`. Windows kills the process a few seconds after `SIGBREAK`, and `atexit` handlers do not run on that path.

`_finish_close(stopControl)`

Disconnects the temperature controller with `disconnect_temp_controller(stopControl=stopControl)`, if it is connected. Then it disconnects the impedance analyzer (`disconnect_device()`). It calls `shutdown(wait=False, cancel_futures=True)` on `dltsc.manual_transientExecutor` and `dltsc.detailed_executor`, then runs `dltsc.root.destroy()`. Each device and executor step ignores its own exceptions. With `stopControl=False` the controller keeps ramping after the program exits.

`_close_now()`

Handler for the **Close now** button and for closing the dialog window. It sets `dltsc.app_closeNowRequested = True` and calls `_finish_close(stopControl=False)`. A second call does nothing.

`_wait_for_room_temp(Tr, ramp)`

Runs on a daemon thread. Every 2 s it reads `dltsc.tempDev.read_temp()`, compares the reading with `Tr`, and pushes the status text to `dltsc.app_closeStatusLabel` through `root.after(0, ...)`. It stops when the stage is within `ROOM_TOLERANCE_C`, when the deadline passes, or when **Close now** is requested. It then schedules `_finish_close(stopControl=(result == 'arrived'))` on the Tk thread. After **Close now** it returns without scheduling anything.

`_show_close_dialog(Tr, ramp)`

Builds the `tk.Toplevel` close dialog described above. The dialog is transient to the root window and cannot be resized. It stores the dialog in `dltsc.app_closeDialog` and the status label in `dltsc.app_closeStatusLabel`.

Startup sequence (`__main__` block)

1. `multiprocessing.freeze_support()`. The Qualitative Analysis extraction (`liveDataTab._process_raw_transients`) and the Detailed Analysis tab use `ProcessPoolExecutor`. On Windows the `spawn` start method re-imports this script in each child process. The `__main__` guard stops a child from opening another GUI, and `freeze_support()` covers frozen executables.
2. Log setup, root window, window size, and notebook style.
3. `dltsc.tabControl = ttk.Notebook(dltsc.root, padding=0)`, then each tab frame is created and its `construct_*` function is called, in the order shown in the table above.
4. `root.protocol("WM_DELETE_WINDOW", on_closing)`, `atexit.register(_emergency_room_return)`, and the signal handlers.
5. `dltsc.root.mainloop()`.

Notes and limitations

- The room-temperature target is whatever `tempDev.Troom` and `tempDev.roomRamp` hold at close time. If you edit **Room Temperature (C)** or **Room Ramp (C/min)** after connecting and do not click **Apply + Push Params**, the close sequence uses the older values.
- If no temperature controller is connected, closing destroys the window at once, even while a run thread is active. The run thread is a daemon and ends with the process.
- On a timeout, the program exits without STOP, so the controller keeps ramping. It stops the controller only when the stage actually reached room temperature.
- The wait thread calls `read_temp()` while a run thread may still be finishing a step. Both calls go through the controller's `_ioLock`, so their serial commands do not interleave.
- `_finish_close` shuts down the process pools with `wait=False`. A child process busy with an extraction is not waited for.
- `dltscConfig.init()` is not called. The tab modules seed the globals they need themselves.
- The log shows only the last 10 lines (`maxTextLineCount = 10`). Earlier messages are deleted from the text box.

runParamsTab

What it's for

`runParamsTab.py` builds the **Input Parameters** tab. On this tab you enter the settings for the Zurich Instruments MFIA impedance analyzer, the Instec temperature controller, and the output files. You connect to each instrument and push the settings to it. The tab also keeps a short history of parameter sets, has a button to export one HDF5 step file to readable text, and holds the application's only log text box.

What the user sees

The tab has two framed areas. The top frame is 860×770 px with a fixed size (`grid_propagate(False)`) and holds all the parameter groups. The bottom frame holds the log text box and grows with the window.

Impedance analyzer parameters (blue, left columns)

Each row is a label and a field. Fields listed with options are read-only dropdowns (`ttk.Combobox`). All other fields are free-text entries. Every value is stored as a string in `dltsc.z_params_vars[name]` (a `tk.StringVar`).

Label	Default	Widget / options
Oscillation Amplitude	0.300	Entry
Oscillation Frequency	501000	Entry
Oscillation ON/OFF	1 - On	0 - Off, 1 - On
Max bandwidth	10000	Entry
Input Control	0 - Manual	0 - Manual, 1 - Auto, 2 - Current Zone
Current Range	0.010	Entry
Voltage Range	3	Entry
Omega Suppression	80	Entry

Label	Default	Widget / options
Filter Harmonic	1	1 to 16
Filter Bandwidth	2	1 to 8
Data Transfer Rate	60000	Entry
Equivalent Circuit Mode	0 - 4-Terminal	0 - 4-Terminal, 1 - 2-Terminal
Threshold Input Signal	59 - TU Output Value	59 - TU Output Value, 58 - Aux Output Overload, 56 - Aux Input Overload, 55 - Output Overload, 54 - Input(I) Overload, 53 - Input(V) Overload, 52 - Trigger Out, 51 - Trigger In, 50 - DIO, 3 - Demod Theta, 2 - Demod R, 1 - Demod Y, 0 - Demod X
State Enable Time	0.006	Entry
State Disable Time	0.003	Entry
Logic Unit Not	1 - On	0 - Off, 1 - On
Aux Output Signal	13 - TU Output Value	0 - Demod X, 1 - Demod Y, 2 - Demod R, 3 - Demod Theta, 11 - TU Filtered Value, 12 - Manual, 13 - TU Output Value
Aux Output Scale	-1	Entry
Aux Output Offset	-0.5	Entry
Aux Output Lower Limit	-10	Entry
Aux Output Upper Limit	0	Entry
Signal Output Add	1 - True	0 - False, 1 - True
Trigger Source Signal	36 - Threshold 1	0 - Off, 1 - Osc Phi Demod 2, 36 - Threshold 1, 37 - Threshold 2, 38 - Threshold 3, 39 - Threshold 4, 52 - MDS Sync Out

`dltsConfig.recast_param_type('impDev', name)` converts each value before it is sent. Dropdown values become the integer before the dash. Filter Harmonic, Filter Bandwidth, and Data Transfer Rate become `int`. All other entries become `float`. The units follow the terminal prompts in `zurichInstruments_Control.ziDevice.set_param_value`: amplitude in V, frequency in Hz, bandwidth in Hz, current range in A, voltage range in V, omega suppression in dB, data transfer rate in Sa/s, State Enable/Disable Time in s, aux output values in V.

State Enable Time is the reverse-bias duration and **State Disable Time** is the fill-pulse duration of the MFIA threshold unit. This follows the docstring of `liveDataTab._legacy_run_timing`. The defaults give a 6 ms reverse bias and a 3 ms fill pulse.

Buttons under the group:

- **Connect + Get Params** calls `connect_and_get_params(devType='impedance')`.
- **Apply + Push Params** calls `apply_and_push_params(devType='impedance')`.

Temperature controller parameters (red, right columns, top)

All seven are free-text entries stored in `dlts.t_params_vars`.

Label	Default	Converted to
Initial Temperature (C)	25	float
Final Temperature (C)	25	float
Temperature Step (C)	5	float
Temperature Ramp (C/min)	5	float
Stability Delay (s)	0	int (a value such as 0.5 raises <code>ValueError</code>)
Room Temperature (C)	25	float
Room Ramp (C/min)	10	float

The run uses Initial, Final, and Step to build the temperature grid (`instecTempStage_Control.mk2000B.build_temp_grid`, inclusive of both ends). **Room Temperature (C)** and **Room Ramp (C/min)** set where, and how fast, the stage returns after a run, after a failed step, and when the GUI closes.

Buttons:

- **Connect + Get Params** calls `connect_and_get_params(devType='temperature')`.
- **Apply + Push Params** calls `apply_and_push_params(devType='temperature')`.

Output parameters (green, right columns, middle)

Stored in `dlts.d_params_vars`.

Label	Default	Widget	Meaning
Number of Points (power of 2)	16	Entry	Exponent. The run acquires <code>2 ** value</code> points per step (<code>runDlts_Tools._run_single_step</code>).
Number of Reps	500	Entry	Passed to <code>pull_data(numReps=...)</code> .

Label	Default	Widget	Meaning
Data File Format	TXT	Read-only dropdown: TXT, HDF5	TXT writes .txt files whose content is JSON. HDF5 writes .h5 files.
Data Root Folder	' '	Browse... button	Root folder for run output. The tab does not display the chosen path; only the log shows it.

The run writes each step to <Data Root Folder>\MMDDYY\HHMMSS\ with names such as p25p0.txt or n10p0.h5 (p/n for the sign, p for the decimal point). It writes runParams.txt there when the main sequence completes (runDlts_Tools.dltsRun.init_experiment and finish_experiment).

Buttons:

- **Apply + Push Params** calls `apply_and_push_params(devType='output')`. It sends nothing to any device (see below).
- **Export HDF5 File...** calls `export_h5_file()`.

Parameter History (purple)

- **Parameter History** label, then a read-only dropdown (`dlts.param_history_inputField`) listing saved sets as HH:MM:SS | <source>, newest first. After each save the newest entry is selected.
- **Save Current** saves the current values of all three groups under the source `Manual Save` and writes `Saved parameter set to history [Manual Save]` to the log.
- **Load Selected** restores the selected set into all three groups and logs `Reloaded parameter set from history [<source>]`. It only changes the GUI fields. It does not push anything to a device.

Connect and Apply actions also save an entry automatically, with sources `Connect/Get impedance`, `Connect/Get temperature`, `Apply/Push impedance`, `Apply/Push temperature`, or `Apply/Push output`. At most 5 entries are kept. History lives in memory only and is lost when the program exits.

Log

A `tk.Text` box (`dlts.textbox`, `wrap='none'`, 10 lines high) fills the bottom frame. It is appended to `dlts.textboxes`, so every `dlts.log_to_textbox()` call from any module writes here. `DLTSGUI_MainWindow` sets `maxTextLineCount = 10`, so only the 10 most recent lines are kept. The box has no scrollbar.

Import

```
import runParamsTab as rpT
```

Module constants and globals

Name	Value / type	Meaning
<code>_exportButton</code>	<code>ttk.Button</code> or <code>None</code>	The Export HDF5 File... button. Set by <code>construct_runParamsTab()</code> . <code>export_h5_file()</code> disables it during an export.

Parameter lists (`z_param_list`, `param_options`, `t_param_list`, `d_param_list`) are local variables inside `construct_runParamsTab()`, not module constants. Their contents are the tables above.

`dltsConfig` globals this module reads or writes:

Name	Access	Meaning
<code>root</code> , <code>runParamsTab</code> , <code>tabControl</code>	read	Tk root, this tab's frame, the notebook.
<code>z_params_vars</code> , <code>t_params_vars</code> , <code>d_params_vars</code>	write	<code>dict[str, tk.StringVar]</code> for each group. The keys are the labels.
<code>z_param_inputField</code> , <code>t_param_inputField</code> , <code>d_param_inputField</code>	write	<code>dict[str, widget]</code> of the entry, combobox, or button for each parameter.
<code>z_params_for_push</code> , <code>t_params_for_push</code>	write	Typed values last synced to the device. <code>runDlts_Tools</code> prefers these over the live GUI fields.
<code>impDev</code>	write	<code>zurichInstruments_Control.ziDevice</code> instance.
<code>tempDev</code>	write	<code>instecTempStage_Control.mk2000B</code> instance.
<code>param_history</code> , <code>param_history_labels</code> , <code>param_history_selection</code> , <code>param_history_inputField</code>	write	Parameter history state and widget.
<code>textbox</code> , <code>textboxes</code>	write	The log text box, and the list <code>log_to_textbox</code> writes to.

Functions

`connect_and_get_params(devType='impedance')`

Creates a device object, connects it, and copies the current GUI values into the device's `params` dict. Despite the button label, it does **not** read any settings from the instrument.

Name	Type	Default	Meaning
<code>devType</code>	str	'impedance'	'impedance' or 'temperature'. Any other value does nothing.

Returns `0`.

Side effects

- 'impedance': `dltsC.impDev = ziC.ziDevice()` (serial `dev32271` by default), then `connect_device()`, which opens a zhinst-toolkit session. Then `time.sleep(1)` and `_sync_param_values_to_device(...)`. The result goes into `dltsC.z_params_for_push`. A history entry `Connect/Get impedance` is saved and the log shows `Connect + Get Params [impedance]: name=value, ...`.
- 'temperature': `dltsC.tempDev = tsC.mk2000B()` (port `COM7`), then `connect_temp_controller()`, which opens the serial port. Then `time.sleep(1)` and the sync into `dltsC.t_params_for_push`. The sync also sets `tRamp`, `tStableDelay`, `Troom`, and `roomRamp` on the device. A history entry `Connect/Get temperature` is saved and the log line is written.

- Runs on the Tk thread, so the GUI freezes for the connection time plus 1 s.
- Each click replaces the previous device object without disconnecting it first.

Called by the two **Connect + Get Params** buttons.

apply_and_push_params(devType='impedance')

Copies the current GUI values to the connected device and sends them to the instrument.

Name	Type	Default	Meaning
devType	str	'impedance'	'impedance', 'temperature', or 'output'.

Returns 0 for 'output', otherwise None.

Side effects

- 'impedance': requires `dltsc.impDev` and `dltsc.impDev.device`. It syncs the values into `impDev.params` and `dltsc.z_params_for_push`. Then it calls `impDev.push_param_to_device(name)` for every parameter, which writes the MFIA nodes through `session.daq_server.set`. It saves history `Apply/Push impedance` and logs the values. Without a device it logs `Apply + Push Params [impedance]: No device connected. ....`
- 'temperature': requires `dltsc.tempDev` and `dltsc.tempDev.dev`. It syncs, then calls `tempDev.load_params(dltsc.t_params_for_push)`. This stores the values, rebuilds `params['Temperature Grid (C)']`, and updates `tRamp`, `tStableDelay`, `Troom`, and `roomRamp`. No serial command is sent here. The controller is commanded only when a run or the close handler moves the stage. It saves history `Apply/Push temperature` and logs the values.
- 'output': if **Data Root Folder** is set, it saves history `Apply/Push output` and logs the values. Otherwise it logs `... No data folder selected`. Nothing is sent to any device. The run reads the output fields straight from `dltsc.d_params_vars`.
- The message `Set ... parameters first by Connect + Get Params!` appears only if the sync returns an empty dict.

Called by the three **Apply + Push Params** buttons.

browse_root_folder()

Opens a folder picker (`filedialog.askdirectory()`) and stores the chosen path in `dltsc.d_params_vars['Data Root Folder']`. It then logs a preview of the folder's contents: `Data Root Folder: selected <path> -- already contains N item(s): a, b, ... (at most 12 names, then ... (M more))`. An empty folder logs `-- empty folder`. and an unreadable one logs `-- cannot list contents: <error>`. Cancelling the dialog changes nothing.

Returns 0.

Side effects Opens a Tk dialog, writes the StringVar, and writes to the log.

Called by the **Browse...** button in the output group.

export_h5_file()

Exports one HDF5 step file to a readable tab-separated `.txt` table or to `.json`, using `convert_h5_to_text.export_h5`.

1. An open dialog, **Select an HDF5 step file to export**, filtered to *.h5. It starts in the Data Root Folder.
2. A save dialog, **Save readable copy as**. It starts in the .h5 file's folder with the default name `<stem>_export.txt`. The file types are .txt and .json. The output format follows the chosen extension: .json gives JSON, anything else gives the text table.
3. The button is disabled and the log shows `Exporting <file> to <out>...`.
4. A daemon thread runs `h5txt.export_h5(h5Path, outPath)`. It then uses `root.after(0, ...)` to re-enable the button and log either `Exported <file> -> <output> (TXT, R rows x C channels, S MB)`. Or `Export of <file> failed: <error>`.

Cancelling either dialog returns without exporting.

Returns `0`.

Side effects Tk dialogs, one background thread, and one output file. Per `convert_h5_to_text`, a failed export leaves no partial file.

Called by the **Export HDF5 File...** button.

construct_runParamsTab()

Builds the whole tab. It adds `dlts.runParamsTab` to the notebook as **Input Parameters**, creates the namespaced ttk styles (`Dlts.Blue.TLabel`, `Dlts.Blue.TButton`, and the same for Red, Green, and Purple), creates every widget and Tk variable listed above, initializes the history, and registers the log text box.

Returns `0`.

Side effects Writes the `dltsConfig` globals listed above and the module global `_exportButton`.

Called by `DLTSGUI_MainWindow` at startup.

Internal helpers

_format_param_snapshot(param_vars)

Returns `"name=value, name=value, ..."` for a dict of Tk variables. A value without a `.get()` is used as it is. Used in log messages.

_get_param_values(param_vars)

Returns a plain `{name: value}` dict read from a dict of Tk variables. A value without a `.get()` is copied as it is.

_apply_param_values(param_vars, values)

Calls `.set(value)` on each Tk variable whose name appears in `values`. Unknown names and `set` errors are ignored.

_ensure_param_history_state()

Creates `dlts.param_history` (list), `dlts.param_history_labels` (list), and `dlts.param_history_selection` (`tk.StringVar`) if any of them is missing or `None`.

`_capture_current_param_set(source='Manual Save')`

Returns one history entry: `{'timestamp': 'HH:MM:SS', 'source': source, 'z_params': {...}, 't_params': {...}, 'd_params': {...}}`, with plain values from the three groups.

`_update_param_history_field()`

Rebuilds `dlts.param_history_labels` as `"<timestamp> | <source>"`, loads the labels into the history dropdown, and selects the first (newest) entry, or clears the selection if the history is empty.

`_save_current_param_set(source='Manual Save', should_log=False)`

Inserts a new entry at the front of `dlts.param_history`, trims the list to 5 entries, and refreshes the dropdown. When `should_log` is true, it writes `Saved parameter set to history [<source>]`.

`_load_selected_param_set()`

Finds the selected label in `dlts.param_history_labels` and writes that entry's values back into all three groups. If nothing is selected, it logs `No parameter history entry selected`. Returns `0`. If two entries have the same label (same second, same source), `list.index` picks the newer one.

`_sync_param_values_to_device(device, param_vars, device_type)`

Builds typed values and stores them in `device.params`. For `'impDev'` and `'tempDev'` each value comes from `dlts.recast_param_type(device_type, name)`, which reads `dlts.z_params_vars` or `dlts.t_params_vars` directly rather than the `param_vars` argument. For each name in `device.params` that has a typed value, it calls `device.set_param_value(name, typed_values)`. Returns the typed dict, or `{}` if `device` or `param_vars` is `None`. A non-numeric entry raises `ValueError` from `recast_param_type`. Nothing catches it, so the button callback stops and Tk prints the traceback to the console.

Notes and limitations

- **Connect + Get Params** does not read settings from either instrument. It pushes the GUI values into the Python device object.
- Connection failures are printed to the console, not written to the log. `ziDevice.connect_device()` and `mK2000B.connect_temp_controller()` catch their own errors with `print(...)`. The tab still logs `Connect + Get Params [...]` and saves a history entry, so a failed connection looks successful in the GUI. `ziDevice.connect_device()` calls `ziDiscovery.find/get` outside its `try`, so an exception there reaches Tk's callback handler and nothing is logged.
- After connecting, a run uses the values from the last **Connect + Get Params** or **Apply + Push Params**. `runDlts_Tools._get_runtime_param_value` prefers `z_params_for_push` and `t_params_for_push`, and the temperature grid comes from `tempDev.params`. Edits made after that without **Apply + Push Params** are ignored for impedance and temperature. The output group is always read live from the GUI.
- The output group's **Apply + Push Params** only logs and saves history. `dlts.d_params_for_push` is never written.
- **Stability Delay (s)** must be an integer.

- `dlts.maxTextLineCount = 10`: the log keeps 10 lines and has no scrollbar, so long messages and bursts of messages scroll out quickly.
- The top frame has a fixed 860 × 770 px size. On a smaller window its lower rows are clipped rather than scrolled.
- The chosen Data Root Folder is shown only in the log, not next to the **Browse...** button.
- **Data Root Folder** starts empty. **Run DLTS** with it empty (or relative) stops before anything is created: the log shows `Error: Data Root Folder is not set, Choose it with Browse...` on the Input Parameters tab, then `Apply + Push Params.` and `Error: Failed to initialize the experiment.` (checked in `runDlts_Tools.dltsRun.init_experiment`). Earlier versions wrote such a run to the root of the current drive.
- Device identifiers are fixed in the device classes: MFIA serial `dev32271`, temperature controller on `COM7`.

liveDataTab

What it's for

`liveDataTab.py` builds the **Live Tools** tab. From this tab you start a DLTS temperature scan and control it: pause, resume, redo steps, or remove and retake them. The tab shows each temperature's emission transients as the files arrive or from a finished run. Its **Qualitative Analysis** section extracts and averages capacitance transients from a data folder in any of the three supported formats. The pure extraction functions (`_compute_*`) are also used by the Detailed Analysis tab and by the tests and benchmarks.

What the user sees

The tab has three stacked areas:

1. The **Run DLTS** button across the top, in a frame 52 px high.
2. A vertical `tk.PanedWindow` with a draggable sash. The top pane is **Automated / Live Data** and the bottom pane is **Qualitative Analysis**. Both start 460 px high and both stretch.
3. No log. Every message goes to the log box on the **Input Parameters** tab.

Run DLTS

Run DLTS (Segoe UI 14 bold, `dltsc.run_button`) calls `start_thread()`. It does nothing if `dltsc.run_busy` is already set. Otherwise, on the Tk thread, it:

- sets `run_busy = True`, `run_pauseRequested = False`, and `run_paused = False`;
- sets the buttons to the `running` state (only **Pause** enabled);
- clears the Live view state (`_reset_live_plot_state('live')`), which bumps `dltsc.livePlot_liveRunToken` and switches the view to **Live**;
- starts the output-file watcher (`_schedule_live_poll`);
- starts a daemon thread running `start_dltsc()`;
- starts the step-list refresh loop (`_poll_step_listbox_while_busy`, every 500 ms while `run_busy`).

`start_dltsc()` creates `runDltsc_Tools.dltscRun()` and stores it in `dltsc.run_dltscInstance`. It calls `init_experiment()`, which checks the devices and parameters, builds the temperature grid, creates the output folder, and publishes `dltsc.run_dataFileNames`. It marks every grid step `pending` and then calls `run_experiment()`. When the run returns, `_handle_run_status(status, isMainSequence=True)` runs on the Tk thread.

The **Input Parameters** tab must be set up first. Both devices must have been created with **Connect + Get Params**, or `init_experiment()` fails with `Error: Failed to initialize the experiment.`

Automated / Live Data (top pane)

Header row, left to right:

Widget	What it does
Automated / Live Data (bold label)	Heading.
View: radio buttons Live / Offline	<code>_on_mode_toggle()</code> . Switches which dataset is drawn. Live and Offline each keep their own data (<code>dlts.livePlot_live*</code> and <code>dlts.livePlot_offline*</code>) and their own last-selected temperature, so switching never discards anything.
Load Existing Run (Offline)	<code>_load_offline_run()</code> . File picker Select DLTS Data Files (Offline Run) with filters <code>*.txt *.json *.h5, *.h5, *.csv</code> , and all files. Select all step files of a run. Loading runs on a background thread and replaces the previous Offline data.
Dataset: dropdown	Read-only. One entry per temperature, labeled <code>K (°C)</code> , for example <code>298.15 (25)</code> . The Kelvin value is <code>impdData.dataTemps</code> (exact K, a float); the Celsius value is <code>round(K - 273.15)</code> . Choosing one redraws from cached results (<code>_on_dataset_selected</code>).
Denoise Method: dropdown	Read-only. <code>pca</code> (default), <code>wavelet</code> , <code>sgolay</code> , <code>lowess</code> . Changing it recomputes <code>impdData.filter_emissions()</code> for the displayed mode on a background thread (<code>_recompute_denoise_and_redraw</code>).
Status label	For example <code>Waiting for data...</code> , <code>Processing N new file(s)...</code> , <code>Live: N temperature(s) loaded.</code> , <code>Run complete: N temperature(s) loaded.</code> , <code>Loading N offline file(s) – this can take a while for many temperatures...</code> , <code>Offline: N temperature(s) loaded.</code>

Run Control column (left, 250 px wide, `tk.LabelFrame`):

Button	Enabled when	What it does
Pause	running	<code>_pause_run()</code> : sets <code>dlts.run_pauseRequested = True</code> , disables itself, and logs <code>Pause requested</code> ; the run will stop after the current temperature step completes. <code>run_experiment()</code> checks the flag before each step. The stage stays at its current temperature.
Resume	paused; or idle after a run, if <code>currentStepIndex < len(tempGrid)</code>	<code>_resume_run()</code> : logs <code>Resuming run...</code> and runs the main sequence from <code>dltsRun.currentStepIndex</code> on a new thread, reusing the same instance and connected devices.
Redo Selected	paused, or idle after a run	<code>_redo_selected_steps()</code> : runs only the selected steps in ascending order and overwrites their files. It does not move the resume point. With nothing selected it logs <code>Redo: select at least one step from the list first.</code>
Remove & Retake Selected	paused, or idle after a run	<code>_retake_selected_steps()</code> : like Redo, but deletes each step's existing file first (<code>deleteFirst=True</code>).

Below the buttons, a multi-select listbox (`dlts.run_stepListbox`) shows each grid step as `N. T.TT °C – status`. The status is `pending`, `running`, `done`, `failed`, `paused`, or `aborted`, taken from `dltsRun.stepStatus`. The selection survives refreshes.

Button states (`_set_run_control_buttons`):

Mode	Run DLTS	Pause	Resume	Redo / Retake
idle (startup)	on	off	off	off

Mode	Run DLTS	Pause	Resume	Redo / Retake
running	off	on	off	off
paused	off	off	on	on
returning (stage ramping to room temperature, or GUI closing)	off	off	off	off
idle_with_instance (finished or failed, devices still connected)	on	off	on if steps remain	on

End of a run (`_handle_run_status`):

- `completed`, main sequence: calls `finish_experiment()`, which writes `runParams.txt` into the run folder on the Tk thread. Logs `DLTS run completed!`, then returns the stage to room temperature.
- `completed`, Redo/Retake: logs `Redo/Retake completed.` and returns the stage to room temperature. `runParams.txt` is not rewritten.
- `paused`: logs `Run paused. Click Resume to continue, or Redo/Remove & Retake specific steps below.` The stage stays where it is.
- `error`: logs `Run stopped due to an error. Devices remain connected; Resume/Redo/Retake are available.` and returns the stage to room temperature.
- `aborted`, or the GUI is closing: logs `Run stopped: GUI is closing.` and does nothing else. The close handler in `DLTSGUI_MainWindow` has already commanded the ramp.

The return to room temperature (`_return_to_room_temp_async`) runs `dltsRun.return_to_room_temp()` on a daemon thread. That call ramps to **Room Temperature (C)** at **Room Ramp (C/min)** and waits until the stage is within 0.5 °C, up to the expected ramp time plus 900 s. Meanwhile `run_busy` stays `True` and all buttons are off (`returning`).

Plot area (right): one matplotlib figure (10 × 4 in, 100 dpi) with a navigation toolbar and two side-by-side axes:

- **Emission 0 (T = K K)**: the raw mean of the first emission cluster (`ymean`, grey) and, if present, the denoised curve (`yFiltered`, orange, legend `Denoised (<method>)`). X axis **Time (s)**, y axis **Impedance Im (F)**.
- **All Emissions Aligned (T = K K)**: every emission block for that temperature, colored with the viridis colormap. A legend (`#1`, `#2`, ...) appears when there are 12 blocks or fewer.

With no data, both axes show `Waiting for data...`.

Live watcher: `_schedule_live_poll` checks `dlts.run_dataFileNames` once a second. It picks up files that exist and have not been ingested yet, and hands them to `_ingest_files_async`. That function reads the first file with `impdData.read_data()` and each later file with `append_data()`. It then runs `cleanup_data()`, a forced re-clustering (`selected_emissions(0)`), `filter_emissions(method)`, and `selected_emissions(-1)` on a background thread. Polling stops once every listed file has been ingested.

Qualitative Analysis (bottom pane)

A left column (230 px, scrollable with the mouse wheel while the pointer is over it) and a plot on the right.

Directory Loader Config

- **Select Source Folder** (`_browse_manual_folder`): folder picker **Select DLTS Source Folder**, starting in the last folder or the working directory. It first logs a one-line preview: `Manual analysis: selected <path> -- N item(s): ... - <format guess>`. Then it scans the folder on a background thread and **replaces** the current dataset.

- **Append Source Folder** (`_append_manual_folder`): folder picker **Select DLTS Source Folder to Append**. It adds that folder's temperatures to the current dataset. A temperature that is already loaded keeps its first-loaded copy, and the log lists the skipped ones. With nothing loaded yet, it behaves like **Select Source Folder**.
- Label: Source: (none selected), then Source: <folder> or Source: N folder(s) combined (latest: <folder>).

Both buttons, and **Extract & Average Transients**, are disabled while a scan or an extraction is running.

Three folder formats are detected, in this order:

Format	Registry tag	How it is recognized	Temperatures from
ZI single CSV	'zi'	A file matching <code>*imps_0_sample_param1_avg_header*.csv</code> directly in the folder	<code>history_name</code> in the header CSV, pattern <code>^(\d+)C_</code> (non-negative whole °C only); one chunk per temperature in one data CSV
ZI subfolder per temperature	'zi_subfolder'	Subfolders named like <code>0C</code> , <code>100C</code> , <code>n10C</code> , <code>120C_000</code> (<code>_ZI_SUBFOLDER_PATTERN</code>), each with its own <code>*imps_0_sample_param1_avg_<n>.csv</code>	Subfolder name; n means negative
Legacy per-temperature files	plain path or 'legacy_chunk'	File names matching <code>_LEGACY_FILENAME_PATTERN</code> , for example <code>p25p0.txt</code> , <code>n10p5.h5</code> , <code>P120C_001.csv</code> ; plus semicolon CSVs with <code>chunk</code> , <code>smoothed_value</code> , or <code>timestamp</code> in their header	File name, or a fixed chunk-to-temperature map for multi-chunk CSVs

The output of **Run DLTS** (`p25p0.txt` or `.h5` files) is the legacy per-temperature format.

Available Temperatures Filter

- A multi-select listbox (3 rows high, scrolls) showing each temperature as `T °C`, sorted. After each scan all entries are selected.
- **Select All** and **Clear All**.

Timing Boundaries (`dlts.manual_paramVars`)

Label	Key	Default	Used for
Filling Duration (ms):	<code>fp_ms</code>	1.0	Display only
Reverse Bias (ms):	<code>rb_ms</code>	500.0	Extraction: legacy window length, and <code>C_infinity</code> at 90 % of it
Analysis Slice Start (ms):	<code>slice_start</code>	2.0	Plot only: the Qualitative plot starts here (<code>_plot_slice_ms</code>)
Analysis Slice End (ms):	<code>slice_end</code>	490.0	Plot only: the Qualitative plot ends here

These fields are filled automatically on a **Select Source Folder** scan, but not on an append:

- Legacy folder: from `runParams.txt` in that folder (`_legacy_run_timing`). Fill = **State Disable Time** × 1000, reverse bias = **State Enable Time** × 1000, slice end = 0.98 × reverse bias. The log shows Manual analysis: Timing Boundaries set from `runParams.txt` (fill F ms, reverse bias R ms). `runParams.txt` exists only after a main sequence completed.

- ZI formats: from the folder name, regexes `FP\w+(\d+(?:\.\d+)?)ms` and `RB[\w\+|-]+\d+(?:\.\d+)?)ms`, for example `..._FP_1ms_RB-5V_500ms`. Slice end = $0.98 \times \text{RB}$.

Execution Action

- **Extract & Average Transients** (`_process_raw_transients`, bold, light green): reads **Reverse Bias (ms)**. A non-numeric value logs `Manual analysis: Reverse Bias (ms) must be numeric`. It splits the selected temperatures by format tag and submits `_compute_mixed_transients` to a single-worker `ProcessPoolExecutor` (`dltsc.manual_transientExecutor`, created on first use and reused). A background thread waits for the result, and the plot is drawn on the Tk thread. The results are stored in `dltsc.manual_processedTransients`. The Quick Analysis tab (`dataAnalysisTab`) reads them from there.
- Status label: Status: Idle, Processing N temperature(s)..., Transients ensembled completely – N traces., or N of M traces; see log for the rest. Every skipped temperature is logged with its reason, for example `Manual analysis: 25.0°C: no fill pulses found: ...`.

Plot: **Averaged Capacitance Transients Profile**, x axis **Time from Reverse Bias Start (ms)**, one line per temperature. Only the Analysis Slice (Start to End) is drawn, so the fill-pulse edge at $t = 0$ does not set the y scale; the extracted data passed to other tabs is not cut. Up to `_MAX_LEGEND_TRACES` (10) traces get a legend ($^{\circ}\text{C}$, upper right); more traces are colored by temperature (plasma colormap) with a **Temperature ($^{\circ}\text{C}$)** color bar instead. The y axis is **Capacitance (pF)**, or nF, μF , or mF chosen from the largest value (`_capacitance_axis_units`). Offset notation is off. Each curve is thinned to at most 3000 points for drawing (`_downsample_for_plot`). With no results, the plot shows `No transients extracted`. See the log for the reason per temperature.

Import

```
import liveDataTab as ldT
```

Importing the module creates no widgets. It imports `numpy`, `pandas`, `matplotlib` (including `FigureCanvasTkAgg`), `dltscConfig`, `impedanceAnalysis_Tools`, and `runDlts_Tools`. The `_compute_*` functions and `_find_reverse_bias_starts`, `_legacy_run_timing`, `_capacitance_axis_units`, and `_downsample_for_plot` need no Tk and no hardware.

Module constants and globals

Name	Value / type	Meaning
<code>DENOISE_METHODS</code>	<code>['pca', 'wavelet', 'sgolay', 'lowess']</code>	Options of the Denoise Method : dropdown, passed to <code>impdData.filter_emissions(method=...)</code> . The first entry is the default.
<code>_LEGACY_FILENAME_PATTERN</code>	<code>re.compile(r'^([npNP])(\d+)(?:[pP](\d+))?[cC](?:_\d+)?\.(txt csv h5)\$')</code> (backslashes before the pipes are table escaping only)	Legacy step file name: sign (p/n), whole degrees, optional p + decimals, optional C, optional <code>_NNN</code> , extension <code>.txt</code> , <code>.csv</code> , or <code>.h5</code> (lower case only). Groups: sign, integer part, decimal part, extension.
<code>_ZI_REGISTRY_TAGS</code>	<code>('zi', 'zi_subfolder')</code>	Registry tags that count as ZI in <code>_registry_format_label</code> .

Name	Value / type	Meaning
<code>_ZI_SUBFOLDER_PATTERN</code>	<code>re.compile(r'^(n?)(\d+)C(?:_\d+)?\$', re.IGNORECASE)</code>	ZI per-temperature subfolder name. Group 1 is <code>n</code> for negative, group 2 is whole degrees.
<code>_MAX_PLOT_POINTS</code>	3000	Default point limit of <code>_downsample_for_plot</code> .
<code>_MAX_LEGEND_TRACES</code>	10	Above this many traces the Qualitative plot uses a temperature color bar instead of a legend.
<code>_manualColorbar</code>	None or matplotlib Colorbar	The Qualitative plot's current color bar; removed before each redraw.
<code>_MIN_PULSE_HEIGHT_V</code>	0.05 (V)	Smallest excitation swing that <code>_find_reverse_bias_starts</code> treats as real pulsing.

Fixed values inside functions:

Where	Value	Meaning
<code>_process_raw_transients</code>	<code>samplingRateS = dltsc.manual_samplingRate</code> or <code>1.8666666666666665e-05</code>	Sample period (s) for legacy files, about 53.57 kSa/s. <code>dltsc.manual_samplingRate</code> is never set anywhere, so the fixed value is always used.
<code>_process_raw_transients</code>	<code>cInfTargetMs = 0.90 * rb_ms</code>	Time at which <code>C_infinity</code> is read.
ZI defaults	<code>gridColOffset = -0.001 s</code> , <code>gridColDelta = 1.86667e-05 s</code> , <code>chunkSize = 32768</code>	Used when a ZI header CSV is missing or unreadable.
<code>_compute_legacy_dataset</code>	chunk map {0: 120.0, 1: 125.0, ..., 8: 160.0}	Temperatures assigned to chunks of a multi-chunk legacy CSV.

`dltscConfig` globals owned by this tab:

Group	Names
Run control	<code>run_button</code> , <code>run_pauseButton</code> , <code>run_resumeButton</code> , <code>run_redoButton</code> , <code>run_retakeButton</code> , <code>run_stepListbox</code> , <code>run_dltscInstance</code> , <code>run_busy</code> , <code>run_pauseRequested</code> , <code>run_paused</code>
Read from other modules	<code>run_dataFileNames</code> (written by <code>runDltscTools.init_experiment</code>), <code>app_closing</code> (written by <code>DLTSGUI_MainWindow</code>)
Automated plot	<code>livePlot_activeMode</code> , <code>livePlot_modeVar</code> , <code>livePlot_denoiseMethodVar</code> , <code>livePlot_datasetVar</code> , <code>livePlot_datasetCombo</code> , <code>livePlot_figure</code> , <code>livePlot_axEmission0</code> , <code>livePlot_axAllEmissions</code> , <code>livePlot_canvas</code> , <code>livePlot_statusLabel</code>
Live state	<code>livePlot_liveImpdData</code> , <code>livePlot_liveEmission0Data</code> , <code>livePlot_liveAllEmissionsData</code> , <code>livePlot_liveDatasetSel</code> , <code>livePlot_liveRunToken</code> , <code>livePlot_pollAfterId</code> , <code>livePlot_processedFiles</code> , <code>livePlot_liveIngestBusy</code>

Group	Names
Offline state	<code>livePlot_offlineImpdData</code> , <code>livePlot_offlineEmission0Data</code> , <code>livePlot_offlineAllEmissionsData</code> , <code>livePlot_offlineDatasetSel</code> , <code>livePlot_offlineRunToken</code> , <code>livePlot_offlineIngestBusy</code>
Qualitative Analysis	<code>manual_dataDirectory</code> , <code>manual_datasetRegistry</code> , <code>manual_ziMode</code> , <code>manual_ziParamByFile</code> , <code>manual_ziDataFile</code> , <code>manual_ziGridColOffset</code> , <code>manual_ziGridColDelta</code> , <code>manual_ziChunkSize</code> , <code>manual_sourceFolders</code> , <code>manual_samplingRate</code> (read only), <code>manual_processedTransients</code> , <code>manual_paramVars</code> , <code>manual_tempListbox</code> , <code>manual_folderLabel</code> , <code>manual_selectFolderButton</code> , <code>manual_appendFolderButton</code> , <code>manual_extractButton</code> , <code>manual_processingBusy</code> , <code>manual_loadingBusy</code> , <code>manual_transientExecutor</code> , <code>manual_figure</code> , <code>manual_ax</code> , <code>manual_canvas</code> , <code>manual_statusLabel</code>

The snapshot dicts (`livePlot_*Emission0Data`, `livePlot_*AllEmissionsData`) map the Kelvin temperature to a copy of that temperature's `impdData.dataEmissions` record. The redraw reads the keys `xTimeStampImps` (or `x`), `ymean` (or `y`), and `yFiltered` for emission 0, and `xTimeStampImps` (or `x`) and `y` (1-D or 2-D, one column per block) for all emissions.

Functions

`start_thread()`

The **Run DLTS** handler. See "Run DLTS" above for the full sequence.

Returns `None`.

Side effects Sets `run_busy`, `run_pauseRequested`, and `run_paused`. Resets the Live state and bumps `livePlot_liveRunToken`. Schedules `root.after` callbacks for the file watcher and the step list. Starts a daemon thread running `start_dlts`.

Called by the **Run DLTS** button.

`start_dlts()`

The body of the run thread. It creates `rdT.dltsRun()` and stores it in `dlts.run_dltsInstance`. It calls `init_experiment()`; on failure (`< 0`) it logs the error and uses `root.after` to clear `run_busy` and return the buttons to `idle`. On success it marks every step `pending`, schedules a step-list refresh, calls `run_experiment()` (blocking: this is the whole scan), and schedules `_handle_run_status(status, isMainSequence=True)`.

Returns `None`.

Side effects Drives the hardware through `runDlts_Tools` (stage moves, MFIA acquisition), writes the step data files, and writes to the log from the background thread.

Called by the thread started in `start_thread()`.

`construct_livePlotTab()`

Adds `dlts.livePlotTab` to the notebook as **Live Tools** and builds the **Run DLTS** button, the paned window, `_build_autoPlotFrame`, and `_build_manualPlotFrame`.

Returns `None`.

Side effects Creates every widget on this tab and writes the widget globals listed above.

Called by `DLTSGUI_MainWindow` at startup.

Internal helpers

Run control

`_pause_run()`

When `run_busy`, sets `dlts.run_pauseRequested = True`, disables **Pause**, and logs the pause request. The run stops before the next step, not during the current one.

`_run_control_thread(indices=None, deleteFirst=False, isMainSequence=False)`

The shared launcher for Resume, Redo, and Retake. It does nothing if there is no `run_dltsInstance` or if `run_busy` is set. Otherwise it sets `run_busy`, clears `run_pauseRequested`, sets the buttons to `running`, and starts a daemon thread calling `run_dltsInstance.run_experiment(indices=indices, deleteFirst=deleteFirst)`. The result goes to `_handle_run_status` through `root.after`. It also starts the step-list refresh loop. It does not restart the Live file watcher.

`_resume_run()`

Logs `Resuming run...` and calls `_run_control_thread(indices=None, isMainSequence=True)`.

`_get_selected_step_indices()`

Returns `list(run_stepListbox.curselection())`, or `[]` if the listbox does not exist.

`_redo_selected_steps()`

For the selected steps, logs `Redoing N step(s)...` and calls `_run_control_thread(indices, deleteFirst=False)`.

`_retake_selected_steps()`

For the selected steps, logs `Removing and retaking N step(s)...` and calls `_run_control_thread(indices, deleteFirst=True)`.

`_set_run_control_buttons(mode)`

Enables or disables the five run buttons according to the table in "What the user sees". `mode` is one of `'idle'`, `'running'`, `'paused'`, `'returning'`, `'idle_with_instance'`; any other value acts as `'idle'`.

`DLTSGUI_MainWindow.on_closing` calls it with `'returning'`.

`_handle_run_status(status, isMainSequence)`

Runs on the Tk thread when a run thread ends. It clears `run_busy`, refreshes the step list, and then acts on `status` (`'completed'`, `'paused'`, `'error'`, `'aborted'`) as described in "End of a run".

`_return_to_room_temp_async()`

Sets `run_busy = True` and the buttons to `returning`. It starts a daemon thread calling `run_dltsInstance.return_to_room_temp()`. When that returns, it clears `run_busy` and, unless the GUI is closing, sets `idle_with_instance`.

`_refresh_step_listbox()`

Rebuilds the step list from `run_dltsInstance.tempDevice.tempGrid` and `run_dltsInstance.stepStatus`, keeping the current selection.

`_poll_step_listbox_while_busy()`

Calls `_refresh_step_listbox()` and reschedules itself every 500 ms while `run_busy` is true.

Automated / Live Data

`_active_impd()`

Returns `livePlot_offlineImpdData` in Offline mode, otherwise `livePlot_liveImpdData`.

`_active_emission0Data()`

Returns the emission-0 snapshot dict of the active mode, or `{}`.

`_active_allEmissionsData()`

Returns the all-emissions snapshot dict of the active mode, or `{}`.

`_mode_label(mode)`

Returns `'Live'` for `'live'`, otherwise `'Offline'`.

`_reset_auto_plot_placeholder()`

Clears both axes, draws `Waiting for data...` with the titles **Emission 0** and **All Emissions Aligned**, and empties the **Dataset:** dropdown.

`_reset_live_plot_state(mode)`

Clears one mode's state and then calls `_set_livePlot_mode(mode)`. For `'live'` it bumps `livePlot_liveRunToken`, clears the Live `impdData`, snapshots, selection, and `livePlot_processedFiles`, clears `livePlot_liveIngestBusy`, and cancels any pending poll. For `'offline'` it bumps `livePlot_offlineRunToken` and clears the Offline state. The other mode is left untouched.

`_set_livePlot_mode(newMode)`

Remembers the old mode's dropdown selection, sets `livePlot_activeMode` and the radio button, and then either fills the dropdown and redraws (if the new mode has data) or shows the placeholder. It also updates the status label.

`_on_mode_toggle()`

Radio-button callback. Calls `_set_livePlot_mode(livePlot_modeVar.get())`.

`_format_dataset_label(t)`

Returns `f"{t} ({round(t - 273.15)})"`, for example `298.15 (25)` or `262.65 (-10)`. A non-numeric `t` is returned as `str(t)`.

`_get_selected_dataset_temp()`

Parses the Kelvin part of the dropdown text back to `int`, or to `float` if that fails. Returns `None` if the text is empty or unparseable.

`_update_dataset_dropdown(temps, preferred=None)`

Sets the dropdown values from `temps`. It selects `preferred` if that label is present, otherwise the last entry.

`_on_dataset_selected(*_args)`

`<<ComboboxSelected>>` handler. Calls `_redraw_auto_plots()`.

`_schedule_live_poll(token)`

The Live file watcher, run on the Tk thread once a second through `root.after(1000, ...)`. It stops if `token` no longer equals `livePlot_liveRunToken`. When no ingest is busy, it passes the files that exist and are not yet in `livePlot_processedFiles` to `_ingest_files_async`. It stops, and sets the status `Run complete: N temperature(s) loaded.` in Live mode, once every name in `run_dataFileNames` has been processed. Each step file is checked with `os.path.exists`, so a `*.h5.tmp` file that is still being written is not picked up.

`_ingest_files_async(paths, token)`

Sets `livePlot_liveIngestBusy` and starts a daemon thread. The thread loads `paths` into the Live `impdData`: `read_data()` for the first file, `append_data()` afterwards. It then runs `cleanup_data()`, sets `dataEmissionClusterParams = None` to force re-clustering, and calls `selected_emissions(emissionIndex=0)`, `filter_emissions(method, emissionIndex=0, recalculate=True, interactivePlot=False)`, and `selected_emissions(emissionIndex=-1)`, taking a snapshot after each. `apply()` runs on the Tk thread. It clears the busy flag, drops the result if the token is stale, logs any errors as `Live plot: ...`, stores the `impdData`, the processed paths, and the snapshots, and redraws if Live is shown. A file that fails to load is not marked processed, so the next tick tries it again.

`_recompute_denoise_and_redraw(*_args)`

`trace_add('write')` callback of the denoise variable. It does nothing if the active mode has no data or is busy. Otherwise it runs `selected_emissions(0)` and `filter_emissions(method, ...)` on a background thread, replaces the active mode's emission-0 snapshot if the token still matches, and redraws if that mode is still shown. The all-emissions snapshot is not recomputed, because it is raw data.

`_redraw_auto_plots(*_args)`

Draws both axes from the active mode's snapshots for the selected temperature, as described under "Plot area". If the selection has no emission-0 snapshot, it switches to the highest temperature that has one and updates the dropdown. With no data it shows the placeholder. It only reads cached results, so it is cheap enough for the Tk thread.

`_load_offline_run()`

Load Existing Run (Offline) handler. After the file dialog it resets the Offline state (switching the view to Offline), sets `livePlot_offlineIngestBusy`, and starts a daemon thread running the pipeline below. `apply()` stores the result if the offline token still matches, and fills the dropdown and redraws if Offline is shown. The busy flag is cleared through `root.after` in a `finally` block. Errors are logged as `Live plot: failed to load the selected offline files.` or `Live plot: error processing offline data: ...`.

The underlying `impedanceAnalysis_Tools` pipeline can be run without the GUI. This is the same sequence the tests use (`tests/test_hdf5_analysis.py::run_offline_pipeline`):

```
import glob
import impedanceAnalysis_Tools as iaT

files = sorted(glob.glob(r'D:\data\092826\141503\p*.txt')) # or *.h5; both may be mixed
impd = iaT.impdData(fName=files)
assert impd.read_data() == 0 # 0 on success
impd.cleanup_data()
impd.selected_emissions(emissionIndex=0)
impd.filter_emissions(method='pca', emissionIndex=0, recalculate=True, interactivePlot=False)
emission0 = {T: dict(rec) for T, rec in impd.dataEmissions.items()} # keys: K (float)
impd.selected_emissions(emissionIndex=-1)
allEmissions = {T: dict(rec) for T, rec in impd.dataEmissions.items()}
print(impd.dataTemps) # e.g. [263.15, 298.15, 323.65] for -10, 25, 50.5 °C
```

`_build_autoPlotFrame(parent)`

Seeds the Live and Offline globals that `dltsConfig.init()` would normally set (mode 'live', tokens 0, empty sets and dicts, busy flags `False`). It then builds the header row, calls `_build_runControlPanel`, and creates the two-axes figure, canvas, and toolbar.

`_build_runControlPanel(parent)`

Builds the **Run Control** frame: **Pause**, **Resume**, **Redo Selected**, and **Remove & Retake Selected** (all disabled at start), and the step listbox with its scrollbar.

Qualitative Analysis: GUI and scanning

`_set_manual_buttons_state(state)`

Sets `state` ('normal' or 'disabled') on **Select Source Folder**, **Append Source Folder**, and **Extract & Average Transients**.

`_describe_folder_contents(dir_path, max_entries=12)`

Returns a one-line preview string: "N item(s): a, b, ... (M more) – <guess>". The guess is looks like ZI single-file format ..., looks like ZI subfolder-per-temperature format, looks like legacy per-temperature files, or format not recognized from filenames It returns folder is empty or cannot list folder: <error> when appropriate. It only lists the folder and reads no files. The Detailed Analysis tab also uses it.

`_browse_manual_folder()`

Select Source Folder handler: dialog, log preview, then `_scan_manual_directory_async(dir_path, isAppend=False)`. It does nothing while a scan or an extraction is busy.

`_append_manual_folder()`

Append Source Folder handler. It falls back to `_browse_manual_folder()` when the registry is empty; otherwise it runs a dialog, logs the preview, and calls `_scan_manual_directory_async(dir_path, isAppend=True)`.

`_registry_format_label(registry)`

Returns 'ZI', 'Legacy', or 'Mixed' according to the tags present in the registry. This is the value stored in `dlts.manual_ziMode` and shown in the status as [ZI], [Legacy], or [Mixed].

`_scan_manual_directory_async(dir_path, isAppend)`

Sets `manual_loadingBusy`, disables the buttons, and starts a daemon thread. The thread detects the format (ZI single CSV, then ZI subfolders, then legacy with `_legacy_run_timing`) and builds the registry. `apply()` on the Tk thread:

- replaces or merges `manual_datasetRegistry` and `manual_sourceFolders`;
- stores the ZI grid parameters in `manual_ziParamsByFile` and the `manual_zi*` globals;
- fills the Timing Boundaries (not on append);
- refills and selects all entries in the temperature listbox;
- logs errors and sets the status [`<format>`] N temperature step(s) from S source(s).

A replacing scan also clears `manual_ziParamsByFile` and sets `manual_dataDirectory`.

`_select_all_manual_temps()`

Selects every entry in the temperature listbox.

`_clear_manual_temps()`

Clears the temperature listbox selection.

Qualitative Analysis: dataset builders (pure computation)

All three return a **registry**, a dict mapping a temperature in °C (`float`) to its data source:

Registry value	Produced by	Meaning
<code>('zi', dataFile, chunkId)</code>	<code>_compute_zi_dataset</code>	One chunk of a combined ZI CSV.

Registry value	Produced by	Meaning
<code>('zi_subfolder', dataFile, None)</code>	<code>_compute_zi_subfolder_dataset</code>	This temperature's own ZI CSV.
<code>('legacy_chunk', filePath, chunkId or None)</code>	<code>_compute_legacy_dataset</code>	A semicolon CSV, optionally filtered by its first column.
<code>filePath (str)</code>	<code>_compute_legacy_dataset</code>	A <code>.txt</code> (JSON), <code>.h5</code> , or plain two-column <code>.csv</code> step file.

`errorMsgs` is a list that each builder appends human-readable problems to. The builders never raise for a bad individual file.

`_compute_zi_dataset(dir_path, header_filename, errorMsgs)`

Reads `header_filename` (semicolon CSV with the columns `chunk_number`, `history_name`, `grid_col_offset`, `grid_col_delta`, `chunk_size`) and finds the first data CSV matching `imps_0_sample_param1_avg_<digits>.csv`.

Name	Type	Default	Meaning
<code>dir_path</code>	<code>str</code>	<code>none</code>	Folder that holds the ZI export.
<code>header_filename</code>	<code>str</code>	<code>none</code>	Name of the header CSV inside <code>dir_path</code> .
<code>errorMsgs</code>	<code>list</code>	<code>none</code>	Receives error strings.

Returns `(registry, ziInfo)`. `ziInfo` is `{'dataFile', 'gridColOffset', 'gridColDelta', 'chunkSize', 'fpMs', 'rbMs', 'sliceEnd'}`. The grid values come from the first header row, with the ZI defaults as fallback. `fpMs`, `rbMs`, and `sliceEnd` are strings parsed from the folder name, or `None`. On a read failure it returns `(registry, None)`. Temperatures come only from `history_name` values that start with `<digits>C_`.

Side effects Reads files only.

Called by `_scan_manual_directory_async` and `detailedAnalysisTab`.

`_compute_zi_subfolder_dataset(dir_path, errorMsgs)`

Scans the subfolders of `dir_path` that match `_ZI_SUBFOLDER_PATTERN`. For each, it takes the first `imps_0_sample_param1_avg_<n>.csv` and reads `grid_col_offset`, `grid_col_delta`, and `chunk_size` from that subfolder's header CSV, with defaults if the header is missing or unreadable.

Returns `(registry, ziParamsByFile)`, where `ziParamsByFile[dataFile] = {'gridColOffset', 'gridColDelta', 'chunkSize'}`.

Called by `_scan_manual_directory_async` and `detailedAnalysisTab`.

Example

```
import liveDataTab as ldT
errors = []
registry, params = ldT._compute_zi_subfolder_dataset(r'D:\zi_export', errors)
# registry -> {-10.0: ('zi_subfolder', r'D:\zi_export\n10C\dev..._avg_00000.csv', None),
#           25.0: ('zi_subfolder', r'D:\zi_export\25C_000\dev..._avg_00000.csv', None)}
# params    -> {r'D:\zi_export\n10C\dev..._avg_00000.csv': {'gridColOffset': -0.001,
#           'gridColDelta': 1e-05, 'chunkSize': 1000}, ...}
```

`_compute_legacy_dataset(dir_path, errorMsgs)`

Lists the files in `dir_path`, without recursing:

- A `.csv` whose first line contains `;`:
 - if the line also contains `chunk`: it reads the first column. With more than one distinct chunk, or with `p120C-p160C` in the file name, it maps chunks 0 to 8 to 120, 125, ..., 160 °C as `('legacy_chunk', path, chunk)`. Other chunk numbers are dropped. With a single chunk, the temperature comes from the file name.
 - else if it contains `smoothed_value` or `timestamp`: the temperature comes from the file name, as `('legacy_chunk', path, None)`.
- Any other file whose name matches `_LEGACY_FILENAME_PATTERN` is registered as a plain path. The temperature is `sign × (integer + 0.<decimals>)`.

Returns `registry` (dict).

Called by `_scan_manual_directory_async`, `detailedAnalysisTab`, the tests, and the benchmarks.

Example (see the full pipeline example under `_compute_legacy_transients`)

```
errors = []
registry = ldT._compute_legacy_dataset(r'D:\data\092826\141503', errors)
# {-10.0: 'D:\...\n10p0.txt', 25.0: 'D:\...\p25p0.txt', 50.5: 'D:\...\p50p5.txt'}
```

`_legacy_run_timing(dir_path)`

Reads `runParams.txt` (JSON) in `dir_path`.

Name	Type	Default	Meaning
<code>dir_path</code>	str	none	Run folder.

Returns (`fpMs`, `rbMs`) as floats, with `fpMs` = State Disable Time × 1000 and `rbMs` = State Enable Time × 1000. It returns `None` if the file or either key is missing or unreadable, or if either value is not positive.

Side effects None.

Called by `_scan_manual_directory_async`, and the benchmarks in `benchmarks/hardware/hw_analyze.py`.

Example

```
ldT._legacy_run_timing(r'D:\data\092826\141503') # runParams.txt with 0.006 / 0.003
# -> (3.0, 6.0)
```

Qualitative Analysis: extraction (pure computation)

All extractors return (processedTransients, executionErrors):

```
processedTransients = {
    25.0: {
        'time_ms': ndarray,      # time axis, ms from reverse-bias start
        'avg_cap_pf': ndarray,    # averaged capacitance, pF
        'C_infinity': float,      # avg_cap_pf at the sample nearest cInfTargetMs
    },
    ...
}
executionErrors = ['30.0°C: no fill pulses found: ...', ...] # one string per skipped temperature
```

They take only plain Python and numpy data, so they can run in a child process. This shape is what `dlts.manual_processedTransients` holds, and what `dataAnalysisTab` and `detailedAnalysisTab` consume.

`_compute_zi_transients(selectedTemps, cInfTargetMs, datasetRegistry, ziParamsByFile)`

For the 'zi' entries: it groups temperatures by data file and reads each file once (`pd.read_csv(sep=';')`). For each temperature it takes the `value` column where `chunk == chunkId`, truncated to `chunkSize`. The time axis is $(\text{gridColOffset} + \text{arange}(n) * \text{gridColDelta}) * 1000$ ms, so it starts at the offset (−1 ms by default), before the trigger. Values below $1e-3$ in magnitude are taken as farads and multiplied by $1e12$.

Name	Type	Default	Meaning
<code>selectedTemps</code>	list of float	none	Temperatures to extract. Each must map to a 'zi' tuple.
<code>cInfTargetMs</code>	float	none	Time for <code>C_infinity</code> , in ms.
<code>datasetRegistry</code>	dict	none	Registry from <code>_compute_zi_dataset</code> .
<code>ziParamsByFile</code>	dict	none	<code>dataFile -> {'gridColOffset', 'gridColDelta', 'chunkSize'}</code> . Missing entries use the defaults.

Returns (processedTransients, executionErrors).

Called by `_compute_mixed_transients`.

`_compute_zi_subfolder_transients(selectedTemps, cInfTargetMs, datasetRegistry, ziParamsByFile)`

For the 'zi_subfolder' entries: it reads each file with the column names forced to `chunk;timestamp;value` and keeps the `chunk == 0` rows (at least 2 are required). The truncation, time axis, and unit conversion are the same as in `_compute_zi_transients`. The parameters are the same too.

Returns (processedTransients, executionErrors).

Called by `_compute_mixed_transients`.

`_compute_mixed_transients(ziTemps, legacyTemps, cInfTargetMs, datasetRegistry, ziParamsByFile, rbDurationMs, samplingRateS, ziSubfolderTemps=None)`

Runs the three extractors on their subsets and merges the results. This is the function submitted to the process pool.

Name	Type	Default	Meaning
<code>ziTemps</code>	list	none	Temperatures tagged ' <code>zi</code> '.
<code>legacyTemps</code>	list	none	Temperatures with a plain path or ' <code>legacy_chunk</code> '.
<code>cInfTargetMs</code>	float	none	Time for <code>C_infinity</code> , in ms (the GUI passes $0.9 \times$ Reverse Bias).
<code>datasetRegistry</code>	dict	none	Combined registry.
<code>ziParamsByFile</code>	dict	none	ZI grid parameters per data file.
<code>rbDurationMs</code>	float	none	Reverse-bias window length for legacy pulse averaging, in ms.
<code>samplingRateS</code>	float	none	Sample period in seconds, despite the name.
<code>ziSubfolderTemps</code>	list or None	None	Temperatures tagged ' <code>zi_subfolder</code> '.

Returns (`processedTransients`, `executionErrors`).

Called by `_process_raw_transients` (in the child process) and `detailedAnalysisTab`.

`_compute_legacy_transients(selectedTemps, rbDurationMs, cInfTargetMs, datasetRegistry, samplingRateS)`

Extracts and averages transients from legacy entries:

- ('legacy_chunk', path, chunk): it reads the semicolon CSV and filters on the first column when chunk is not None. It drops rows with any NaN and uses the last column whose name contains value, smoothed, cap, or impedance (otherwise the last column). F values are converted to pF. The time axis is `arange(n) * samplingRateS * 1000` ms.
- .txt (JSON) or .h5: it reads `AuxInput1` (excitation, V) and `ImpedanceIm` (F). For .h5 it reads only these two channels, through `iaT.read_h5_record`. It converts to float32 and multiplies `ImpedanceIm` by $1e12$ (pF). It finds the reverse-bias starts with `_find_reverse_bias_starts` and cuts `int(rbDurationMs * 1e-3 / samplingRateS)` samples after each start, keeping only full windows. The windows are averaged. Time is `arange(len) * samplingRateS * 1000` ms, with `t = 0` at the last fill sample.
- .csv (plain): column 0 is taken as `time_ms` and column 1 as `avg_cap_pF`, both as they are.
- Any other extension: logged as unsupported file type.

The skip reasons appended to `executionErrors` include no excitation (`AuxInput1`) samples ..., no fill pulses found: the excitation (`AuxInput1`) stays between L and H V., ... never falls through X V., and found N fill pulse(s), but none is followed by a full R ms reverse-bias window before the data ends (T ms recorded); lower Reverse Bias (ms). Any exception becomes `T°C: <exception>`.

Name	Type	Default	Meaning
<code>selectedTemps</code>	list of float	none	Temperatures to extract (keys of <code>datasetRegistry</code>).
<code>rbDurationMs</code>	float	none	Reverse-bias window, in ms.
<code>cInfTargetMs</code>	float	none	Time for <code>C_infinity</code> , in ms.
<code>datasetRegistry</code>	dict	none	Registry from <code>_compute_legacy_dataset</code> .

Name	Type	Default	Meaning
samplingRates	float	none	Sample period, in s (the GUI uses 1.8666666666666665e-05).

Returns (processedTransients, executionErrors).

Called by `_compute_mixed_transients`, the tests, and the benchmarks.

Example (checked by running it: a synthetic 3 ms fill / 6 ms reverse-bias file)

```
import json, os, tempfile
import numpy as np
import liveDataTab as ldT

dt = 1.8666666666666665e-05          # s per sample
fillN, rbN = int(3e-3 / dt), int(6e-3 / dt)  # 160, 321
k = np.arange(20000) % (fillN + rbN)
aux = np.where(k < fillN, -0.5, -1.5)      # fill -0.5 V, reverse bias -1.5 V
cap = 1e-10 + 2e-12 * np.exp(-np.maximum(k - fillN, 0) * dt / 1e-3)  # F

folder = tempfile.mkdtemp()
with open(os.path.join(folder, 'p25p0.txt'), 'w') as f:
    json.dump({'AuxInput1': aux.tolist(), 'ImpedanceIm': cap.tolist()}, f)
with open(os.path.join(folder, 'runParams.txt'), 'w') as f:
    json.dump({'State Enable Time': 0.006, 'State Disable Time': 0.003}, f)

errors = []
registry = ldT._compute_legacy_dataset(folder, errors)      # {25.0: '...p25p0.txt'}
fpMs, rbMs = ldT._legacy_run_timing(folder)                 # (3.0, 6.0)
transients, extractErrors = ldT._compute_legacy_transients(
    sorted(registry), rbMs, 0.9 * rbMs, registry, dt)
rec = transients[25.0]
len(rec['time_ms'])      # 321
rec['time_ms'][-1]       # 5.9733 ms
rec['avg_cap_pf'][:2]    # [102., 102.]
rec['C_infinity']        # 100.009
extractErrors             # []
```

`_find_reverse_bias_starts(auxV)`

Finds where the excitation drops from the fill level to the reverse-bias level. It estimates the two levels from the finite samples only, but searches the full array (a NaN counts as not-forward), so the returned indices line up with `ImpedanceIm`. It takes the two levels as the 0.01th and 99.99th percentiles, which still catch a fill pulse that is 0.2 % of the samples. If the levels differ by less than `_MIN_PULSE_HEIGHT_V`, it reports no pulses. Otherwise the threshold is the midpoint $(low + high) / 2$. It returns the indices `i` where sample `i` is above the threshold and sample `i + 1` is not, that is, the last fill sample of each pulse.

Name	Type	Default	Meaning
auxV	array-like	none	Excitation voltage (AuxInput1), V.

Returns (indices, message). `indices` is an int array. `message` is `None` on success, otherwise a sentence starting `no excitation ...` Or no fill pulses found: `...`

Called by `_compute_legacy_transients`.

Example

```
ldT._find_reverse_bias_starts(np.array([0, 0, -5, -5, -5, 0, -5, -5]))
# -> (array([1, 5]), None)      threshold -2.5 V
ldT._find_reverse_bias_starts(np.full(100, -1.0))
# -> (array([]), 'no fill pulses found: the excitation (AuxInput1) stays between -1.000 and -1.000 V.')
```

`_capacitance_axis_units(curves)`

Picks a display unit for curves stored in pF, from the largest finite absolute value `peak` across all curves: `peak >= 1e9` gives `(1e-9, 'mF')`, `>= 1e6` gives `(1e-6, 'µF')`, `>= 1e3` gives `(1e-3, 'nF')`, and anything smaller, or no finite data, gives `(1.0, 'pF')`. Multiply the pF values by `scale` to plot them.

Name	Type	Default	Meaning
<code>curves</code>	iterable of arrays	none	Curves in pF. Empty and all-NaN curves are skipped.

Returns `(scale, unit)`.

Called by `_process_raw_transients`.

Example

```
ldT._capacitance_axis_units([np.array([150.0])]) # (1.0, 'pF')
ldT._capacitance_axis_units([np.array([4.5e3])]) # (0.001, 'nF')
ldT._capacitance_axis_units([np.array([4.5e6])]) # (1e-06, 'µF')
ldT._capacitance_axis_units([])                  # (1.0, 'pF')
```

Qualitative Analysis: extraction driver and plot

`_plot_slice_ms()`

Returns `(start, end)` in ms from **Analysis Slice Start/End (ms)**, each `None` when the field is empty, not a number, or the Timing Boundaries fields do not exist yet. `_process_raw_transients` uses it to window the Qualitative plot.

```
lo, hi = ldT._plot_slice_ms() # (2.0, 490.0) with the default fields
```

`_downsample_for_plot(x, y, maxPoints=_MAX_PLOT_POINTS)`

Returns `(x, y)` unchanged if `len(x) <= maxPoints`. Otherwise it returns every `ceil(n / maxPoints)`-th sample, for example 7000 points become 2334. It is used only for drawing; the stored data keeps full resolution.

`_get_transient_executor()`

Returns `dlts.manual_transientExecutor`, creating a `ProcessPoolExecutor(max_workers=1)` on first use. The pool is reused, so only the first extraction pays the child's import start-up time. `DLTSGUI_MainWindow._finish_close` shuts it down.

`_process_raw_transients()`

Extract & Average Transients handler. It returns silently if the registry is empty or a worker is busy. It logs an error if no temperature is selected or **Reverse Bias (ms)** is not numeric. Otherwise it sets `manual_processingBusy`, disables the buttons, and splits the selection into `ziTemps`, `ziSubfolderTemps`, and `legacyTemps` by registry tag. A daemon thread submits `_compute_mixed_transients(...)` to the process pool and blocks on `future.result()`. `apply()` on the Tk thread stores `manual_processedTransients`, draws the plot, logs each error, and sets the status. A crash in the child process is reported as `extraction process failed: <error>`.

`_build_manualPlotFrame(parent)`

Seeds `manual_processedTransients`, `manual_processingBusy`, and `manual_loadingBusy`, then builds the whole **Qualitative Analysis** pane: the scrollable left column (with a mouse-wheel binding active while the pointer is over it), the four group boxes, the figure, the canvas, and the toolbar.

Threading model

- **Tk main thread:** every widget update, every matplotlib draw, the button callbacks, `_schedule_live_poll`, `_poll_step_listbox_while_busy`, `_handle_run_status`, and `finish_experiment()` (which writes `runParams.txt`).
- **Run threads** (daemon `threading.Thread`): `start_dltsc` for **Run DLTS**, and the `_run_control_thread` worker for Resume, Redo, and Retake. They drive the hardware through `runDltsc_Tools`. `dltsc.run_busy` allows only one at a time: every launcher returns early while it is set, and `_return_to_room_temp_async` keeps it set during the ramp back. Pausing sets `run_pauseRequested`, and the loop in `run_experiment()` returns 'paused' before the next step.
- **Room-return thread:** `_return_to_room_temp_async` runs `return_to_room_temp()`, which can block for many minutes.
- **Plot worker threads:** `_ingest_files_async`, `_load_offline_run`, and `_recompute_denoise_and_redraw` do `impdData` loading, clustering, and denoising. `livePlot_liveIngestBusy` and `livePlot_offlineIngestBusy` stop a second worker on the same mode.
- **Qualitative threads:** `_scan_manual_directory_async` (folder scan) and the waiter thread in `_process_raw_transients`. `manual_loadingBusy` and `manual_processingBusy` gate them, and the three buttons are disabled meanwhile.
- **Process pool:** one worker process (`dltsc.manual_transientExecutor`) runs `_compute_mixed_transients`. It has its own interpreter and GIL, so long extractions do not slow the Tk thread. This is why `DLTSGUI_MainWindow` guards GUI construction with `__name__ == '__main__'` and calls `multiprocessing.freeze_support()`.
- **Marshaling:** workers hand results back with `dltsc.root.after(0, callback)`, and Tk calls happen only in those callbacks. The exception is `dltsc.log_to_textbox`, which the run threads (inside `runDltsc_Tools`) and `start_dltsc` call directly from their background threads.
- **Run tokens:** `livePlot_liveRunToken` and `livePlot_offlineRunToken` are incremented on each reset. Pollers and workers carry the token they started with and discard their results if it has changed, so a stale watcher or loader from an earlier run cannot overwrite newer data.

Notes and limitations

- **NaNs in `AuxInput1`** no longer shift the pulse windows: `[nan, 0, 0, -5, -5, -5, 0, -5, -5]` gives `[2, 6]` (fixed after this page was first written; covered by `tests/test_transient_extraction.py`).

- **Fixed sample period.** Legacy extraction always uses `1.8666666666666665e-05` s (about 53.57 kSa/s), because `dlts.manual_samplingRate` is never set. If the MFIA delivered a different rate, for example after changing **Data Transfer Rate**, the legacy time axis and the reverse-bias window length are wrong.
- **Timing Boundaries.** Only **Reverse Bias (ms)** affects extraction. **Analysis Slice Start/End (ms)** window the Qualitative plot only; **Filling Duration (ms)** is display only.
- `runParams.txt` is written only when a main sequence completes. A paused, failed, or closed-early run folder has none, so the Timing Boundaries keep their previous values for it.
- **Folder-name timing regex.** The ZI FP/RB patterns need at least one character between FP/RB and the number. `FP_1ms_RB_500ms` gives 1 and 500 ms. `FP1ms_RB500ms` gives FP = 500 and RB = 00. `FP10ms` gives FP = 0.
- ZI single-CSV temperatures must be non-negative whole degrees (`^(\\d+)C_` on `history_name`).
- Multi-chunk legacy CSVs use a hard-coded chunk map from 120 to 160 °C in 5 °C steps. Chunks above 8 are dropped.
- `_LEGACY_FILENAME_PATTERN` needs a leading `p/n` and a lower-case extension. `p25.TXT`, `25C.txt`, and `p25p0.json` are not recognized. **Load Existing Run (Offline)** does offer `*.json` in its filter.
- Plain two-column legacy `.csv` files are taken as they are: column 0 must already be in ms and column 1 in pF.
- **Possible Live-view mix-up on a second run.** `start_thread` starts the watcher before `init_experiment()` publishes the new `dlts.run_dataFileNames`. On a second **Run DLTS** in the same session, the first ticks can still see the previous run's file list. Those files exist, so they are ingested into the new Live view. If they finish ingesting before the new list is published, the watcher decides that all files are done and stops, and the new run's files are never shown. This is inferred from the code and was not observed in the GUI.
- The Live watcher is started only by **Run DLTS**. Files rewritten by **Redo Selected** or **Remove & Retake Selected** are already marked processed and are not reloaded. Use **Load Existing Run (Offline)** to view them.
- A step file that always fails to load is retried every second for as long as the watcher runs, and each failure is logged.
- If `init_experiment()`, `run_experiment()`, or `return_to_room_temp()` raises an exception instead of returning a status, the `root.after` callback is never scheduled. `run_busy` then stays `True` and the buttons stay in `running` or `returning` until restart. `run_experiment()` catches step errors itself, so this needs an error outside its `try`.
- `dlts.log_to_textbox` is called from background threads (the run threads and `start_dIts`), which updates a Tk widget off the main thread.
- The ZI transients' time axis starts at `grid_col_offset` (–1 ms by default), so ZI curves include pre-trigger samples. Legacy curves start at the last fill sample.
- The y label of the Live plots is **Impedance Im (F)**. The values are whatever `impdData` stores; no unit conversion is done here.
- `_recompute_denoise_and_redraw` leaves `impd.dataEmissions` holding the emission-0 selection, whereas after an ingest it holds the all-emissions selection. Other tabs that read `livePlot_*ImpdData.dataEmissions` directly can therefore see either one.

dataAnalysisTab

What it's for

`dataAnalysisTab.py` builds the **Quick Analysis** tab. It turns averaged capacitance transients (one per temperature) into double-boxcar DLTS spectra for five rate windows, finds the peak temperature of each spectrum, and fits an Arrhenius line through the peaks. From the fit it reports the defect activation energy, the apparent capture cross-section and the trap density. The code is a tkinter port of tabs 2 and 3 ("Rate Window Analysis" and "Arrhenius Defect Mapping") of `DrKayisScript.py`.

The analysis steps (C_∞ , $S(T)$, peak finding, the Arrhenius fit and N_T) are the functions of `detailedAnalysisTab`. On averaged transients, Quick Analysis therefore gives the same E_t , σ and N_T as the **Standard** result of Detailed Analysis. This needs the same data, the same windows (the defaults are the same), the same search range, the same reverse-bias duration and the smoothing spline. See "Matching Detailed Analysis" below.

What the user sees

The tab is a horizontal `tk.PanedWindow` with two resizable frames of equal starting width (700 px each). The left frame is **Rate Window Analysis**. The right frame is **Arrhenius Defect Mapping**. Each frame has a plot with a matplotlib navigation toolbar on top and a scrollable control area underneath (requested height `QUICK_CONTROLS_HEIGHT` = 260 px). The mouse wheel scrolls the control area while the pointer is over it.

Left frame: "Rate Window Analysis"

Plot: title "Multi-Window DLTS Signal Spectrum (Pseudo-Voigt Refinement)" before the first run, "Multi-Window DLTS Signal Spectrum ()" after a run. X axis "Temperature (K)", Y axis "DLTS Signal ($\Delta C / C_\infty$)".

Controls, top row (three groups side by side):

Group	Widget	Default	What it does
Data Source	read-only combobox	Auto (first available)	Chooses where the transients come from. Options: Auto (first available), Qualitative Analysis, Automated/Live Data – Live, Automated/Live Data – Offline. See "Data sources" below.
Data Source	status label	No data source resolved yet.	After each Compute Boxcar Spectrums click it shows Using: <code><source> / <method> (<n> temperature(s))</code> , or the reason nothing was computed.
DLTS Signal Calculation	Method: combobox	Measured C (nearest sample)	Options: Measured C (nearest sample), Smoothed C (impdData, spline-interpolated).
DLTS Signal Calculation	Denoise (Live/Offline-backed Measured C, or Smoothed C, only): combobox	None (raw)	Options: None (raw), pca, wavelet, sgolay, lowess. Only used when the signal is computed through an <code>impdData</code> instance (Live/Offline). It has no effect for Qualitative Analysis data.

Group	Widget	Default	What it does
Peak-Finding Method	combobox	Smoothing Spline	Options: Smoothing Spline, Curve Fit (Pseudo-Voigt), Curve Fit (Gaussian), Curve Fit (Lorentzian), Curve Fit (Voigt).
Peak-Finding Method	Tp search range (K, blank = no bound): two entries	250.0 to 400.0	Only points with $\text{lower} \leq T \leq \text{upper}$ (in K) go to the peak finder. A blank entry means no bound on that side. Bounds are drawn as dotted gray vertical lines.

Configure Rate Windows (Double Boxcar) group: five sets, two per row, each with a `t1 (ms):` and `t2 (ms):` entry. The defaults are Detailed Analysis' standard windows (`detailedAnalysisTab.DEFAULT_STD_WINDOWS`):

Set	t1 (ms)	t2 (ms)
Set 1	5.0	25.0
Set 2	10.0	50.0
Set 3	20.0	100.0
Set 4	50.0	250.0
Set 5	100.0	490.0

The number of sets is the length of `DEFAULT_RATE_WINDOWS`.

Compute Boxcar Spectrums (blue button) runs `_calculate_rate_windows()`. It clears the plot and the peak table, computes the DLTS signal $S(T)$ for every set, finds each peak, fills the table and redraws the plot. Each set is drawn as points `RW <n>` (with error bars when errors exist), the fitted curve as a dashed line `Fit <n>`, and the peak as a black `x` with a `T_peak` error bar. A peak with no `T_peak` error (on the edge of the search range) is a gray `x`. It logs a summary line to the app log.

Extracted Peaks table (`ttk.Treeview`, 4 rows visible, scrolls): columns `Window (Set n)`, `Emission  $e_n$  ( $s^{-1}$ )` (2 decimals), `T_peak (K)` (2 decimals, $\pm \text{err}$ when available), `Max Extrema  $\Delta C/C_\infty$` (5 decimals, the largest $|S|$ in the search range, with its sign), and `Arrhenius fit`. The last column reads `skipped (no peak)` for a skipped window and `no Tp error (edge)` for an edge peak. After the solver runs it reads `used` or `excluded (edge peak)`.

Right frame: "Arrhenius Defect Mapping"

Plot: title "Arrhenius Plot Representation for Trap Signature Extraction", X axis "Reciprocal Temperature ($1000 / T$) (K^{-1})", Y axis " $\ln(e_n / T^2)$ ".

Group	Widget	Default	What it does
Material Parameters	Background Doping Nd (cm^{-3}): entry	$3.2e+14$ (<code>DEFAULT_ND</code> , the same as Detailed Analysis)	Used only for the trap density. Must be a positive number.
Material Parameters	Pre-factor γ ($cm^{-2} s^{-1} K^{-2}$): entry	$1.66e+21$ (<code>DEFAULT_GAMMA</code>)	Used only for the capture cross-section. Must be positive. A gray hint below reads (<code>SiC $\approx 1.66e21$, Si $\approx 3.256e21$</code>).
(no group)	Execute Arrhenius Signature Solver (yellow button)		Runs <code>_run_arrhenius_solver()</code> on the peaks from the last Compute Boxcar Spectrums . Needs at least 2 peaks.

Group	Widget	Default	What it does
Extracted Microscopic Trap Signatures	Defect Activation Energy E_t (eV): value label	Waiting...	Shows $E \pm \text{err eV}$ (3 decimals).
Extracted Microscopic Trap Signatures	Apparent Capture Cross-Sec σ (cm^2): value label	Waiting...	Shows $\sigma \pm \text{err cm}^2$ (.2e).
Extracted Microscopic Trap Signatures	Calculated Trap Density N_t (cm^{-3}): value label	Waiting...	Shows $N_T \text{ cm}^{-3}$ (.2e).

The solver plots the fitted points as red squares "Experimental Extrema Points" (with x and y error bars from the T_{peak} error), excluded points as gray x "Excluded (peak on search-range edge)", and the fit as a blue line "Linear Fit Reference", drawn from $0.95 \cdot \min(1000/T)$ to $1.05 \cdot \max(1000/T)$. It logs `Arrhenius solver (T_peak-error weighted|unweighted, n windows[, m excluded]): Et=..., sigma=... ( $\gamma$ =...), Nt=...`

Data sources

Selection	Where the data comes from
Qualitative Analysis	<code>dlts.manual_processedTransients</code> , filled by Extract & Average Transients in the Qualitative Analysis frame of the Live Tools tab. Keys are °C.
Automated/Live Data – Live	The Run DLTS session in progress: <code>dlts.livePlot_liveImpdData</code> (an <code>impdData</code> instance) or, if that is <code>None</code> , the averaged snapshot <code>dlts.livePlot_liveAllEmissionsData</code> .
Automated/Live Data – Offline	A previously loaded run: <code>dlts.livePlot_offlineImpdData</code> or <code>dlts.livePlot_offlineAllEmissionsData</code> .
Auto (first available)	Measured C: Qualitative Analysis first, then Live, then Offline. Smoothed C: Live <code>impdData</code> , then Offline <code>impdData</code> (never Qualitative Analysis).

Smoothed C always needs an `impdData` instance (Live or Offline). With Qualitative Analysis, or with no instance, the status label shows the error and nothing is computed.

Import

```
import dataAnalysisTab as daT      # alias used in DLTS GUI_MainWindow.py
daT.construct_dataAnalysisTab()
```

`dataAnalysisTab` itself imports `detailedAnalysisTab` as `da` for the shared analysis functions and defaults. Elsewhere in the codebase the alias `daT` means `detailedAnalysisTab` (tests and benchmarks use `import detailedAnalysisTab as daT`), so check which module a `daT` refers to before reading code.

Module constants and globals

Name	Value / type	Meaning
<code>K_BOLTZMANN</code>	<code>da.KB_EV = 8.617333262145e-5</code> (float)	Boltzmann constant, eV/K.

Name	Value / type	Meaning
DEFAULT_GAMMA_A	'1.66e+21' (str, from dA.DEFAULT_GAMMA)	Default emission pre-factor γ , $\text{cm}^{-2} \text{s}^{-1} \text{K}^{-2}$ (SiC value).
DEFAULT_ND	'3.2e+14' (str, from dA.DEFAULT_ND)	Default background doping Nd, cm^{-3} .
DEFAULT_TPEAK_LO	'250.0' (str, from dA.DEFAULT_TPEAK_LO)	Default lower peak-search bound, K.
DEFAULT_TPEAK_HI	'400.0' (str, from dA.DEFAULT_TPEAK_HI)	Default upper peak-search bound, K.
QUICK_CONTROL_HEIGHT	260 (int)	Requested height (px) of the scrollable control area under each plot.
DEFAULT_RATE_WINDOWS	[('5.0', '25.0'), ('10.0', '50.0'), ('20.0', '100.0'), ('50.0', '250.0'), ('100.0', '490.0')]	Default (t1, t2) per set, ms, built from dA.DEFAULT_STD_WINDOWS. Its length sets the number of sets.
DATA_SOURCE_AUTO	'Auto (first available)'	Data Source option.
DATA_SOURCE_QUALITATIVE	'Qualitative Analysis'	Data Source option.
DATA_SOURCE_LIVE	'Automated/Live Data – Live'	Data Source option.
DATA_SOURCE_OFFLINE	'Automated/Live Data – Offline'	Data Source option.
DATA_SOURCE_OPTIONS	list of the four above	Combobox values.
SIGNAL_METHOD_MEASURED	'Measured C (nearest sample)'	Signal method option (default).
SIGNAL_METHOD_SMOOTHED	'Smoothed C (impdData, spline-interpolated)'	Signal method option.
SIGNAL_METHOD_OPTIONS	list of the two above	Combobox values.
DENOISE_NONE	'None (raw)'	No denoising.
DENOISE_OPTIONS	['None (raw)', 'pca', 'wavelet', 'sgolay', 'lowess']	Combobox values.
PEAK_METHOD_SPLINE	'Smoothing Spline'	Peak method (default). The same finder as Detailed Analysis' Smoothing Spline (bootstrap).
PEAK_METHOD_PSEUDOVOIGT	'Curve Fit (Pseudo-Voigt)'	Peak method.
PEAK_METHOD_GAUSSIAN	'Curve Fit (Gaussian)'	Peak method.
PEAK_METHOD_LORENTZIAN	'Curve Fit (Lorentzian)'	Peak method.
PEAK_METHOD_VOIGT	'Curve Fit (Voigt)'	Peak method.

Name	Value / type	Meaning
PEAK_METHOD_OPTIONS	list of the five above	Combobox values.
PEAK_METHOD_CURVETYPE	{Pseudo-Voigt: 'pseudoVoigt', Gaussian: 'gaussian', Lorentzian: 'lorentzian', Voigt: 'voigt'}	Maps a curve-fit option to the <code>curveType</code> argument of <code>impdData._curveFit_peakFinder</code> .

Shared state in `dltsConfig` (`dltsC`) that this module reads or writes:

Name	Type	Meaning
<code>rateWindow_dataSourceVar</code> , <code>rateWindow_signalMethodVar</code> , <code>rateWindow_denoiseVar</code> , <code>rateWindow_peakMethodVar</code>	<code>tk.StringVar</code>	Combobox selections.
<code>rateWindow_tpLoVar</code> , <code>rateWindow_tpHiVar</code>	<code>tk.StringVar</code>	Peak-search bounds (K), blank allowed.
<code>rateWindow_windowVars</code>	list of 5 (<code>t1Var</code> , <code>t2Var</code>) <code>tk.StringVar</code> pairs	Rate-window entries (ms).
<code>rateWindow_statusLabel</code>	<code>ttk.Label</code>	Data Source status text.
<code>rateWindow_peakTable</code>	<code>ttk.Treeview</code>	Extracted Peaks table. Row ids are <code>w<i></code> (<code>i</code> = set index from 0).
<code>rateWindow_figure</code> , <code>rateWindow_ax</code> , <code>rateWindow_canvas</code>	matplotlib Figure, Axes, FigureCanvasTkAgg	Left plot.
<code>rateWindow_signals</code>	dict <code>i</code> -> {'T_k', 'Signal', 'Signal_err'}	Per-set S(T) curves from the last run (T in K). <code>Signal_err</code> is None when not every point has a usable error.
<code>rateWindow_extractedPeaks</code>	dict <code>i</code> -> {'T_peak', 'S_peak', 'e_n', 'T_peak_err', 't1', 't2'}	Per-set peaks, input to the Arrhenius solver. Skipped sets are left out. <code>T_peak_err</code> is None for an edge peak.
<code>rateWindow_transients</code>	(records, temps, <code>rb_ms</code>) or None	The averaged transients of the last run as Detailed Analysis records. <code>N_T</code> is computed from them.
<code>rateWindow_denoisedEmissions</code>	dict T -> dict	Copy of <code>impd.dataEmissions</code> after the last <code>impdData</code> -backed window (raw and denoised emissions).
<code>arrhenius_ndVar</code> , <code>arrhenius_gammaVar</code>	<code>tk.StringVar</code>	Nd and γ entries.

Name	Type	Meaning
arrhenius_energyLabel, arrhenius_captureLabel, arrhenius_densityLabel	ttk.Label	Result labels.
arrhenius_figure, arrhenius_ax, arrhenius_canvas	matplotlib objects	Right plot.
read only: manual_processedTransients, manual_paramVars['rb_ms'], livePlot_liveImpdData, livePlot_offlineImpdData, livePlot_liveAllEmissionsData, livePlot_offlineAllEmissionsData, livePlot_liveIngestBusy, livePlot_offlineIngestBusy, tabControl, dataAnalysisTab	various	Inputs from other tabs and the main window.

Analysis method

All times typed in the GUI are in ms. Temperatures are in K inside the analysis (Qualitative Analysis keys are °C and get +273.15).

Step 1. Get one averaged transient per temperature. (`_get_processed_transients_for_source`, `_processed_transients_from_automated`, or `impdData.selected_emissions` inside `calculate_delC_normalized`.)

For an averaged Live/Offline snapshot (`_processed_transients_from_automated`):

```
time_ms      = x * 1000                # x is pulse-relative time in s
avg_cap_pf   = ymean * 1e12  if max|ymean| < 1e-3  (Farads -> pF), else ymean
T_C          = round(T_K - 273.15, 6)   # impdData keys are exact K (298.15 -> 25.0)
```

(That function still stores a `C_infinity` at 90 % of the span, for other callers. The analysis below recomputes C_{∞} .)

Step 2. Reverse bias and C_{∞} (`_rb_ms_for_source`, `_transient_records`). The reverse-bias duration `rb_ms` is the **Reverse Bias (ms)** field of Qualitative Analysis for that source; for a Live/Offline snapshot, or when that field is not a positive number, it is the longest transient's span. C_{∞} is Detailed Analysis' convention:

```
C_inf = mean C(t) over 0.40 * rb_ms <= t <= 0.90 * rb_ms    # dA._cinf_range_mean
```

Step 3. Emission rate of each rate window (`dA._emission_rate`):

```
e_n = ln(t2 / t1) / ((t2 - t1) * 1e-3)    # t1, t2 in ms -> e_n in s^-1
```

$0 < t1 < t2$ is enforced for every set before anything is computed.

Step 4. DLTS signal $S(T)$. Two code paths:

a. No `impdData` instance (Qualitative Analysis, or an averaged Live/Offline snapshot), Measured C only. This is `dA._rate_window_signal` on the records of Step 2:

```
C(t)      = nearest measured sample to t
S(T)      = (C(t2) - C(t1)) / C_inf          # dimensionless
sigma_S   = hypot(noise(t1), noise(t2)) / |C_inf|
```

`noise(t)` is `dA._cap_noise_at(time_ms, C, t)`: the Rice second-difference estimate $\sqrt{\text{sum}(d2^2) / (6 * \text{len}(d2))}$ with $d2 = C[k-1] - 2 * C[k] + C[k+1]$, over the ± 25 samples around t .

b. `impdData` instance (Live/Offline; always the case for Smoothed C). The code calls

```
impd.calculate_dclC_normalized(t1=t1/1000.0, t2=t2/1000.0, emissionIndex=-1,
    denoiseEmission=(denoise != 'None (raw)'), denoiseMethod=(denoise or 'pca'),
    smoothCapacitance=(method == 'Smoothed C'), plot=False)
```

and uses column 0 (T, K) as x , column 3 ($\Delta C/C_\infty$) as S , and error column 1 as σ_S . Inside that method (in `impedanceAnalysis_Tools.py`): `emissionIndex=-1` is the ensemble average over all reverse-bias repeats; $t1/t2$ are clipped to the data range; $C(t1)$, $C(t2)$ and C_∞ are the nearest samples (Measured C) or `CubicSpline` values (Smoothed C); **C_∞ is the last sample, $C(t[-1])$** ; errors come from the cross-repeat standard deviation `yerr`, propagated with the `uncertainties` package; $S = (C(t2) - C(t1)) / C_\infty$.

Step 5. Peak of S(T) (`dA._peak_from_signal`, the step Detailed Analysis uses too):

- Only points with `Tp_lo <= T <= Tp_hi` are used. Fewer than 4 points gives a log warning; the peak is then the raw maximum.
- If the largest $|S|$ in the range is negative, S is flipped so the peak is positive.
- A window whose peak is at most `dA.PEAK_MIN_FRAC` (5 %) of $\max|S|$ over all temperatures is **skipped**: it gets no peak and does not go to the Arrhenius fit.
- Smoothing Spline: `impdData._smoothingSpline_peakFinder`: `scipy.interpolate.UnivariateSpline(k=3)`. With point errors: weights $1/\sigma_S$ and smoothing factor $s = n$. Without: $s = n * \text{noiseVar}$ where `noiseVar` is the Rice estimate over $S(T)$ itself. The peak is the maximum on a 100 000-point grid. The T_{peak} error is the standard deviation of 200 parametric-bootstrap refits (seed 0, 2000-point grid).
- Curve Fit (...): `impdData._curveFit_peakFinder` with `lmfit` `PseudoVoigtModel`/`GaussianModel`/`LorentzianModel`/`VoigtModel`, weights $1/\sigma_S$ when available. $T_{\text{peak}} = \text{center}$, its error = `lmfit.stderr`. If the fit raises or returns `-1`, the raw grid maximum is used with no error.
- Edge guard** (`dA._guard_peak_error`): a peak within 1 % of a grid step of either end of the range (`dA.EDGE_PEAK_FRAC`), or with an error below `dA.MIN_TP_ERR_K` ($1e-4$ K), keeps its T_{peak} but loses its error. The signal is still rising out of the range, so it is not a real peak. A spline peak pinned to the edge would otherwise report an error of about $1e-13$ K and dominate the weighted fit.

Step 6. Arrhenius fit (`_quick_arrhenius` → `dA._arrhenius_fit` → `dA._fit_arrhenius_line`):

```
x = 1000 / T_peak          # K^-1 (times 1000)
y = ln(e_n / T_peak^2)    # ln(s^-1 K^-2)
dx = 1000 * dT / T_peak^2 # plotted error bars only
dy = 2 * dT / T_peak
```

Windows with no T_{peak} error are left out of the fit (shown as gray x). If fewer than 3 windows have errors, all windows are fitted unweighted. Fewer than 2 fittable windows: no result. The fit is `lmfit` `LinearModel`, weighted by the effective variance $\text{sig_eff} = |-2/T + 1000 * \text{slope}/T^2| * dT$, re-fitted up to 10 times until the slope changes by $\leq 1e-9 \cdot \max(|\text{slope}|, 1)$. With fewer than 3 fitted windows, slope and intercept errors are `None`.

Step 7. Defect parameters:

```

E_a      = -slope * 1000 * K_BOLTZMANN          # eV
dE_a     = 1000 * K_BOLTZMANN * slope_stderr    # eV
sigma    = exp(intercept) / gamma              # cm^2   (gamma in cm^-2 s^-1 K^-2)
dsigma   = sigma * intercept_stderr            # cm^2
N_T      = 2 * max_T |(C(rb_ms) - C(2 ms)) / C_inf| * Nd  # cm^-3, dA._trap_density

```

N_T uses the averaged transients of Step 2 (`dltsc.rateWindow_transients`). For an `impdData` source that has no averaged snapshot, it falls back to $2 \cdot \max|S_{\text{peak}}| \cdot Nd$. N_T has no error estimate.

Matching Detailed Analysis

On averaged transients (path a), both tabs call the same functions for every step, so the numbers agree to rounding. Set up both tabs like this:

Setting	Quick Analysis	Detailed Analysis
Data	Qualitative Analysis extraction of the run folder	Load Data on the same folder
Reverse bias	Qualitative Analysis Reverse Bias (ms)	RB duration (ms) , same value
C_∞ range	fixed 0.40 to 0.90	Start / End (frac RB) 0.40 / 0.90 (defaults)
Signal	Measured C, source Qualitative Analysis	Measured C (nearest sample), denoise None (raw)
Windows	the five sets	the five Standard Windows (same defaults)
Search range	Tp search range	T min / T max (same defaults)
Peak method	Smoothing Spline	Smoothing Spline (bootstrap)
N_d , γ	same values	same values

Then Quick Analysis' E_t , σ and N_T equal the **Standard** block of the Detailed results. `tests/test_quick_vs_detailed.py` checks this. A Detailed standard row of $0 / 0$ is unused, so fewer windows can be compared too.

The `impdData` path (b) normalizes by the last sample and uses cross-repeat errors, so its $S(T)$ differs slightly. The curve-fit peak methods have no Detailed Analysis equivalent. The edge guard, the skip rule and the fit rule still apply to both.

Functions

construct_dataAnalysisTab()

Adds the **Quick Analysis** tab to `dltsc.tabControl` and builds both frames.

Name	Type	Default	Meaning
(none)			

Returns None.

Side effects Calls `dltsc.tabControl.add(dltsc.dataAnalysisTab, text='Quick Analysis')` and `tabControl.pack(expand=1, fill="both")`; creates the `tk.PanedWindow`, then calls `_build_rateWindowFrame` and `_build_arrheniusFrame`, which create every widget and `dltsc.rateWindow_*` / `dltsc.arrhenius_*` global listed above.

`dltsc.dataAnalysisTab` must already be a `ttk.Frame` inside `dltsc.tabControl`.

Called by `DLTSGUI_MainWindow.py` (`daT.construct_dataAnalysisTab()`).

Example Needs a Tk root and notebook; not useful outside the GUI.

Internal helpers

`_resolve_impd_for_source(source)`

Maps a Data Source label to an `impdData` instance.

Name	Type	Default	Meaning
<code>source</code>	<code>str</code>		One of the <code>DATA_SOURCE_*</code> labels.

Returns (`impd`, `resolvedLabel`), Or (`None`, `None`) when that source has no instance. Qualitative Analysis always gives (`None`, `None`). Auto tries `livePlot_liveImpdData`, then `livePlot_offlineImpdData`.

Side effects None.

Called by `_calculate_rate_windows`.

`_processed_transients_from_automated(mode)`

Converts the Automated/Live Data frame's "All Emissions Aligned" snapshot into the Qualitative Analysis shape.

Name	Type	Default	Meaning
<code>mode</code>	<code>str</code>		'live' reads <code>dltsc.livePlot_liveAllEmissionsData</code> ; any other value reads <code>dltsc.livePlot_offlineAllEmissionsData</code> .

Returns dict `T_C -> {'time_ms', 'avg_cap_pf', 'C_infinity'}` (keys in °C, rounded to 1e-6 so a 298.15 K key gives 25.0). `C_infinity` here is the sample nearest 90 % of the span. Entries with empty or mismatched `x/ymean` are skipped. Returns `{}` when there is no snapshot.

Side effects None.

Called by `_get_processed_transients_for_source`, `_calculate_rate_windows` (for `N_T` on the `impdData` path).

Example (runs offline, verified):

```

import numpy as np
import matplotlib; matplotlib.use('Agg')
import dltsConfig as dltsC
import dataAnalysisTab as daT

t = np.arange(0, 0.5, 1.8666666666666665e-05)          # s
snap = {}
for TK in np.arange(300.0, 360.0, 5.0):
    en = 1.66e21 * 1e-15 * TK**2 * np.exp(-0.6 / (8.617333e-5 * TK))
    snap[TK] = {'x': t, 'ymean': 100e-12 * (1 - 0.01 * np.exp(-en * t))} # Farads
dltsC.livePlot_offlineAllEmissionsData = snap

pt = daT._processed_transients_from_automated('offline')
print(sorted(pt)[0], pt[sorted(pt)[0]]['C_infinity']) # 26.85 (degC), ~99.996 (pF)

```

`_get_processed_transients_for_source(source)`

Resolves a Data Source label to averaged transients.

Name	Type	Default	Meaning
source	str		One of the <code>DATA_SOURCE_*</code> labels.

Returns (processedTransients, resolvedLabel, errorReason). On success `errorReason` is `None`; on failure the first two are `None` and `errorReason` is a message such as "Qualitative Analysis has no extracted transients yet.". Auto order: Qualitative Analysis, Live snapshot, Offline snapshot.

Side effects None.

Called by `_calculate_rate_windows`.

Example Continuing the example above: `daT._get_processed_transients_for_source(daT.DATA_SOURCE_AUTO)` returns the 12-temperature dict and `'Automated/Live Data – Offline'`.

`_rb_ms_for_source(label, processedTransients)`

Name	Type	Default	Meaning
label	str		Resolved Data Source label.
processedTransients	dict		<code>T_C -> {'time_ms', ...}</code> .

Returns the reverse-bias duration in ms (see Step 2).

`_transient_records(processedTransients, rb_ms)`

Name	Type	Default	Meaning
processedTransients	dict		<code>T_C -> {'time_ms', 'avg_cap_pf', ...}</code> .
rb_ms	float		Reverse-bias duration, ms.

Returns (records, temps): `da._prepare_signal_data` records (Measured C, no denoise) with C_{∞} from `da._cinf_range_mean`, and the sorted °C keys.

`_peak_method_for(peakMethod)` and `_curve_fit_peak_method(curveType)`

`_peak_method_for` turns a `PEAK_METHOD_*` label into a method for `dA._peak_from_signal`: `dA.PEAK_METHOD_SPLINE` for the smoothing spline, or the callable made by `_curve_fit_peak_method(curveType)`. That callable takes `(T, S, S_err)` and returns `(Tp, Tp_err, xFit, yFit)` from `impdData._curveFit_peakFinder`, or the raw grid maximum with no error and no curve if the fit fails.

`_window_peaks(records, temps, windows, tpLo, tpHi, peakMethod)`

Name	Type	Default	Meaning
<code>records, temps</code>			From <code>_transient_records</code> .
<code>windows</code>	list of (t1, t2)		ms.
<code>tpLo, tpHi</code>	float		Search range, K (<code>-inf/inf</code> for no bound).
<code>peakMethod</code>	str		A <code>PEAK_METHOD_*</code> label.

Returns one dict per window: `t1, t2, e_n, T_k, Signal, Signal_err`, plus `dA._peak_from_signal`'s keys (`skipped`, and unless skipped `Tp, Tp_err, S_peak, sign, xFit, yFit`).

`_quick_arrhenius(peaks, gamma, nd, transients=None)`

Name	Type	Default	Meaning
<code>peaks</code>	list of dict		Non-skipped peaks with <code>T_peak, T_peak_err, e_n, S_peak</code> .
<code>gamma</code>	float		$\text{cm}^{-2} \text{s}^{-1} \text{K}^{-2}$.
<code>nd</code>	float		cm^{-3} .
<code>transients</code>	<code>(records, temps, rb_ms)</code> or <code>None</code>	<code>None</code>	For <code>N_T</code> by <code>dA._trap_density</code> .

Returns the `dA._arrhenius_fit` dict (`Et, Et_se, sigma, sigma_se, R2, N, N_excluded, fit_mask, weighted, slope, intercept, x, y, Tp_arr, Tp_err_arr, ...`) plus `Nt`, or `None` if fewer than 2 peaks can be fitted.

Example (runs offline, verified; the synthetic run of `tests/test_detailed_peaks.py`, a 0.60 eV trap):

```
import sys; sys.path.insert(0, 'tests')
import matplotlib; matplotlib.use('Agg')
import dataAnalysisTab as qa, detailedAnalysisTab as dA
from test_detailed_peaks import synthetic_run

pt = {tc: {'time_ms': t, 'avg_cap_pf': c} for tc, (t, c, _) in synthetic_run().items()}
records, temps = qa._transient_records(pt, 500.0)
peaks = [dict(T_peak=p['Tp'], T_peak_err=p['Tp_err'], e_n=p['e_n'], S_peak=p['S_peak'])
          for p in qa._window_peaks(records, temps, dA.DEFAULT_STD_WINDOWS, 290.0, 400.0, qa.PEAK_METHOD_SPLINE)
          if not p['skipped']]
res = qa._quick_arrhenius(peaks, dA.DEFAULT_GAMMA, dA.DEFAULT_ND, (records, temps, 500.0))
print(f"{res['Et']:.4f} +/- {res['Et_se']:.4f} eV, {res['N_excluded']} excluded")
# 0.5957 +/- 0.0005 eV, 1 excluded (the 100/490 ms window peaks on the 290 K edge)
```

`_calculate_rate_windows()`

Command of **Compute Boxcar Spectrums**. Implements Steps 2 to 5 for each set.

Name	Type	Default	Meaning
(none)			Reads all inputs from the <code>dltsc.rateWindow_*</code> Tk variables.

Returns None.

Side effects Clears and redraws `dltsc.rateWindow_ax`; resets and refills `dltsc.rateWindow_signals`, `dltsc.rateWindow_extractedPeaks`, `dltsc.rateWindow_transients` and the Extracted Peaks table; sets `dltsc.rateWindow_denoisedEmissions` on the `impdData` path; updates the status label; writes to the app log via `dltsc.log_to_textbox`. It stops early (with a log message, before clearing anything) when: no data source has data; Smoothed C has no `impdData`; the resolved Live/Offline source is still ingesting (`livePlot_*IngestBusy`); a Tp bound is non-numeric or upper $\leq$ lower; a t1/t2 is non-numeric or not $0 < t1 < t2$. It stops part-way if `calculate_delC_normalized` raises.

Called by the **Compute Boxcar Spectrums** button.

`_run_arrhenius_solver()`

Command of **Execute Arrhenius Signature Solver**. Implements Steps 6 and 7.

Name	Type	Default	Meaning
(none)			Reads <code>dltsc.rateWindow_extractedPeaks</code> , <code>dltsc.rateWindow_transients</code> , <code>dltsc.arrhenius_ndVar</code> , <code>dltsc.arrhenius_gammaVar</code> .

Returns None.

Side effects Sets the three result labels, fills the `Arrhenius_fit` column of the peak table, clears and redraws `dltsc.arrhenius_ax`, logs the result. Stops with a log message when fewer than 2 peaks exist, Nd or γ is not a positive number, the fit raises, or fewer than 2 windows can be fitted.

Called by the **Execute Arrhenius Signature Solver** button.

`_build_rateWindowFrame(parent)`

Builds the left frame (plot, toolbar, scrollable controls, button, table) inside `parent` and creates the `dltsc.rateWindow_*` widgets and variables. Seeds `dltsc.rateWindow_signals` and `dltsc.rateWindow_extractedPeaks` as empty dicts if they are None, because `dltscConfig.init()` is not called by the main window.

`_build_arrheniusFrame(parent)`

Builds the right frame (plot, toolbar, Material Parameters, solver button, results group) inside `parent` and creates the `dltsc.arrhenius_*` widgets and variables. Uses the same row weights and control height as the left frame so both plots are the same height.

`_on_mousewheel(event)` (nested, one in each `_build_*` function)

Scrolls that frame's control canvas by `-event.delta / 120` units. Bound with `bind_all` while the pointer is over the canvas and unbound when it leaves.

Notes and limitations

- C_∞ on the `impdData` path is the last sample of the (trimmed) emission, not the 40–90 % mean. Results from that path are close to, but not the same as, the averaged-transient path and Detailed Analysis.
- The sign convention is $S = (C(t_2) - C(t_1)) / C_{\text{inf}}$. A negative peak is flipped before peak finding; `S_peak` keeps the original sign.
- $e_n = \ln(t_2/t_1) / (t_2 - t_1)$ assumes t_1 and t_2 in ms (the `1e-3` factor). The `impdData` path passes t_1/t_2 in s and may clip them to the data range, but the table and Arrhenius plot still use the e_n computed from the unclipped GUI values.
- The initial plot title says "(Pseudo-Voigt Refinement)" although the default peak method is `Smoothing Spline`. The title is corrected after the first run.
- On the `impdData` path with `Denoise = None (raw)`, `calculate_deIC_normalized` still runs the `pca` filter (`filter_emissions(..., recalculate=True)`) and discards it. This costs time but does not change the result.
- If `calculate_deIC_normalized` fails on a later set, the function returns before `canvas.draw()`. `rateWindow_extractedPeaks` then holds only the earlier sets.
- If every window's peak lies on the edge of the search range, fewer than 3 windows have errors and all are fitted unweighted. The result is then not meaningful: widen the range or choose windows whose peaks fall inside it.
- With only 2 fitted windows the fit has no residual degrees of freedom; E_a and σ are reported with no error.
- N_T is $2 \cdot \max|\Delta C/C_\infty| \cdot N_d$ from $C(\text{rb}) - C(2 \text{ ms})$, the small-signal approximation $N_T \approx 2 (\Delta C/C) N_d$. It ignores the λ -region correction.
- σ is the apparent cross-section `exp(intercept)/ $\gamma$` . It is only as good as the γ entered; the hint gives SiC 1.66e21 and Si 3.256e21.
- `rateWindow_signals` stores T in K under the key `'T_k'`.

detailedAnalysisTab

What it's for

`detailedAnalysisTab.py` builds the **Detailed Analysis** tab, a port of the standalone `DLTS_APP.py` ("DLTS Multiwindow Analysis") into this app. It loads a whole temperature sweep from a data folder, computes DLTS spectra for many log-spaced rate windows (plus five fixed "standard" windows for comparison), finds each peak with an error estimate, and fits an error-weighted Arrhenius line to get E_a , σ and N_T . It also draws a DLTS spectra panel, a $\Delta C/C_0(t, T)$ transient map with a $\tau(T)$ overlay, and a rate-window map. Loading and analysis run in the tab's own worker process so the GUI stays responsive.

The per-window steps (C_∞ , $S(T)$, the peak with its edge guard, the Arrhenius fit with its exclusion rule, N_T) are also what [Quick Analysis](#) calls. The **Standard** result of this tab therefore equals the Quick Analysis result for the same transients, windows, search range and reverse bias; see [Matching Detailed Analysis](#).

What the user sees

A header reads **Detailed Analysis** (DLTS Multiwindow Analysis – ZI MFIA temperature sweep). Below it a horizontal `ttk.PanedWindow` has three parts:

1. a scrollable control column on the left (295 px wide, mouse wheel scrolls it while the pointer is over it);
2. the figure area in the middle (takes all extra width), with a matplotlib toolbar under the figure and a status label at the bottom (initial text `Load a data folder to begin.`);
3. a **Results** text box on the right (Consolas 9, dark background, read-only, 57 characters wide, no wrapping, horizontal and vertical scroll bars).

Control column

Section headings are shown in upper case in the GUI. Spinboxes accept typed values outside the listed range; the range only limits the arrow buttons.

DATA

Widget	Default	What it does
Data folder: entry	empty	Path of the run folder.
Browse		Opens a folder dialog ("Select DLTS Data Folder"), puts the path in the entry and logs a description of the folder contents (<code>liveDataTab._describe_folder_contents</code>).
Load Data		Runs <code>_detailed_load_data()</code> : loads every temperature in the folder in the worker process. Disabled while a load or run is in progress.
info label	No data loaded.	After a load: <code><n> temperatures loaded: <Tmin> °C to <Tmax> °C</code> (blue).

ZI MFIA GRID

Label	Variable	Default	Spinbox range, step	Used?
Grid offset (s)	detailed_gridOffsetVar (DoubleVar)	-0.001 (DEFAULT_GRID_OFFSET)	-0.01 to 0, 0.0001	No. Passed to <code>_load_detailed_data</code> but ignored there.
Grid dt (s)	detailed_gridDtVar (DoubleVar)	1.86667e-5 (DEFAULT_GRID_DT)	1e-6 to 1e-3, 1e-6	No. Ignored.
Chunk size	detailed_chunkSizeVar (IntVar)	32768 (DEFAULT_CHUNK_SIZE)	1024 to 131072, 1024	No. Ignored.
RB duration (ms)	detailed_rbMsVar (DoubleVar)	500.0 (DEFAULT_RB_MS)	10 to 2000, 10	Yes: C _∞ window, legacy-file cycle length, N_T reference time, transient-map time range.

C₀ ESTIMATION WINDOW (C_∞ is the mean capacitance over [Start·RB, End·RB])

Label	Variable	Default	Range, step
Start (frac RB)	detailed_cinfLoVar	0.40	0.1 to 0.8, 0.05
End (frac RB)	detailed_cinfHiVar	0.90	0.3 to 0.99, 0.05

These two values are applied at **Load Data** time. Changing them later has no effect until the data is loaded again.

PHYSICAL CONSTANTS (4H-SiC)

Label	Variable	Default	Range, step
γ (cm ⁻² s ⁻¹ K ⁻²)	detailed_gammaVar	1.66e21	1e20 to 1e22, 1e20
N ^d (cm ⁻³)	detailed_ndVar	3.2e14	1e12 to 1e17, 1e13

DLTS SIGNAL CALCULATION

Label	Variable	Default	Options
Method :	detailed_signalMethodVar	Measured C (nearest sample)	Measured C (nearest sample), Smoothed C (spline-interpolated)
Denoise e:	detailed_denoiseVar	None (raw)	None (raw), pca, wavelet, sgolay, lowess

PEAK SEARCH RANGE

Label	Variable	Default	Range, step
T min (K)	detailed_tpeakLoVar	250.0	100 to 350, 5
T max (K)	detailed_tpeakHiVar	400.0	200 to 600, 5
Peak method: combobox	detailed_peakMethodVar	Smoothing Spline (bootstrap)	Parabolic fit (lmfit), Smoothing Spline (bootstrap)

MULTI-WINDOW PARAMETERS

Label	Variable	Default	Range, step	Meaning
N windows	detailed_nWinVar (IntVar)	16	3 to 60, 1	Number of rate windows.
t ₁ min (ms)	detailed_t1MinVar	5.0	1 to 200, 1	Smallest t1.
t ₁ max (ms)	detailed_t1MaxVar	90.0	5 to 450, 5	Largest t1.
Ratio t ₂ /t ₁	detailed_ratioVar	5.0	2 to 20, 0.5	t2 = ratio · t1 for every window.

STANDARD WINDOWS (REFERENCE)

Widget	Default	What it does
Show standard-window comparison (0 / 0 = unused) checkbox (detailed_stdWinsVar)	on	Draws the standard-window fit (orange dashed line, diamond markers) on the Arrhenius panel. The standard-window fit is computed and written to Results whether or not this is checked.
t1 / t2 (ms): five spinbox pairs (detailed_stdEntries)	5/25, 10/50, 20/100, 50/250, 100/490 (DEFAULT_STD_WINDOWS, the same as Quick Analysis' window sets)	t1 range 0 to 490, t2 range 0 to 499, step 1. A row with t1 ≤ 0 or t2 ≤ t1 (for example 0 / 0) is not used; with no usable row there is no Standard result.

DISPLAY

Widget	Variable	Default	What it does
Show DLTS spectra panel	detailed_showSpectraVar	on	Adds the DLTS Spectra panel.
Spectra to plot spinbox	detailed_nSpectraVar (IntVar)	5	Range 2 to 16. Number of multi-windows drawn in the spectra panel.
Show 2D transient map	detailed_showTmapVar	on	Adds the ΔC/C ₀ transient map panel.
Overlay τ(T) curve on map	detailed_showTauVar	on	Draws τ(T) from the fit and the (T _p , 1/e _n) points on the transient map.
Show Rate-Window Analysis map	detailed_showRwmVar	on	Adds the rate-window map panel.

Actions (under an empty section divider)

Button	What it does
Run Analysis	Disabled until data has been loaded. Runs <code>_detailed_run_analysis()</code> : snapshots all parameters, computes everything in the worker process, then draws the figure and fills Results. Status becomes Done. Et = <E ± err> eV Nt = <N _T > cm ⁻³ .
Edit Labels & Legends	Opens the "Edit Labels & Legends" dialog (<code>_detailed_open_label_editor</code>). Logs a hint if no analysis has been run yet.
Save Figure...	Save dialog (default name <code>DLTS_MultiWindow.png</code>). Formats: PNG, PDF, SVG, EPS, JPEG, TIFF, BMP. Raster formats are saved at 180 dpi with the figure background; vector formats without a dpi.

Button	What it does
Export Results to TXT	Save dialog (default name <code>DLTS_Results.txt</code>); writes the Results text as UTF-8.

Figure

The figure is rebuilt on every run. It always has the **Arrhenius Plot** panel. The other panels depend on the three Display checkboxes:

Spectra	Map	RW map	figsize (in)	Layout
on	on	on	14 × 11.0	2×2: Arrhenius, Spectra / Transient map, RW map
on	on	off	13 × 10.5	Arrhenius, Spectra (width 1.5:1) / Transient map across the bottom
on	off	on	13 × 10.5	Arrhenius, Spectra / RW map across the bottom
off	on	on	14 × 10.5	Arrhenius across the top / Transient map, RW map
on	off	off	13 × 5.4	Arrhenius, Spectra side by side
off	on	off	10 × 10.5	Arrhenius / Transient map
off	off	on	10 × 10.5	Arrhenius / RW map
off	off	off	8 × 5.4	Arrhenius only

Every panel uses `set_box_aspect(0.9)`. The figure title is "DLTS Multiwindow Analysis".

- **Arrhenius Plot:** x " $1000 / T$ (K^{-1})", y " $\ln(e_n / T^2)$ ($s^{-1} K^{-2}$)". Multi-window points colored by $\log_{10} e_n$ (plasma colormap, colorbar " $\log_{10} e_n$ (s^{-1})"), error bars from T_p errors, blue fit line labeled `Multi (<N> win) Et=<E> eV  $R^2=<R^2>$` with a shaded 2σ confidence band. Excluded windows are gray x markers "Excluded (peak at range edge, no T_p error)". Optional standard-window line and diamonds. A top axis shows $^{\circ}C$ ticks for 240 to 360 K in 10 K steps (only those inside the x range). A yellow box in the lower-left lists E_a , σ , N_T , R^2 and (if weighted) χ^2 .
- **DLTS Spectra:** x "Temperature ($^{\circ}C$)", y " $\Delta C/C_0$ ($\times 10^{-3}$)". One line per selected window (cool colormap), labeled `t=<t1>/<t2> ms  $e_n=<e_n> s^{-1}$` ; dotted vertical lines at each multi-window T_p .
- **$\Delta C/C_0$ Transient Map:** filled contours (jet, 64 levels, black contour every 6th level) of $\Delta C/C_0 \times 10^{-5}$ vs "Temperature (K)" and log "Time (s)". With the τ overlay: white $\tau(T)$ line labeled `Z12  $\tau(T)$   $E_t=<E> eV$` , cyan circles "Rate-window peaks (T_p , $1/e_n$)", and a cyan " Z_{12} defect" arrow at the middle point. Shows "Not enough time points" when the map cannot be built.
- **Rate-Window Analysis Map:** black background, x " $1/kT$ (eV^{-1})", log y " T^2/e_n (K^2s)", color N_T/N_D on a log scale (jet). White dashed Arrhenius line and 22 cyan circles " $Z_{1/2}$ (fitted)". Shows "Insufficient data for Rate-Window map" when it cannot be built.

All legends and the results box can be dragged with the left mouse button.

Edit Labels & Legends dialog

A scrollable `TopLevel` (500×580, minimum 460×340). For each existing panel (Arrhenius Panel, DLTS Spectra Panel, Transient Map Panel, Rate-Window Map Panel) it offers `Title`, `X label`, `Y label` text entries with a point-size spinbox (6 to 28). For each existing legend (Arrhenius Legend, Spectra Legend, Transient Map Legend, Rate-Window Legend) it

offers one `Font size` spinbox (6 to 18) and one entry per legend item (`Entry n`). Buttons: **Apply** (applies and redraws), **Apply & Close**, **Close** (discards unapplied edits). Edits are lost on the next **Run Analysis**, which rebuilds the figure.

Results text

Example layout (numbers from the synthetic example further below):

```
=====
DLTS Multiwindow Analysis  -  Results
=====

Signal      : Measured C (nearest sample)
Denoise     : None (raw)
Peak method: Smoothing Spline (bootstrap)

Multi-window (16 windows)
  fit  : weighted by Tpeak errors
  Et   = 0.5968 +/- 0.0032 eV
  sn   = 8.904e-16 +/- 1.076e-16 cm2
  slope = ... K
  intercept = ...
  R2    = ...
  red. chi2 = ...
  Nt    ~ 6.236e+12 cm-3

Standard (5 windows)
...
Et uncertainty improvement: <Et_se(std) / Et_se(multi)>x

Window detail (multi):
  t1(ms)   t2(ms)   en(s-1)   Tpeak(C)   +/- (K)
-----
      5.0      25.0      80.47      ...      ...
```

excl. marks a window left out of the fit. n/a means the peak fit gave no `Tp` error. The fit line reads `UNWEIGHTED` (no `Tpeak errors`) when no weighting was possible, and adds , <k> window(s) excluded when windows were excluded.

Import

```
import detailedAnalysisTab as deT # alias used in DLTSGUI_MainWindow.py
deT.construct_detailedAnalysisTab()

import detailedAnalysisTab as daT # alias used in tests/test_hdf5_analysis.py and benchmarks/
data, temps, errs = daT._load_detailed_data(folder, None, None, None, 500.0, 0.4, 0.9)
```

`dataAnalysisTab.py` also imports this module, as `dA`, for `_cap_noise_at`, `_valid_errors` and `_fit_arrhenius_line`. In `DLTSGUI_MainWindow.py` the alias `daT` means `dataAnalysisTab`, not this module.

Module constants and globals

Name	Value / type	Meaning
C0	'#2a78d6'	Blue: multi-window fit, info label text.
C1	'#eb6834'	Orange: standard-window fit.
SURFACE	'#fcfcfb'	Figure/axes background; raster save background.
TEXT_PRI, TEXT_SEC, TEXT_MUT	'#0b0b0b', '#52514e', '#898781'	Text colors.
GRIDLINE, BASELINE	'#e1e0d9', '#c3c2b7'	Grid and axis-spine colors.
CTRL_BG	'#f0f0f0'	Control column and dialog background.
DEFAULT_GRID_OFF	-0.001	Default Grid offset (s) (not used by the loader).
DEFAULT_GRID_DT	1.86667e-5	Default Grid dt (s) (not used by the loader).
DEFAULT_CHUNK_SIZE	32768	Default Chunk size (not used by the loader).
DEFAULT_RB_MS	500.0	Default RB duration (ms).
KB_EV	8.617333262145e-5	Boltzmann constant, eV/K (CODATA 2018). Quick Analysis uses the same value.
DEFAULT_CINF_LO, DEFAULT_CINF_HI	0.40, 0.90	C_{∞} window as fractions of the reverse bias (<code>_cinf_range_mean</code>).
DEFAULT_ND	3.2e14	Default Nd, cm ⁻³ (both tabs).
DEFAULT_GAMMA	1.66e21	Default γ , cm ⁻² s ⁻¹ K ⁻² (4H-SiC; both tabs).
PEAK_MIN_FRAC	0.05	A window whose peak is at most this fraction of max S is skipped (<code>_peak_from_signal</code>).
DEFAULT_TPEAK_LO, DEFAULT_TPEAK_HI	250.0, 400.0	Default peak search range, K (both tabs).
DEFAULT_STD_WINDOWS	[(5, 25), (10, 50), (20, 100), (50, 250), (100, 490)] (ms, floats)	Default standard windows, and Quick Analysis' default window sets.
_FOLDER_RE	<code>re.compile(r'^(n?)(\d+)C(?:_\d+)?\$', re.I)</code>	Temperature subfolder names: 25C_001, 85C, n10C.
SIGNAL_METHOD_MEASURED	'Measured C (nearest sample)'	Signal method (default).
SIGNAL_METHOD_SMOOTHED	'Smoothed C (spline-interpolated)'	Signal method.
SIGNAL_METHOD_OPTIONS	list of the two above	Combobox values.
DENOISE_NONE	'None (raw)'	No denoising.
DENOISE_OPTIONS	['None (raw)', 'pca', 'wavelet', 'sgolay', 'lowess']	Combobox values.

Name	Value / type	Meaning
PEAK_METHOD_PARABOLIC	'Parabolic fit (lmfit)'	Peak method.
PEAK_METHOD_SPLINE	'Smoothing Spline (bootstrap)'	Peak method (default).
PEAK_METHOD_OPTIONS	[PEAK_METHOD_PARABOLIC, PEAK_METHOD_SPLINE]	Combobox values.
EDGE_PEAK_FRAC	0.01	A peak within this fraction of a grid step of either end of the peak search range is an edge peak (<code>_peak_on_edge</code>) and gets no <code>T_p</code> error.
MIN_TP_ERR_K	1e-4 (K)	Smallest <code>T_p</code> error accepted as real; smaller ones are treated as no error.
_FIGURE_SAVE_FILE_TYPES	list of 8 (label, pattern) tuples	Save Figure dialog file types.

Shared state in `dltsConfig` (`dltsC`):

Name	Type	Meaning
<code>detailed_data</code>	dict <code>T_C</code> -> (<code>t_{ms}</code> , <code>cap_{pF}</code> , <code>C_{inf}</code>)	Loaded sweep.
<code>detailed_temps</code>	sorted list of float, °C	Loaded temperatures.
<code>detailed_figure</code> , <code>detailed_canvas</code> , <code>detailed_figFrame</code>	Figure, FigureCanvasTkAgg, tk.Frame	Current figure and its container.
<code>detailed_ax1</code> ... <code>detailed_ax4</code>	Axes or None	Arrhenius, Spectra, Transient map, RW map.
<code>detailed_leg1</code> , <code>detailed_leg2</code> , <code>detailed_legM</code> , <code>detailed_legRW</code>	Legend or None	Legends of those panels.
<code>detailed_annBox</code>	Annotation	Arrhenius results box.
<code>detailed_lastResMw</code> , <code>detailed_lastResStd</code> , <code>detailed_lastNt</code>	dict, dict/None, float	Last run's results.
<code>detailed_loadingBusy</code> , <code>detailedProcessingBusy</code>	bool (or None before first use)	Load / Run in progress.
<code>detailed_executor</code>	ProcessPoolExecutor or None	The tab's single worker process. Shut down by <code>DLTSGUI_MainWindow.py</code> on exit.
<code>detailed_hitTestStale</code>	bool	True between a canvas resize and its redraw (drag hit-testing).
<code>detailed_*Var</code> , <code>detailed_stdEntries</code>	Tk variables	All control values (see tables above).
<code>detailed_loadButton</code> , <code>detailed_runButton</code> , <code>detailed_loadInfoLabel</code> , <code>detailed_statusLabel</code> , <code>detailed_resultsText</code>	widgets	Buttons, labels, Results text.

Name	Type	Meaning
read: <code>tabControl</code> , <code>detailedAnalysisTab</code> , <code>root</code> , <code>log_to_textbox</code>		Main-window objects.

Analysis method

Times are in ms unless noted; temperatures are stored in °C and converted with $T_K = T_C + 273.15$; capacitance is in pF (as returned by the extractors).

Step 1. Load transients (`_load_detailed_data`). The folder format is detected in this order:

1. **ZI single-file export**: a file matching `imps_0_sample_param1_avg_header*.csv` directly in the folder. Parsed by `liveDataTab._compute_zi_dataset`; temperatures come from the header's `history_name (<n>C_...)`, one chunk per temperature; grid offset/delta/chunk size come from the header.
2. **ZI subfolder-per-temperature export** (the original `DLTS_APP.py` format): subfolders named like `0C`, `n10C`, `120C_000`, each with its own `imps_0_sample_param1_avg_<n>.csv` and header. Parsed by `liveDataTab._compute_zi_subfolder_dataset`; chunk 0 of each file is used; grid parameters from each subfolder's header (defaults -0.001 s, $1.86667e-5$ s, 32768 if unreadable).
3. **Legacy per-temperature files**: names matching `^[npNP]\d+([pP]\d+)?[cC]?(_\d+)?\.(txt|csv|h5)$` (e.g. `p25p0.h5`, `n10p0.txt`). Parsed by `liveDataTab._compute_legacy_dataset`. For `.txt` (JSON) and `.h5`: ImpedanceIm $\cdot 1e12$ is split at each fall of `AuxInput1` through the midpoint of its 0.01/99.99 percentile levels, cut into blocks of `int(rb_ms * 1e-3 / 1.8666666666666665e-05)` samples, and averaged; time axis `k * 1.8666666666666665e-05 * 1000` ms, starting at 0 at the trigger.

For the ZI formats the time axis is `(grid_col_offset + k * grid_col_delta) * 1000` ms, and values below $1e-3$ in magnitude are scaled by $1e12$ (F to pF).

C_∞ is then recomputed for every temperature, independent of format:

```
mask = (t_ms >= cinf_lo * rb_ms) & (t_ms <= cinf_hi * rb_ms)
C_inf = nanmean(cap[mask])      if any sample is in the mask
      = nanmean(cap)           otherwise
```

With the defaults: mean over 200 to 450 ms of a 500 ms reverse bias.

Step 2. Prepare the C(t) read-off (`_prepare_signal_data`, `_denoise_transient`, `_cap_at`). Once per run, each transient is optionally denoised with the `impdData` filters at the same settings as `impdData.filter_emissions: pca` (`window_size=100`), `wavelet (db4, level=4, mode='soft')`, `sgolay (window_size=None, order=2)`, `lowess (fraction=None)`. For Smoothed C a `scipy.interpolate.CubicSpline` is built through the (denoised) curve. `_cap_at(rec, t)` returns the spline value (Smoothed C) or the nearest sample (Measured C; ties go to the earlier sample). C_∞ is not recomputed after denoising.

Step 3. Rate windows (`_compute_detailed_analysis`):

```
t1_k = logspace(log10(t1_min), log10(t1_max), N)      # ms
t2_k = ratio * t1_k                                  # ms
e_n  = ln(t2 / t1) / ((t2 - t1) * 1e-3)              # s^-1
```

Defaults give 16 windows from 5/25 ms ($e_n = 80.47 \text{ s}^{-1}$) to 90/450 ms ($e_n = 4.47 \text{ s}^{-1}$). The five standard windows are processed the same way.

Step 4. DLTS signal and its error (`_rate_window_signal`, `_cap_noise_at`):

```
S(T)      = (C(t2) - C(t1)) / C_inf          # dimensionless
sigma_S   = hypot(noise(t1), noise(t2)) / |C_inf|
noise(t)  = sqrt( sum(d2^2) / (6 * len(d2)) ), d2 = c[k-1] - 2 c[k] + c[k+1]
```

`noise(t)` is the Rice (1984) second-difference estimator over the samples `cap_raw[i-25 : i+26]`, where `i = searchsorted(t_ms, t)`. It always uses the raw (not denoised) transient. It returns NaN when fewer than 5 samples are available.

Step 5. Peak per window (`_peak_from_signal`, `_find_peak_full`):

```
mask = (T_K >= T_min) & (T_K <= T_max)
S_m  = S[mask]; if max(S_m) < |min(S_m)|: S_m = -S_m          # make the peak positive
skip the window if max(S_m) <= 0.05 * max(|S|) (over all temperatures)
```

- Smoothing Spline (bootstrap) (`_find_peak_spline`): `impdData._smoothingSpline_peakFinder(T, S_m, signalYErr=sigma_S)`. Weighted UnivariateSpline($k=3$, $s=n$) when all errors are valid, otherwise $s = n \cdot \text{Rice noise variance of } S$. T_p is the argmax of $|spline|$ on a 100 000-point grid; T_p error is the standard deviation of 200 bootstrap refits. On an exception: the grid point with the largest S , no error.
- Parabolic fit (lmfit) (`_find_peak_parabolic`, `_vertex_stderr`): `lmfit QuadraticModel` on the 5 points ($hw = 2$) around the grid maximum, in $x = T - T_{ref}$, weights $1/\sigma_S$ when valid. If $a < 0$ and the vertex lies inside those points: $T_p = T_{ref} - b/(2a)$, with

```
J      = (b / (2 a^2), -1 / (2 a))
var(T_p) = J · Cov(a, b) · J^T
```

Otherwise $T_p = T_{ref}$ with no error. No error either when `result.covar` is None or `nfree < 1`.

Step 6. Arrhenius fit (`_arrhenius_fit`, `_fit_arrhenius_line`):

```
x = 1000 / T_p          # K^-1 (x 1000)
y = ln(e_n / T_p^2)    # ln(s^-1 K^-2)
```

`_find_peak` reports no T_p error for a peak on the edge of the search range (the signal is still rising out of the range, so the grid end is not a real peak) and for any error below `MIN_TP_ERR_K`, whichever peak method is used. Windows with no T_p error are excluded from the fit (`fit_mask`) unless fewer than 3 windows have errors; then all windows are fitted. Fewer than 2 windows returns None.

`_fit_arrhenius_line` first fits $y = \text{slope} \cdot x + \text{intercept}$ unweighted (`lmfit LinearModel`). If every T_p error is finite and > 0 , it iterates up to 10 times:

```
sig_eff = | -2/T_p + 1000 * slope / T_p^2 | * sigma_Tp      # effective variance
refit with weights = 1 / sig_eff
stop when |slope_new - slope_old| <= 1e-9 * max(|slope_old|, 1)
```

Step 7. Defect parameters (`_arrhenius_fit`, `kB = kB_EV = 8.617333262145e-5 eV/K`):

```
E_a      = -slope * 1000 * kB          # eV
dE_a     = slope_stderr * 1000 * kB    # eV
sigma_n  = exp(intercept) / gamma      # cm^2
dsigma_n = sigma_n * intercept_stderr  # cm^2
R^2      = lmfit rsquared
```

With fewer than 3 fitted points, slope/intercept errors and the covariance are set to `None`.

Step 8. Trap density (`_trap_density`):

```
S_ref(T) = (C(rb_ms) - C(2.0 ms)) / C_inf
N_T      = 2 * max_T |S_ref(T)| * Nd      # cm^-3 (Nd in cm^-3)
```

This uses all loaded temperatures, not only the peak-search range.

Step 9. Display arrays.

- Spectra (`_prepare_spectra`): windows at indices `round(linspace(0, N-1, min(nSpectra, N)))`; curve $S(T) \cdot 1e3$.
- Transient map (`_prepare_transient_map`): time range `max(2.0, t[1])` to `0.95 * rb_ms` ms; up to 250 log-spaced sample indices; $Z = (C(t) - C_{inf}) / C_{inf}$, displayed as $Z \cdot 1e5$; 64 color levels between the 1st and 99th percentile. Needs at least 10 time points.
- τ overlay (`_detailed_plot_transient_map`): $e_n(T) = \gamma \cdot \sigma_n \cdot T^2 \cdot \exp(-E_a / (kB \cdot T))$, $\tau = 1/e_n$ (s), over T from `min(T) - 20` to `max(T) + 20` K.
- Rate-window map (`_prepare_rate_window_map`): 100 log-spaced $t1$ from $t1_{min}$ to $t1_{max}$, $t2 = ratio \cdot t1$; for every (window, temperature): $x = 1/(kB \cdot T)$ (eV^{-1}), $y = T^2 / e_n$ ($K^2 s$), $z = 2 |S|$ ($= N_T/N_D$); points with $z < 1e-8$ or $|C_{inf}| < 1e-30$ skipped. Linearly interpolated (`scipy.interpolate.griddata`) onto 220×220 points in (x , $\log_{10} y$); log color scale from the 2nd percentile ($\geq 1e-8$) to the 98th percentile ($\geq 10 \times$ low). Overlay line $y = \exp(E_a \cdot x) / (\gamma \cdot \sigma_n)$. Needs at least 6 points.
- Confidence band on the Arrhenius panel: $half = 2 \cdot \sqrt{var_m \cdot x^2 + var_b + 2 \cdot x \cdot cov_{mb}}$.
- Arrhenius error bars (`_detailed_arrhenius_errorbars`): $dx = 1000 \cdot dT / T^2$, $dy = 2 \cdot dT / T$.

Executor model

`_detailed_load_data` and `_detailed_run_analysis` each start a daemon `threading.Thread`. That thread submits the pure function (`_load_detailed_data` or `_compute_detailed_analysis`) to `dltsc.detailed_executor`, a `ProcessPoolExecutor(max_workers=1)` created on first use, and blocks on `future.result()`. The result is handed back to the Tk main thread with `dltsc.root.after(0, apply)`, where widgets are updated and the figure is drawn. The worker process has its own GIL, so parsing and fitting do not stall the GUI. If the worker dies (`BrokenProcessPool`), the executor is dropped and the next Load/Run creates a new one. Load and Run are mutually exclusive (`detailed_loadingBusy`, `detailedProcessingBusy`); both buttons are disabled while either runs. The whole data dict is pickled to the worker on every Run.

Functions

construct_detailedAnalysisTab()

Adds the **Detailed Analysis** tab to `dltsc.tabControl` and builds all its widgets.

Name	Type	Default	Meaning
(none)			

Returns `None`.

Side effects `dltsc.tabControl.add(dltsc.detailedAnalysisTab, text='Detailed Analysis')`, `tabControl.pack(expand=1, fill="both")`; creates every `dltsc.detailed_*` variable and widget listed above with the defaults listed above. Run starts disabled. `dltsc.detailedAnalysisTab` must already exist as a `ttk.Frame` in the notebook.

Called by `DLTSGUI_MainWindow.py` (`deT.construct_detailedAnalysisTab()`).

Example Needs a Tk root and notebook; not useful outside the GUI.

Internal helpers

Computation helpers (no Tk; safe outside the GUI)

`_folder_to_tempC(name)`

Name	Type	Default	Meaning
<code>name</code>	<code>str</code>		Folder name.

Returns Temperature in °C as float (`'n10C' → -10.0`, `'120C_000' → 120.0`), or `None` if `name` does not match `_FOLDER_RE`.

Side effects `None`. **Called by** nothing in the current code (the loader uses `LiveDataTab`'s own identical pattern).

`_denoise_transient(t_ms, cap, method)`

Name	Type	Default	Meaning
<code>t_ms</code>	<code>ndarray</code>		Time axis, ms.
<code>cap</code>	<code>ndarray</code>		Capacitance, pF.
<code>method</code>	<code>str</code>		<code>'pca'</code> , <code>'wavelet'</code> , <code>'sgolay'</code> , <code>'lowess'</code> ; anything else returns <code>cap</code> unchanged.

Returns Denoised copy of `cap` (float `ndarray`, truncated to `len(cap)`). Filter settings are in Step 2.

Side effects `None`. **Called by** `_prepare_signal_data`.

`_prepare_signal_data(data, temps, signal_method, denoise)`

Name	Type	Default	Meaning
<code>data</code>	dict		<code>T_C -> (t_ms, cap, c_inf)</code> from <code>_load_detailed_data</code> .
<code>temps</code>	list of float		Temperatures to prepare (°C).
<code>signal_method</code>	str		<code>SIGNAL_METHOD_MEASURED</code> or <code>SIGNAL_METHOD_SMOOTHED</code> .
<code>denoise</code>	str		One of <code>DENOISE_OPTIONS</code> .

Returns dict `T_C -> (t_ms, cap_used, c_inf, cap_raw, spline)`. `cap_used` is denoised (or raw for `None` (raw)); `spline` is a `CubicSpline` through `cap_used` for Smoothed C, else `None`.

Side effects None. **Called by** `_compute_detailed_analysis`.

`_cap_at(rec, tv_ms)`

Name	Type	Default	Meaning
<code>rec</code>	tuple		One <code>_prepare_signal_data</code> record.
<code>tv_ms</code>	float or ndarray		Time(s), ms.

Returns C at `tv_ms`: spline value, or nearest sample (ties to the earlier sample). Scalar in, float out; array in, array out. Times outside the data give the end sample (Measured C) or a spline extrapolation (Smoothed C).

Side effects None. **Called by** `_rate_window_signal`, `_prepare_spectra`, `_prepare_transient_map`, `_prepare_rate_window_map`, `_compute_detailed_analysis`.

`_cap_noise_at(t_ms, cap, tv_ms, hw=25)`

Name	Type	Default	Meaning
<code>t_ms</code>	ndarray		Time axis, ms (sorted).
<code>cap</code>	ndarray		Raw capacitance, pF.
<code>tv_ms</code>	float		Time where the noise is wanted, ms.
<code>hw</code>	int	25	Half-width of the sample window.

Returns 1σ sample noise (same unit as `cap`) from the Rice second-difference estimator (Step 4), or `NaN` if fewer than 5 samples fall in the window.

Side effects None. **Called by** `_rate_window_signal`; `dataAnalysisTab._calculate_rate_windows`.

`_rate_window_signal(temps, data, t1, t2)`

Name	Type	Default	Meaning
<code>temps</code>	list of float		Temperatures, °C.
<code>data</code>	dict		<code>_prepare_signal_data</code> records.
<code>t1, t2</code>	float		Gate times, ms.

Returns (`S`, `S_err`), two ndarrays in `temps` order: $S = (C(t_2) - C(t_1)) / C_{inf}$ and its 1σ error (Step 4).

Side effects None. **Called by** `_compute_arrhenius`.

`_valid_errors(err)`

Name	Type	Default	Meaning
<code>err</code>	ndarray or None		Candidate 1σ errors.

Returns `err` if it is not `None` and every entry is finite and > 0 , else `None`.

Side effects None. **Called by** `_find_peak_spline`, `_find_peak_parabolic`, `_fit_arrhenius_line`; `dataAnalysisTab._calculate_rate_windows`.

`_find_peak(T_arr, S_arr, S_err, method)`

Name	Type	Default	Meaning
<code>T_arr</code>	ndarray		Temperatures, K.
<code>S_arr</code>	ndarray		Signal (positive-going peak expected).
<code>S_err</code>	ndarray		1σ errors of <code>S_arr</code> .
<code>method</code>	str or callable		<code>PEAK_METHOD_SPLINE</code> selects the spline; <code>PEAK_METHOD_PARABOLIC</code> (or any other string) the parabola; a callable <code>(T, S, S_err) -> (Tp, Tp_err, xFit, yFit)</code> is used as is (Quick Analysis passes its Imfit curve-fit finders this way).

Returns `(Tp, Tp_err)` in K. `Tp_err` is `None` when the method gave none, when the peak is on the edge of `T_arr` (`_peak_on_edge`), or when the error is below `MIN_TP_ERR_K` (`_guard_peak_error`); `_arrhenius_fit` then leaves the window out of the weighted fit.

`_find_peak_full(T_arr, S_arr, S_err, method)` takes the same arguments and returns `(Tp, Tp_err, xFit, yFit)`: the same peak plus the fitted curve for plotting (`None, None` for the parabola). `_find_peak` is `_find_peak_full(...)[2]`.

Side effects None. **Called by** `_peak_from_signal`.

Example

```
T = np.arange(300.0, 330.0, 5.0)
deT._find_peak(T, (T - 290) / 100, np.full(T.size, 1e-3), deT.PEAK_METHOD_SPLINE) # (325.0, None): edge peak
```

`_peak_on_edge(T_arr, Tp)`

True if `Tp` is within `EDGE_PEAK_FRAC` $\times$ the smallest grid step of either end of `T_arr`, or outside it, or if `T_arr` has fewer than 2 points.

`_guard_peak_error(T_arr, Tp, Tp_err)`

Returns `float(Tp_err)`, or `None` when `_peak_on_edge(T_arr, Tp)` or when `Tp_err` is `None`, not finite, or below `MIN_TP_ERR_K`. **Called by** `_find_peak_full`.

`_find_peak_spline(T_arr, S_arr, S_err)`

Parameters as `_find_peak`. **Returns** (float(Tp), Tp_err, xFit, yFit) from `impdData._smoothingSpline_peakFinder` (Step 5), or (T at `argmax(S_arr)`, None, None, None) on an exception. **Side effects** None. **Called by** `_find_peak_full`.

`_find_peak_parabolic(T_arr, S_arr, S_err=None, hw=2)`

Name	Type	Default	Meaning
T_arr	ndarray		Temperatures, K.
S_arr	ndarray		Signal.
S_err	ndarray or None	None	1 σ errors; used as weights <code>1/S_err</code> when valid.
hw	int	2	Points on each side of the grid maximum (5 points in total).

Returns (Tp, Tp_err): the vertex and its error from `_vertex_stderr` when the parabola opens downward and the vertex is inside the fitted points; otherwise (T_ref, None). Also (T_ref, None) when fewer than 3 points are available or the fit raises.

Side effects None. **Called by** `_find_peak_full`.

`_vertex_stderr(result, a, b)`

Name	Type	Default	Meaning
result	lmfit <code>ModelResult</code>		Quadratic fit.
a, b	float		Fitted quadratic and linear coefficients.

Returns $\sqrt{J \cdot \text{Cov}(a,b) \cdot J^T}$ with $J = (b/(2a^2), -1/(2a))$, or None if `covar` is None, `nfree` < 1, or the variance is negative or not finite.

Side effects None. **Called by** `_find_peak_parabolic`.

`_fmt_pm(val, err, fmt, sep=' ± ')`

Name	Type	Default	Meaning
val	float		Value.
err	float or None		Error.
fmt	str		Format spec, e.g. <code>'.3f'</code> .
sep	str	' ± '	Separator.

Returns 'val ± err' or 'val' when `err` is None. `_fmt_pm(1.23456, 0.012, '.3f') → '1.235 ± 0.012'`.

Side effects None. **Called by** plotting, results and status code.

`_load_detailed_data(base, grid_off, grid_dt, chunk_size, rb_ms, cinf_lo, cinf_hi)`

Loads every temperature in a run folder (Step 1).

Name	Type	Default	Meaning
base	str		Run folder path.
grid_off	float		Ignored.
grid_dt	float		Ignored.
chunk_size	int		Ignored.
rb_ms	float		Reverse-bias duration, ms. Sets the legacy block length and the C^∞ window.
cinf_lo	float		Start of the C^∞ window, fraction of rb_ms.
cinf_hi	float		End of the C^∞ window, fraction of rb_ms.

Returns (data, temps, errorMsgs): data[T_C] = (t_ms, cap_pF, C_inf); temps sorted °C list; errorMsgs list of per-file problems. ({}, [], errorMsgs) when nothing is recognized.

Side effects Reads files only. os.listdir(base) raises if base does not exist.

Called by _detailed_load_data (in the worker process); tests/test_hdf5_analysis.py; benchmarks/bench_hdf5.py; benchmarks/hardware/hw_analyze.py.

Example (verified on the HDF5 run folder: 6 temperatures, 321 samples each, 0 to 5.973 ms):

```
import os
os.environ['LOKY_MAX_CPU_COUNT'] = '8'
import matplotlib; matplotlib.use('Agg')
import detailedAnalysisTab as daT

data, temps, errs = daT._load_detailed_data(
    r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655", None, None, None, 6.0, 0.4, 0.9)
print(temps)          # [25.0, 30.0, 35.0, 40.0, 45.0, 50.0]
t_ms, cap, c_inf = data[25.0]
print(len(t_ms), t_ms[1], t_ms[-1], c_inf)  # 321 0.018666... 5.9733... ~5.09e6
```

The JSON run folder C:\Users\spencer\Desktop\DATA\DLTS\092326\101901 (p100p0.json etc.) returns ({}, [], []): .json is not a recognized extension, and no error message is produced.

_cinf_range_mean(t_ms, cap, rb_ms, cinf_lo=DEFAULT_CINF_LO, cinf_hi=DEFAULT_CINF_HI)

C^∞ of one averaged transient: nanmean(cap) over $\text{cinf_lo} \cdot \text{rb_ms} \leq t_ms \leq \text{cinf_hi} \cdot \text{rb_ms}$, or over the whole transient if no sample falls in that range. **Called by** _load_detailed_data; dataAnalysisTab._transient_records.

_fit_arrhenius_line(x, y, Tp, Tp_err)

Name	Type	Default	Meaning
x	ndarray		1000 / Tp.
y	ndarray		$\ln(e_n / T_p^2)$.
Tp	ndarray		Peak temperatures, K.
Tp_err	ndarray		1σ errors of Tp, K (NaN allowed; any NaN makes the fit unweighted).

Returns (lmfit ModelResult, weighted). weighted is False when Tp_err fails _valid_errors. Algorithm in Step 6.

Side effects None. **Called by** `_arrhenius_fit`.

Example (verified; recovers 0.600 eV from exact points):

```
import numpy as np
import detailedAnalysisTab as daT
Tp = np.array([300., 310., 320., 330.])
en = 1.66e21 * 1e-15 * Tp**2 * np.exp(-0.6 / (8.617333e-5 * Tp))
res, weighted = daT._fit_arrhenius_line(1000 / Tp, np.log(en / Tp**2), Tp, np.full(4, 0.5))
print(-res.params['slope'].value * 1000 * 8.617333e-5, weighted) # 0.6000000000000011 True
```

`_emission_rate(t1, t2)`

$\ln(t_2/t_1) / ((t_2 - t_1) \cdot 1e-3)$, s^{-1} , for t_1 and t_2 in ms.

`_peak_from_signal(T_K, S_full, S_err_full, t_peak_lo, t_peak_hi, peak_method=PEAK_METHOD_SPLINE, peak_min_frac=PEAK_MIN_FRAC)`

One window's peak from its $S(T)$ curve (Step 5). Shared with Quick Analysis.

Name	Type	Default	Meaning
<code>T_K</code>	ndarray		Temperatures, K.
<code>S_full</code> , <code>S_err_full</code>	ndarray, ndarray or None		$S(T)$ and its 1σ errors over all temperatures.
<code>t_peak_lo</code> , <code>t_peak_hi</code>	float		Search range, K ($\pm\infty$ allowed).
<code>peak_method</code>	str or callable	<code>PEAK_METHOD_SPLINE</code>	As <code>_find_peak</code> .
<code>peak_min_frac</code>	float	<code>0.05</code>	Skip threshold, fraction of $\max S $.

Returns `dict(skipped=True)` when no point is in range or the peak is at most `peak_min_frac · max|S_full|`; otherwise `dict(skipped=False, Tp, Tp_err, S_peak, sign, xFit, yFit)`. `S_peak` is the in-range grid maximum of `sign · S`, given back in the curve's own sign; `yFit` is in the curve's own sign too.

Called by `_compute_arrhenius`; `dataAnalysisTab._window_peaks`, `dataAnalysisTab._calculate_rate_windows`.

`_arrhenius_fit(en_list, Tp_list, Tp_err_list, gamma)`

Steps 6 and 7 on a list of peaks. Shared with Quick Analysis.

Name	Type	Default	Meaning
<code>en_list</code>	list of float		e_n , s^{-1} .
<code>Tp_list</code>	list of float		T_p , K.
<code>Tp_err_list</code>	list of float/None		T_p errors, K; <code>None</code> or <code>NaN</code> = no error.
<code>gamma</code>	float		γ , $cm^{-2} s^{-1} K^{-2}$.

Returns `None` when fewer than 2 windows can be fitted; otherwise the dict described under `_compute_arrhenius` below, without `Sp_arr`, `peak_method` and `detail`.

Called by `_compute_arrhenius`; `dataAnalysisTab._quick_arrhenius`.

`_trap_density(data, temps, rb_ms, nd)`

Step 8: $2 \cdot \max_T |(C(rb_ms) - C(2\ ms)) / C_\infty| \cdot nd, \text{cm}^{-3}$, from `_prepare_signal_data` records. **Called by** `_compute_detailed_analysis`; `dataAnalysisTab._quick_arrhenius`.

`_compute_arrhenius(windows, temps, data, gamma, t_peak_lo, t_peak_hi, peak_method=PEAK_METHOD_SPLINE, peak_min_frac=0.05)`

Steps 4 to 7 for one set of windows: `_rate_window_signal` and `_peak_from_signal` per window, then `_arrhenius_fit`.

Name	Type	Default	Meaning
<code>windows</code>	list of (t1, t2)		Gate times, ms.
<code>temps</code>	list of float		Temperatures, °C.
<code>data</code>	dict		<code>_prepare_signal_data</code> records.
<code>gamma</code>	float		$\gamma, \text{cm}^{-2} \text{s}^{-1} \text{K}^{-2}$.
<code>t_peak_lo</code> , <code>t_peak_hi</code>	float		Peak-search range, K.
<code>peak_method</code>	str	PEAK_METHOD_SPLINE	Peak finder.
<code>peak_min_frac</code>	float	0.05	Skip a window if its in-range peak is $\leq$ this fraction of max S over all temperatures.

Returns `None` when fewer than 2 windows can be fitted; otherwise a dict:

Key	Meaning
<code>en_arr</code> , <code>Tp_arr</code> , <code>Tp_err_arr</code> , <code>Sp_arr</code>	e_n (s^{-1}), T_p (K), T_p error (K, NaN if none), peak S, for every kept window.
<code>x</code> , <code>y</code>	$1000/T_p$ and $\ln(e_n/T_p^2)$ for every kept window.
<code>fit_mask</code>	bool array, windows used in the fit.
<code>N</code> , <code>N_excluded</code>	Fitted and excluded window counts.
<code>Et</code> , <code>Et_se</code>	E_a and its error, eV.
<code>sigma</code> , <code>sigma_se</code>	σ_n and its error, cm^2 .
<code>slope</code> , <code>slope_se</code> , <code>intercept</code> , <code>intercept_se</code> , <code>covar</code>	Line parameters; <code>covar</code> is the <code>lmfit [slope, intercept]</code> covariance (or <code>None</code>).
<code>R2</code> , <code>redchi</code> , <code>weighted</code> , <code>peak_method</code>	Fit quality and settings.
<code>detail</code>	list of (t1, t2, e_n , T_p in °C, T_p error or <code>None</code> , peak S).

Side effects `None`. **Called by** `_compute_detailed_analysis`.

Example see `_compute_detailed_analysis`.

`_prepare_spectra(mw, temps, data, n_spectra)`

Name	Type	Default	Meaning
<code>mw</code>	list of (t1, t2)		Multi-windows, ms.
<code>temps</code>	list of float		°C.
<code>data</code>	dict		<code>_prepare_signal_data</code> records.
<code>n_spectra</code>	int		Number of spectra.

Returns list of (k, t1, t2, e_n, S·1e3) (Step 9). **Side effects** None. **Called by** `_compute_detailed_analysis`.

`_prepare_transient_map(temps, data, rb_ms)`

Name	Type	Default	Meaning
<code>temps</code>	list of float		°C.
<code>data</code>	dict		<code>_prepare_signal_data</code> records.
<code>rb_ms</code>	float		Reverse-bias duration, ms.

Returns `dict(t_s, Z_disp, levels)` (time in s, $(C - C_{\text{inf}})/C_{\text{inf}} \cdot 1e5$ with shape (n_t, n_T), 64 levels), or `None` if fewer than 10 time points fall in the range. Uses the first temperature's time axis for all. **Side effects** None. **Called by** `_compute_detailed_analysis`.

`_prepare_rate_window_map(temps, data, t1_min, t1_max, ratio)`

Name	Type	Default	Meaning
<code>temps</code>	list of float		°C.
<code>data</code>	dict		<code>_prepare_signal_data</code> records.
<code>t1_min, t1_max</code>	float		t1 range, ms.
<code>ratio</code>	float		t2/t1.

Returns `dict(Xi, Yi, Zi, z_lo, z_hi, x_min, x_max)`: 220 × 220 grids (Xi in eV⁻¹, Yi in K² s, Zi masked N_T/N_D), color limits, and x range; or `None` if fewer than 6 points. **Side effects** None. **Called by** `_compute_detailed_analysis`.

`_compute_detailed_analysis(data, temps, p)`

Everything **Run Analysis** computes. Runs in the worker process.

Na me	Type	Def ault	Meaning
<code>data</code>	dict		<code>_load_detailed_data</code> output.
<code>temps</code>	list of float		°C.

Na me	Type	Def ault	Meaning
p	dict		Parameter snapshot with keys <code>signalMethod</code> , <code>denoise</code> , <code>t1Min</code> , <code>t1Max</code> , <code>nWin</code> , <code>ratio</code> , <code>stdWins</code> , <code>gamma</code> , <code>nd</code> , <code>tpLo</code> , <code>tpHi</code> , <code>peakMethod</code> , <code>rbMs</code> , <code>nSpectra</code> , <code>showSpectra</code> , <code>showTmap</code> , <code>showRwm</code> (the GUI also passes <code>showStd</code> , <code>showTau</code> , used only for plotting).

Returns `dict(resMw, resStd, mw, T_K, Nt, spectra, tmap, rwm)`. `resStd` may be `None`; `spectra`, `tmap`, `rwm` are `None` when their panel is off or cannot be built.

Side effects `None`. Raises `ValueError('Multi-window: no peaks found. Check t1 range and temperature bounds.')` when the multi-window fit returns `None`.

Called by `_detailed_run_analysis` (through `_run_in_detailed_process`).

Example (verified; synthetic 0.60 eV, $\sigma = 1\text{e-}15\text{ cm}^2$ trap, 0 to 120 °C, GUI defaults):

```
import os
os.environ['LOKY_MAX_CPU_COUNT'] = '8'
import numpy as np
import matplotlib; matplotlib.use('Agg')
import detailedAnalysisTab as daT

rng = np.random.default_rng(1)
t_ms = np.arange(0, 500.0, 1.8666666666666665e-02)
data = {}
for tc in np.arange(0.0, 121.0, 5.0):
    TK = tc + 273.15
    en = 1.66e21 * 1e-15 * TK**2 * np.exp(-0.60 / (8.617333e-5 * TK))
    cap = 100.0 * (1 - 0.01 * np.exp(-en * t_ms * 1e-3)) + rng.normal(0, 1e-3, t_ms.size)
    mi = (t_ms >= 200) & (t_ms <= 450)
    data[float(tc)] = (t_ms, cap, float(np.mean(cap[mi])))
temps = sorted(data)

p = dict(signalMethod=daT.SIGNAL_METHOD_MEASURED, denoise=daT.DENOISE_NONE,
        nWin=16, t1Min=5.0, t1Max=90.0, ratio=5.0,
        stdWins=[(5, 25), (10, 50), (20, 100), (50, 250), (100, 400)],
        gamma=1.66e21, nd=3.2e14, tpLo=250.0, tpHi=400.0,
        peakMethod=daT.PEAK_METHOD_SPLINE, rbMs=500.0, nSpectra=5,
        showSpectra=True, showTmap=True, showRwm=True)
out = daT._compute_detailed_analysis(data, temps, p)
r = out['resMw']
print(daT._fmt_pm(r['Et'], r['Et_se'], '.4f'))      # 0.5968 ± 0.0032
print(daT._fmt_pm(r['sigma'], r['sigma_se'], '.3e')) # 8.904e-16 ± 1.076e-16
print(f"{out['Nt']:.3e}")                          # 6.236e+12
```

On the same data the five standard windows give $E_a = 0.5979 \pm 0.0042\text{ eV}$.

Worker process helpers

`_get_detailed_executor()`

Returns `dltsc.detailed_executor`, creating a `ProcessPoolExecutor(max_workers=1)` on first use. It is this tab's own executor, separate from the Qualitative Analysis one.

`_run_in_detailed_process(fn, *args)`

Submits `fn(*args)` to the worker and blocks for the result. Call it only from a background thread. On `BrokenProcessPool` it sets `dltsc.detailed_executor = None` and re-raises.

GUI helpers

`_set_detailed_buttons_state(state)`

Sets **Load Data** to `state`. **Run Analysis** is disabled for `'disabled'`; for any other state it is enabled only if `dltsc.detailed_data` is non-empty.

`_detailed_browse_folder()`

Browse command. Folder dialog; sets `detailed_baseVar` and logs the folder contents.

`_detailed_load_data()`

Load Data command. Returns silently if a load or run is in progress; logs `please select a valid data folder.` if the path is not a directory. Otherwise reads the Grid, RB and C_∞ variables, runs `_load_detailed_data` in the worker (from a daemon thread), and on the main thread stores `dltsc.detailed_data` / `dltsc.detailed_temps`, updates the info and status labels, and logs each load error. With zero temperatures it reports `No recognizable data found (checked ZI single-file, ZI subfolder-per-temperature, and legacy per-temperature formats).`

`_detailed_run_analysis()`

Run Analysis command. Logs `load data first.` without data; returns silently when busy. Snapshots every parameter into a dict, runs `_compute_detailed_analysis` in the worker, then on the main thread calls `_detailed_plot` and `_detailed_write_results`, stores `detailed_lastResMw`, `detailed_lastResStd`, `detailed_lastNt`, and sets the status label. Errors appear as `Error: <message>` in the status label and the log.

`_detailed_plot(out, params)`

Destroys the old canvas, builds a new `Figure` with the layout from the Figure table, draws the Arrhenius panel (fit line, 2σ band, colored points, excluded points, optional standard fit, colorbar, results box, $^{\circ}\text{C}$ top axis, legend) and the optional panels, embeds it with `FigureCanvasTkAgg` and a toolbar, and installs drag handlers on each legend and the results box. Uses `Figure()` rather than `plt.figure()` so no extra pyplot window opens. It does not call `draw()`; the resize after packing triggers the single render.

`_detailed_arrhenius_errorbars(ax, res, color)`

Draws x/y error bars ($dx = 1000 \cdot dT/T^2$, $dy = 2 \cdot dT/T$) for fitted windows that have a T_p error.

`_detailed_install_drag(canvas, fig, artist, kind)`

Connects `button_press_event`, `motion_notify_event` and `button_release_event` so a left-drag moves a legend (`kind='legend'`, updates `artist._loc`) or annotation (`kind='annot'`, updates `artist.xyann`). Redraws once before hit-testing only when `dltsc.detailed_hitTestStale` is `True`. Contains nested `_on_press`, `_on_motion`, `_on_release`. `_detailed_plot` also defines nested `_mark_stale` / `_mark_fresh` that set `detailed_hitTestStale` on resize and draw events.

`_detailed_plot_transient_map(ax, res_mw, T_K, tmap, params)`

Draws the transient map contours, colorbar and optional $\tau(T)$ overlay (Step 9). Returns the legend when the overlay was drawn, else `None`.

`_detailed_plot_rate_window_map(ax, res_mw, rwm, params)`

Draws the rate-window map with `pcolormesh` (log color scale), colorbar, Arrhenius line and 22 circles. Returns the legend.

`_detailed_style_rwm_axes(ax)`

Applies labels, title, white ticks and gray spines to the rate-window map axes.

`_detailed_open_label_editor()`

Edit Labels & Legends command. Builds the dialog described above. Nested helpers: `section(title)` (divider and heading), `axis_row(parent, label, current_text, current_size, setter_fn)` (text entry plus size spinbox), `apply_all()` (applies every edit and calls `draw_idle()`).

`_detailed_write_results(res_mw, res_std, Nt, params)`

Builds the Results text shown above and writes it into `dltsc.detailed_resultsText`. Nested `fit_block(res)` formats one fit.

`_detailed_save_figure()`

Save Figure... command. Logs a hint if there is no figure. `.svg/.eps/.pdf`: `savefig(path, bbox_inches='tight')`; other extensions: `savefig(path, dpi=180, bbox_inches='tight', facecolor=SURFACE)`. Save errors are logged.

`_detailed_export_txt()`

Export Results to TXT command. Logs a hint if Results is empty; otherwise writes it to the chosen file (UTF-8).

Nested helpers in `construct_detailedAnalysisTab`

`section(title)` draws a divider and an upper-case heading; `row(label, widget_factory, **kw)` packs a 15-character label and a widget; `spinbox(parent_, var, lo, hi, inc, fmt='%g', width=9)` builds a `ttk.Spinbox`.

Notes and limitations

- The **ZI MFIA GRID** fields `Grid offset (s)`, `Grid dt (s)` and `Chunk size` have no effect. `_load_detailed_data` accepts them as `grid_off`, `grid_dt`, `chunk_size` and never uses them; ZI grid values come from each header CSV (or `LiveDataTab` defaults).
- Legacy `.txt/.h5` files use a hard-coded sampling interval `samplingRateS=1.8666666666666666e-05` s. A run recorded at another rate gets a wrong time axis and wrong `e_n`.
- `.json` run files are not loaded (the legacy filename pattern accepts `.txt`, `.csv`, `.h5` only), and the loader returns no error message for them.

- **C₀ Estimation Window** and **RB duration (ms)** are applied at load time for C_∞. Changing them without reloading only changes the N_T reference time and the transient-map range.
- The legacy `.h5/.txt` extractor multiplies `ImpedanceIm` by 1e12 unconditionally. On the HDF5 test run this gives values near 5e6 "pF", so the stored channel is not a capacitance in F. $S = \Delta C/C_\infty$ is a ratio and is unaffected by the scale, but the pF label is not meaningful there.
- Labels say $\Delta C/C_0$, but every quantity is normalized by C_∞ (the late-time mean).
- Edge peaks no longer dominate the fit. The spline finder puts a peak that is still rising out of the search range on the last grid point in every bootstrap sample, with an "error" of about 1e-13 K; earlier versions kept such windows in the weighted fit, where they swamped the rest. On the 23 Sep run with the GUI-default windows this gave $E_t = 90.967 \pm 7.9e9$ eV; now those six edge windows are excluded and the result is 0.693 ± 0.013 eV (0.685 ± 0.015 eV with windows suited to the data). Tests: `tests/test_detailed_peaks.py`.
- N_T is $2 \cdot \max|S_{\text{ref}}| \cdot N_d$ with $S_{\text{ref}} = (C(\text{rb_ms}) - C(2 \text{ ms}))/C_\infty$, taken over all loaded temperatures, not the peak range. Quick Analysis uses the same definition (`_trap_density`). If `rb_ms` is past the end of a transient, `_cap_at` returns the last sample (Measured C) or extrapolates the spline (Smoothed C).
- The standard-window fit is always computed and written to Results. **Show standard-window comparison** only controls drawing, and the Arrhenius x range includes the standard windows even when they are hidden.
- The Results line `slope = ... K` has the wrong unit. x is $1000/T$, so the slope is in units of 1000 K (`slope = -E_a/(1000·kB)`).
- The °C top axis uses `k - 273` (not 273.15) and only has ticks for 240 to 360 K.
- The τ overlay and the rate-window map are labeled $z_{1/2} / z_{1/2}$ (the 4H-SiC Z1/2 defect) for any data.
- If `_prepare_transient_map` finds equal 1st and 99th percentiles, it falls back to levels $\pm 1e-4$ in the $\times 1e5$ display units, which is a tiny range.
- `_prepare_transient_map` builds its time axis from the first temperature and assumes all transients share it.
- In `_find_peak_parabolic` a vertex outside the 5 fitted points, or an upward-opening parabola, falls back to the grid maximum with no error; that window is then excluded from the weighted fit.
- A failed load keeps the previously loaded data, and **Run Analysis** stays enabled for that old data.
- `_detailed_export_txt` does not catch file-write errors.
- Figure edits made in **Edit Labels & Legends** are lost on the next run.
- Both analysis tabs use `KB_EV = 8.617333262145e-5` eV/K. (Before, this tab used `8.617333e-5`; the difference in E_a is below 1e-7 relative.)
- The whole `detailed_data` dict is pickled to the worker process on every Run. For long sweeps with 500 ms transients this is tens of MB per run.

runDlts_Tools

`runDlts_Tools.py` runs a DLTS temperature scan. Its class, `dltRun`, checks that both instruments are connected and all parameters are set, builds the temperature grid and the output file names, then, for each set-point, ramps the stage, waits for it to stabilize, acquires one triggered MFIA record and writes it to its own file. The same object handles Pause/Resume, Redo and Remove & Retake of selected steps, and the return to room temperature after a run.

GUI callers (all in `liveDataTab.py`, **Live Tools** tab):

- **Run DLTS** (`start_thread` → `start_dlt`): creates `dltRun()`, stores it in `dltsc.run_dltInstance`, calls `init_experiment()`, sets every `stepStatus[i] = 'pending'`, then `run_experiment()` on a background thread.
- **Pause** sets `dltsc.run_pauseRequested = True`; **Resume** calls `run_experiment()` again on the same instance.
- **Redo** calls `run_experiment(indices=selected, deleteFirst=False)`; **Remove & Retake** calls `run_experiment(indices=selected, deleteFirst=True)`.
- `_handle_run_status` calls `finish_experiment()` after a completed main sequence, then `_return_to_room_temp_async()` runs `return_to_room_temp()` on a background thread after any completed or failed run (not after a pause or an abort).
- `_refresh_step_listbox` shows `stepStatus` in the step list.
- The window close handler (`DLTSGUI_MainWindow._send_room_ramp`) sets `dltsc.run_abortRequested = True`, which this module checks.

Import

```
import runDlts_Tools as rdT
```

The module imports `dltConfig` as `dltsc` and works entirely through its globals: `dltsc.impDev`, `dltsc.tempDev`, `dltsc.z_params_vars`, `dltsc.z_params_for_push`, `dltsc.t_params_vars`, `dltsc.t_params_for_push`, `dltsc.d_params_vars`, `dltsc.run_pauseRequested`, `dltsc.run_abortRequested`, `dltsc.recast_param_type`, `dltsc.log_to_textbox`. It writes `dltsc.run_dataFolder`, `dltsc.run_dataFileNames` and `dltsc.run_outputFileType`.

On import it also calls `warnings.filterwarnings("ignore", category=FutureWarning, module="uncertainties")`.

Module constants and globals

Name	Value / type	Meaning
(none)		The module defines no constants. It has one module-level helper, <code>_get_runtime_param_value</code> (see Internal helpers).

Data folder and file naming

`init_experiment()` builds the paths from **Data Root Folder** and the current local time:

```
<Data Root Folder>\MMDYY\HHMMSS\<prefix><|T|>.<ext>
```

- **Data Root Folder** must be set and be a full path (for example `C:\Users\spencer\Desktop\DATA\DLTS`). Otherwise `init_experiment()` logs `Error: Data Root Folder is not set (or ... is not a full path)` and returns `-1` before creating anything, and the GUI reports `Error: Failed to initialize the experiment`.
- `MMDDYY`: month, day, two-digit year. `HHMMSS`: 24-hour time, both from one `datetime.now()` call. Paths are joined with `os.path.join`, for example `C:\DATA\DLTS\092826\143015\p25p0.h5`.
- `<prefix>`: `n` if `str(T)` contains `-`, otherwise `p`.
- `<|T|>`: `str(abs(T))` with `.` replaced by `p`. Grid values are floats, so `25 °C` gives `p25p0`, `27.5 °C` gives `p27p5`, `-10 °C` gives `n10p0`.
- `<ext>` from **Data File Format** (lower-cased): `json` → `.json`, `hdf5` → `.h5`, `txt` → `.txt`, anything else → `.json`.
- Run parameters go to `<dataFolder>\runParams.txt` (JSON content).

The GUI offers only `TXT` and `HDF5`. `TXT` files contain JSON (written by `writeDataJson`), not a text table.

Step status values

`stepStatus` maps a `tempGrid` index to one of:

Value	Set by	Meaning
'pending'	the caller (<code>liveDataTab.start_dlts</code> , <code>hw_run.py</code>) after <code>init_experiment()</code>	Not yet run. <code>dltsRun</code> never sets it.
'running'	<code>_run_single_step</code>	Ramping, stabilizing or acquiring.
'done'	<code>_run_single_step</code>	File written.
'failed'	<code>run_experiment</code>	An exception occurred during this step.
'paused'	<code>run_experiment</code>	The main sequence paused before this step; Resume starts here.
'aborted'	<code>_run_single_step</code>	<code>dlts.run_abortRequested</code> was set after the ramp or during acquisition; no file written.

The unused global `dlts.run_stepStatus` is not this dict; the GUI reads `dlts.run_dltsInstance.stepStatus`.

class dltsRun

Attributes

Name	Type	Default	Meaning
<code>impDevice</code>	<code>ziC.ziDevice</code> or <code>None</code>	<code>None</code>	Set from <code>dlts.impDev</code> by <code>check_device_connections()</code> when both instruments are connected.
<code>tempDevice</code>	<code>tsC.mK2000B</code> or <code>None</code>	<code>None</code>	Set from <code>dlts.tempDev</code> at the same time.
<code>impDeviceParams</code>	<code>dict</code> or <code>None</code>	<code>None</code>	MFIA parameter values found by <code>check_device_parameters('impDev')</code> .

Name	Type	Default	Meaning
tempDeviceParams	dict or None	None	Temperature parameter values.
outputParams	dict or None	None	Output parameters (Number of Points (power of 2), Number of Reps, Data File Format, Data Root Folder).
dataFolder	str or None	None	Run folder, ends with \.
runOutputFileType	str or None	None	Lower-cased Data File Format.
dataFileNames	list[str] or None	None	One path per tempGrid index.
paramsFileName	str or None	None	dataFolder + 'runParams.txt'.
currentStepIndex	int	0	Next index the main sequence runs; the resume point.
stepStatus	dict[int, str]	{}	See "Step status values".
_everPulled	bool	False	True after the first acquisition on this instance. Before every later acquisition the MFIA gets a factory reset.

init(self, fName=None)

Creates an empty run object. `fName` is accepted and ignored.

Name	Type	Default	Meaning
fName	any	None	Unused.

Returns a new `dltsRun`.

Side effects None.

Example

```
run = rdT.dltsRun()
```

check_device_connections(self)

Reports which instruments exist in `dltsConfig`. When both `dltsConfig.impDev` and `dltsConfig.tempDev` are not `None`, it copies them into `impDevice` and `tempDevice`.

No parameters.

Returns `[imp, temp]`: for each, 1 connected (not `None`), 0 `None`, -1 attribute missing from `dltsConfig` (cannot happen with the current `dltsConfig`, which defines both).

Side effects Prints one status line with `print()` (not the GUI log). Sets `impDevice/tempDevice` only when both are present. It checks for `None` only; it does not check that the connections work.

Example

```
print(run.check_device_connections()) # [1, 1]
```

check_device_parameters(self, device='impDev')

Collects the parameter values for one group and counts the missing ones.

Name	Type	Default	Meaning
device	str	'impDev'	'impDev': keys of <code>dltsc.z_params_vars</code> , values preferring <code>dltsc.z_params_for_push</code> . 'tempDev': keys of <code>dltsc.t_params_vars</code> , values preferring <code>dltsc.t_params_for_push</code> . 'output': keys and values of <code>dltsc.d_params_vars</code> .

A value is "missing" when it is `None`. An empty string (for example no Data Root Folder) counts as set here; `init_experiment()` then rejects an empty or relative Data Root Folder.

Returns the number of missing values (int), or `None` for any other `device` string.

Side effects Sets `impDeviceParams`, `tempDeviceParams` or `outputParams`. Logs `Error: Parameter <name> does not exist.` for each missing value.

Example

```
missing = run.check_device_parameters('output')
```

check_setup(self)

Runs `check_device_connections()` and the three `check_device_parameters()` calls.

No parameters.

Returns 2 when both instruments are present and no value is missing; 0 or -2 otherwise (+1/-1 for devices, +1/-1 for parameters).

Side effects Logs `All devices are connected.` Or `Error: One or more devices are not connected.`, and `All device parameters are set.` Or `Error: One or more device parameters are not set.`. Sets the `*Params` attributes and the device references.

Example

```
if run.check_setup() < 2:  
    raise RuntimeError('setup incomplete')
```

init_experiment(self)

Prepares a new run: checks the setup, builds the temperature grid, creates the data folders and fixes every file name.

No parameters.

Returns 0 on success; -1 if `check_setup()` returned less than 2, if **Data Root Folder** is empty or not an absolute path, or if the run folder cannot be created.

Side effects

- Calls `tempDevice.set_temp_grid()`, which rebuilds `tempGrid` from `tempDevice.params` (needs `tempDevice.dev` set).
- Checks **Data Root Folder** (stripped of spaces): empty or relative logs an error plus `Choose it with Browse...` on the Input Parameters tab, then Apply + Push Params. and returns -1 without creating anything.
- Creates `<root>\MMDDYY\HHMMSS\` with `os.makedirs(exist_ok=True)`; an `OSError` (no permission, missing drive) logs `Error: cannot create the run folder ...` and returns -1.
- Sets `runOutputFileType`, `dataFolder`, `dataFileNames`, `paramsFileName`.
- Publishes `dltsc.run_dataFolder`, `dltsc.run_dataFileNames` (a copy of the list) and `dltsc.run_outputFileType`; the live-data watcher polls `run_dataFileNames`.
- Logs 1. Hardware initialized., 2. Temperature grid set., 3. Output file names set.
- Does not reset `currentStepIndex`, `stepStatus` or `_everPulled`; use a new `dltscRun` for a new run (the GUI does).

Example

```
if run.init_experiment() < 0:
    raise RuntimeError('init_experiment failed')
print(run.dataFolder, run.dataFileNames)
```

run_experiment(self, indices=None, deleteFirst=False)

Runs temperature steps and returns why it stopped.

Name	Type	Default	Meaning
indices	iterable of int or None	None	None: main sequence, from <code>currentStepIndex</code> to the end of <code>tempGrid</code> . A list: only those indices, sorted ascending (fewer thermal cycles), without touching <code>currentStepIndex</code> (Redo / Remove & Retake).
deleteFirst	bool	False	True ("Remove & Retake"): delete each step's existing file before re-acquiring. False ("Redo"): overwrite in place.

Before each step it checks `dltsc.run_abortRequested` (returns 'aborted'). In the main sequence only, it then checks `dltsc.run_pauseRequested`: if set, it stores the index in `currentStepIndex`, marks it 'paused', logs `Run paused before T = ... C (step i/N)`. and returns 'paused'. Devices stay connected and the grid is untouched, so calling `run_experiment()` again resumes. After each completed main-sequence step, `currentStepIndex = i + 1`.

Returns 'completed', 'paused', 'aborted' or 'error'. An exception from any step marks that step 'failed', logs `Error during step T = ... C: <exception>` and returns 'error'.

Side effects Everything `_run_single_step` does, for each step. Blocks for the whole run; the GUI runs it on a daemon thread. It does not clear `dlts.run_pauseRequested`; the caller must set it back to `False` before resuming (the GUI does). It does not return the stage to room temperature.

Example

```
# Stage heats/cools and the sample is biased for every step.
status = run.run_experiment()                # main sequence
status = run.run_experiment(indices=[4], deleteFirst=False) # Redo step 5
status = run.run_experiment(indices=[3], deleteFirst=True)  # Remove & Retake step 4
```

finish_experiment(self)

Writes the run parameters and logs the end of the run.

No parameters.

Returns `0`.

Side effects Writes `runParams.txt` (JSON: MFIA, temperature and output parameters merged) with `impDevice.writeDataJson()`, if `impDevice` and `paramsFileName` are set. Logs `Run complete: devices remain connected for the next run until the GUI is closed.` It does not move the stage or disconnect anything.

Example

```
if run.run_experiment() == 'completed':
    run.finish_experiment()
```

return_to_room_temp(self)

Ramps the stage to `tempDevice.Troom` at `tempDevice.roomRamp` and waits (`go_to_room_temp(Tr, ramp)`, 0.5 °C tolerance, deadline = expected ramp time + 900 s).

No parameters.

Returns `0` reached, `1` gave up waiting (stage still ramping or holding at room temperature), `-1` not connected, no `tempDevice`, or an exception.

Side effects One ramp command, then a `read_temp()` every 5 s. Blocks for minutes; call it from a background thread in a GUI. Logs `Returning stage to room temperature: ...` and the outcome.

Example

```
# The stage heats or cools to Room Temperature (C).
status = run.return_to_room_temp()
```

Internal helpers

- `_get_runtime_param_value(param_vars, param_name, fallback_bucket=None)` (module level): returns `fallback_bucket[param_name]` if present; else `param_vars[param_name].get()`, or the raw entry if it has no `.get()`; else `None` if the name is missing.
- `dltsRun._read_stage_temp(self)`: `tempDevice.read_temp()`, or `None` on any exception. Used only for the HDF5 `stage_temperature_C` attribute.
- `dltsRun._current_run_params(self)`: `{**impDeviceParams, **tempDeviceParams, **outputParams}` when all three are set, otherwise `{}`. Written to `runParams.txt` and to each HDF5 file's `run_params` attribute.
- `dltsRun._run_single_step(self, i, deleteFirst=False)`: acquires and writes step `i`. In order: if `deleteFirst`, deletes `dataFileNames[i]` (logs a warning if that fails); marks 'running'; `tempDevice.go_to_temp(tempGrid[i], tRamp, tStableDelay)`; returns 'aborted' if `dlts.run_abortRequested`; sleeps 1 s; calls `impDevice.device.factory_reset()` if `_everPulled`; `impDevice.reload_params()`; reads `numPoints = 2 ** recast_param_type('output', 'Number of Points (power of 2)')` and `numReps = recast_param_type('output', 'Number of Reps')`; for `txt/json/hdf5` reads the stage temperature, runs `pull_data(plot=False, trigger=True, numPoints, numReps)`, reads the stage temperature again and averages the readings that succeeded; returns 'aborted' (and discards the data) if `dlts.run_abortRequested` was set during acquisition; writes with `writeDataH5(data, fName, setpoint_C=tempGrid[i], stage_temperature_C=avg, runParams=_current_run_params())` for `hdf5` or `writeDataJson(data, fName)` otherwise; sets `_everPulled = True`, marks 'done', returns `0`.

Standalone use

Safety: this script heats and cools the stage and biases the mounted sample for every step. Only run it when the sample and temperature range are safe.

The working pattern is `benchmarks/hardware/hw_common.py` and `hw_run.py`. `dltsRun` reads its inputs from `dltsConfig`, so a script must fill the same globals the Input Parameters tab fills:

- `dlts.z_params_vars`, `dlts.t_params_vars`, `dlts.d_params_vars`: dicts of objects with `.get()` holding the GUI's **string labels** ('1 - On', '36 - Threshold 1', ...). `recast_param_type()` calls `.get()` on them, so plain values do not work for `d_params_vars`.
- `dlts.z_params_for_push`, `dlts.t_params_for_push`: the numeric values, built by `runParamsTab._sync_param_values_to_device()` exactly as the **Connect + Get Params / Apply + Push Params** buttons do.

```

import sys, os
REPO = r'C:\Users\spencer\Documents\GitHub\DLtsOnHallProbeSetup'
sys.path.insert(0, REPO)
import matplotlib; matplotlib.use('Agg')
import dltsConfig as dltsc
import runParamsTab as rpT
import zurichInstruments_Control as ziC
import instecTempStage_Control as tsC
import runDLts_Tools as rdT

class Var:
    """tk.StringVar stand-in."""
    def __init__(self, value): self.value = value
    def get(self): return self.value
    def set(self, value): self.value = value

# Input Parameters tab defaults, as the tab stores them (runParamsTab.construct_runParamsTab).
Z = [('Oscillation Amplitude', '0.300'), ('Oscillation Frequency', '501000'),
      ('Oscillation ON/OFF', '1 - On'), ('Max bandwidth', '10000'), ('Input Control', '0 - Manual'),
      ('Current Range', '0.010'), ('Voltage Range', '3'), ('Omega Suppression', '80'),
      ('Filter Harmonic', '1'), ('Filter Bandwidth', '2'), ('Data Transfer Rate', '60000'),
      ('Equivalent Circuit Mode', '0 - 4-Terminal'), ('Threshold Input Signal', '59 - TU Output Value'),
      ('State Enable Time', '0.006'), ('State Disable Time', '0.003'), ('Logic Unit Not', '1 - On'),
      ('Aux Output Signal', '13 - TU Output Value'), ('Aux Output Scale', '-1'),
      ('Aux Output Offset', '-0.5'), ('Aux Output Lower Limit', '-10'), ('Aux Output Upper Limit', '0'),
      ('Signal Output Add', '1 - True'), ('Trigger Source Signal', '36 - Threshold 1')]
T = [('Initial Temperature (C)', '25'), ('Final Temperature (C)', '40'),
      ('Temperature Step (C)', '5'), ('Temperature Ramp (C/min)', '5'),
      ('Stability Delay (s)', '0'), ('Room Temperature (C)', '25'), ('Room Ramp (C/min)', '10')]
D = [('Number of Points (power of 2)', 16), ('Number of Reps', 100),
      ('Data File Format', 'HDF5'), ('Data Root Folder', r'C:\DATA\DLTS')]

dltsc.init()                # run-control flags False, empty dicts; call it before filling the dicts
                           # (no GUI textbox, so log_to_textbox() prints to the console)
dltsc.z_params_vars = {k: Var(v) for k, v in Z}
dltsc.t_params_vars = {k: Var(v) for k, v in T}
dltsc.d_params_vars = {k: Var(v) for k, v in D}

# Connect + push, as the Input Parameters buttons do.
dltsc.tempDev = tsC.mK2000B()
dltsc.tempDev.connect_temp_controller()
if not dltsc.tempDev.state:
    raise SystemExit('temperature controller connection failed')
dltsc.t_params_for_push = rpT._sync_param_values_to_device(dltsc.tempDev, dltsc.t_params_vars, 'tempDev')
dltsc.tempDev.load_params(dltsc.t_params_for_push)

dltsc.impDev = ziC.ziDevice()
dltsc.impDev.connect_device()
if dltsc.impDev.device is None:
    dltsc.tempDev.disconnect_temp_controller(stopControl=False)
    raise SystemExit('MFIA connection failed')
dltsc.z_params_for_push = rpT._sync_param_values_to_device(dltsc.impDev, dltsc.z_params_vars, 'impDev')
for pName in dltsc.impDev.params:
    dltsc.impDev.push_param_to_device(pName)

roomStatus = None
try:

```

```

run = rdT.dltsRun()
dltsc.run_dltsInstance = run
if run.init_experiment() < 0:
    raise RuntimeError('init_experiment failed')
for i in range(len(run.tempDevice.tempGrid)):
    run.stepStatus[i] = 'pending'

status = run.run_experiment()          # 'completed' / 'paused' / 'error' / 'aborted'
print('main sequence:', status, run.stepStatus)
if status == 'completed':
    run.finish_experiment()             # writes runParams.txt
    # Optional: redo the second step in place.
    # run.run_experiment(indices=[1], deleteFirst=False)
    roomStatus = run.return_to_room_temp() # 0 arrived, 1 gave up, -1 failed
finally:
    # Send TEMPerature:STOP only if the stage is at room temperature;
    # otherwise the controller keeps ramping there on its own.
    dltsc.tempDev.disconnect_temp_controller(stopControl=(roomStatus == 0))
    dltsc.impDev.disconnect_device()

```

Pause, resume and abort from another thread:

```

dltsc.run_pauseRequested = True        # run_experiment() returns 'paused' before the next step
# ... later, in the run thread:
dltsc.run_pauseRequested = False
status = run.run_experiment()          # continues from run.currentStepIndex

# Abort (what the GUI close handler does): set both flags.
dltsc.run_abortRequested = True
dltsc.tempDev.abort()                 # ends a go_to_temp() wait
# Before any further motion with the same objects:
dltsc.tempDev._aborted = False
dltsc.run_abortRequested = False

```

`hw_run.py` adds a watchdog thread that sets both abort flags when one step exceeds a time limit, because `go_to_temp()` has no overall time limit on stabilizing. (A silent controller is a separate case: `read_temp()` raises `TimeoutError` after about 4 s, and the step fails.)

Notes and limitations

- **Data File Format** `TXT` writes JSON into `.txt` files. A format string other than `txt`, `json` or `hdf5` (not reachable from the GUI) gets a `.json` extension in `init_experiment()`, but `_run_single_step` then acquires nothing, writes nothing and still marks the step 'done'.
- An empty or relative **Data Root Folder** stops the run in `init_experiment()` (earlier versions wrote the run to `\MMDDYY\HHMMSS\` at the root of the current drive, with a doubled backslash in the path). `check_setup()` itself still accepts an empty value. Tested in `tests/test_run_folder_and_serial.py`.
- `check_device_connections()` prints with `print()`, not `log_to_textbox()`, so its messages do not reach the GUI log.
- `check_device_parameters()` takes the MFIA and temperature keys from `*_params_vars`, not from `*_params_for_push`. With empty `*_params_vars` the check passes and the MFIA/temperature parameters are missing from `runParams.txt` and the HDF5 `run_params` attribute.

- `_run_single_step` ignores `go_to_temp()`'s return value and checks only `dltsc.run_abortRequested`. `tempDev.abort()` alone does not stop a run; the step acquires at the current stage temperature.
- `_run_single_step` factory-resets the MFIA before every acquisition except the first on the instance, then `reload_params()` re-sends parameters from `dltsc.z_params_for_push`. Parameters not in that dict (for example `imps/0/model`) return to factory values.
- `run_experiment()` catches `Exception` only. `KeyboardInterrupt` leaves the step marked `'running'`.
- An out-of-range index in `indices` raises `IndexError` inside the `except` handler itself (the log line indexes `tempGrid[i]`), so `run_experiment()` raises instead of returning `'error'`.
- `run_experiment(indices=[])` returns `'completed'` without doing anything.
- `init_experiment()` does not reset `currentStepIndex`, `stepStatus` or `_everPulled`. `run_experiment()` on a completed instance runs nothing and returns `'completed'`, even after a new `init_experiment()`.
- `finish_experiment()` writes `runParams.txt` with JSON content; Temperature Grid (C) is not included because it is not a `t_params_vars` key.
- `__init__`'s `fName` argument is unused.

zurichInstruments_Control

`zurichInstruments_Control.py` drives the Zurich Instruments MFIA impedance analyzer (serial `dev32271`) through the `zhinst.core` and `zhinst.toolkit` packages. Its single class, `ziDevice`, connects to the LabOne data server, pushes the 23 MFIA settings shown on the **Input Parameters** tab, acquires one capacitance transient record per temperature step, and writes that record to disk as HDF5 or JSON. It also works without the GUI: `DEFAULT_PARAMS` and `ziDevice.configure()` set and push every setting from a script.

GUI callers:

- **Input Parameters** tab (`runParamsTab.connect_and_get_params`, `runParamsTab.apply_and_push_params`): creates `dltsc.impDev = ziDevice()`, calls `connect_device()`, `set_param_value()` and `push_param_to_device()`.
- **Live Tools** tab, Run DLTS (`runDlts_Tools.dltsRun._run_single_step`): calls `reload_params()`, `pull_data(plot=False, trigger=True, ...)`, `writeDataH5()` or `writeDataJson()` for each temperature step, and `device.factory_reset()` before every step except the first.
- `runDlts_Tools.dltsRun.finish_experiment` calls `writeDataJson()` to write `runParams.txt`.
- Window close (`DLTSGUI_MainWindow._finish_close`): calls `disconnect_device()`.
- `convert_json_to_h5.py` calls `writeDataH5()` with `acquired_at` and `extraAttrs` to convert older files.

Import

```
import zurichInstruments_Control as ziC
```

The module imports `dltscConfig` as `dltsc` itself. It reads `dltsc.z_params_for_push`, `dltsc.z_params_vars` and `dltsc.log_to_textbox`.

Module constants and globals

Name	Value / type	Meaning
<code>DEFAULT_PARAMS</code>	<code>dict</code> , 23 entries (table below)	The Input Parameters tab's MFIA defaults, as the numeric values pushed to the device. <code>ziDevice.configure()</code> starts from these.
<code>ziDevice.H5_FOR_MAT_VERSION</code>	<code>int</code> , 1 (class attribute of <code>ziDevice</code> , not a module global)	Written to the <code>format_version</code> root attribute of every HDF5 file. Bumped when the on-disk layout of <code>writeDataH5()</code> changes.

MFIA parameters

Every key of `ziDevice.params` and `DEFAULT_PARAMS`, in push order. All node paths are hard-coded with the serial `dev32271`. "Allowed values" come from the terminal prompts in `set_param_value()` and the dropdown lists in `runParamsTab.construct_runParamsTab()`. The GUI label is the string the Input Parameters tab stores; `dltscConfig.recast_param_type()` turns it into the number in the "Default" column.

Parameter	Device node it is pushed to	Default (DEFAULT_PARAMS)	Allowed values / meaning
Oscillation Amplitude	/dev32271/imps/0/output/amplitude	0.3	Test-signal amplitude in V (free entry).
Oscillation Frequency	/dev32271/imps/0/freq	501000.0	Test-signal frequency in Hz (free entry).
Oscillation ON/OFF	/dev32271/imps/0/auto/output	1	0 Off, 1 On (GUI: 0 - Off, 1 - On). The node is the impedance module's auto/output switch, not imps/0/output/on; the code never writes output/on. Check the LabOne node documentation for what auto/output controls on your firmware.
Max bandwidth	/dev32271/imps/0/maxbandwidth	10000.0	Maximum demodulator bandwidth in Hz (free entry).
Input Control	/dev32271/imps/0/auto/inputrange	0	0 Manual, 1 Auto, 2 Current Zone (GUI: 0 - Manual, 1 - Auto, 2 - Current Zone).
Current Range	/dev32271/imps/0/current/range	0.01	Current input range in A (free entry). Used when Input Control is Manual.
Voltage Range	/dev32271/imps/0/voltage/range	3.0	Voltage input range in V (free entry).
Omega Suppression	/dev32271/imps/0/omegasuppression	80.0	Omega suppression in dB (free entry).
Filter Harmonic	/dev32271/imps/0/demod/harmonic	1	Demodulator harmonic. GUI dropdown 1 to 16.
Filter Bandwidth	/dev32271/imps/0/demod/order	2	Demodulator filter order , not a bandwidth (the prompt says "Filter Order Bandwidth"). GUI dropdown 1 to 8.
Data Transfer Rate	/dev32271/imps/0/demod/rate	60000	Demodulator sample rate in Sa/s (free entry).
Equivalent Circuit Mode	/dev32271/imps/0/mode	0	0 4-Terminal, 1 2-Terminal (GUI: 0 - 4-Terminal, 1 - 2-Terminal).
Threshold Input Signal	/dev32271/tu/thresholds/0/input	59	Input of threshold unit 0: 59 TU Output Value, 58 Aux Output Overload, 56 Aux Input Overload, 55 Output Overload, 54 Input(I) Overload, 53 Input(V) Overload, 52 Trigger Out, 51 Trigger In, 50 DIO, 3 Demod Theta, 2 Demod R, 1 Demod Y, 0 Demod X.

Parameter	Device node it is pushed to	Default (DEFAULT_PARAMS)	Allowed values / meaning
State Enable Time	/dev32271/tu/thresholds/0/activationtime	0.006	Threshold activation time in s (free entry).
State Disable Time	/dev32271/tu/thresholds/0/deactivationtime	0.003	Threshold deactivation time in s (free entry).
Logic Unit Not	/dev32271/tu/logicunits/0/inputs/0/not	1	Invert input 0 of logic unit 0: 0 Off, 1 On (GUI: 0 - Off, 1 - On).
Aux Output Signal	/dev32271/auxo uts/0/outputselect	13	Signal on Aux Output 1: 0 Demod X, 1 Demod Y, 2 Demod R, 3 Demod Theta, 11 TU Filtered Value, 12 Manual, 13 TU Output Value.
Aux Output Scale	/dev32271/auxo uts/0/scale	-1.0	Aux output scale (prompt unit: V).
Aux Output Offset	/dev32271/auxo uts/0/offset	-0.5	Aux output offset in V.
Aux Output Lower Limit	/dev32271/auxo uts/0/limitlower	-10.0	Aux output lower limit in V.
Aux Output Upper Limit	/dev32271/auxo uts/0/limitupper	0.0	Aux output upper limit in V.
Signal Output Add	/dev32271/sig o uts/0/add	1	Add the Aux output to Signal Output 1: 0 False, 1 True (GUI: 0 - False, 1 - True).
Trigger Source Signal	/dev32271/trig gers/out/0/source	36	Source of Trigger Out 1: 0 Off, 1 Osc Phi Demod 2, 36 Threshold 1, 37 Threshold 2, 38 Threshold 3, 39 Threshold 4, 52 MDS Sync Out.

With the defaults, threshold unit 0 watches its own output (59), the logic unit inverts it, and the TU output (13) drives Aux Output 1 with scale -1 and offset -0.5 V, limited to -10 V ... 0 V. `Signal Output Add = 1` adds that aux voltage to the signal output, so the sample sees a pulsed DC bias. Trigger Out 1 follows Threshold 1, and `pull_data(trigger=True)` triggers on it. The enable/disable times set the two pulse durations. This reading of the defaults follows from the node names; the code has no comment that states it.

class ziDevice

One MFIA connection plus the parameter values last set for it.

Attributes

Name	Type	Default	Meaning
devSerial	str	'dev32271'	Serial used by <code>connect_device()</code> and <code>disconnect_device()</code> . Every other method uses the hard-coded <code>/dev32271/</code> path instead.
session	zhinst.toolkit.Session or None	None	Connection to the LabOne data server.
device	toolkit device object or None	None	The connected MFIA. None means not connected.
rm	None	None	Unused.
params	dict	the 23 names above, all 0	Values the next <code>push_param_to_device()</code> will write.
configured Params	dict or None	None	The full parameter set of the last <code>configure()</code> call; <code>set_param_value()</code> falls back to it when there is no GUI.
H5_FORMAT_VERSION	int (class attribute)	1	See above.

init(self, devSerial = None)

Creates an unconnected device object. No hardware is contacted.

Name	Type	Default	Meaning
devSerial	str or None	None	Device serial; None means 'dev32271'.

Returns a new `ziDevice`.

Side effects None.

Example

```
mfia = ziC.ziDevice()          # dev32271
```

connect_device(self)

Finds the device with `zhinst.core.ziDiscovery().find(devSerial)`, reads its data-server address and port, opens a `zhinst.toolkit.Session(..., allow_version_mismatch=True)` and calls `session.connect_device(devSerial)`.

No parameters.

Returns 0 in every case. Check `self.device` is not None to know whether it worked.

Side effects Opens a network session to the LabOne data server. If the session or device connection fails, it prints `Error connecting to device: ...` and leaves `session` and `device` at None. The discovery calls are outside the `try` block, so a device that discovery cannot find raises an exception from `zhinst.core` instead of printing. No settings are changed on the instrument.

Example

```
mfia = ziC.ziDevice()
mfia.connect_device()
if mfia.device is None:
    raise RuntimeError('MFIA connection failed')
```

disconnect_device(self)

Drops the device object, calls `session.disconnect_device(devSerial)`, and calls `session.close()` if the session has one (toolkit 1.3.0 sessions do not). Every step is wrapped in `try/except: pass`.

No parameters.

Returns `0`.

Side effects Sets `session` and `device` to `None`. It does not change any instrument setting: the test signal, the aux output and the added bias stay as last configured.

Example

```
mfia.disconnect_device()
```

set_param_value(self, pName='Oscillation Frequency', valueDict=None)

Stores one parameter value in `self.params`. It does not talk to the device.

The value source, in order:

1. `valueDict`, if given: `self.params[pName] = valueDict.get(pName)` (a missing key stores `None`).
2. If `valueDict` is `None` and `pName` is in `dlts.z_params_for_push`: that dict.
3. Otherwise, if `dlts.z_params_vars` is non-empty: a dict of `var.get()` (or the raw value) for each entry. These are the GUI's label strings such as '1 - On', not numbers.
4. Otherwise, if `configure()` has been called: `self.configuredParams` (the values it set).
5. Otherwise: an interactive `input()` prompt on the console for `pName`. An empty answer stores the prompt's default (the same values as `DEFAULT_PARAMS`). The typed text is converted with `float()` or `int()`.

Name	Type	Default	Meaning
<code>pName</code>	<code>str</code>	'Oscillation Frequency'	Parameter name from the table above.
<code>valueDict</code>	<code>dict</code> or <code>None</code>	<code>None</code>	<code>{name: value}</code> source.

Returns `0`.

Side effects Writes `self.params[pName]`. Unknown names print `Unknown Parameter <name>!!!` (prompt path) or `Parameter <name> not found!` (dict path). Path 4 blocks on console input.

Example

```
mfia.set_param_value('State Enable Time', {'State Enable Time': 0.5})
```

push_param_to_device(self, pName='Oscillation Frequency')

Writes `self.params[pName]` to its node (table above) with `session.daq_server.set(path, value)`.

Name	Type	Default	Meaning
pName	str	'Oscillation Frequency'	Parameter to push.

Returns `0`.

Side effects One `set` on the instrument. Needs a connected `session`; with `session = None` it raises `AttributeError`. Unknown names print `Unknown Parameter!!!`. A value of `None` is passed to `set` unchanged.

Example

```
mfia.params['Oscillation Frequency'] = 1.0e6
mfia.push_param_to_device('Oscillation Frequency')
```

check_param(self, pName='Oscillation Frequency')

Reads the whole node tree with `session.daq_server.get('*')` and compares the parameter's node value with `self.params[pName]` using `np.isclose(..., rtol=1e-03)`.

Name	Type	Default	Meaning
pName	str	'Oscillation Frequency'	Parameter to check.

Returns `numpy.bool_` (`True` when the device value is within 0.1 % of the stored value), or `False` for an unknown name.

Side effects One full-tree `get('*')` per call. Prints `Unknown Parameter!!!` for an unknown name.

Example

```
ok = mfia.check_param('Data Transfer Rate')
```

configure(self, values=None, push=True)

Sets every parameter without the GUI. It starts from `DEFAULT_PARAMS`, overrides it with `values`, stores all 23 values with `set_param_value(pName, merged)`, and, when `push` is true, pushes all 23 to the device in the order of the table.

Name	Type	Default	Meaning
values	dict or None	None	{parameter name: number} overrides. Use numbers, not GUI label strings.
push	bool	True	Push to the device after setting. <code>False</code> only fills <code>self.params</code> .

Returns a copy of `self.params` (dict), the values now in effect.

Side effects

- Raises `KeyError('unknown MFIA parameter(s): [...]')` before changing anything if `values` holds a name not in `self.params`.

- With `push=True` and no session, raises `RuntimeError("MFIA not connected: call connect_device() first")`. `self.params` is already updated at that point.
- With `push=True`: 23 `daq_server.set` calls.
- Stores the merged values in `self.configuredParams`, so a later `reload_params()` (for example after `device.factory_reset()`) re-sends these values instead of prompting on the console when there is no GUI. It does not write `dlts.z_params_for_push`; GUI values, when present, still take priority.

Example

```
mfia.connect_device()
params = mfia.configure({'State Enable Time': 0.5, 'State Disable Time': 0.001})
offline = ziC.ziDevice().configure(push=False) # no hardware needed
```

load_params(self)

Calls `set_param_value(pName)` with no dict, then `push_param_to_device(pName)`, for every parameter.

No parameters.

Returns `0`.

Side effects 23 `set` calls. Values come from `dlts.z_params_for_push`, else `dlts.z_params_vars`, else console prompts (see `set_param_value`). Nothing in the codebase calls it.

Example

```
import dltsConfig as dltsC
dltsC.z_params_for_push = dict(ziC.DEFAULT_PARAMS)
mfia.load_params()
```

reload_params(self)

For every parameter, calls `check_param(pName)`. When the device value differs, it calls `set_param_value(pName)` with no dict and pushes the result.

No parameters.

Returns `0`.

Side effects 23 full-tree reads plus one `set` per mismatch. A mismatched value is re-read from `dltsC.z_params_for_push` (or `z_params_vars`, or a console prompt), not from `self.params`. `dltsRun._run_single_step` calls it before every acquisition, right after `device.factory_reset()`, so after a factory reset every parameter that differs from the factory value is re-sent.

Example

```
mfia.device.factory_reset()
mfia.reload_params()
```

pull_data(self, plot=True, trigger=False, numPoints=1024, numReps=1)

Acquires one record from the MFIA and returns it as a dict of NumPy arrays.

Name	Type	Default	Meaning
plot	bool	True	Show a 2×2 matplotlib figure of the channels (<code>plt.show()</code>).
trigger	bool	False	True: triggered, averaged acquisition with the DAQ module (the path used by DLTS runs). False: short untriggered poll.
numPoints	int	1024	Triggered path only: grid columns (samples per record). The run uses $2 \times N$ from the Input Parameters tab.
numReps	int	1	Triggered path only: grid repetitions averaged into the record.

Triggered path (trigger=True)

The managed DAQ module (`session.modules.daq`, one instance reused for the whole session) is configured as:

Setting	Value	Meaning
type	6	Hardware trigger.
triggernode	/dev32271/demods/0/sample.TrigOut1	Trigger on Trigger Out 1 (set by Trigger Source Signal).
clearhistory	1	Clear previous records.
bandwidth	0	DAQ module low-pass bandwidth; the code gives no comment on this value.
grid.mode	4	Exact (on-grid) sampling.
grid.cols	numPoints	Samples per record.
grid.repetitions	numReps	Records averaged.
endless	0	Single acquisition.

It enables `imps[0]`, subscribes to `demods/0/sample.AuxIn0.avg`, `demods/0/sample.R.avg`, `imps/0/sample.Param0.avg` and `imps/0/sample.Param1.avg`, calls `daq_module.forcetrigger()`, waits 1 s, calls `execute()`, waits 0.5 s, then polls `progress()` every 0.05 s until it reaches 1.0. After 60 s it logs `Warning: DAQ acquisition did not complete within 60s; proceeding with whatever data is available.` through `dltscl.log_to_textbox` and continues. It then waits 1 s, calls `read()`, waits 5 s and unsubscribes everything.

Fixed sleeps: $1 + 0.5 + 1 + 5 = 7.5$ s per call, on top of the acquisition itself (about `numReps` trigger periods). The `sample.R.avg` subscription is read back but not returned.

Channels returned (each a 1-D array of `numPoints` values, the averaged grid row `value[0]` of the first burst):

Key	Content	Units
tickStampImps	Time axis of the <code>param1</code> record (<code>DAQResult.time</code> : grid timestamps minus the first, divided by 60 MHz). Floats, despite the name.	s
tickStampDemods	Same array as <code>tickStampImps</code> (also taken from <code>param1</code> , not from the demodulator node).	s
timeStampImps	Copy of <code>tickStampImps</code> .	s
timeStampDemods	Copy of <code>tickStampDemods</code> .	s
ImpedanceRe	<code>imps/0/sample.param0</code> , averaged.	Depends on the impedance module's representation (<code>imps/0/model</code> , not set by this code). The analysis code treats it as a resistance in Ω .
ImpedanceIm	<code>imps/0/sample.param1</code> , averaged.	Same dependency. The analysis code (<code>impedanceAnalysis_Tools</code> , <code>liveDataTab</code>) treats it as capacitance in F and multiplies by $1e12$ for pF.
AuxInput1	<code>demods/0/sample.auxin0</code> , averaged. The fill-pulse (excitation) waveform used to locate transients.	V
AbsZ	<code>sqrt(ImpedanceRe**2 + ImpedanceIm**2)</code> computed in Python. Only a true $ Z $ when <code>param0/param1</code> are the real and imaginary parts.	as the params

Untriggered path (`trigger=False`)

Enables `demods[0]`, waits 2 s, enables `imps[0]`, waits 2 s, subscribes `demods[0].sample`, calls `session.poll()` (toolkit default: 0.1 s recording time, 0.5 s timeout), unsubscribes, waits 2 s, does the same for `imps[0].sample`, waits 2 s. Fixed sleeps: 8 s. `numPoints` and `numReps` are ignored; the record length is whatever arrived during the poll.

Key	Content	Units
tickStampImps	Raw <code>imps</code> sample timestamps.	60 MHz clock ticks (integers)
tickStampDemods	Raw <code>demods</code> sample timestamps.	clock ticks
timeStampImps	<code>tickStampImps / 60e6</code> , minus its first value.	s
timeStampDemods	<code>tickStampDemods / 60e6</code> , minus its first value.	s
ImpedanceRe	<code>imps</code> sample <code>param0</code> .	per <code>imps/0/model</code>
ImpedanceIm	<code>imps</code> sample <code>param1</code> .	per <code>imps/0/model</code>
AbsZ	<code>abs(z)</code> of the complex impedance sample.	Ω
AuxInput1	<code>demods</code> sample <code>auxin0</code> .	V

Returns dict of the 8 keys above.

Side effects Enables the impedance module (and, untriggered, demodulator 0). Blocks for at least 7.5 s (triggered) or 8 s (untriggered). With `plot=True`, opens a matplotlib window; `plt.show()` blocks in a non-interactive backend.

Example

```
# The stage and bias must be safe: the sample is pulsed during acquisition.
data = mfia.pull_data(plot=False, trigger=True, numPoints=2**16, numReps=100)
print(data['ImpedanceIm'].size, data['timeStampImps'][-1])
```

defaultJsonConverter(self, obj)

default= hook for `json.dump`: converts a NumPy array to a list and raises `TypeError` for anything else.

Name	Type	Default	Meaning
obj	any	required	Object <code>json</code> cannot serialize.

Returns `obj.tolist()` for an `np.ndarray`. The trailing `return 0` is unreachable.

Side effects None.

Example

```
import json, numpy as np
json.dumps({'a': np.arange(3)}, default=mfia.defaultJsonConverter)
```

writeDataJson(self, data, fName)

Writes `data` as indented JSON (`indent=4`) to `fName`, creating missing parent folders.

Name	Type	Default	Meaning
data	dict	required	Usually a <code>pull_data()</code> result or the run parameters.
fName	str or path	required	Output file. The extension is not checked.

Returns `0`.

Side effects Creates folders and overwrites `fName` directly (no temporary file, so a crash can leave a truncated file). NumPy arrays become lists; NumPy integer scalars raise `TypeError`. DLTS runs with Data File Format **TEXT** use this method, so their `.txt` step files contain JSON. `finish_experiment()` also writes `runParams.txt` with it.

Example

```
mfia.writeDataJson(data, r'C:\DATA\test\p25p0.txt')
```

writeDataH5(self, data, fName, setpoint_C=None, stage_temperature_C=None, runParams=None, acquired_at=None, extraAttrs=None)

Writes one temperature step's `pull_data()` output to its own HDF5 file. One file per step means Redo / Remove & Retake overwrites only that step, and a crash can only cost the step being written.

Name	Type	Default	Meaning
<code>data</code>	<code>dict</code>	required	<code>{channel name: array-like}</code> .
<code>fName</code>	<code>str</code> or path	required	Final file path, usually <code>... \p25p0.h5</code> .
<code>setpoint_C</code>	number or <code>None</code>	<code>None</code>	Temperature set-point in °C.
<code>stage_temperature_C</code>	number or <code>None</code>	<code>None</code>	Measured stage temperature in °C (the run averages the readings before and after acquisition).
<code>runParams</code>	<code>dict</code> or <code>None</code>	<code>None</code>	Run parameters, stored as a JSON string.
<code>acquired_at</code>	<code>datetime</code> or <code>None</code>	<code>None</code>	Acquisition time; <code>None</code> means now.
<code>extraAttrs</code>	<code>dict</code> or <code>None</code>	<code>None</code>	More root attributes (used by <code>convert_json_to_h5.py</code>).

File layout:

Item	Kind	Content
<code>format_version</code>	root attribute	<code>H5_FORMAT_VERSION</code> (1).
<code>acquired_at</code>	root attribute	ISO 8601 string, seconds precision.
<code>setpoint_C</code>	root attribute	<code>float</code> , <code>NaN</code> if not given.
<code>stage_temperature_C</code>	root attribute	<code>float</code> , <code>NaN</code> if not given.
<code>run_params</code>	root attribute	<code>json.dumps(runParams, default=str)</code> ; absent if <code>runParams</code> is <code>None</code> .
any <code>extraAttrs</code> key	root attribute	as given.
one dataset per <code>data</code> key	1-D dataset	Keys starting with <code>tickStamp</code> whose array has an integer dtype are stored as <code>uint64</code> ; everything else is cast to <code>float64</code> . Non-empty datasets use gzip level 4 with shuffle. Empty arrays are stored uncompressed.

Triggered `pull_data()` output has `float tickStamp*` arrays, so those are stored as `float64` seconds.

Returns `0`.

Side effects Creates parent folders. Writes to `<fName>.tmp`, then renames it onto `fName` with `os.replace`, so the live-data watcher never sees a half-written file. On `PermissionError` (another process has `fName` open) it retries up to 10 times, 0.5 s apart, then re-raises. If writing fails (for example a non-numeric channel), the exception propagates, `fName` is untouched, and the `.tmp` file is left on disk.

Example

```
mfia.writeDataH5(data, r'C:\DATA\test\p25p0.h5', setpoint_C=25.0,  
stage_temperature_C=25.02, runParams=params)
```

runSweep(self, sweepType='freq')

Runs a LabOne sweeper measurement. Not used by the GUI.

Name	Type	Default	Meaning
sweepType	str	'freq'	'freq': sweep /dev32271/oscs/0/freq from 10 Hz to 510 kHz. 'cv': sweep /dev32271/auxouts/0/offset from 0 to 1 V. Any other value leaves the grid node unset.

Fixed sweeper settings: 200 points, linear x mapping (xmapping 0), filtermode 0, endless 0, settling inaccuracy 0.01, averaging 20 samples / 15 TC / 0.1 s, bandwidth 10, maxbandwidth 100, bandwidthoverlap 1, omegasuppression 80, order 8.

Returns a tuple (allData, d): the full sweep_module.read() result and allData['/dev32271/imps/0/sample'][0][0].

Side effects Enables imps[0], subscribes three nodes, runs the sweep, prints progress() once per second until done (no timeout), unsubscribes. The 'cv' sweep changes the DC offset on the aux output, which biases the sample, and leaves it at the last swept value.

Example

```
# Biases the sample (cv) or drives it across 10 Hz-510 kHz (freq).  
allData, d = mfia.runSweep('freq')
```

Internal helpers

The module has no _name functions. set_param_value's interactive prompt path and defaultJsonConverter act as helpers but are public.

Standalone use

Safety: these scripts bias the mounted sample (aux output added to the signal output) and, in the second script, heat or cool the stage.

MFIA only

```
import sys
sys.path.insert(0, r'C:\Users\spencer\Documents\GitHub\DltsOnHallProbeSetup')
import matplotlib; matplotlib.use('Agg')
import dltsConfig as dltsC
import zurichInstruments_Control as ziC

mfia = ziC.ziDevice()
mfia.connect_device()
if mfia.device is None:
    raise SystemExit('MFIA connection failed')
try:
    params = mfia.configure({'State Enable Time': 0.5, 'State Disable Time': 0.001})

    data = mfia.pull_data(plot=False, trigger=True, numPoints=2**16, numReps=100)
    mfia.writeDataH5(data, r'C:\DATA\standalone\p25p0.h5', setpoint_C=None, runParams=params)
finally:
    mfia.disconnect_device()    # instrument settings stay as configured
```

MFIA plus temperature stage, one step

```
import sys
sys.path.insert(0, r'C:\Users\spencer\Documents\GitHub\DltsOnHallProbeSetup')
import matplotlib; matplotlib.use('Agg')
import dltsConfig as dltsC
import zurichInstruments_Control as ziC
import instecTempStage_Control as tsC

stage = tsC.mK2000B()          # COM7
stage.connect_temp_controller()
if not stage.state:
    raise SystemExit('temperature controller connection failed')
mfia = ziC.ziDevice()
mfia.connect_device()
if mfia.device is None:
    stage.disconnect_temp_controller(stopControl=False)
    raise SystemExit('MFIA connection failed')

roomStatus = None
try:
    tParams = stage.configure({'Temperature Ramp (C/min)': 5, 'Room Temperature (C)': 25})
    tParams.pop('Temperature Grid (C)')          # numpy array; not needed in the metadata
    params = mfia.configure()

    T = 30.0
    stage.go_to_temp(T)                          # blocks until stable (no overall time limit)
    before = stage.read_temp()
    data = mfia.pull_data(plot=False, trigger=True, numPoints=2**16, numReps=100)
    after = stage.read_temp()
    mfia.writeDataH5(data, r'C:\DATA\standalone\p30p0.h5', setpoint_C=T,
                     stage_temperature_C=(before + after) / 2,
                     runParams=({'params', **tParams}))
    roomStatus = stage.go_to_room_temp()          # 0 arrived, 1 gave up, -1 not connected
finally:
    # Send TEMPerature:STOP only if the stage reached room temperature;
    # otherwise leave the controller ramping there on its own.
    stage.disconnect_temp_controller(stopControl=(roomStatus == 0))
    mfia.disconnect_device()
```

`stage.configure()` sends nothing to the controller; `go_to_temp()` sends the ramp command.

Notes and limitations

- Every node path is hard-coded to `/dev32271/`. `devSerial` only affects `connect_device()` and `disconnect_device()`; a different MFIA would connect but every set, get and subscribe would address `dev32271`.
- `connect_device()` always returns `0`. Discovery errors are raised, session errors are printed.
- `reload_params()` (called by every run step) re-reads mismatched values from `dltsC.z_params_for_push`, then `dltsC.z_params_vars`, then `self.configuredParams` (set by `configure()`), and only then `input()` prompts. A script that calls `configure()` first never reaches the prompts (covered by `tests/test_standalone_helpers.py`).
- If `dltsC.z_params_vars` holds GUI label strings and `z_params_for_push` lacks a name, `set_param_value(pName)` stores the label string (for example `'1 - On'`) and the push fails.
- `check_param()` reads the entire node tree (`get('*')`) once per parameter; `reload_params()` does this 23 times.

- Oscillation ON/OFF is pushed to `imps/0/auto/output`, not an output on/off node. Filter Bandwidth sets the filter order.
- Triggered `pull_data()`: `tickStampDemods` is the `param1` time axis, not the demodulator's own; `tickStamp*` hold seconds, not clock ticks; `AbsZ` is $\sqrt{\text{param0}^2 + \text{param1}^2}$, which is not $|Z|$ when the representation is $Rp||Cp$; the `R.avg` subscription is unused.
- `daq_module.forcetrigger()` is called with no argument. In `zhinst-toolkit`, calling a node without an argument reads it, so this line most likely does not force a trigger.
- After the 60 s timeout, `pull_data()` still calls `read()` and indexes `[0]` of each node's result. If no burst arrived, that raises an exception, which the run records as a failed step.
- `pull_data()` never calls `daq_module.finish()`; the managed module is reused between calls. The 0.5 s settle after `execute()` exists because `progress()` can report the previous run's 1.0.
- The code never sets `imps/0/model`, so the meaning of `param0/param1` depends on the instrument's current or factory setting.
- `disconnect_device()` leaves the bias and test signal running.
- `H5_FORMAT_VERSION` is a class attribute (`ziC.ziDevice.H5_FORMAT_VERSION`), not a module constant.
- `writeDataH5()` leaves `<fName>.tmp` behind when writing fails.
- `runSweep()` has no timeout and the 'cv' sweep leaves the aux offset at its last value.

instecTempStage_Control

`instecTempStage_Control.py` controls the Instec mK2000B temperature controller of the Hall-probe stage over a serial port (COM7 by default). Its class, `mK2000B`, opens the port, commands temperature ramps, reads the stage temperature, waits for the stage to stabilize at each set-point of the DLTS temperature grid, and returns the stage to room temperature after a run. It also works without the GUI: `DEFAULT_PARAMS` and `mK2000B.configure()` set every temperature parameter from a script.

GUI callers:

- **Input Parameters** tab: `runParamsTab.connect_and_get_params('temperature')` creates `dltsctrl.tempDev = mK2000B()` and calls `connect_temp_controller()` and `set_param_value()`; `apply_and_push_params('temperature')` calls `load_params()`.
- **Live Tools** tab, Run DLTS (`runDltsTools.dltsRun`): `set_temp_grid()`, `go_to_temp()`, `read_temp()` (HDF5 metadata), `go_to_room_temp()`, and the attributes `tempGrid`, `tRamp`, `tStableDelay`, `Troom`, `roomRamp`.
- Window close (`DLTSGUI_MainWindow`): `abort()`, `start_ramp(..., lockTimeout=...)`, `read_temp()` while waiting, and `disconnect_temp_controller(stopControl=...)`.

Import

```
import instecTempStage_Control as tsC
```

The module does not import `dltsConfig`.

Module constants and globals

Name	Value / type	Meaning
<code>SERIAL_TIMEOUT_S</code>	2.0 (s)	Read and write timeout set on the port by <code>connect_temp_controller()</code> . The longest <code>read_temp()</code> waits for one reply.
<code>READ_RETRIES</code>	1	Extra queries <code>read_temp()</code> sends after a missed or unreadable reply before raising <code>TimeoutError</code> .
<code>DEFAULT_PARAMS</code>	dict, 7 entries (below)	The Input Parameters tab's temperature defaults. <code>mK2000B.configure()</code> starts from these.
<code>mK2000B._PARAM_ATTRS</code>	dict (class attribute)	Maps parameter names to the attributes the motion methods read: 'Temperature Ramp (C/min)' → <code>tRamp</code> , 'Stability Delay (s)' → <code>tStableDelay</code> , 'Room Temperature (C)' → <code>Troom</code> , 'Room Ramp (C/min)' → <code>roomRamp</code> .

`DEFAULT_PARAMS`:

Key	Value	Meaning
Initial Temperature (C)	25.0	First grid set-point, °C.
Final Temperature (C)	25.0	Last grid set-point, °C.

Key	Value	Meaning
Temperature Step (C)	5.0	Grid step, °C (sign ignored).
Temperature Ramp (C/min)	5.0	Ramp rate between set-points.
Stability Delay (s)	0.0	Extra wait after the stage is stable.
Room Temperature (C)	25.0	Where the stage goes after a run.
Room Ramp (C/min)	10.0	Ramp rate for that return.

`params` also has an eighth key, `Temperature Grid (C)`, which is always computed from the first three; it is not in `DEFAULT_PARAMS` and `configure()` rejects it.

Serial commands

The port uses pyserial defaults for the line settings (9600 baud, 8 data bits, no parity, 1 stop bit).

`connect_temp_controller()` sets a read and a write timeout of `SERIAL_TIMEOUT_S` (2 s); the controller normally answers in 10–15 ms. Every command ends with `\n`.

Command sent	Sent by	Reply	Effect
<code>TEMPerature:RAMP <T>, <ramp></code>	<code>start_ramp()</code> (so <code>go_to_temp()</code> , <code>go_to_room_temp()</code> , GUI close)	none read	Ramp to <code><T></code> °C at <code><ramp></code> °C/min. The controller carries the ramp out by itself. <code><T></code> and <code><ramp></code> are Python <code>str()</code> of the values, for example <code>TEMPerature:RAMP 30.0,5.0</code> . No leading colon.
<code>:TEMPerature:CTEMPerature?</code>	<code>read_temp()</code>	one line, parsed with <code>float()</code>	Current stage temperature, °C. Sent again (up to <code>READ_RETRIES</code> times) if no readable reply arrives within 2 s.
<code>:TEMPerature:STOP</code>	<code>disconnect_temp_controller(stopControl=True)</code>	none read	Ends temperature control and cancels any ramp in progress.

Stabilization logic

`go_to_temp(Tf)` sends the ramp, waits 0.5 s, then compares the measured deviation `|Tf - T_stage|` with a tolerance from `expected_del_t(Tf)`:

1. Take one reading. If the deviation is at or below the tolerance, the wait ends right away (no confirmation reading).
2. Otherwise loop: take a reading. If it is below the tolerance, wait 10 s and take a confirmation reading; the loop ends only if that reading is also within tolerance. If it is not below the tolerance, wait 5 s.
3. After the loop, sleep `tStableDelay` seconds.

There is no overall time limit on stabilizing: the loop ends only on stabilization, `abort()`, or an exception. If the controller stops answering, `read_temp()` raises `TimeoutError` after about 4 s (2 tries × 2 s), which ends the wait with that exception instead of hanging.

`expected_de1_t(T)` builds the tolerance from a sensor calibration table: set-points [19.648, 100, 199.99, 300, 400, 500, 600] °C and measured [19.648, 99.679, 199.633, 299.235, 398.98, 498.725, 598.47] °C, so `de1T` = [0, 0.321, 0.357, 0.765, 1.02, 1.275, 1.53] °C. Then:

Range of T	Tolerance formula
<code>T < 19.648</code>	Straight line from 0 at 19.648 °C to $2 \times 1.53 = 3.06$ at -190 °C: $-0.0145959 \cdot T + 0.286780$
<code>19.648 ≤ T ≤ 25</code>	<code>0.1</code>
<code>T > 25</code>	Straight line from 0 at 19.648 °C to 1.53 at 600 °C: $0.00263633 \cdot T - 0.0517986$

It also runs an `lmfit` linear fit of `de1T` on every call (about 1 ms) and discards the result.

Tolerances computed offline with the function itself:

Set-point (°C)	Tolerance (°C)
-190	3.060
-100	1.746
-50	1.017
0	0.287
10	0.141
19	0.0095
19.648 to 25	0.100
25.01	0.0141
30	0.0273
40	0.0537
50	0.0800
75	0.1459
100	0.2118
150	0.3437
200	0.4755
300	0.7391
400	1.0027
500	1.2664
600	1.5300

The tolerance jumps from 0.1 °C at 25 °C to 0.014 °C just above it, and falls to nearly 0 just below 19.648 °C. Set-points slightly above 25 °C or slightly below 19.6 °C need the stage to hold within a few hundredths of a degree for two readings 10 s apart.

`go_to_room_temp()` does not use this tolerance. It uses a plain `tolerance` argument (0.5 °C by default) and a deadline.

Thread safety

`_ioLock` is a `threading.RLock`. `start_ramp()`, `read_temp()` and `disconnect_temp_controller()` hold it for each command (and, for `read_temp()`, the command plus its reply), so the run thread, the GUI close handler and any monitor thread never interleave bytes on the port. `go_to_temp()` and `go_to_room_temp()` do not hold the lock between polls. `write()`, `read()` and `query()` do not use the lock.

class mK2000B

Attributes

Name	Type	Default	Meaning
port	str	'COM7'	Serial port name.
dev	serial.Serial or None	None	Open port. Not reset to None by <code>disconnect_temp_controller()</code> .
rm	None	None	Unused.
state	bool	False	True while the port is open.
Tinitial	number	25	First set-point, updated by <code>set_temp_grid()</code> and <code>configure()</code> .
Tfinal	number	25	Last set-point.
tempStep	number	5	Grid step.
numTemps	int	1	Number of grid points.
tRamp	float	5	Ramp rate <code>go_to_temp()</code> uses, °C/min.
tStableDelay	float	0	Wait after stabilization, s.
tempGrid	numpy.ndarray	25 (an int until a grid is built)	Set-points, °C.
Troom	float	25	Room-temperature target, °C.
roomRamp	float	10	Room-return ramp rate, °C/min.
_ioLock	threading.RLock	new lock	Serial I/O lock.
_aborted	bool	False	Set by <code>abort()</code> . Never cleared by the class.
params	dict	8 keys, all None	The seven <code>DEFAULT_PARAMS</code> names plus <code>Temperature Grid (C)</code> .

init(self, port = None)

Creates an unconnected controller object.

Name	Type	Default	Meaning
port	str or None	None	Serial port; None means 'COM7'.

Returns a new `mK2000B`.

Side effects None. The port is not opened.

Example

```
stage = tsC.mK2000B()          # COM7
stage3 = tsC.mK2000B('COM3')
```

`build_temp_grid(Tinit, Tfin, step)`

Static method. Builds the set-point grid from `Tinit` to `Tfin` inclusive, stepping by `|step|`. The number of intervals is `round(|Tfin - Tinit| / step)` (at least 1), and the last point is forced to `Tfin`, so the last interval can be shorter or longer than `step`.

Name	Type	Default	Meaning
<code>Tinit</code>	number or numeric string	required	First set-point, °C.
<code>Tfin</code>	number or numeric string	required	Last set-point, °C. Can be below <code>Tinit</code> (cooling grid).
<code>step</code>	number or numeric string	required	Step, °C; sign ignored.

Returns `numpy.ndarray` of floats. `Tinit == Tfin` gives `[Tinit]`; `step == 0` gives `[Tinit, Tfin]`.

Side effects None.

Example

```
tsC.mK2000B.build_temp_grid(25, 50, 5)  # [25. 30. 35. 40. 45. 50.]
tsC.mK2000B.build_temp_grid(25, 52, 5)  # [25. 30. 35. 40. 45. 52.]
tsC.mK2000B.build_temp_grid(50, 25, 5)  # [50. 45. 40. 35. 30. 25.]
```

`read(self)`

Returns `self.dev.read()`: one byte from the port (pyserial reads 1 byte by default).

No parameters.

Returns `bytes`. Raises `AttributeError` if `dev` is `None`.

Side effects Waits up to `SERIAL_TIMEOUT_S` for a byte; returns `b''` on timeout. Not locked. Not used by the codebase.

Example

```
b = stage.read()
```

`write(self, writeStr = None)`

Writes `writeStr` to the port if `dev` is set.

Name	Type	Default	Meaning
<code>writeStr</code>	<code>bytes</code>	<code>None</code>	Raw bytes to send, including <code>\n</code> . A <code>str</code> raises <code>TypeError</code> in pyserial.

Returns `0` if written, `-1` if `dev` is `None`.

Side effects Sends bytes. Not locked. Not used by the codebase.

Example

```
stage.write(b':TEMPerature:CTEMperature?\n')
```

`query(self, queryStr = None)`

Calls `self.dev.query(queryStr)`. `serial.Serial` has no `query` method, so this raises `AttributeError` when connected, and returns `None` when `dev` is `None`. It is a leftover from a VISA-based version.

Name	Type	Default	Meaning
<code>queryStr</code>	<code>any</code>	<code>None</code>	Unused in practice.

Returns `None` (or raises).

Side effects None.

Example

```
# Do not use; call read_temp() instead.
```

`connect_temp_controller(self)`

Opens `self.port` with `serial.Serial()`, with `timeout` and `write_timeout` set to `SERIAL_TIMEOUT_S`, waits 0.2 s and sets `state = True`. If the controller is already connected it prints `Temperature controller is already connected!`.

No parameters.

Returns `0` in every case. Check `self.state`.

Side effects Opens the serial port. On failure prints `Error connecting to device: ...` and resets `state`, `dev`, `rm`. It sends no command, so any device on the port "connects".

Example

```
stage.connect_temp_controller()
if not stage.state:
    raise RuntimeError('temperature controller connection failed')
```

`disconnect_temp_controller(self, stopControl=True, lockTimeout=5.0)`

Closes the serial port. With `stopControl=True` it first sends `:TEMPerature:STOP` and waits 0.5 s.

Name	Type	Default	Meaning
stopControl	bool	True	True: end temperature control (and cancel any ramp) before closing. False: leave the controller running on its own, for example still ramping to room temperature after the program exits.
lockTimeout	float	5.0	Seconds to wait for _ioLock. If it expires, the method closes the port anyway.

Returns 0.

Side effects Optional STOP command; closes the port; state = False. dev keeps the closed port object. Prints Already disconnected! if state is already False, or Nothing to do! if dev is None. The GUI passes stopControl=True only when the stage reached room temperature (or no ramp was needed); "Close now" passes False.

Example

```
stage.disconnect_temp_controller(stopControl=False) # stage keeps ramping
```

abort(self)

Sets self._aborted = True. go_to_temp() then refuses to send a new set-point and its wait loop exits; go_to_room_temp() stops waiting (the ramp it already sent keeps going).

No parameters.

Returns None.

Side effects Sets the flag only. Nothing clears it; set stage._aborted = False before using the object again (as benchmarks/hardware/hw_run.py does). In a dltsRun, also set dlts.run_abortRequested = True, or the run will acquire at whatever temperature the stage has.

Example

```
import threading
threading.Timer(40 * 60, stage.abort).start() # watchdog for go_to_temp()
```

start_ramp(self, T, ramp, lockTimeout=None)

Sends TEMPERATURE:RAMP <T>,<ramp> and returns without waiting.

Name	Type	Default	Meaning
T	number	required	Target, °C.
ramp	number	required	Rate, °C/min.
lockTimeout	float or None	None	None: wait for the lock indefinitely. A number: wait that long, then send anyway. The GUI close handler uses 5 s (2 s for the exit backstop): reaching room temperature matters more than a garbled concurrent reply.

Returns None.

Side effects One serial command. Does not check `state` or `_aborted`. Raises `AttributeError` if `dev` is `None`, and `serial.SerialTimeoutException` if the write cannot complete within `SERIAL_TIMEOUT_S`.

Example

```
# The stage starts heating or cooling immediately.
stage.start_ramp(30.0, 5.0)
```

read_temp(self, retries=READ_RETRIES)

Clears any unread bytes from the port, sends `:TEMPerature:CTemperature?` and reads one reply line, all under the lock. A missing (timed out) or non-numeric reply is retried.

Name	Type	Default	Meaning
<code>retries</code>	<code>int</code>	<code>READ_RETRIES</code> (1)	Extra attempts after the first. Each waits at most <code>SERIAL_TIMEOUT_S</code> (2 s).

Returns `float`, stage temperature in °C.

Side effects One to `retries + 1` command/response pairs. Clearing the input buffer first means a late reply to an earlier query is never taken for this one. When no attempt gives a number, it raises `TimeoutError("temperature controller on COM7 did not answer the temperature query (2 tries, 2 s each; last reply b'')")`, about 4 s after the call, and releases the lock. Callers handle it: a run marks the step failed and returns to room temperature, `dltsRun._read_stage_temp()` records the stage temperature as missing, and the GUI close dialog keeps polling until its deadline.

Example

```
try:
    print(stage.read_temp())
except TimeoutError as exc:
    print('controller not answering:', exc)
```

expected_del_t(self, T=25)

Returns the stabilization tolerance for set-point `T` (see "Stabilization logic").

Name	Type	Default	Meaning
<code>T</code>	number	25	Set-point, °C.

Returns `float`, tolerance in °C.

Side effects None (runs an unused `lmfit` fit). Works offline.

Example

```
tsC.mK2000B().expected_del_t(50)  # 0.0800...
```

go_to_temp(self, Tf=25, ramp=5, delayTime = 0)

Ramps to `Tf` and blocks until the stage is stable, then sleeps the stability delay.

Name	Type	Default	Meaning
<code>Tf</code>	number	25	Target set-point, °C.
<code>ramp</code>	number	5	Ignored when <code>dev</code> is set: replaced by <code>self.tRamp</code> .
<code>delayTime</code>	number	0	Ignored when <code>dev</code> is set: replaced by <code>self.tStableDelay</code> . Used only on the <code>dev is None</code> path.

Returns 0 when done. -1 if `_aborted` was set before the call, during the wait, or during the stability delay. Also 0 when `dev` is `None` (prints `Nothing to do!`) or `state` is `False` (prints `T-Controller is disconnected!`), without moving the stage.

Side effects Sends one ramp command, then `read_temp()` every 5 s (plus confirmation readings 10 s apart). Prints `Wait for T = <Tf> stabilization!`. Has no overall time limit; other than stabilizing, it ends on `abort()` from another thread, or with `TimeoutError` from `read_temp()` if the controller stops answering.

Example

```
# The stage heats or cools to 30 C.
stage.configure({'Temperature Ramp (C/min)': 5, 'Stability Delay (s)': 60})
status = stage.go_to_temp(30.0)
```

go_to_room_temp(self, Tr=None, ramp=None, tolerance=0.5, extraWaitS=900)

Ramps to room temperature and waits until the stage is within `tolerance` of it, or a deadline passes.

Name	Type	Default	Meaning		---	---	---	---		Tr	number or None		None		Target, °C; None means self.Troom.	
ramp	number or None	None	Rate, °C/min; None means self.roomRamp.			tolerance	float	0.5		Arrival band, °C.						
extraWaitS	float	900	Deadline = Tr - T _{first} / ramp × 60 s + extraWaitS, from the first reading.													

Returns 0 once within tolerance (after a final 10 s sleep); 1 if aborted or the deadline passed (prints `Gave up waiting for room temperature ... on timeout`); -1 if `dev` is `None` or `state` is `False`.

Side effects Sends the ramp command even if `_aborted` is set, then polls `read_temp()` every 5 s. Returning 1 leaves the controller ramping or holding at `Tr`. A `TimeoutError` from `read_temp()` propagates (`dltsRun.return_to_room_temp()` catches it and returns -1).

Example

```
# The stage heats or cools to 25 C.
status = stage.go_to_room_temp(Tr=25, ramp=10)
```

set_temp_grid(self, userInput = False)

Builds `tempGrid` from the initial/final/step values and stores `Tinitial`, `Tfinal`, `tempStep`, `numTemps`.

Name	Type	Default	Meaning
userInput	bool	False	False: read the values from <code>self.params</code> . True: prompt on the console with <code>input()</code> ; an empty answer keeps the current attribute.

Returns `0`.

Side effects Updates the grid attributes. Does nothing but print `Nothing to do!` when `dev` is `None`, so it needs a connected (or at least constructed) port. `dltsRun.init_experiment()` calls it.

Example

```
stage.set_temp_grid()
print(stage.tempGrid)
```

set_param_value(self, pName='Initial Temperature (C)', valueDict=None)

Stores one parameter in `self.params`, then copies the ramp, delay and room values into the motion attributes (`_sync_attrs_from_params()`).

Name	Type	Default	Meaning
pName	str	'Initial Temperature (C)'	One of the 8 <code>params</code> keys.
valueDict	dict or None	None	<code>{name: value}</code> . None: prompt on the console (empty answer = the <code>DEFAULT_PARAMS</code> value).

For `Temperature Grid (C)` the value is always `build_temp_grid()` of the dict's (or `params`) initial, final and step values. Any other name takes `valueDict.get(pName)` (a missing key stores `None`, which `_sync_attrs_from_params()` then skips).

Returns `0`.

Side effects Writes `params` and possibly `tRamp`, `tStableDelay`, `Troom`, `roomRamp` (cast to `float`). Unknown names print `Unknown Parameter <name>!!!`. No serial traffic.

Example

```
stage.set_param_value('Room Temperature (C)', {'Room Temperature (C)': 22})
```

configure(self, values=None)

Sets every parameter without the GUI: `DEFAULT_PARAMS` overridden by `values`, then `load_params()`, then `Tinitial`, `Tfinal`, `tempStep`, `tempGrid` and `numTemps`.

Name	Type	Default	Meaning
values	dict or None	None	<code>{parameter name: number}</code> overrides. Names must be <code>DEFAULT_PARAMS</code> keys.

Returns a copy of `self.params` (dict), including the `Temperature Grid (C)` array.

Side effects Raises `KeyError('unknown temperature parameter(s): [...]')` for unknown names before changing anything. Updates `params`, the motion attributes and the grid. Needs no connection and sends nothing to the controller.

Example

```
stage = tsC.mk2000B()
p = stage.configure({'Initial Temperature (C)': 25, 'Final Temperature (C)': 50})
stage.tempGrid      # array([25., 30., 35., 40., 45., 50.]
```

load_params(self, valueDict)

Calls `set_param_value(pName, valueDict)` for all 8 parameters.

Name	Type	Default	Meaning
valueDict	dict	required	Must contain the initial, final and step values (the grid is built from them).

Returns `0`.

Side effects As `set_param_value`. The **Apply + Push Params** button calls it with `dltsc.t_params_for_push`.

Example

```
stage.load_params(dict(tsC.DEFAULT_PARAMS))
```

measure_proxy_temp(self, Tf)

Reads the stage once and returns the deviation and the tolerance.

Name	Type	Default	Meaning
Tf	number	required	Set-point, °C.

Returns tuple `(abs(Tf - T_stage), expected_del_t(Tf))`, both °C.

Side effects One `read_temp()`. With `dev = None` it prints `Nothing to do!` and then raises `UnboundLocalError`.

Example

```
dev, tol = stage.measure_proxy_temp(30.0)
```

Internal helpers

- `_sync_attrs_from_params(self)`: for each entry of `_PARAM_ATTRS`, copies `float(params[name])` into the attribute if the value is not `None`. Without it, `go_to_temp()` would keep the constructor's 5 °C/min and 0 s.

Standalone use

Safety: these scripts heat and cool the stage; the last one also biases the sample.

Read the temperature

```
import sys
sys.path.insert(0, r'C:\Users\spencer\Documents\GitHub\DLTSOnHallProbeSetup')
import instecTempStage_Control as tsC

stage = tsC.mK2000B()
stage.connect_temp_controller()
if not stage.state:
    raise SystemExit('connection failed')
try:
    print('stage at', stage.read_temp(), 'C')
finally:
    stage.disconnect_temp_controller(stopControl=False)  # do not change control state
```

Ramp through a grid with a watchdog

```
import sys, threading
sys.path.insert(0, r'C:\Users\spencer\Documents\GitHub\DLTSOnHallProbeSetup')
import instecTempStage_Control as tsC

stage = tsC.mK2000B()
stage.configure({'Initial Temperature (C)': 25, 'Final Temperature (C)': 40,
                'Temperature Step (C)': 5, 'Temperature Ramp (C/min)': 5,
                'Stability Delay (s)': 30, 'Room Temperature (C)': 25})
stage.connect_temp_controller()
if not stage.state:
    raise SystemExit('connection failed')

roomStatus = None
try:
    for T in stage.tempGrid:
        watchdog = threading.Timer(30 * 60, stage.abort)  # go_to_temp has no overall time limit
        watchdog.start()
        status = stage.go_to_temp(T)
        watchdog.cancel()
        if status == -1:
            print(f'gave up stabilizing at {T} C')
            break
        print(f'{T} C reached, stage reads {stage.read_temp()} C')
finally:
    stage._aborted = False  # abort() is sticky
    roomStatus = stage.go_to_room_temp()  # 0 arrived, 1 gave up, -1 not connected
    stage.disconnect_temp_controller(stopControl=(roomStatus == 0))
```

`stopControl=(roomStatus == 0)` follows the GUI: send STOP only when the stage is at room temperature, otherwise leave the controller ramping there.

One DLTS step with the MFIA

```
import sys
sys.path.insert(0, r'C:\Users\spencer\Documents\GitHub\DltsOnHallProbeSetup')
import matplotlib; matplotlib.use('Agg')
import dltsConfig as dltsC
import instecTempStage_Control as tsC
import zurichInstruments_Control as ziC

stage, mfia = tsC.mk2000B(), ziC.ziDevice()
stage.configure({'Room Temperature (C)': 25})
stage.connect_temp_controller()
mfia.connect_device()
if not stage.state or mfia.device is None:
    raise SystemExit('connection failed')

roomStatus = None
try:
    params = mfia.configure()          # also what reload_params() falls back to
    T = 30.0
    stage.go_to_temp(T)
    tBefore = stage.read_temp()
    data = mfia.pull_data(plot=False, trigger=True, numPoints=2*16, numReps=100)
    tAfter = stage.read_temp()
    mfia.writeDataH5(data, r'C:\DATA\standalone\p30p0.h5', setpoint_C=T,
                    stage_temperature_C=(tBefore + tAfter) / 2, runParams=params)
    roomStatus = stage.go_to_room_temp()
finally:
    stage.disconnect_temp_controller(stopControl=(roomStatus == 0))
    mfia.disconnect_device()
```

Notes and limitations

- A controller that stops answering no longer hangs anything: reads time out after 2 s, `read_temp()` retries once and then raises `TimeoutError` (earlier versions set no serial timeout and blocked forever). Tested in `tests/test_run_folder_and_serial.py`.
- `go_to_temp()` still has no overall time limit. A set-point the stage cannot hold within tolerance, while the controller keeps answering, waits until another thread calls `abort()`.
- `go_to_temp()` ignores its `ramp` and `delayTime` arguments when connected and uses `tRamp` and `tStableDelay`.
- `go_to_temp()` returns 0 without moving the stage when the port is closed (`state == False`) or `dev` is `None`, so callers cannot tell "done" from "not connected".
- If the stage is already within tolerance at the first reading, `go_to_temp()` skips the 10 s confirmation reading.
- `expected_del_t()` is discontinuous: 0.1 °C at 25 °C, 0.014 °C at 25.01 °C, and it drops to about 0 just below 19.648 °C. At 30 °C the tolerance is 0.027 °C. The `lmfit` fit it runs is unused.
- `abort()` is permanent for the object; reset `_aborted` by hand to reuse it. `go_to_room_temp()` sends its ramp even when aborted.
- `dltsRun` checks `dltsC.run_abortRequested`, not `go_to_temp()`'s return value. Calling only `stage.abort()` during a run makes the step acquire at the current temperature.
- `connect_temp_controller()` does not verify that the device on the port is an mk2000B.
- `read()`, `write()` and `query()` are unlocked leftovers; `query()` always fails on a `serial.Serial`.
- `measure_proxy_temp()` raises `UnboundLocalError` when not connected.

- `set_temp_grid()` does nothing without an open port object, but `configure()` builds the same grid offline.
- `tempGrid` is the integer 25 until a grid is built.
- `disconnect_temp_controller()` leaves `dev` pointing at the closed port.
- The header comment's example (`temperatureTools`, `connectTempController`, `goToTemp`, `disconnTController`) names functions that do not exist in this module.
- The module imports `matplotlib.pyplot` and `from lmfit.models import *` without using them beyond `lmfit.models.LinearModel`.

impedanceAnalysis_Tools

`impedanceAnalysis_Tools.py` is the analysis library for DLTS runs. It loads the per-temperature step files a run writes (`.h5`, JSON in `.txt`/`.json`, or Zurich Instruments LabOne `.csv` exports), repairs the time axis, splits each record into excitation (fill pulse) and emission blocks by clustering the pulse signal, averages the emission transients, denoises them, and computes the double-boxcar DLTS signal with error propagation. It also holds the peak finders used to locate the DLTS peak temperature.

Where the GUI uses it:

GUI file	What it calls
<code>liveDataTab.py</code>	Live mode: <code>impdData(fName=[path]) + read_data()</code> for the first file, then <code>append_data(fName=[path])</code> for each new step, on a worker thread; then <code>cleanup_data()</code> , <code>selected_emissions(emissionIndex=0)</code> , <code>filter_emissions(method=..., emissionIndex=0, recalculate=True, interactivePlot=False)</code> , <code>selected_emissions(emissionIndex=-1)</code> . Offline mode: the same chain after <code>read_data()</code> on the chosen files. Also <code>read_h5_record(filePath, keys=('AuxInput1', 'ImpedanceIm'))</code> .
<code>dataAnalysisTab.py</code> (Quick Analysis)	<code>impd.calculate_delC_normalized(...)</code> for each rate window, then <code>impdData._curveFit_peakFinder(...)</code> or <code>impdData._smoothingSpline_peakFinder(...)</code> for the peak.
<code>detailedAnalysisTab.py</code>	<code>impdData._pca_denoise</code> , <code>_wavelet_denoise</code> , <code>_savitzkyGolay_denoise</code> , <code>_lowess_denoise</code> on averaged transients, and <code>impdData._smoothingSpline_peakFinder</code> for Tp.
<code>runParametersTab.py</code> , <code>runDLTS.py</code>	Import the module; no live calls (comments and commented-out code only).

Import

```
import impedanceAnalysis_Tools as iaT

impd = iaT.impdData(fName=[r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5"])
```

Importing the module also imports `zhinst.core`, `zhinst.toolkit`, `torch`, `zurichInstruments_Control` and `instecTempStage_Control`, so those packages must be installed even for offline analysis. The import does not connect to any hardware.

Module constants and globals

The module defines no named constants. The table lists every module-level name and import-time setting, followed by the literal values hard-coded inside functions that act as constants.

Name	Value / type	Meaning
<code>GAP_FACTOR</code>	<code>100</code>	<code>cleanup_data()</code> / <code>_gap_remove</code> : a time step larger than this many typical (median) steps is a gap and is closed.
<code>_STEP_STEM_PATTERN</code>	regex <code>^([npNP]?)(\d+)(?:[pP](\d+))?[cC]?(?:_\d+)?\$</code>	Step file name without extension → sign, whole degrees, decimals. Same rule as <code>LiveDataTab._LEGACY_FILENAME_PATTERN</code> ; a missing sign letter means positive.
<code>_CSV_NAME_PATTERN</code>	regex <code>^([npNP]?)(\d+)(?:[pP.](\d+))?[C_]</code>	ZI export history name (<code>25C_000</code> , <code>n10C_000</code> , <code>p25p5C_000</code>) → sign, whole degrees, decimals.
<code>read_h5_record</code>	function	Reads one HDF5 step file. See Module-level functions .
<code>impdData</code>	class	The analysis class.
<code>warnings.filterwarnings("ignore", category=FutureWarning, module="uncertainties")</code>	import-time setting	Silences <code>FutureWarnings</code> raised by the <code>uncertainties</code> package. Other warnings (for example <code>UserWarning: Using UFloat objects with std_dev==0</code>) still print.
<code>zi</code> , <code>zt</code>	modules <code>zhinst.core</code> , <code>zhinst.toolkit</code>	Imported, not used in this file.
<code>ziC</code> , <code>tsC</code>	modules <code>zurichInstruments_Control</code> , <code>instecTempStage_Control</code>	Imported, not used in this file.
<code>np</code> , <code>plt</code> , <code>pd</code> , <code>h5py</code> , <code>json</code> , <code>os</code> , <code>re</code> , <code>pywt</code>	modules	Used.
<code>sm</code> , <code>torch</code> , <code>lmfit</code> , <code>time</code> , <code>itertools</code> , <code>statistics</code>	modules	Imported; <code>lmfit</code> is used only through <code>lmfit.models</code> ; the others are unused.
<code>askopenfilenames</code>	<code>tkinter.filedialog</code> function	Opens the file-picker dialog when no file names are given.
<code>GaussianMixture</code> , <code>KMeans</code> , <code>PCA</code>	scikit-learn classes	Used for level clustering and PCA denoising.
<code>savgol_filter</code> , <code>UnivariateSpline</code> , <code>CubicSpline</code> , <code>differential_evolution</code>	SciPy functions/classes	Used.
<code>GaussianModel</code> , <code>PseudoVoigtModel</code> , <code>VoigtModel</code> , <code>LorentzianModel</code>	lmfit models	Used by <code>_curveFit_peakFinder</code> .
<code>ufloat</code>	<code>uncertainties</code> function	Used for error propagation in <code>calculate_delC_normalized</code> .
<code>Lowess</code>	<code>fastlowess</code> class	Used by <code>_lowess_denoise</code> .
<code>apply_along_axis</code> , <code>unumpy</code> , <code>FastICA</code> , <code>weibull_min</code> , <code>mode</code> , <code>quad</code> , <code>BSpline</code> , <code>interp2d</code> , <code>LognormalModel</code>	various	Imported, not used.

Hard-coded values inside functions:

Value	Where	Meaning
<code>60 * 10**6</code>	<code>_ticks_per_second</code> , CSV branch of <code>read_data/append_data</code>	Instrument clock rate in ticks per second (60 MHz).
<code>+ 273.15</code> , rounded to 1e-6 K	<code>_celsius_to_kelvin_key</code>	°C to K conversion of the setpoint in a file or history name.
<code>20.0 * second</code> minimum	<code>find_data_levels_scikit</code>	A minimum more than 20 times larger in magnitude than the second-smallest value marks the record unusable.
<code>n_components=2</code> , <code>reg_covar=1e-8</code> , <code>random_state=0</code> ; <code>n_clusters=2</code> , <code>n_init=10</code>	<code>find_data_levels_scikit</code>	GMM and K-Means settings (two levels).
<code>headerKeys[16]</code>	CSV branch of <code>read_data/append_data</code>	Column 17 of the LabOne header file holds the per-chunk data names that carry the temperature.
<code>trimHead=10</code> , <code>trimTail=10</code>	<code>selected_emissions</code> , <code>filter_emissions</code> , <code>calculate_delC_normalized</code>	Samples dropped at each end of an emission block. <code>calculate_delC_normalized</code> always uses 10/10.
<code>window_size=100</code> ; <code>wavelet="db4"</code> , <code>level=4</code> , <code>mode="soft"</code> ; <code>window_size=None</code> , <code>order=2</code> ; <code>fraction=None</code> (becomes 0.1)	<code>filter_emissions</code>	Fixed denoiser settings.
<code>iterations=3</code> , <code>robustness_method="bisquare"</code>	<code>_lowess_denoise</code>	LOWESS settings.
100000 points; 2000 points	<code>_smoothingSpline_peakFinder</code>	Fine grid for the spline peak; coarser grid for bootstrap peaks.
2000 points	<code>_curveFit_peakFinder</code>	Grid for the fitted curve.

Data model

All data lives in attributes of one `impdData` instance. Temperatures are kelvin floats everywhere; they come from the file name in °C, keep their decimals and are converted with `round(T_C + 273.15, 6)` (`p25p0` → 298.15, `p50p5` → 323.65, `n10p0` → 263.15). Convert back with `round(T_K - 273.15, 6)`.

```

impd.dataTemps = [298.15, 303.15, 308.15, ...] # list[float], K, in file-selection order (not sorted)

impd.dataValues = {                                # one record per temperature
    298.15: {
        'tickStampImps': array(N)                # 60 MHz ticks (int) or seconds (float), see below
        'tickStampDemods': array(N)
        'timeStampImps': array(N)                # seconds
        'timeStampDemods': array(N)             # seconds
        'ImpedanceRe': array(N)                 # real part, Ohm (per the code comment)
        'ImpedanceIm': array(N)                # imaginary part in Ohm, or capacitance in F,
                                                # depending on the equivalent-circuit mode
        'AuxInput1': array(N)                  # pulse / excitation signal, V
        'AbsZ': array(N)                       # |Z|, Ohm
    },
    303: {...},
}

impd.dataParams = {...}                          # runParams.txt as a dict, or None

impd.dataExcitationLevelParams = {                # also dataEmissionLevelParams (same shape)
    'means': array(nT, 2)                        # column 0 = higher level, column 1 = lower level
    'stds': array(nT, 2)
    'labels': array(nT, N) float                 # 0 = higher level, 1 = lower level, -1 = dropped
}

impd.dataEmissionClusterParams = {               # also dataExcitationClusterParams
    'clusterBlocks': {
        298.15: {
            'high': {0: [start, end, end-start], 1: [...], ...} # label-0 blocks
            'low': {0: [start, end, end-start], 1: [...], ...}  # label-1 blocks = emission windows
        }, ...
    },
    'clusterSizesFreqs': {
        298.15: {
            'high': [array([length, count]), ...]
            'low': [array([length, count]), ...]
        }, ...
    }
}

impd.dataEmissions = {                          # built by selected_emissions()
    298.15: {
        'x': array(n)                            # s, time from the first kept sample of the block
        'xTimeStampImps': array(n)               # s, the same samples on the record's own time axis
        'y': array(n) or array(n, R)            # ImpedanceIm; one column per repeat
        'ymean': array(n)                       # mean over repeats (or y itself for one repeat)
        'yerr': array(n)                       # std over repeats (zeros for one repeat)
        # added by filter_emissions():
        'yFiltered': array(n)                   # denoised trace
        'yRaw': array(n)                       # the trace that was denoised
        'filterMethod': 'pca' | 'wavelet' | 'sgolay' | 'lowess'
        'filterIndex': int                     # emissionIndex used for the filter
    }, ...
}

```

Details:

- **Channel values.** `.h5` records hold numpy arrays (`read_h5_record` converts `uint64` to `int64`). JSON records hold Python lists until `cleanup_data()` converts them to arrays. CSV records hold only the channels found in the CSV set (`tickStampImps`, `timeStampImps`, `ImpedanceIm`, `tickStampDemods`, `timeStampDemods`, `AuxInput1`, and `AbsZ` plus an all-zero `ImpedanceRe` if the `_r_avg_` files are present).
- **tickStamp channels.** They hold either integer 60 MHz clock ticks (untriggered acquisitions) or a float time axis in seconds starting at 0 (triggered acquisitions, which runs use; step 18.67 μ s in the sample run). `_is_seconds_axis` tells them apart.
- **Block entries.** `[start, end, end-start]` are sample indices into the record, both ends inclusive. The third value is `end - start`, one less than the number of samples. `align_clusters()` shortens `end` and the third value in place.
- **Which blocks are emissions.** Label 0 is the higher mean level of the clustered signal and label 1 the lower one. The code uses the 'low' (label 1) blocks as emission windows. In the sample run `AuxInput1` sits at -0.5 V and -1.5 V; the -1.5 V blocks are the emission windows.
- **Repeats.** In `dataEmissions[T]['y']` for `emissionIndex=-1`, rows are time samples and columns are repeats (pulses). The sample HDF5 run gives `y.shape == (301, 542)`: 320-sample blocks, trimmed by 10 + 10.
- **Rate-window output.** `calculate_delC_normalized()` returns `(N, 4)` rows of `[T (K), tau (s), deltaC, deltaC/C_inf]` and `(N, 2)` rows of `[deltaC_err, (deltaC/C_inf)_err]`, sorted by temperature.

class impdData

Holds one DLTS run (one or many temperatures) and the analysis state derived from it. Every analysis step writes its result into an attribute, and later steps read those attributes.

Name	Type	Set by	Meaning
<code>fileName</code>	<code>str</code> , <code>list[str]</code> or <code>None</code>	<code>__init__</code> , <code>read_data</code> (normalized to a list, digit-less names removed, set to <code>None</code> on <code>FileNotFoundError</code>), <code>append_data</code> (extended)	Step files loaded so far.
<code>rootFolder</code>	<code>str</code> ending in <code>os.sep</code> , or <code>None</code>	<code>read_data</code> (only when <code>None</code>), CSV branch of <code>append_data</code>	Folder of the first file. <code>runParams.txt</code> is read from here.
<code>dataValues</code>	<code>dict[int, dict]</code>	<code>read_data</code> , <code>append_data</code> , <code>cleanup_data</code>	Raw channel data per temperature.
<code>dataTemps</code>	<code>list[float]</code>	<code>read_data</code> , <code>append_data</code>	Temperatures in K (exact: 298.15 for 25 °C), in load order.
<code>dataParams</code>	<code>dict</code> or <code>None</code>	<code>read_data</code> , <code>append_data</code> (txt/h5 only)	Contents of <code>runParams.txt</code> .
<code>dataExcitationLevelParams</code>	<code>dict</code> or <code>None</code>	<code>find_data_levels(dataType='excitation', recalculate=True)</code>	Level means, stds, labels of <code>AuxInput1</code> .
<code>dataEmissionLevelParams</code>	<code>dict</code> or <code>None</code>	<code>find_data_levels(dataType='emission', recalculate=True)</code>	Level means, stds, labels of <code>ImpedanceIm</code> .
<code>dataExcitationClusterParams</code>	<code>dict</code> or <code>None</code>	<code>find_clusters(dataType='excitation')</code> , <code>align_clusters</code>	Blocks and block-length statistics from <code>AuxInput1</code> .

Name	Type	Set by	Meaning
dataEmissionClusterParams	dict or None	find_clusters(dataType='emission'), align_clusters	Blocks used to cut emission transients. Set it to <code>None</code> to force re-clustering.
dataEmissions	dict[int, dict] or None	selected_emissions, filter_emissions	Cut, averaged and denoised emission transients.

`__init__(self, fName=None)`

Creates an empty instance. Nothing is read until `read_data()`.

Name	Type	Default	Meaning
fName	str, list of str, or None	None	Step-file path(s). <code>None</code> makes <code>read_data()</code> open a file dialog. Pass files, not a folder.

Returns the instance.

Example

```
import glob
import impedanceAnalysis_Tools as iaT

files = sorted(glob.glob(r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\*.h5"))
impd = iaT.impdData(fName=files)
```

`read_data(self)`

Loads every file in `self.fileName` into `dataValues`, keyed by temperature. The extension of the first file picks the loader.

- `.txt`, `.json`, `.h5`: files whose base name has no digit are skipped with a message. The temperature comes from the file name (`p25p0.h5` → 25.0 °C → 298.15 K; see `_extract_txt_temperature`); a file whose name encodes no temperature is skipped with `Warning: Could not extract temperature from filename`. Each file is read with `_load_record`, so `.h5` and JSON files can be mixed in one list. `runParams.txt` is then loaded from `rootFolder` into `dataParams`.
- `.csv` (LabOne export): files are sorted by name fragment into emission (`_imps_0`), excitation (`_auxin0`) and impedance (`_r_avg`) sets, each split into header (`_header`) and data files. Temperatures come from column 17 of the emission header; the `chunk` column maps rows to temperatures. Time stamps are `timestamp / 60e6`. `runParams.txt` is not read.

Name	Type	Default	Meaning
(none)			Uses <code>self.fileName</code> .

Returns `0` on success, `-1` if no file was chosen, no temperature could be parsed, the type is unsupported, or a file is missing.

Side effects

- Opens a Tk file dialog when `fileName` is `None`. In the CSV branch it opens more dialogs if a header or data file of the emission/excitation set is missing.
- Replaces `fileName`, `dataValues`, `dataTemps`, `dataParams`; sets `rootFolder` if it was `None`.
- Prints warnings for skipped files, unparseable temperatures and a missing `runParams.txt`.

Call order first call on a new instance. Use `append_data()` to add files later.

Example

```
impd = iaT.impdData(fName=files)
if impd.read_data() != 0:
    raise RuntimeError("load failed")
print(impd.dataTemps)           # [298.15, 303.15, 308.15, 313.15, 318.15, 323.15]
print(impd.dataParams['Number of Reps'])
```

append_data(self, fName=None)

Adds more step files to an instance that already holds data. Uses the same loaders as `read_data()`. A new temperature gets its own record. A temperature already present is merged with `_append_records`: arrays and lists of the same channel are concatenated end to end.

Name	Type	Default	Meaning
<code>fName</code>	<code>str</code> , list of <code>str</code> , or <code>None</code>	<code>None</code>	Files to add. <code>None</code> opens a file dialog. <code>self.fileName</code> is not used as input.

Returns `0` on success, `-1` on no files, no parsable temperature or unsupported type.

Side effects

- Updates `dataValues`, `dataTemps` (new temperatures appended at the end), and extends `fileName`.
- txt/h5/json: reads `runParams.txt` from the folder of the first appended file and merges it into `dataParams` (new values overwrite old ones). Does not set `rootFolder`.
- CSV: sets `rootFolder` if `None`; may open file dialogs; does not touch `dataParams`.
- Does not reset cluster or emission results. Set `dataEmissionClusterParams = None` before `selected_emissions()` so the new temperatures are clustered (this is what `liveDataTab.py` does).

Call order after `read_data()` (or on a fresh instance; `_append_records` accepts `dataValues=None`).

Example

```
impd = iaT.impdData(fName=[files[0]])
impd.read_data()
impd.append_data(fName=files[1:])
impd.cleanup_data()
impd.dataEmissionClusterParams = None      # force re-clustering
impd.selected_emissions(emissionIndex=-1)
```

cleanup_data(self)

Repairs the time axis of every record. For each temperature it runs `_zero_time_fill` (replaces zero time stamps by interpolated values) and then `_gap_remove` (closes every jump in the time stamps larger than `GAP_FACTOR` typical steps).

How the two kinds of `tickStamp*` channel are handled:

- **Clock ticks (integer, untriggered acquisitions).** Every zero tick is a missing sample. It is filled by linear interpolation between its valid neighbours, or extrapolated with the median step at the ends. `timeStamp*` is recomputed as `ticks / 60e6`.
- **Seconds (float, triggered acquisitions, which runs use).** The zero at index 0 is the real $t = 0$ and is kept. Only zeros after index 0 are filled, using a float median step. `timeStamp*` is recomputed as `ticks / 1.0`, so the axis stays in seconds.

`timeStamp*` and `tickStampDemods` are rewritten only when a zero was filled or a gap was closed; after a fix, `tickStampDemods/timeStampDemods` become copies of the `Imps` channels.

Name	Type	Default	Meaning
(none)			Works on <code>dataValues</code> for every temperature in <code>dataTemps</code> .

Returns `0`.

Side effects mutates `dataValues[T]` in place; converts list channels to numpy arrays.

Call order after `read_data()/append_data()`, before clustering. `selected_emissions()` calls it itself when `dataEmissionClusterParams` is `None`.

Example

```
impd.read_data()
impd.cleanup_data()
t = impd.dataValues[298.15]['timeStampImps'] # seconds
```

`find_data_levels_scikit(self, dataType = 'emission', model='gmm', interactivePlot=False)`

Splits each record's samples into two levels. For every temperature it divides the signal by its minimum, fits both a 2-component Gaussian mixture and 2-cluster K-Means, and relabels both so label 0 is the higher level. Non-finite samples are replaced by the median first. A record whose minimum is more than 20 times the second-smallest value in magnitude is marked unusable (all labels -1 , means/stds NaN).

Name	Type	Default	Meaning
<code>dataType</code>	<code>str</code>	<code>'emission'</code>	<code>'emission'</code> clusters <code>ImpedanceIm</code> ; <code>'excitation'</code> clusters <code>AuxInput1</code> . Any other value fails with <code>UnboundLocalError</code> .
<code>model</code>	<code>str</code>	<code>'gmm'</code>	<code>'gmm'</code> , <code>'kmeans'</code> , or <code>'hybrid'</code> (keeps only samples where GMM and K-Means agree; the rest get label -1). Case-insensitive.
<code>interactivePlot</code>	<code>bool</code>	<code>False</code>	Shows a 3-panel figure (K-Means, GMM, Hybrid) per temperature; left/right arrow keys step through temperatures. Only works with <code>model='hybrid'</code> ; otherwise prints a message.

Returns (`means`, `stds`, `labels`): `means` and `stds` are $(nT, 2)$ arrays (column 0 higher level, column 1 lower level); `labels` is an (nT, N) float array of 0, 1 or -1 .

Side effects prints warnings for unusable records; with `interactivePlot=True` opens a blocking matplotlib window. Does not store its result (see `find_data_levels`).

Raises `ValueError` if no data is loaded, if `model` is invalid, or if a record's length differs from the first record's.

Call order after `read_data()`, ideally after `cleanup_data()`.

Example

```
means, stds, labels = impd.find_data_levels_scikit(dataType='excitation', model='hybrid')
print(means[0])      # e.g. [-0.501, -1.500] V
```

find_data_levels(self, dataType = 'excitation', algorithm='gmm', recalculate=True, interactivePlot=False)

Wrapper around `find_data_levels_scikit` that stores the result and trims labels. After getting the labels it sets to `-1` every sample before the first label-0 sample and every sample from the last label-1 sample to the end, so partial blocks at the record edges are dropped.

Name	Type	Default	Meaning
<code>dataType</code>	<code>str</code>	<code>'excitation'</code>	<code>'excitation'</code> or <code>'emission'</code> .
<code>algorithm</code>	<code>str</code>	<code>'gmm'</code>	<code>'gmm'</code> / <code>'gaussianmixture'</code> , <code>'kmeans'</code> , or <code>'hybrid'</code> / <code>'gmmkmeans'</code> / <code>'kmeansgmm'</code> . Case-insensitive.
<code>recalculate</code>	<code>bool</code>	<code>True</code>	<code>False</code> reuses stored levels if present (recomputes and prints a message if not).
<code>interactivePlot</code>	<code>bool</code>	<code>False</code>	Passed to <code>find_data_levels_scikit</code> .

Returns (`means`, `stds`, `labels`) as above, with edge samples set to `-1`. Returns `-1` for an invalid `dataType` when `recalculate=False`. Raises `ValueError` for an unknown `algorithm`.

Side effects with `recalculate=True`, replaces `dataExcitationLevelParams` or `dataEmissionLevelParams` with `{'means', 'stds', 'labels'}`. The edge trimming is applied to the same `labels` array that was stored, so the stored labels are trimmed too.

Call order after `read_data()` and `cleanup_data()`. Called by `find_clusters()`.

Example

```
m, s, l = impd.find_data_levels(dataType='excitation', algorithm='hybrid')
print(impd.dataExcitationLevelParams['labels'].shape)  # (nT, N)
```

find_clusters(self, dataType='excitation', method='free', recalculate=False, align=False)

Turns the level labels into blocks of consecutive samples and records how often each block length occurs. It always calls `find_data_levels(..., algorithm="hybrid", recalculate=True)`, then `_find_cluster_stats`.

Name	Type	Default	Meaning
dataType	str	'excitation'	Which cluster set to build: 'excitation' or 'emission'.
method	str	'free'	Emission only. 'free' clusters ImpedanceIm itself. 'synced' clusters AuxInput1 and uses those blocks for the emission set (the pulse defines the emission windows). Ignored for excitation.
recalculate	bool	False	False does nothing if the chosen cluster set already exists.
align	bool	False	Runs align_clusters(dataType) after building the blocks.

Returns 0, or -1 for an invalid dataType or method.

Side effects replaces dataExcitationClusterParams or dataEmissionClusterParams with {'clusterBlocks', 'clusterSizesFreqs'}; also replaces the level params that find_data_levels stores (method='synced' updates dataExcitationLevelParams, not dataEmissionLevelParams). Prints messages.

Call order read_data() → cleanup_data() → find_clusters(). selected_emissions() calls find_clusters(dataType='emission', method='synced', recalculate=True, align=True) when no clusters exist.

Example

```
impd.find_clusters(dataType='emission', method='synced', recalculate=True, align=True)
blocks = impd.dataEmissionClusterParams['clusterBlocks'][298.15]['low']
print(blocks[0])          # e.g. [55, 375, 320]
```

align_clusters(self, dataType='excitation', method='free')

Makes all blocks of one kind the same length so they can be stacked and averaged. Steps:

1. Calls find_clusters(dataType, method) (no-op if clusters exist).
2. Per temperature, shortens every 'high' and 'low' block to the shortest length in that temperature's frequency table.
3. Rebuilds clusterSizesFreqs as [length, count] pairs (in first-seen order, no longer sorted by count).
4. If a temperature has more than one distinct 'low' length, deletes its last 'low' block and last frequency entry.
5. Shortens blocks at every temperature to the shortest first-block length over all temperatures.

Only the end index and the length field change; start indices stay.

Name	Type	Default	Meaning
dataType	str	'excitation'	'excitation' or 'emission'.
method	str	'free'	Passed to find_clusters() if clusters must be built.

Returns 0, or -1 for an invalid dataType.

Side effects mutates clusterBlocks and clusterSizesFreqs of the chosen cluster set.

Call order after find_clusters(); usually invoked through find_clusters(..., align=True).

Example

```
impd.find_clusters(dataType='emission', method='synced', recalculate=True)
impd.align_clusters(dataType='emission')
```

selected_emissions(self, emissionIndex=0, trimHead = 10, trimTail = 10, plot=False)

Cuts the emission transients out of `ImpedanceIm` using the 'low' blocks of `dataEmissionClusterParams` and stores them in `dataEmissions`. Each block is sliced as `[start + trimHead : end + 1 - trimTail]`. `x` is the matching `timeStampImps` slice shifted to start at 0.

If `dataEmissionClusterParams` is `None`, it first runs `cleanup_data()` and `find_clusters(dataType='emission', method='synced', recalculate=True, align=True)`.

Name	Type	Default	Meaning
<code>emissionIndex</code>	int	0	-1: all blocks, stacked as columns of <code>y</code> , with <code>ymean/yerr</code> the mean and std over repeats (<code>x</code> from the first block). 0 ... <code>maxIndex-1</code> : that single block, <code>ymean = y</code> , <code>yerr</code> zeros. See limits below.
<code>trimHead</code>	int	10	Samples dropped after each block start.
<code>trimTail</code>	int	10	Samples dropped before each block end.
<code>plot</code>	bool	False	Unused.

`maxIndex` is `count - 1` of the first entry in `clusterSizesFreqs[first T]['low']` (1 if that is 0). Index handling:

- `emissionIndex > maxIndex`: prints a warning and uses `maxIndex`.
- `emissionIndex == maxIndex` (including after that clamp): nothing is built; `dataEmissions` keeps its previous value (or stays `None`).
- `emissionIndex < -1`: prints "Using min index" but uses 0.

Returns 0.

Side effects replaces `dataEmissions` (dropping any `yFiltered/yRaw`); may run `cleanup_data()` and clustering; prints warnings.

Call order `read_data()` → `cleanup_data()` → `(find_clusters(...))` → `selected_emissions()` → `filter_emissions()`.

Example

```
impd.selected_emissions(emissionIndex=-1)
e = impd.dataEmissions[298.15]
print(e['y'].shape, e['ymean'].shape)    # (301, 542) (301,)
```

filter_emissions(self, method='pca', emissionIndex=-1, recalculate=False, trimHead=10, trimTail=10, interactivePlot=True)

Denoises one trace per temperature from `dataEmissions` and stores it as `yFiltered`, with the input trace as `yRaw`. The settings per method are fixed:

method	Call	Settings
'pca'	<code>_pca_denoise</code>	<code>window_size=100</code>
'wavelet'	<code>_wavelet_denoise</code>	<code>wavelet="db4", level=4, mode="soft"</code>
'sgolay'	<code>_savitzkyGolay_denoise</code>	<code>window_size=None (automatic), order=2</code>
'lowess'	<code>_lowess_denoise</code>	<code>fraction=None (0.1)</code>

Name	Type	Default	Meaning
<code>method</code>	str	'pca'	One of the four above (case-insensitive). Otherwise <code>ValueError</code> .
<code>emissionIndex</code>	int	-1	Which trace to denoise: -1 uses <code>ymean</code> ; 0 ... R-2 uses column <code>emissionIndex</code> of <code>y</code> ; a larger value uses the last column; < -1 uses column 0. Also passed to <code>selected_emissions()</code> if <code>dataEmissions</code> is <code>None</code> .
<code>recalculate</code>	bool	False	False skips the work if <code>yFiltered</code> exists and <code>filterMethod</code> matches <code>method</code> . A changed <code>emissionIndex</code> alone does not trigger a recompute.
<code>trimHead</code>	int	10	Used only if <code>selected_emissions()</code> must run.
<code>trimTail</code>	int	10	Used only if <code>selected_emissions()</code> must run.
<code>interactivePlot</code>	bool	True	Opens a blocking plot of filtered (red line) vs raw (black points) per temperature; left/right arrows step through temperatures.

Returns `0`. Raises `ValueError` if no emissions exist.

Side effects adds `yFiltered`, `yRaw`, `filterMethod`, `filterIndex` to each `dataEmissions[T]`; may call `selected_emissions()`; opens a matplotlib window by default. Pass `interactivePlot=False` in scripts and GUI code.

Call order after `selected_emissions()`.

Example

```
impd.selected_emissions(emissionIndex=-1)
impd.filter_emissions(method='wavelet', emissionIndex=-1, recalculate=True, interactivePlot=False)
yF = impd.dataEmissions[298.15]['yFiltered']
```

calculate_delC_normalized(self, t1=0.003, t2=0.203, emissionIndex=-1, denoiseEmission=False, denoiseMethod='pca', smoothCapacitance=True, plot=False)

Computes the double-boxcar DLTS signal for one rate window at every temperature: $\Delta C = C(t_2) - C(t_1)$ and $\Delta C / C_{\text{inf}}$, where `C_inf` is the last sample of the trimmed emission window. Uncertainties from `yerr` are propagated with `uncertainties.ufloat`. The time constant is $\tau = (t_2 - t_1) / \ln(t_2 / t_1)$.

It always rebuilds the emissions first: `selected_emissions(emissionIndex, trimHead=10, trimTail=10)` then `filter_emissions(method=denoiseMethod, emissionIndex, recalculate=True, interactivePlot=False)`. `t1` is raised to `min(x)` and `t2` lowered to `max(x)` per temperature if they fall outside the window, without a warning.

Name	Type	Default	Meaning
<code>t1</code>	float	0.003	First gate, s after the start of the trimmed window.
<code>t2</code>	float	0.203	Second gate, s.
<code>emissionIndex</code>	int	-1	-1 averages all repeats and gives meaningful errors; a single pulse (0) gives zero <code>yerr</code> , so all errors are 0.
<code>denoiseEmission</code>	bool	False	True reads C from <code>yFiltered</code> , False from <code>yRaw</code> .
<code>denoiseMethod</code>	str	'pca'	Filter used by <code>filter_emissions()</code> . The filter runs even when <code>denoiseEmission=False</code> .
<code>smoothCapacitance</code>	bool	True	True reads C and its error at <code>t1</code> , <code>t2</code> , <code>x[-1]</code> from cubic splines of C and <code>yerr</code> . False uses the nearest sample (and computes <code>tau</code> from those sample times).
<code>plot</code>	bool	False	Shows normalized ΔC vs T with error bars (blocking window).

Returns (`delCNormalized`, `delCNormalizedErr`): (N, 4) array [T (K), tau (s), `deltaC`, `deltaC/C_inf`] and (N, 2) array [`deltaC_err`, (`deltaC/C_inf`)_err], both sorted by temperature.

Side effects replaces `dataEmissions` (and whatever `selected_emissions/filter_emissions` touch); may emit `uncertainties` warnings when errors are zero; opens a plot if `plot=True`.

Call order after `read_data()` and `cleanup_data()`. Clustering runs automatically if needed.

Example

```
delC, delCErr = impd.calculate_delC_normalized(t1=0.001, t2=0.004, emissionIndex=-1,
                                              denoiseEmission=True, denoiseMethod='sgolay')
T, S, Serr = delC[:, 0], delC[:, 3], delCErr[:, 1]
```

test(self, t1=None, t2=None, plot=True)

Experimental scan over rate-window pairs. For every pair with `t2[j] > t1[i]` it runs `calculate_delC_normalized(t1, t2, emissionIndex=0, denoiseEmission=False, smoothCapacitance=False)`, fits a cubic spline to normalized ΔC vs T, and runs `differential_evolution` on that spline over the temperature range.

Name	Type	Default	Meaning
<code>t1</code>	array of float	None → [0.01, 0.02]	First-gate values, s.
<code>t2</code>	array of float	None → [0.01, 0.02]	Second-gate values, s.
<code>plot</code>	bool	True	Draws two <code>tricontourf</code> maps over (t1, t2): peak T and peak ΔC .

Returns array of rows `[t1, t2, T_at_extremum, -min_value]`, with `len(t1)*(len(t2)-1)//2` rows.

Side effects replaces `dataEmissions`; opens two blocking plots when `plot=True`.

`differential_evolution` minimizes the spline, so the third column is the temperature of the minimum and the fourth is the negated minimum value, not the maximum. The row count formula only matches the number of valid pairs when `t1` and `t2` are the same array. With the defaults there is one row, and `plot=True` fails with `ValueError: arange: cannot compute length`.

Example

```
import numpy as np
C = impd.test(t1=np.array([0.001, 0.002, 0.003]), t2=np.array([0.001, 0.002, 0.003]), plot=False)
```

Module-level functions

`read_h5_record(file_path, keys=None)`

Reads one temperature step written by `ziDevice.writeDataH5()` into `{channel name: numpy array}`, the same layout a JSON step file gives. `uint64` datasets (tick stamps) are returned as `int64` so that `np.diff()` in `cleanup_data()` does not wrap around.

Name	Type	Default	Meaning
<code>file_path</code>	<code>str</code>	required	Path to a <code>.h5</code> step file.
<code>keys</code>	iterable of <code>str</code> or <code>None</code>	<code>None</code>	Channels to read. <code>None</code> reads all datasets. A missing key raises <code>KeyError</code> .

Returns `dict[str, numpy.ndarray]`.

Side effects none (opens the file read-only and closes it).

Example

```
rec = iaT.read_h5_record(r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5",
                        keys=('AuxInput1', 'ImpedanceIm'))
print(rec['ImpedanceIm'].shape)      # (262144,)
```

Internal helpers

All are `@staticmethod`s of `impdData` unless noted.

Loading and merging

- `_normalize_file_selection(file_selection)` returns `[]` for `None`, `[s]` for a string, otherwise `list(file_selection)`.
- `_load_record(file_path)` returns `read_h5_record(file_path)` for `.h5`, otherwise `json.load()` of the file (UTF-8).

- `_extract_txt_temperature(file_path)` matches the file name without extension against `_STEP_STEM_PATTERN` (optional sign letter `p/n` in either case, whole degrees, optional `p` + decimals, optional `C`, optional `_NNN`; no sign letter means positive) and returns the key from `_celsius_to_kelvin_key`, or `None` if the name encodes no temperature. `p25p0.h5` → 298.15, `n10p5.txt` → 262.65, `p25C_1.txt` → 298.15, `x25p0.txt` and `p-3.h5` → `None`.
- `_extract_csv_temperature(data_name)` matches a LabOne data name against `_CSV_NAME_PATTERN` (optional sign letter, whole degrees, optional `p` or `.` + decimals, then `C_`) and returns the key, or `None`. `25C_000` → 298.15, `n10C_000` → 263.15, `p25p5C_000` → 298.65.
- `_celsius_to_kelvin_key(sign, integerPart, fracPart)` turns the matched parts into `round(±T_C + 273.15, 6)`, so the same setpoint always gives the same float key.
- `_merge_nested_dict(existing, incoming, concatenate_arrays=False)` merges dicts recursively. With `concatenate_arrays=True`, same-key arrays are joined with `np.concatenate` and same-key lists with `+`; otherwise the incoming value wins. Returns `incoming` if either side is not a dict.
- `_append_records(existing_records, new_records)` merges per-temperature records: new temperatures are added, existing ones merged with `_merge_nested_dict(..., concatenate_arrays=True)`.
- `_append_unique_temps(existing_temps, new_temps)` appends temperatures not already in the list, keeping order.

Time-axis repair

- `_is_seconds_axis(tick)` returns `True` when `tick` is a non-empty float array with at least one finite non-integer value, meaning a time axis in seconds. Integer arrays, and float arrays holding only whole numbers, count as clock ticks.
- `_ticks_per_second(tick)` returns `1.0` for a seconds axis, else `60 * 10**6`.
- `_zero_time_fill(signal=None)` fills zero entries of `tickStampImps` by interpolation or median-step extrapolation (keeping index 0 on a seconds axis), then recomputes `timeStampImps` and copies both into the `Demods` channels. Converts list channels to arrays. Returns the record, or `-1` if `signal` is `None`.
- `_zero_time_remove(signal=None)` deletes every sample whose `tickStampImps` is 0 from all channels. Not used; the call in `cleanup_data()` is commented out. Returns the record or `-1`.
- `_gap_remove(signal=None)` takes the typical step as the median of `|diff(tickStampImps)|`. Every step larger than `GAP_FACTOR` (100) typical steps, forward (dropped samples) or backward (a second record of the same temperature appended by `append_data`, whose clock restarts), is replaced by one typical step, and the axis is rebuilt from its first sample. The result is continuous and increasing; the data samples keep their order. It then recomputes `timeStampImps` (with `_ticks_per_second`) and copies both into the `Demods` channels. Unsigned ticks are converted to `int64` first so backward jumps don't wrap. A record without gaps is returned unchanged. Returns the record, or `-1` for `signal=None`.

Clustering

- `_find_nearest(array, value)` returns the index of the element closest to `value`.
- `_find_signal_blocks(arr, val)` returns `[(start, end), ...]` inclusive index ranges of runs equal to `val`. Runs of `-1` lying strictly between two `val` samples are counted as `val` first. Uses the nested helper `get_blocks_for_target(target)`.
- `_find_cluster_stats(dataLabels, classLabels)` builds `clusterBlocks` (label 0 → 'high', label 1 → 'low', entries `[start, end, end-start]`) and `clusterSizesFreqs` (`[length, count]` sorted by count, descending) for each temperature. For 'low' it drops the last length added to the frequency table (`popitem()`), which removes the length of the final low block when that length is unique. Returns `(blocks, clusterSizesFreqs)`.

Denoisers (also called directly by `detailedAnalysisTab.py`)

All four take `signal`, a `dataEmissions[T]`-style dict with `'x'`, `'y'`, `'ymean'`, and `index` which picks the trace: `-1` → `ymean`; `0 ≤ index < maxIndex` → column index (or the whole `y` if `maxIndex == 1`); `index > maxIndex` → last column; `index < -1` → column 0. `maxIndex` is `y.shape[1] - 1`, or 1 when `y` is 1-D. `index == maxIndex` leaves the trace undefined and raises `UnboundLocalError`. Each returns `(x, yDenoised, yRaw)`.

- `_wavelet_denoise(signal, index=-1, wavelet="db4", level=1, mode="soft")` decomposes with `pywt.wavedec`, estimates noise from the finest detail coefficients (`MAD / 0.6745`), applies the universal threshold `sigma * sqrt(2 * ln n)` to all detail levels, and reconstructs.
- `_pca_denoise(signal, index=-1, window_size=None)` frames the trace into overlapping windows, keeps the first principal component, and averages the overlapping reconstructions. `window_size=None` → `max(2, n // 1000)` for `n ≥ 1000`, else `max(2, n // 20)`; it is capped at `n - 2`. The last sample is set to the mean of the two before it.
- `_savitzkyGolay_denoise(signal, index=-1, window_size=None, order=2)` applies `scipy.signal.savgol_filter`. `window_size=None` → `max(order + 2, n // 1000)` for `n ≥ 1000`, else `max(order + 2, n // 20)`; forced odd, capped at the trace length, and kept above `order`.
- `_lowess_denoise(signal, index=-1, fraction=None)` fits `fastlowess.Lowess(fraction, iterations=3, robustness_method="bisquare")` against `x`. `fraction=None` → 0.1.

Peak finders (also called directly by `dataAnalysisTab.py` and `detailedAnalysisTab.py`)

Both return `(maxX, maxY, xf, yf, maxXErr, maxYErr)`, or `-1` (after printing) if `signalX` or `signalY` is `None`. Typical input is `x = delC[:, 0]` (T in K) and `y = delC[:, 3]`.

`_smoothingSpline_peakFinder(signalX = None, signalY = None, smoothingFactor = None, nBootstrap = 200, randomState = 0, signalYErr = None)` fits a cubic `UnivariateSpline` and returns the point of largest absolute value on a 100000-point grid, so negative peaks are found too.

Name	Type	Default	Meaning
<code>signalX, signalY</code>	array	None	Data; sorted by x internally.
<code>smoothingFactor</code>	float	None	Spline <code>s</code> . <code>None</code> → <code>len(x)</code> when valid errors are given, otherwise <code>noiseVar * len(x)</code> with <code>noiseVar</code> estimated from second differences (plain variance for fewer than 5 points).
<code>nBootstrap</code>	int	200	Parametric bootstrap refits for the errors; 0 skips them.
<code>randomState</code>	int	0	Seed for the bootstrap.
<code>signalYErr</code>	array	None	Per-point errors; used as weights <code>1/err</code> and as bootstrap noise. Ignored unless all finite and <code>> 0</code> .

`maxXErr/maxYErr` are the standard deviations of the bootstrap peaks (2000-point grid), or `None` if the bootstrap was skipped or fewer than `max(10, nBootstrap // 4)` refits succeeded.

`_curveFit_peakFinder(signalX = None, signalY = None, curveType = "pseudoVoigt", signalYErr = None)` fits an `lmfit` model after `model.guess()` and returns its `center` and `height` parameters with their `stderr` (which may be `None`). `curveType` is one of `pseudoVoigt`, `gaussian`, `lorenzian/lorentzian`, `voigt` (case-insensitive); anything else raises

ValueError. `signalYErr` sets `weights = 1/err` when all finite and > 0 . `xf/yf` is the fitted curve on 2000 points.

Nested functions

- `_safe_ylim(values, pad_frac=0.1)` (inside `find_data_levels_scikit`) returns padded y-limits from the finite values, `(-1, 1)` if none.
- `update_cluster_plot(idx)` and `on_key(event)` (inside `find_data_levels_scikit`) redraw the K-Means / GMM / Hybrid panels for temperature `idx` and handle the arrow keys. `update_cluster_plot` refits both models for the displayed temperature.
- `update_filtered_plot(idx)` and `on_key(event)` (inside `filter_emissions`) redraw filtered vs raw traces and handle the arrow keys.
- `get_blocks_for_target(target)` (inside `_find_signal_blocks`) does the actual block search.

Typical workflows

Run these outside the GUI with a non-interactive backend so no windows block:

```
import os
os.environ.setdefault("LOKY_MAX_CPU_COUNT", "8")
import matplotlib
matplotlib.use("Agg")
```

1. Load an HDF5 run, cut and average the emissions

```
import glob
import numpy as np
import impedanceAnalysis_Tools as iaT

runFolder = r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655"
files = sorted(glob.glob(os.path.join(runFolder, "*.h5"))) # step files only; runParams.txt is read
automatically

impd = iaT.impdData(fName=files)
assert impd.read_data() == 0
print("Temperatures (K):", impd.dataTemps)
print("Reps:", impd.dataParams["Number of Reps"])

impd.cleanup_data()
impd.find_clusters(dataType='emission', method='syncd', recalculate=True, align=True)
impd.selected_emissions(emissionIndex=-1, trimHead=10, trimTail=10)

T = impd.dataTemps[0]
e = impd.dataEmissions[T]
print(e['x'].shape, e['y'].shape) # (301,) (301, 542)
print("window length (s):", e['x'][-1])
```

2. Denoise with each method and compare

```
methods = ['pca', 'wavelet', 'sgolay', 'lowess']
results = {}
for m in methods:
    impd.filter_emissions(method=m, emissionIndex=-1, recalculate=True, interactivePlot=False)
    results[m] = {T: impd.dataEmissions[T]['yFiltered'].copy() for T in impd.dataTemps}

T = impd.dataTemps[0]
raw = impd.dataEmissions[T]['yRaw']          # ymean, because emissionIndex=-1
for m in methods:
    print(m, "rms change:", np.sqrt(np.mean((results[m][T] - raw) ** 2)))
```

To denoise a single averaged trace without the class (as `detailedAnalysisTab.py` does):

```
sig = {'x': e['x'], 'y': e['ymean'], 'ymean': e['ymean']}
x, ySG, yIn = iaT.impdData._savitzkyGolay_denoise(sig, index=-1, window_size=None, order=2)
```

3. Rate-window DLTS spectrum and peak temperature (JSON run)

```
jsonFolder = r"C:\Users\spencer\Desktop\DATA\DLTS\092326\101901"
jfiles = sorted(glob.glob(os.path.join(jsonFolder, "*.json")))

run = iaT.impdData(fName=jfiles)
assert run.read_data() == 0          # 47 temperatures
run.cleanup_data()

delC, delCErr = run.calculate_delC_normalized(t1=0.010, t2=0.050, emissionIndex=-1,
                                              denoiseEmission=True, denoiseMethod='sgolay',
                                              smoothCapacitance=True)
T, S, Serr = delC[:, 0], delC[:, 3], delCErr[:, 1]
tau = delC[0, 1]

yErr = Serr if np.all(np.isfinite(Serr)) and np.all(Serr > 0) else None
Tp, Sp, xf, yf, TpErr, SpErr = iaT.impdData._smoothingSpline_peakFinder(T, S, signalYErr=yErr)
print(f"tau = {tau:.4g} s, Tp = {Tp:.1f} K +/- {TpErr}")

# Parametric alternative:
Tp2, Sp2, xf2, yf2, TpErr2, SpErr2 = iaT.impdData._curveFit_peakFinder(T, S, curveType='gaussian',
                               signalYErr=yErr)
```

4. Several rate windows and an Arrhenius plot

The module has no Arrhenius function; this combines its outputs with plain numpy.

```

windows = [(0.005, 0.025), (0.010, 0.050), (0.020, 0.100), (0.040, 0.200)] # s, must fit inside the emission
window
taus, Tps = [], []
for t1, t2 in windows:
    delC, delCErr = run.calculate_delC_normalized(t1=t1, t2=t2, emissionIndex=-1,
                                                  denoiseEmission=True, denoiseMethod='sgolay')

    Tp, _, _, _, _ = iaT.impdData._smoothingSpline_peakFinder(delC[:, 0], delC[:, 3], nBootstrap=0)
    taus.append(delC[0, 1])
    Tps.append(Tp)

taus, Tps = np.array(taus), np.array(Tps)
kB = 8.617333e-5 # eV/K
slope, intercept = np.polyfit(1.0 / Tps, np.log(taus * Tps**2), 1)
print(f"Ea = {slope * kB:.3f} eV")

```

Notes and limitations

- **tickStamp channels have two meanings.** Untriggered acquisitions store integer 60 MHz clock ticks; triggered acquisitions (used by runs) store the DAQ grid's float time axis in seconds under the same `tickStamp*` names. `cleanup_data()` detects this with `_is_seconds_axis` and uses `_ticks_per_second` (60e6 or 1.0) for conversion; on a seconds axis the leading `t = 0` is kept rather than treated as a missing sample. A float axis that holds only whole numbers is treated as ticks.
- **Gap removal is continuous now.** Earlier versions shifted one segment by the wrong offset (the segments overlapped) and could apply both of their two shifts on a backward jump, leaving a non-monotonic axis; they also closed only the single largest gap and compared against the smallest step. Tests: `tests/test_temperatures_and_gaps.py`. The rebuilt axis starts at the record's first time stamp, so after a gap the later samples' absolute times change; only relative times matter to the analysis.
- **Temperature keys are exact kelvin** (`T_C + 273.15`, decimals kept). Earlier versions used `int(float(T_C)) + 273` (`-10.5 °C → 263 K`, `50.5 °C → 323 K`), read any first character other than `p` as a minus sign (`25p0.txt → 268 K`), and raised `ValueError` for `p-3.h5` or `p25C.txt`. Code that indexed `dataValues` with integers (`dataValues[298]`) must use the float key (`dataValues[298.15]`).
- **Pass files, not a folder.** `fName` must be file paths. `read_data()` picks the loader from the first file's extension; `.h5`, `.txt` and `.json` can be mixed, CSV cannot. Files without a digit in the name are skipped.
- **dataTemps keeps load order.** With `sorted(glob(...))`, `p100p0` sorts before `p25p0`. Only `calculate_delC_normalized()` sorts its output by temperature.
- `runParams.txt` is read only for txt/h5/json loads, from `rootFolder` (the first file's folder). `append_data()` merges a new one over the old; CSV loads never read it.
- **CSV specifics.** Temperatures come from header column index 16, time is `timestamp / 60e6`, and `ImpedanceRe` is filled with zeros. Missing header/data files trigger extra file dialogs.
- **Rate-window gates are clipped silently.** `t1/t2` are forced into `[x.min(), x.max()]` of the trimmed emission window. In the sample HDF5 run the window is only about 5.6 ms (301 samples of 18.67 μ s), so the default `t2=0.203 s` becomes 5.6 ms, and `test()` with its defaults gives `t1 == t2`, `tau = NaN` and `$\Delta C = 0$` . With `smoothCapacitance=False`, a gate at `t = 0` gives `log(inf)` in `tau`.
- **`C_inf` is the last sample of the trimmed window**, not an extrapolated steady-state value. With short windows it can be far from the true steady-state capacitance.
- **`selected_emissions()` index quirks.** `maxIndex` is taken from the first temperature only. `emissionIndex == maxIndex`, including after clamping a too-large index, builds nothing and leaves `dataEmissions` unchanged (possibly `None`, so `filter_emissions()` then raises). `emissionIndex < -1` prints "Using min index" but uses 0.

plot is unused.

- **Denoiser index quirks.** `index == maxIndex` raises `UnboundLocalError`. With exactly two repeats (`y` of shape `(n, 2)`) and `index=0`, `maxIndex == 1` and the whole 2-D `y` is passed to the filter instead of one column. The sample JSON run has two repeats per temperature (`y.shape == (26780, 2)`), so use `emissionIndex=-1` or build emissions with `emissionIndex=0` first. The `except: fallback` returns `-1`, which callers then fail to unpack.
- `filter_emissions()` uses fixed settings per method, ignores a change of `emissionIndex` unless `recalculate=True`, and opens a blocking plot by default (`interactivePlot=True`).
- `_find_cluster_stats` **drops a length with** `popitem()` from the `'low'` table. If every low block has the same length, the table ends up empty and `align_clusters()` fails on `np.min` of an empty array.
- `find_data_levels()` **edge trimming** raises `IndexError` if a record has no label-0 or no label-1 samples (for example a record marked unusable). It also mutates the stored labels.
- `find_data_levels_scikit()` always fits both GMM and K-Means, whatever `model` is. It divides by the record minimum, so a minimum of exactly 0 produces `inf/NaN`. All records must have the same length as the first. `dataType` values other than `'emission' / 'excitation'` fail with `UnboundLocalError`.
- **Re-clustering after appends.** `selected_emissions()` only clusters when `dataEmissionClusterParams` is `None`; set it to `None` after `append_data()`. `selected_emissions()` also calls `cleanup_data()` again in that case.
- `test()` **is experimental**. It minimizes rather than maximizes, reports the negated minimum as "max ΔC ", mis-sizes its output for unequal `t1 / t2` arrays, and its plots fail with one row.
- **Interactive plots block.** `find_data_levels_scikit(interactivePlot=True)`, `filter_emissions(interactivePlot=True)`, `calculate_delC_normalized(plot=True)` and `test(plot=True)` call `plt.show()`. Do not use them from a worker thread.
- **Heavy imports.** The module imports hardware-control modules, `torch` and several unused packages at import time. The module creates no threads; `liveDataTab.py` runs it on a worker thread.
- **Units of** `ImpedanceIm` depend on the instrument's equivalent-circuit mode (code comment: imaginary impedance in Ohm or capacitance in F). The module does not check or convert them.

dltsConfig

`dltsConfig.py` is the shared state of the application. Every tab module, the run code and the instrument classes import it and read or write its module-level globals: the Tk root and tabs, the two connected instruments, the Input Parameters tab's variables, the run-control flags, and each analysis tab's widgets and results. It also provides the shared log function `log_to_textbox()` and `recast_param_type()`, which turns the Input Parameters tab's display strings into the numbers sent to the instruments.

Every module in the GUI uses it: `DLTSGUI_MainWindow`, `runParamsTab` (Input Parameters), `liveDataTab` (Live Tools), `dataAnalysisTab`, `detailedAnalysisTab`, plus `runDlts_Tools` and `zurichInstruments_Control`. The benchmark scripts in `benchmarks/hardware/` fill it by hand to run without the GUI.

Import

```
import dltsConfig as dltsc
```

Importing it calls `sys.setswitchinterval(0.001)` (the interpreter default is 5 ms), so a CPU-heavy background thread hands the GIL back to the Tk main thread more often. This applies to the whole process.

Module constants and globals

All globals are listed below, grouped by the section comments in the file. Abbreviations in the "Set by" and "Read by" columns:

Abbreviation	Module
main	<code>DLTSGUI_MainWindow.py</code> (main alone = its <code>if __name__ == '__main__':</code> block)
rpT	<code>runParamsTab.py</code>
ldT	<code>liveDataTab.py</code>
daT	<code>dataAnalysisTab.py</code>
deT	<code>detailedAnalysisTab.py</code>
rdT	<code>runDlts_Tools.py</code>
ziC	<code>zurichInstruments_Control.py</code>
hw	<code>benchmarks/hardware/hw_common.py</code> , <code>hw_run.py</code> , <code>hw_preflight.py</code>

A function name followed by "(callback)" means a nested function inside it (usually a `worker` thread or the `apply` it schedules on the Tk thread with `root.after()`). The "Initial / `init()`" column gives the value at import, then the value `init()` assigns, if it assigns one. The GUI never calls `init()` (see below), so in the GUI the import value is the starting value.

"Set by" and "Read by" were found by parsing every `.py` file in the repository root and `benchmarks/hardware/` for `dltsc.<name>`, `getattr/hasattr/setattr(dltsc, '<name>')`. Tests are not listed.

GUI

Name	Initial / init()	Set by	Read by	Meaning
root	None	main (tk.Tk())	main, main._finish_close, main._show_close_dialog, main._wait_for_room_temp; ldT (start_dlts, start_thread, _run_control_thread, _poll_step_listbox_while_busy, _reset_live_plot_state, _schedule_live_poll, construct_livePlotTab and several worker callbacks); deT._detailed_open_label_editor and worker callbacks; rpT.construct_runParamsTab, rpT.export_h5_file (callback)	Tk root window. Worker threads use root.after(0, ...) to run GUI updates on the Tk thread.
tabControl	None	main (ttk.Notebook)	main, rpT.construct_runParamsTab, ldT.construct_livePlotTab, daT.construct_dataAnalysisTab, deT.construct_detailedAnalysisTab	The notebook that holds the tabs.
textbox	None	rpT.construct_runParamsTab	rpT.construct_runParamsTab, log_to_textbox	The log tk.Text at the bottom of the Input Parameters tab.
textboxes	None	main ([]), rpT.construct_runParamsTab (appends textbox)	rpT.construct_runParamsTab, log_to_textbox	Every log text box. Only textbox is ever added.
textlinecount	None	main (0), log_to_textbox	log_to_textbox	Lines currently in the log.
maxTextLineCount	None	main (10)	log_to_textbox	Lines kept in the log; older lines are deleted.
runParametersTab	None	main (ttk.Frame)	rpT.construct_runParamsTab	Frame of the Input Parameters tab.
livePlotTab	None	main	ldT.construct_livePlotTab	Frame of the Live Tools tab.
dataAnalysisTab	None	main	daT.construct_dataAnalysisTab	Frame of the rate-window / Arrhenius tab.
detailedAnalysisTab	None	main	deT.construct_detailedAnalysisTab	Frame of the Detailed Analysis tab.

Name	Initial / init()	Set by	Read by	Meaning
post processingTab	None	main	nothing	Frame created but never added to the notebook.

Devices (RUNTIME)

Na me	Initial / init()	Set by	Read by	Meaning
tempDev	None	rpT.connect_and_get_params('temperature'), hw.connect_temperature	rdT.check_device_connections, rdT.init_experiment, rpT.apply_and_push_params, rpT.connect_and_get_params, main._send_room_ramp, main._wait_for_room_temp, main._finish_close, hw	The connected instecTempStage_Control.mk2000B.
impDev	None	rpT.connect_and_get_params('impedance'), hw.connect_impedance	rdT.check_device_connections, rpT.apply_and_push_params, rpT.connect_and_get_params, main._finish_close, hw	The connected zurichInstruments_Control.ziDevice.

Parameter variables (section "TEST" in the file)

Name	Initial / init()	Set by	Read by	Meaning
sourcePrefixSelection	None	nothing	nothing	Unused.
z_params_vars	{ } / { }	rpT.construct_runParamsTab (one tk.StringVar per MFIA parameter), hw.setup_gui_state	rpT._sync_param_values_to_device (via recast_param_type), rpT.connect_and_get_params, rpT.apply_and_push_params, rpT._capture_current_param_set, rpT._load_selected_param_set, rdT.check_device_parameters, ziC.ziDevice.set_param_value, recast_param_type, hw.connect_impedance	MFIA parameter name → variable holding the GUI string, for example '1 - On' or '0.300'.

Name	Initial / init ()	Set by	Read by	Meaning
z_params_force_push	{ } / { }	rpT.connect_and_get_params, rpT.apply_and_push_params, hw.connect_impedance	ziC.ziDevice.set_param_value, rdT.check_device_parameters, rpT.apply_and_push_params, hw.connect_impedance	MFIA parameter name → numeric value (output of recast_param_type). set_param_value() and reload_params() take values from here first.
z_param_inputField	None	rpT.construct_runParamsTab	rpT.construct_runParamsTab	MFIA parameter name → its ttk.Combobox or ttk.Entry.
t_params_vars	{ } / { }	rpT.construct_runParamsTab, hw.setup_gui_state	rpT._sync_param_values_to_device (via recast_param_type), rpT.connect_and_get_params, rpT.apply_and_push_params, rpT._capture_current_param_set, rpT._load_selected_param_set, rdT.check_device_parameters, recast_param_type, hw.connect_temperature	Temperature parameter name → tk.StringVar.
t_params_force_push	{ } / { }	rpT.connect_and_get_params, rpT.apply_and_push_params, hw.connect_temperature	rdT.check_device_parameters, rpT.apply_and_push_params (passed to tempDev.load_params), hw.connect_temperature	Temperature parameter name → number.
t_param_inputField	None	rpT.construct_runParamsTab	rpT.construct_runParamsTab	Temperature parameter name → ttk.Entry.
d_params_vars	{ } / { }	rpT.construct_runParamsTab, hw.setup_gui_state	rdT.check_device_parameters, recast_param_type, rpT.apply_and_push_params, rpT.browse_root_folder, rpT.export_h5_file, rpT._capture_current_param_set, rpT._load_selected_param_set	Output parameters: Number of Points (power of 2) (GUI default 16), Number of Reps (500), Data File Format ('TXT' or 'HDF5'), Data Root Folder ('', set by Browse...).
d_params_force_push	{ } (not touched by init ())	nothing	nothing	Unused.
d_param_inputField	None	rpT.construct_runParamsTab	rpT.construct_runParamsTab	Output parameter name → widget (Entry, format Combobox, or the Browse button).

Name	Initial / init ()	Set by	Read by	Meaning
root_data_folder	None	nothing	nothing	Unused; the folder is <code>d_params_vars['Data Root Folder']</code> .
param_history	None	<code>rpT._ensure_param_history_state([])</code> , <code>rpT._save_current_param_set</code>	<code>rpT._ensure_param_history_state</code> , <code>rpT._save_current_param_set</code> , <code>rpT._load_selected_param_set</code> , <code>rpT._update_param_history_field</code>	Up to 5 saved parameter sets, newest first.
param_history_labels	None	<code>rpT._ensure_param_history_state</code> , <code>rpT._update_param_history_field</code>	<code>rpT._ensure_param_history_state</code> , <code>rpT._load_selected_param_set</code> , <code>rpT._update_param_history_field</code>	Display labels of <code>param_history</code> .
param_history_selection	None	<code>rpT._ensure_param_history_state(tk.StringVar)</code>	<code>rpT._ensure_param_history_state</code> , <code>rpT._load_selected_param_set</code> , <code>rpT._update_param_history_field</code> , <code>rpT.construct_runParamsTab</code>	Selected history label.
param_history_inputField	None	<code>rpT.construct_runParamsTab</code>	<code>rpT._update_param_history_field</code> , <code>rpT.construct_runParamsTab</code>	History <code>ttk.Combobox</code> .
run_button	None	<code>ldT.construct_livePlotTab</code>	<code>ldT._set_run_control_buttons</code> , <code>ldT.construct_livePlotTab</code>	The Run DLTS button.

Run control (pause / resume / redo / retake)

Name	Initial / init ()	Set by	Read by	Meaning
run_dlts_Instance	None	<code>ldT.start_dlts</code> , <code>hw</code> (<code>hw_run.py</code>)	<code>ldT._handle_run_status</code> , <code>ldT._resume_run</code> , <code>ldT._redo_selected_steps</code> , <code>ldT._retake_selected_steps</code> , <code>ldT._run_control_thread</code> , <code>ldT._return_to_room_temp_async</code> , <code>ldT._refresh_step_listbox</code> , <code>ldT._set_run_control_buttons</code>	The <code>runDltsTools.dltsRun</code> of the current or paused run. Resume, Redo and Retake reuse it.

Name	Initial / init()	Set by	Read by	Meaning
run_busy	None / False	ldT.start_thread (True), ldT._run_control_thread (True), ldT._return_to_room_temp_async (True, then False in its callback), ldT._handle_run_status (False), ldT.start_dlts (callback, False on init failure)	ldT.start_thread, ldT._run_control_thread, ldT._pause_run, ldT._resume_run, ldT._redo_selected_steps, ldT._retake_selected_steps, ldT._poll_step_listbox_while_busy	True while a run thread or the room-temperature return is using the hardware.
run_pauseRequested	None / False	ldT._pause_run (True), ldT.start_thread and ldT._run_control_thread (False)	rdT.run_experiment	Pause button flag; checked between main-sequence steps.
run_paused	None / False	ldT._handle_run_status, ldT.start_thread	nothing	True once the run has stopped at a pause point.
run_stepStatus	None / {}	nothing	nothing	Unused. The step status is run_dltsInstance.stepStatus.
run_stepListbox	None	ldT._build_runControlPanel	ldT._refresh_step_listbox, ldT._get_selected_step_indices	tk.Listbox of grid steps and their status.
run_pauseButton	None	ldT._build_runControlPanel	ldT._set_run_control_buttons, ldT._pause_run	Pause button.
run_resumeButton	None	ldT._build_runControlPanel	ldT._set_run_control_buttons	Resume button.
run_redoButton	None	ldT._build_runControlPanel	ldT._set_run_control_buttons	Redo button.
run_retakeButton	None	ldT._build_runControlPanel	ldT._set_run_control_buttons	Remove & Retake button.

Name	Initial / init()	Set by	Read by	Meaning
run_abor_tReq_uested	None / False	main._send_room_ramp (True on window close), hw (hw_run.watchdog)	rdT.run_experiment, rdT._run_single_step, hw	Stop the run at the next check and discard a step whose acquisition overlapped the close. Never reset in the GUI.

App close (return to room temperature)

Name	Initial / init()	Set by	Read by	Meaning
app_closing	None / False	main.on_closing	main.on_closing, ldT._handle_run_status, ldT._return_to_room_temp_async (callback)	True once the window close was requested; blocks repeat clicks and new runs.
app_roomReturnSent	None / False	main._send_room_ramp	main._emergency_room_return	True once the room-temperature ramp was commanded on the way out, so the exit backstop does not send it again.
app_closeDialog	None	main._show_close_dialog	main.on_closing	The "returning to room temperature" Toplevel.
app_closeStatusLabel	None	main._show_close_dialog	main._show_close_dialog, main._wait_for_room_temp	Its live "Stage at X °C → room temperature Y °C" label.
app_closeNowRequested	None / False	main._close_now	main._close_now, main._wait_for_room_temp	"Close now" was clicked: stop waiting and leave the controller ramping.

Run file watch

Name	Initial / init()	Set by	Read by	Meaning
run_dataFolder	None	rdT.init_experiment	nothing	Folder of the current run (ends with \).
run_dataFileNames	None / []	rdT.init_experiment	ldT._schedule_live_poll, hw (hw_run.live_worker)	One file path per grid step; the live watcher waits for each to exist.
run_outputFileType	None	rdT.init_experiment	nothing	Lower-cased Data File Format of the current run.

Live plot (automated)

Name	Initial / init()	Set by	Read by	Meaning
livePlot_activeMode	None / 'live'	ldT._build_autoPlotFrame, ldT._set_livePlot_mode	ldT._active_impd, ldT._active_emission0Data, ldT._active_allEmissionsData, ldT._ingest_files_async, ldT._load_offline_run (callback), ldT._recompute_denoise_and_redraw, ldT._schedule_live_poll, ldT._set_livePlot_mode	'live' or 'offline': which dataset is drawn.
livePlot_modeVar	None	ldT._build_autoPlotFrame	ldT._on_mode_toggle, ldT._set_livePlot_mode	tk.StringVar of the Live/Offline radio buttons.
livePlot_denoiseMethodVar	None	ldT._build_autoPlotFrame	ldT._ingest_files_async, ldT._load_offline_run, ldT._recompute_denoise_and_redraw, ldT._redraw_auto_plots	tk.StringVar of the denoise dropdown.
livePlot_datasetVar	None	ldT._build_autoPlotFrame	ldT._get_selected_dataset_temp, ldT._redraw_auto_plots, ldT._reset_auto_plot_placeholder, ldT._set_livePlot_mode, ldT._update_dataset_dropdown	tk.StringVar of the dataset (temperature) dropdown.
livePlot_datasetCombo	None	ldT._build_autoPlotFrame	ldT._reset_auto_plot_placeholder, ldT._update_dataset_dropdown	The dataset dropdown widget.
livePlot_figure	None	ldT._build_autoPlotFrame	ldT._redraw_auto_plots	Matplotlib figure of the automated plots.
livePlot_axEmission0	None	ldT._build_autoPlotFrame	ldT._redraw_auto_plots, ldT._reset_auto_plot_placeholder	"Emission 0" axes.
livePlot_axAllEmissions	None	ldT._build_autoPlotFrame	ldT._redraw_auto_plots, ldT._reset_auto_plot_placeholder	"All Emissions Aligned" axes.
livePlot_canvas	None	ldT._build_autoPlotFrame	ldT._redraw_auto_plots, ldT._reset_auto_plot_placeholder	Tk canvas of the figure.
livePlot_statusLabel	None	ldT._build_autoPlotFrame	ldT._ingest_files_async, ldT._load_offline_run, ldT._schedule_live_poll, ldT._set_livePlot_mode	Status text under the plots.

Name	Initial / init()	Set by	Read by	Meaning
livePlot_liveImpdData	None	ldT._reset_live_plot_state, ldT._ingest_files_async (callback)	ldT._active_impd, ldT._ingest_files_async (worker), daT._calculate_rate_windows, daT._resolve_impd_for_source	The accumulating impedanceAnalysisTools.impdData of the current run.
livePlot_liveEmission0Data	None / {}	ldT._reset_live_plot_state, ldT._ingest_files_async (callback), ldT._recompute_denoise_and_redraw (callback)	ldT._active_emission0Data	Temperature → emission-0 block {x, y/ymean, yFiltered}.
livePlot_liveAllEmissionsData	None / {}	ldT._reset_live_plot_state, ldT._ingest_files_async (callback)	ldT._active_allEmissionsData, daT._calculate_rate_windows, daT._processed_transients_from_automated	Temperature → all emission blocks {x, y (2-D, one column per block)}.
livePlot_liveDatasetSel	None	ldT._reset_live_plot_state, ldT._set_livePlot_mode	ldT._set_livePlot_mode	Last dataset selected in Live mode.
livePlot_liveRunToken	None / 0	ldT._build_autoPlotFrame, ldT._reset_live_plot_state (increments)	ldT.start_thread, ldT._schedule_live_poll, ldT._ingest_files_async (callback), ldT._recompute_denoise_and_redraw	Run counter; a watcher or worker with an older token stops.
livePlot_pollAfterId	None / None	ldT._reset_live_plot_state, ldT._schedule_live_poll	ldT._reset_live_plot_state	root.after id of the live file poll, for cancellation.
livePlot_processedFiles	None / set()	ldT._build_autoPlotFrame, ldT._reset_live_plot_state; added to in ldT._ingest_files_async (callback)	ldT._schedule_live_poll, ldT._ingest_files_async	Data files already ingested into the live impdData.
livePlot_liveIngestBusy	None / False	ldT._build_autoPlotFrame, ldT._ingest_files_async, ldT._recompute_denoise_and_redraw, ldT._reset_live_plot_state	ldT._schedule_live_poll, ldT._recompute_denoise_and_redraw, daT._calculate_rate_windows	True while a live-ingest worker runs.
livePlot_offlineImpdData	None	ldT._reset_live_plot_state, ldT._load_offline_run (callback)	ldT._active_impd, daT._calculate_rate_windows, daT._resolve_impd_for_source	impdData loaded by Load Existing Run (Offline) .

Name	Initial / init()	Set by	Read by	Meaning
livePlot_offlineEmission0Data	None / {}	ldT._reset_live_plot_state, ldT._load_offline_run (callback), ldT._recompute_denoise_and_redraw (callback)	ldT._active_emission0Data	Offline counterpart of livePlot_liveEmission0Data.
livePlot_offlineAllEmissionsData	None / {}	ldT._reset_live_plot_state, ldT._load_offline_run (callback)	ldT._active_allEmissionsData, daT._calculate_rate_windows, daT._processed_transients_from_automated	Offline counterpart of livePlot_liveAllEmissionsData.
livePlot_offlineDatasetSel	None	ldT._reset_live_plot_state, ldT._set_livePlot_mode	ldT._set_livePlot_mode	Last dataset selected in Offline mode.
livePlot_offlineRunToken	None / 0	ldT._build_autoPlotFrame, ldT._reset_live_plot_state	ldT._load_offline_run, ldT._recompute_denoise_and_redraw	Offline load counter, as livePlot_liveRunToken.
livePlot_offlineIngestBusy	None / False	ldT._build_autoPlotFrame, ldT._load_offline_run, ldT._recompute_denoise_and_redraw, ldT._reset_live_plot_state	ldT._recompute_denoise_and_redraw, daT._calculate_rate_windows	True while an offline load worker runs.

Manual / qualitative analysis

Name	Initial / init()	Set by	Read by	Meaning
manual_dataDirectory	None	ldT._scan_manual_directory_async (callback)	ldT._browse_manual_folder, ldT._append_manual_folder	Last loaded source folder.
manual_datasetRegistry	None / {}	ldT._scan_manual_directory_async (callback)	ldT._append_manual_folder, ldT._process_raw_transients	Temperature → a file path (legacy file), ('legacy_chunk', file_path, chunk_id) or ('zi', data_file, chunk_id).
manual_ziMode	None	ldT._scan_manual_directory_async (callback)	the same callback	True, False or 'mixed': format of the most recently loaded data.
manual_ziParamsByFile	None / {}	ldT._scan_manual_directory_async (callback)	ldT._process_raw_transients	ZI data file → {'gridColOffset', 'gridColDelta', 'chunkSize'}.

Name	Initial / init ()	Set by	Read by	Meaning
manual_ziDataFile	None	ldT._scan_manual_directory_async (callback)	nothing	Most recently loaded ZI file (display only).
manual_ziGridColOffset	None	same	nothing	Its grid column offset.
manual_ziGridColDelta	None	same	nothing	Its grid column spacing.
manual_ziChunkSize	None	same	nothing	Its chunk size.
manual_sourceFolders	None / []	ldT._scan_manual_directory_async (callback)	the same callback	Folders combined into the registry so far.
manual_samplingRate	None	nothing	ldT._process_raw_transients	Always None; the reader falls back to 1.8666666666666665e-05 (a sample interval in s, despite the name).
manual_processedTransients	None / {}	ldT._build_manualPlotFrame, ldT._process_raw_transients (callback)	daT._calculate_rate_windows, daT._get_processed_transients_for_source	Temperature → {time_ms, avg_cap_pf, C_infinity} from Extract & Average Transients .
manual_paramVars	None / {}	ldT._build_manualPlotFrame	ldT._process_raw_transients, ldT._scan_manual_directory_async (callback)	tk.StringVar s fp_ms (Filling Duration, default '1.0'), rb_ms (Reverse Bias, '500.0'), slice_start ('2.0'), slice_end.
manual_tempListbox	None	ldT._build_manualPlotFrame	ldT._process_raw_transients, ldT._clear_manual_temps, ldT._select_all_manual_temps, ldT._scan_manual_directory_async (callback)	Temperature list box.
manual_folderLabel	None	ldT._build_manualPlotFrame	ldT._scan_manual_directory_async	Shows the loaded folder(s).
manual_selectFolderButton	None	ldT._build_manualPlotFrame	ldT._set_manual_buttons_state	Select Source Folder button.
manual_appendFolderButton	None	ldT._build_manualPlotFrame	ldT._set_manual_buttons_state	Append Source Folder button.

Name	Initial / init()	Set by	Read by	Meaning
manual_extractButton	None	ldT._build_manualPlotFrame	ldT._set_manual_buttons_state	Extract & Average Transients button.
manual_processingBusy	None	ldT._build_manualPlotFrame, ldT._process_raw_transients	ldT._process_raw_transients, ldT._browse_manual_folder, ldT._append_manual_folder, ldT._scan_manual_directory_async	True while the transient-extraction worker runs.
manual_loadingBusy	None	ldT._build_manualPlotFrame, ldT._scan_manual_directory_async	ldT._browse_manual_folder, ldT._append_manual_folder, ldT._process_raw_transients, ldT._scan_manual_directory_async	True while the folder-scan worker runs.
manual_transientExecutor	None	ldT._get_transient_executor	ldT._get_transient_executor, main._finish_close (shutdown)	Persistent ProcessPoolExecutor for transient extraction.
manual_figure	None	ldT._build_manualPlotFrame	ldT._process_raw_transients (callback)	Qualitative Analysis figure.
manual_axes	None	ldT._build_manualPlotFrame	ldT._process_raw_transients (callback)	Its axes.
manual_canvas	None	ldT._build_manualPlotFrame	ldT._process_raw_transients (callback)	Its Tk canvas.
manual_statusLabel	None	ldT._build_manualPlotFrame	ldT._process_raw_transients, ldT._scan_manual_directory_async	Status label.

Data analysis (rate window / Arrhenius)

Name	Initial / init()	Set by	Read by	Meaning
rateWindow_dataSourceVar	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	tk.StringVar: Qualitative Analysis / Automated Live / Automated Offline / Auto.
rateWindow_statusLabel	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Which data source was used and how many temperatures.
rateWindow_signalMethodVar	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Measured C (default) / Smoothed C.
rateWindow_denoiseVar	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	'None (raw)' (default) / pca / wavelet / sgolay / lowess.

Name	Initial / init()	Set by	Read by	Meaning
rateWindow_denoisedEmissions	None	daT._calculate_rate_windows	nothing	Last denoised emission snapshot, $T \rightarrow \{ 'x', 'yRaw', 'yFiltered', 'yerr', 'filterMethod' \}$.
rateWindow_peakMethodVar	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	'Smoothing Spline' (default) or lmfit curve fit.
rateWindow_windowVars	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	List of 5 (t1Var, t2Var) pairs.
rateWindow_tpLoVar	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Peak-search lower bound, K (blank = none).
rateWindow_tpHiVar	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Peak-search upper bound, K.
rateWindow_signals	None / {}	daT._build_rateWindowFrame, daT._calculate_rate_windows	daT._calculate_rate_windows	Window index $\rightarrow \{ 'T_k', 'Signal' \}$.
rateWindow_extractedPeaks	None / {}	daT._build_rateWindowFrame, daT._calculate_rate_windows	daT._calculate_rate_windows, daT._run_arrhenius_solver	Window index $\rightarrow \{ 'T_{peak}', 'S_{peak}', 'e_n', 'T_{peak_err}', 't1', 't2' \}$; skipped windows are left out.
rateWindow_transientS	None	daT._calculate_rate_windows	daT._run_arrhenius_solver	(records, temps, rb_ms): the averaged transients of the last Compute as Detailed Analysis records, for N_T; None when there are none.
rateWindow_peakTable	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	ttk.Treeview of per-window results.
rateWindow_figure	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Rate-window figure.
rateWindow_ax	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Its axes.
rateWindow_canvas	None	daT._build_rateWindowFrame	daT._calculate_rate_windows	Its canvas.
arrhenius_ndVar	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Background doping Nd, cm ⁻³ .
arrhenius_gammaVar	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Emission pre-factor γ , cm ⁻² s ⁻¹ K ⁻² .
arrhenius_energyLabel	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Activation energy result label.

Name	Initial / init()	Set by	Read by	Meaning
arrhenius_captureLabel	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Capture cross-section result label.
arrhenius_densityLabel	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Trap density result label.
arrhenius_figure	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Arrhenius figure.
arrhenius_ax	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Its axes.
arrhenius_canvas	None	daT._build_arrheniusFrame	daT._run_arrhenius_solver	Its canvas.

Detailed analysis

State and results:

Name	Initial / init()	Set by	Read by	Meaning
detailed_data	None / {}	deT._detailed_load_data (callback)	deT._detailed_run_analysis, deT._set_detailed_buttons_state	Temperature (°C) → (t_ms array, capacitance array in pF, C_infinity).
detailed_temps	None / []	deT._detailed_load_data (callback)	deT._detailed_run_analysis	Sorted temperatures, °C.
detailed_figure	None	deT._detailed_plot	deT._detailed_open_label_editor, deT._detailed_plot_rate_window_map, deT._detailed_plot_transient_map, deT._detailed_save_figure	Current figure, rebuilt each run.
detailed_canvas	None	deT._detailed_plot	deT._detailed_plot, deT._detailed_open_label_editor	Canvas embedding it.
detailed_figFrame	None	deT.construct_detailedAnalysisTab	deT._detailed_plot	Frame the canvas and toolbar go in.
detailed_ax1	None	deT._detailed_plot	deT._detailed_open_label_editor	Arrhenius panel.
detailed_ax2	None	deT._detailed_plot	deT._detailed_open_label_editor	DLTS spectra panel (optional).
detailed_ax3	None	deT._detailed_plot	deT._detailed_open_label_editor	2-D transient map (optional).

Name	Initial / init ()	Set by	Read by	Meaning
<code>detailed_ax4</code>	None	<code>deT._detailed_plot</code>	<code>deT._detailed_open_label_editor</code>	Rate-window analysis map (optional).
<code>detailed_leg1</code>	None	<code>deT._detailed_plot</code>	<code>deT._detailed_open_label_editor</code>	Legend.
<code>detailed_leg2</code>	None	<code>deT._detailed_plot</code>	<code>deT._detailed_open_label_editor</code>	Legend.
<code>detailed_legM</code>	None	<code>deT._detailed_plot</code>	<code>deT._detailed_open_label_editor</code>	Legend.
<code>detailed_legRW</code>	None	<code>deT._detailed_plot</code>	<code>deT._detailed_open_label_editor</code>	Legend of the rate-window map.
<code>detailed_annBox</code>	None	<code>deT._detailed_plot</code>	nothing through <code>dltsc</code>	Draggable Arrhenius results annotation.
<code>detailed_lastResMw</code>	None	<code>deT._detailed_run_analysis</code> (callback)	nothing	Last multi-window <code>compute_arrhenius()</code> result.
<code>detailed_lastResStd</code>	None	<code>deT._detailed_run_analysis</code> (callback)	nothing	Last standard-window result.
<code>detailed_lastNt</code>	None	<code>deT._detailed_run_analysis</code> (callback)	nothing	Last trap density.
<code>detailed_loadingBusy</code>	None / False	<code>deT._detailed_load_data</code> (and callback)	<code>deT._detailed_load_data</code> , <code>deT._detailed_run_analysis</code>	True while the load worker runs.
<code>detailed_ProcessingBusy</code>	None / False	<code>deT._detailed_run_analysis</code> (and callback)	<code>deT._detailed_load_data</code> , <code>deT._detailed_run_analysis</code>	True while the analysis worker runs. The name has no underscore after <code>detailed</code> .
<code>detailed_executor</code>	None / None	<code>deT._get_detailed_executor</code> , <code>deT._run_in_detailed_process</code>	<code>deT._get_detailed_executor</code> , <code>main._finish_close</code> (shutdown)	Single-worker <code>ProcessPoolExecutor</code> of this tab.
<code>detailed_hitTestStale</code>	None / True	<code>deT._detailed_plot</code> (and its inner <code>_mark_fresh</code> , <code>_mark_stale</code>)	<code>deT._detailed_install_drag</code> (inner <code>_on_press</code>)	True between a canvas resize and its redraw; the drag handler redraws only then.

Control variables, all created in `deT.construct_detailedAnalysisTab` and read by `deT._detailed_run_analysis` unless stated:

Name	Initial / init()	Set by	Read by	Meaning
detailed_baseVar	None	construct (StringVar(''))	deT._detailed_browse_folder, deT._detailed_load_data	Data folder.
detailed_gridOffsetVar	None	construct (DoubleVar(-0.001))	deT._detailed_load_data	ZI grid offset, s.
detailed_gridDtVar	None	construct (DoubleVar(1.86667e-5))	deT._detailed_load_data	ZI grid dt, s.
detailed_chunkSizeVar	None	construct (IntVar(32768))	deT._detailed_load_data	Samples per chunk.
detailed_reverseBiasMsVar	None	construct (DoubleVar(500.0))	deT._detailed_load_data, deT._detailed_run_analysis	Reverse-bias duration, ms.
detailed_c0WindowStartVar	None	construct (DoubleVar(0.40))	deT._detailed_load_data	C ₀ window start, fraction of RB.
detailed_c0WindowEndVar	None	construct (DoubleVar(0.90))	deT._detailed_load_data	C ₀ window end, fraction of RB.
detailed_gammaVar	None	construct (DoubleVar(1.66e21))	deT._detailed_run_analysis	γ , cm ⁻² s ⁻¹ K ⁻² (4H-SiC).
detailed_ndVar	None	construct (DoubleVar(3.2e14))	deT._detailed_run_analysis	Nd, cm ⁻³ .
detailed_peakSearchTMinVar	None	construct (DoubleVar(250.0))	deT._detailed_run_analysis	Peak search T min, K.
detailed_peakSearchTMaxVar	None	construct (DoubleVar(400.0))	deT._detailed_run_analysis	Peak search T max, K.
detailed_peakFinderMethodVar	None	construct ('Smoothing Spline (bootstrap)')	deT._detailed_run_analysis	Per-window peak finder: Imfit parabola or smoothing spline.
detailed_signalMethodVar	None	construct ('Measured C (nearest sample)')	deT._detailed_run_analysis	C(t) read-off: nearest sample or spline-interpolated.
detailed_denoiseMethodVar	None	construct ('None (raw)')	deT._detailed_run_analysis	Per-transient denoise method.
detailed_multiWindowsVar	None	construct (IntVar(16))	deT._detailed_run_analysis	Number of multi-windows.
detailed_t1MinVar	None	construct (DoubleVar(5.0))	deT._detailed_run_analysis	t ₁ min, ms.
detailed_t1MaxVar	None	construct (DoubleVar(90.0))	deT._detailed_run_analysis	t ₁ max, ms.

Name	Initial / init()	Set by	Read by	Meaning
detailed_ratioVar	None	construct (DoubleVar(5.0))	deT._detailed_run_analysis	Ratio t_2/t_1 .
detailed_stdWinsVar	None	construct (BooleanVar(True))	deT._detailed_run_analysis	Show the 5-window comparison.
detailed_stdEntries	None	construct (list)	deT._detailed_run_analysis	5 (t1Var, t2Var) DoubleVar pairs, defaults (5, 25), (10, 50), (20, 100), (50, 250), (100, 490) ms.
detailed_showSpectraVar	None	construct (BooleanVar(True))	deT._detailed_run_analysis	Show the DLTS spectra panel.
detailed_nSpectraVar	None	construct (IntVar(5))	deT._detailed_run_analysis	Number of spectra drawn.
detailed_showTmapVar	None	construct (BooleanVar(True))	deT._detailed_run_analysis	Show the 2-D transient map.
detailed_showTauVar	None	construct (BooleanVar(True))	deT._detailed_run_analysis	Overlay $\tau(T)$ on the map.
detailed_showRwmVar	None	construct (BooleanVar(True))	deT._detailed_run_analysis	Show the rate-window analysis map.

Widgets referenced outside their constructor:

Name	Initial / init()	Set by	Read by	Meaning
detailed_loadButton	None	deT.construct_detailedAnalysisTab	deT._set_detailed_buttons_state	Load Data button.
detailed_runButton	None	deT.construct_detailedAnalysisTab	deT._set_detailed_buttons_state	Run Analysis button.
detailed_loadInfoLabel	None	deT.construct_detailedAnalysisTab	deT._detailed_load_data (callback)	"No data loaded." / load summary.
detailed_statusLabel	None	deT.construct_detailedAnalysisTab	deT._detailed_load_data, deT._detailed_run_analysis, deT._detailed_save_figure, deT._detailed_export_txt	Status line.
detailed_resultsText	None	deT.construct_detailedAnalysisTab	deT._detailed_write_results, deT._detailed_export_txt	Results text box.

Module-level functions

init()

Declares all globals except `d_params_for_push` as `global` and resets a subset of them to "empty" values:

Global	Value set by init()
<code>z_params_vars</code> , <code>z_params_for_push</code> , <code>t_params_vars</code> , <code>t_params_for_push</code> , <code>d_params_vars</code>	<code>{}</code>
<code>run_busy</code> , <code>run_pauseRequested</code> , <code>run_paused</code> , <code>run_abortRequested</code>	<code>False</code>
<code>app_closing</code> , <code>app_roomReturnSent</code> , <code>app_closeNowRequested</code>	<code>False</code>
<code>run_stepStatus</code>	<code>{}</code>
<code>run_dataFileNames</code>	<code>[]</code>
<code>livePlot_activeMode</code>	<code>'live'</code>
<code>livePlot_liveRunToken</code> , <code>livePlot_offlineRunToken</code>	<code>0</code>
<code>livePlot_pollAfterId</code>	<code>None</code>
<code>livePlot_processedFiles</code>	<code>set()</code>
<code>livePlot_liveEmission0Data</code> , <code>livePlot_liveAllEmissionsData</code> , <code>livePlot_offlineEmission0Data</code> , <code>livePlot_offlineAllEmissionsData</code>	<code>{}</code>
<code>livePlot_liveIngestBusy</code> , <code>livePlot_offlineIngestBusy</code>	<code>False</code>
<code>manual_datasetRegistry</code> , <code>manual_ziParamsByFile</code> , <code>manual_processedTransients</code> , <code>manual_paramVars</code>	<code>{}</code>
<code>manual_sourceFolders</code>	<code>[]</code>
<code>rateWindow_signals</code> , <code>rateWindow_extractedPeaks</code>	<code>{}</code>
<code>detailed_data</code>	<code>{}</code>
<code>detailed_temps</code>	<code>[]</code>
<code>detailed_loadingBusy</code> , <code>detailedProcessingBusy</code>	<code>False</code>
<code>detailed_executor</code>	<code>None</code>
<code>detailed_hitTestStale</code>	<code>True</code>

Every other global keeps its current value.

No parameters.

Returns `None`.

Side effects Rebinds the globals above. It replaces the parameter-variable dicts, so calling it after the tabs are built disconnects `dltsConfig` from the GUI's `StringVars`. The GUI (`DLTSGUI_MainWindow`) never calls it; the tab modules instead guard against the `None` starting values. `benchmarks/hardware/hw_common.setup_gui_state` calls it before filling the parameter dicts.

Example

```
import dltsConfig as dltsC
dltsC.init() # script use: clean flags and empty dicts
assert dltsC.run_abortRequested is False
```

log_to_textbox(message)

Appends `message` and a newline to every text box in `textboxes` (or to `textbox` if `textboxes` is empty), and keeps at most `maxTextLineCount` lines by deleting the first line. When there is no text box (no GUI), it prints `message` to the console instead.

Name	Type	Default	Meaning
<code>message</code>	<code>str</code>	required	Text to log.

Returns `None`.

Side effects Inserts into Tk text widgets, increments `textlinecount`, or prints. It does not marshal onto the Tk thread: callers in worker threads (`runDlts_Tools`, `zurichInstruments_Control.pull_data`) touch the widget directly from those threads. With text boxes present but `textlinecount` or `maxTextLineCount` still `None`, it raises `TypeError`.

`hw_common.setup_gui_state` replaces the whole function with a logger that prints and appends to a file; every caller uses `dltsC.log_to_textbox(...)`, so the replacement takes effect everywhere.

Example

```
dltsC.log_to_textbox('Stage at 25.0 C') # prints when no GUI is running
```

recast_param_type(device, pname)

Reads `<group>_params_vars[pname].get()` and converts the GUI string to the value sent to the instrument.

Name	Type	Default	Meaning
<code>device</code>	<code>str</code>	required	'impDev' (reads <code>z_params_vars</code>), 'tempDev' (<code>t_params_vars</code>) or 'output' (<code>d_params_vars</code>).
<code>pname</code>	<code>str</code>	required	Parameter name.

Returns the converted value (`float` or `int`).

Side effects `None`. Raises `UnboundLocalError` when `device` is unknown, `pname` is not in the dict, or a dropdown string matches no label below (for example `'1'` instead of `'1 - On'`). Raises `ValueError` when `float()/int()` cannot parse the text. The entry must have a `.get()` method.

Numeric conversions:

Group	Parameter	Conversion
impDev	Oscillation Amplitude, Oscillation Frequency, Max bandwidth, Current Range, Voltage Range, Omega Suppression, State Enable Time, State Disable Time, Aux Output Scale, Aux Output Offset, Aux Output Lower Limit, Aux Output Upper Limit	<code>float()</code>

Group	Parameter	Conversion
impDev	Filter Harmonic, Filter Bandwidth, Data Transfer Rate	int()
tempDev	Initial Temperature (C), Final Temperature (C), Temperature Step (C), Temperature Ramp (C/min), Room Temperature (C), Room Ramp (C/min)	float()
tempDev	Stability Delay (s)	int() (so '0.5' raises ValueError)
output	Number of Points (power of 2), Number of Reps	int()

output entries Data File Format and Data Root Folder have no conversion and raise UnboundLocalError if passed.

Label → value mappings (impDev):

Parameter	Label	Value
Oscillation ON/OFF	1 - On	1
	0 - Off	0
Input Control	0 - Manual	0
	1 - Auto	1
	2 - Current Zone	2
Equivalent Circuit Mode	0 - 4-Terminal	0
	1 - 2-Terminal	1
Threshold Input Signal	59 - TU Output Value	59
	58 - Aux Output Overload	58
	56 - Aux Input Overload	56
	55 - Output Overload	55
	54 - Input(I) Overload	54
	53 - Input(V) Overload	53
	52 - Trigger Out	52
	51 - Trigger In	51
	50 - DIO	50
	3 - Demod Theta	3
Logic Unit Not	2 - Demod R	2
	1 - Demod Y	1
	0 - Demod X	0
	0 - Off	0
	1 - On	1

Parameter	Label	Value
Aux Output Signal	0 - Demod X	0
	1 - Demod Y	1
	2 - Demod R	2
	3 - Demod Theta	3
	11 - TU Filtered Value	11
	12 - Manual	12
	13 - TU Output Value	13
Signal Output Add	1 - True	1
	0 - False	0
Trigger Source Signal	0 - Off	0
	1 - Osc Phi Demod 2	1
	36 - Threshold 1	36
	37 - Threshold 2	37
	38 - Threshold 3	38
	39 - Threshold 4	39
	52 - MDS Sync Out	52

These labels are exactly the dropdown options in `runParamsTab.construct_runParamsTab()`. The `Filter Harmonic` (1–16) and `Filter Bandwidth` (1–8) dropdowns hold plain digits and go through `int()`.

Callers: `runParamsTab._sync_param_values_to_device` (every MFIA and temperature parameter, on **Connect + Get Params** and **Apply + Push Params**) and `runDltsTools.dltsRun._run_single_step` (`Number of Points` (power of 2), `Number of Reps`).

Example

```
class Var:
    def __init__(self, v): self.v = v
    def get(self): return self.v

dlts.z_params_vars = {'Trigger Source Signal': Var('36 - Threshold 1')}
dlts.recast_param_type('impDev', 'Trigger Source Signal') # 36
dlts.d_params_vars = {'Number of Reps': Var('100')}
dlts.recast_param_type('output', 'Number of Reps') # 100
```

Internal helpers

The module has no `_name` functions.

Notes and limitations

- The GUI never calls `init()`. Flags such as `run_busy`, `run_abortRequested` and `app_closing` start as `None` and work only because `None` is falsy; `livePlot_liveRunToken` and similar are guarded with `or 0` in `liveDataTab`.
- `init()` does not declare `d_params_for_push` as `global` and does not reset it (it is unused anyway).
- Unused globals: `sourcePrefixSelection`, `d_params_for_push`, `root_data_folder`, `run_stepStatus`. Written but never read: `run_dataFolder`, `run_outputFileType`, `run_paused`, `rateWindow_denoisedEmissions`, `manual_ziDataFile`, `manual_ziGridColOffset`, `manual_ziGridColDelta`, `manual_ziChunkSize`, `detailed_lastResMw`, `detailed_lastResStd`, `detailed_lastNt`, `postprocessingTab`.
- `manual_samplingRate` is read but never set, so `liveDataTab` always uses its fallback of `1.8666666666666665e-05` s.
- `detailedProcessingBusy` breaks the `detailed_` naming pattern.
- `log_to_textbox()` updates Tk widgets from worker threads, which Tkinter does not guarantee to be safe.
- The log keeps only 10 lines in the GUI (`maxTextLineCount = 10`).
- `recast_param_type()` has no `else` branches: unknown names or labels raise `UnboundLocalError` instead of a clear error. `Stability Delay (s)` accepts only whole seconds, while the instrument class stores it as a float.
- `sys.setswitchinterval(0.001)` runs on import and affects every thread of the process, including scripts that only use the instrument classes.

convert_json_to_h5

What it's for

`convert_json_to_h5.py` converts existing DLTS run folders from JSON step files (`p25p0.txt`, `n10p0.txt`, `p25p0.json`, ...) to HDF5 step files (`p25p0.h5`, ...). It is a batch tool you run from the command line. It does not talk to any instrument.

Each converted file is written with the same writer a run uses when **Data File Format** is `HDF5` (`zurichInstruments_Control1.ziDevice.writeDataH5()`). The GUI therefore reads a converted run exactly like a run that was recorded in HDF5. See [data_formats.md](#) for both file layouts.

What goes into each `.h5` file:

HDF5 content	Taken from
The 8 channel datasets	The JSON step file, value for value
<code>setpoint_C</code> attribute	The file name (<code>p25p0.txt</code> gives <code>25.0</code> , <code>n10p0.txt</code> gives <code>-10.0</code>)
<code>run_params</code> attribute	<code>runParams.txt</code> in the same folder (left out if that file is missing)
<code>acquired_at</code> attribute	The JSON file's modification time
<code>stage_temperature_C</code> attribute	Always <code>NaN</code> : JSON runs never recorded the measured stage temperature
<code>converted_from</code> attribute	The JSON file name, for example <code>p25p0.txt</code>
<code>format_version</code> attribute	<code>ziDevice.H5_FORMAT_VERSION</code> (currently 1)

Every `.h5` file is read back and compared value by value with its JSON source before the JSON file is touched. Only then is the JSON file moved, deleted, or left in place, depending on the options. If anything fails, the `.h5` file (and its `.tmp` file) is removed and the JSON file stays where it was.

By default a verified JSON file is moved into a `json_originals\` subfolder of the run folder. Leaving it next to the new `.h5` file would put two copies of the same temperature in the folder, and the folder scans in the analysis tabs would pick up both. `runParams.txt` is never moved, because the analysis code reads it from the run folder.

Command line

```
python convert_json_to_h5.py RUN_FOLDER [RUN_FOLDER ...]
python convert_json_to_h5.py --recursive DATA_ROOT_FOLDER
```

Option	Meaning
<code>folders</code> (positional, one or more)	Run folders to convert. With <code>--recursive</code> , folders to search below. Each must be an existing folder, or the script stops with a usage error (exit code 2).
<code>--recursive</code>	Also convert every folder below the given folders that directly holds at least one step file. <code>json_originals</code> folders are skipped.
<code>--delete-json</code>	Delete each verified JSON file instead of moving it to <code>json_originals\</code> . This is the only option that frees disk space right away.

Option	Meaning
<code>--keep-json</code>	Leave each verified JSON file where it is. Cannot be combined with <code>--delete-json</code> .
<code>--overwrite</code>	Reconvert steps that already have a <code>.h5</code> file. Without it such steps are skipped.
<code>--dry-run</code>	Only list what would be converted. Nothing is written, moved, or deleted.

The exit code is `1` if any file failed and `0` otherwise (also `0` when no step files were found).

Typical use:

```
python convert_json_to_h5.py --dry-run --recursive "C:\Users\spencer\Desktop\DATA\DLTS"
python convert_json_to_h5.py --recursive "C:\Users\spencer\Desktop\DATA\DLTS"
```

Real dry-run output for a JSON run folder (shortened):

```
C:\Users\spencer\Desktop\DATA\DLTS\091526\133851: 21 step file(s)
p100p0.json: would convert
p105p0.json: would convert
...
p95p0.json: would convert
Dry run: 21 file(s) (0.1 MB) would be converted, 0 skipped.
```

A real conversion prints one header line per folder, one line per skipped or failed file, and a summary. This is the real output for a temporary copy of the run folder `092526\114016` (three 32768-point `.txt` step files, no `runParams.txt`):

```
C:\Users\spencer\AppData\Local\Temp\convtest_run: 3 step file(s) (no runParams.txt: run_params not stored)
Converted 3 file(s): 21.0 MB of JSON -> 3.1 MB of HDF5. Skipped 0, failed 0.
Originals were moved to json_originals/ in each run folder; delete those folders to free the space.
```

Afterwards the folder held `p300p0.h5`, `p304p0.h5`, `p308p0.h5` (about 1.0 to 1.2 MB each) and `json_originals\p300p0.txt` etc. (about 7.3 MB each).

To get the disk space back after checking the converted runs in the GUI, delete the `json_originals` folders, or run with `--delete-json` in the first place.

Import

```
import convert_json_to_h5 as conv

stats = conv.convert_folder(r"C:\Users\spencer\Desktop\DATA\DLTS\091526\133851", jsonAction='keep')
print(stats)
```

Importing the module imports `zurichInstruments_Control`, which imports `zhinst.core`, `zhinst.toolkit`, `matplotlib` and `dltsConfig`. Those packages must be installed even though no instrument is used.

Module constants and globals

Name	Value / type	Meaning
STEP_FILE_PATTERN	<code>re.compile(r'^([npNP])(\d+)(?:[pP](\d+))?[cC](?:_\d+)?\.(txt json)\$')</code>	A JSON step file name: sign letter <code>p/n</code> , integer part, optional <code>p</code> + fraction digits, optional <code>C</code> , optional <code><n></code> suffix, extension <code>.txt</code> or <code>.json</code> . Matches <code>p25p0.txt</code> , <code>n10p0.txt</code> , <code>p120.txt</code> , <code>p25C.txt</code> , <code>p25p0_1.txt</code> , <code>p25p0.json</code> . Does not match <code>runParams.txt</code> , <code>notes.txt</code> , <code>.h5</code> or <code>.csv</code> files.
ORIGINALS_DIR	<code>'json_originals'</code>	Subfolder that receives verified JSON files in the default mode.
PARAMS_FILE	<code>'runParams.txt'</code>	Name of the run-parameter file read from each run folder.

Functions

`setpoint_from_name(fileName)`

The setpoint in °C encoded in a step file name.

Parameter	Type	Meaning
<code>fileName</code>	<code>str</code>	A bare file name, not a path.

Returns `float`, or `None` if the name does not match `STEP_FILE_PATTERN`. The value is `integer.fraction`, negated for an `n` prefix. The fraction digits are read as decimal digits: `p50p5` is `50.5`, `p25p05` is `25.05`.

Side effects None.

Example

```
>>> conv.setpoint_from_name('n10p0.txt')
-10.0
>>> conv.setpoint_from_name('p25p0.h5') is None
True
```

`convert_h5_to_text.py` also uses this function to refuse output names that look like step files.

`step_files(folder)`

The JSON step files directly inside one folder.

Parameter	Type	Meaning
<code>folder</code>	<code>str</code>	Folder to list.

Returns `list[str]` of bare file names that match `STEP_FILE_PATTERN` and are regular files, sorted as strings (so `p100p0.txt` comes before `p25p0.txt`).

Side effects None.

Example

```
names = conv.step_files(r"C:\data\091526\133851") # ['p100p0.json', 'p105p0.json', ...]
```

find_run_folders(roots, recursive)

The folders to convert.

Parameter	Type	Meaning
roots	iterable of str	Folders given on the command line.
recursive	bool	<code>False</code> : return <code>roots</code> unchanged. <code>True</code> : walk each root with <code>os.walk()</code> and return every folder (the root included) that directly holds at least one step file. Subfolders named <code>json_originals</code> are not entered. Subfolders are visited in sorted order.

Returns `list[str]`.

Side effects None.

Example

```
folders = conv.find_run_folders([r"C:\Users\spencer\Desktop\DATA\DLTS"], recursive=True)
```

Without `recursive`, a folder with no step files is still returned. It then prints `0 step file(s)` and converts nothing.

verify(jsonData, h5Path)

Check that an `.h5` file holds exactly the values of a JSON step file.

Parameter	Type	Meaning
jsonData	dict	The parsed JSON step file: channel name to list of values.
h5Path	str	The <code>.h5</code> file to check.

Returns `None` when everything matches.

Side effects Opens `h5Path` read-only. Raises `ValueError` if:

- the set of datasets differs from the set of JSON keys (`channels differ: ...`);
- a `uint64` dataset's JSON source has a negative value (`negative tick stamps can't be stored as uint64`);
- any channel's shape or values differ (`<name>: values changed on conversion`). `NaN` equals `NaN` for float datasets.

`uint64` datasets are cast to `int64` before the comparison, so the JSON integers compare directly.

Example

```
import json
with open('p25p0.txt') as f:
    conv.verify(json.load(f), 'p25p0.h5')
```

load_run_params(folder)

Read the folder's `runParams.txt`.

Parameter	Type	Meaning
folder	str	Run folder.

Returns the parsed JSON (normally a `dict`), or `None` if the file does not exist. A file that exists but is not valid JSON raises `json.JSONDecodeError`, which is not caught in `convert_folder()`.

Side effects None.

Example

```
params = conv.load_run_params(r"C:\data\092826\114655")
```

convert_file(dev, folder, name, runParams, jsonAction, overwrite, dryRun)

Convert one step file.

Parameter	Type	Meaning
dev	<code>zurichInstruments_Control.ziDevice</code>	Used only for its <code>writeDataH5()</code> method. It need not be connected.
folder	str	Run folder.
name	str	Step file name inside <code>folder</code> .
runParams	dict or None	Stored as the <code>run_params</code> attribute; <code>None</code> leaves the attribute out.
jsonAction	'move', 'delete' or 'keep'	What to do with the JSON file after verification.
overwrite	bool	Reconvert even if the <code>.h5</code> file exists.
dryRun	bool	Only report what would happen.

The output path is the JSON path with its extension replaced by `.h5`.

Returns a tuple (`status`, `message`, `bytesBefore`, `bytesAfter`):

status	When	message	Byte counts
'skipped'	The <code>.h5</code> exists and <code>overwrite</code> is <code>False</code>	'<name>.h5 already exists'	0, 0
'skipped'	<code>jsonAction == 'move'</code> and <code>json_originals\<name></code> already exists	'json_originals/<name> already exists'	0, 0
'converted'	<code>dryRun</code> is <code>True</code> (and neither skip applies)	'would convert'	JSON size, 0
'failed'	Reading, writing or verification raised	The exception text	0, 0

status	When	message	Byte counts
'converted'	Success	''	JSON size, .h5 size

The skip checks run before the dry-run check, so a dry run reports skips correctly. A dry run does not open the JSON files, so a file that would fail is still listed as `would convert`.

Side effects Writes `<stem>.h5` (through `<stem>.h5.tmp`). On failure, deletes `<stem>.h5` and `<stem>.h5.tmp` if present. On success, moves the JSON file to `json_originals\` (creating the folder), deletes it, or leaves it, as `jsonAction` says. A JSON file whose top level is not a non-empty object fails with `not a DLTS step file` (expected an object of channels).

Example

```
import zurichInstruments_Control as ziC
dev = ziC.ziDevice()
status, msg, before, after = conv.convert_file(dev, r"C:\data\run", 'p25p0.txt', None, 'keep', False, False)
```

convert_folder(folder, jsonAction='move', overwrite=False, dryRun=False, log=print)

Convert every step file in one run folder.

Parameter	Type	Meaning
folder	str	Run folder.
jsonAction	'move', 'delete' or 'keep'	See <code>convert_file()</code> .
overwrite	bool	See <code>convert_file()</code> .
dryRun	bool	See <code>convert_file()</code> .
log	callable taking one str	Receives the progress lines.

Returns dict with keys 'converted', 'skipped', 'failed' (counts) and 'bytesBefore', 'bytesAfter' (sums in bytes).

Side effects Creates one `ziDevice()` (no connection), reads `runParams.txt`, calls `convert_file()` for each name from `step_files()`. Logs `<folder>: <n> step file(s)`, with `(no runParams.txt: run_params not stored)` appended when that file is missing. In a dry run it logs every file; otherwise it logs only skipped and failed files.

Example

```
lines = []
stats = conv.convert_folder(r"C:\data\run", jsonAction='keep', log=lines.append)
```

main(argv=None)

The command-line entry point.

Parameter	Type	Meaning
argv	list[str] or None	Arguments without the program name. None uses <code>sys.argv[1:]</code> .

Returns `int` exit code: 1 if any file failed, else 0. Argument errors raise `SystemExit(2)` from `argparse`.

Side effects Everything `convert_folder()` does, for each folder from `find_run_folders()`, plus the summary printed to `stdout`.

Example

```
conv.main(['--dry-run', '--recursive', r"C:\Users\spencer\Desktop\DATA\DLTS"])
```

Internal helpers

None. All functions are listed above.

Notes and limitations

- Converting twice is safe. The second pass finds no JSON step files (they were moved) or skips every step whose `.h5` exists.
- `--overwrite` only helps while the JSON file is still in the run folder. After a default (`move`) conversion the JSON file is in `json_originals\`, so there is nothing left to reconvert. Move the files back first. With `--overwrite` and the default `move`, a step is still skipped if `json_originals\<name>` already exists.
- If a folder holds both `p25p0.txt` and `p25p0.json`, both map to `p25p0.h5`. The first one in sorted order (`p25p0.json`) is converted and the other is skipped as `p25p0.h5` already exists.
- `acquired_at` is the JSON file's modification time. If the file was copied or unzipped, that can be the copy time, not the acquisition time.
- Files in the older layout, directly in a date folder (for example `DLTS\062626\n10p0.txt`), convert fine. They have no `runParams.txt` next to them, so `run_params` is not stored.
- The step file pattern accepts `.json`, but the Live Tools folder scan (`liveDataTab._LEGACY_FILENAME_PATTERN`) accepts only `.txt`, `.csv` and `.h5`. Converting old `.json` runs to `.h5` makes them visible to that scan.
- A failure in one file does not stop the others. Check the summary line and the exit code.
- Tests: `tests/test_convert_json_to_h5.py` (see [other_scripts.md](#)).

convert_h5_to_text

What it's for

`convert_h5_to_text.py` exports one HDF5 step file (written by `zurichInstruments_Control.ziDevice.writeDataH5()`) to a human-readable file for inspection. Two output formats exist:

For mat	Content
<code>.txt</code>	# header lines with the file's root attributes (setpoint, measured stage temperature, acquisition time, run parameters), then a tab-separated table with one column per channel. Opens in Notepad, Excel or Origin.
<code>.json</code>	<code>{"source": ..., "attributes": {...}, "channels": {name: [values]}}</code> , indented by 2 spaces.

Values are written at full precision (Python `repr()` of each float, which round-trips exactly). Every export is read back and compared with the `.h5` data before it is kept. If the comparison fails, no output file is left behind.

The export is a copy for people to read. The analysis tabs do not read it. To keep it that way, the tool refuses output names that look like a step file (`p25p0.txt`, `n10p0.json`, ...), and the default output name is `<name>_export.txt` or `<name>_export.json`.

The same function is used by the **Export HDF5 File...** button on the Input Parameters tab (see [GUI button](#) below).

Command line

```
python convert_h5_to_text.py FILE.h5 [FILE.h5 ...] [--format txt|json]
python convert_h5_to_text.py FILE.h5 -o OUTPUT.txt
```

Option	Meaning
<code>files</code> (positional, one or more)	<code>.h5</code> step files to export.
<code>--format {txt,json}</code>	Output format. Default: taken from the <code>-o</code> extension (<code>.json</code> gives JSON, anything else gives the text table); without <code>-o</code> , <code>txt</code> .
<code>-o, --output OUTPUT</code>	Output file. Allowed only with a single input file; otherwise the script stops with <code>-o/--output</code> takes a single input file (exit code 2). Without <code>-o</code> , each file goes to <code><stem>_export.<format></code> next to its <code>.h5</code> file.

For each input file the script prints one line, either

```
<input> -> <output> (<rows> rows x <channels> channels, <size> MB)
```

or `<input>: failed: <reason>`. The exit code is `1` if any file failed, else `0`. A failure does not stop the remaining files.

Real example

The command below exports the 25 °C step of the run 092826\114655 (a 2¹⁸-point HDF5 run, 8.7 MB .h5 file):

```
python convert_h5_to_text.py "C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5" -o
"%TEMP%\p25p0_export.txt"
```

It printed:

```
C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5 ->
C:\Users\spencer\AppData\Local\Temp\p25p0_export.txt (262144 rows x 8 channels, 35.9 MB)
```

and took about 3 s. The first lines of the output file (lines longer than about 160 characters are cut here and marked ...):

```
# DLTS step exported from p25p0.h5
# acquired_at: 2026-09-28T11:48:02
# format_version: 1
# run_params: {"Oscillation Amplitude": 0.3, "Oscillation Frequency": 501000.0, "Oscillation ON/OFF": 1, "Max
bandwidth": 10000.0, "Input Control": 0, "Current ...
# setpoint_C: 25.0
# stage_temperature_C: 25.06325
# 262144 rows; columns are tab-separated, empty cells mean a shorter channel
timeStampImps   timeStampDemods tickStampImps   tickStampDemods ImpedanceRe      ImpedanceIm      AbsZ
AuxInput1
0.0      0.0      0.0      0.08808358165588324      5.1615895110309485e-06  0.08808358180711465
-0.4995596262122852
1.8666666666666665e-05  1.8666666666666665e-05  1.8666666666666665e-05  1.8666666666666665e-05
0.08546414090096481  5.091041104975873e-06  0.08546414105259975  -0.499 ...
3.7333333333333333e-05  3.7333333333333333e-05  3.7333333333333333e-05  3.7333333333333333e-05
0.08363013272873036  5.1797908965950025e-06  0.08363013288914045  -0.499503 ...
```

The text export is about 4 times larger than the .h5 file (35.9 MB against 8.7 MB here). In this triggered run all four time columns hold the same seconds axis; see [data_formats.md](#).

GUI button: Export HDF5 File...

The button sits next to **Apply + Push Params** in the output-parameter group of the **Input Parameters** tab (`runParamsTab.export_h5_file()`). It:

1. Opens a file dialog for a .h5 file, starting in the **Data Root Folder** if one is set.
2. Opens a save dialog in the same folder, with `<stem>_export.txt` as the suggested name. The file types are "Text table, tab-separated (.txt)" and "JSON (.json)". The format follows the extension of the name you save as.
3. Disables the button, runs `export_h5(h5Path, outPath)` on a background thread, then re-enables the button and writes one line to the log box: Exported <file> -> <output> (TXT, <rows> rows x <channels> channels, <size> MB). Or Export of <file> failed: <reason>.

The refusal of step-file-like names applies here too. Choosing `p25p0.txt` in the save dialog gives a "failed" log line, not a file.

Import

```
import convert_h5_to_text as h5txt

result = h5txt.export_h5(r"C:\data\092826\114655\p25p0.h5") # -> p25p0_export.txt
attrs, channels = h5txt.read_h5_file(r"C:\data\092826\114655\p25p0.h5")
```

The module needs only `h5py` and `numpy` (plus the standard library); it no longer imports `convert_json_to_h5` or any instrument module.

Module constants and globals

Name	Value / type	Meaning
FORMATS	('txt', 'json')	Accepted output formats.
_STEP_NAME	regex <code>^[npNP]\d+(?:[pP]\d+)?[cC]?(?:_\d+)?\.(txt json csv h5)\$</code> , case-insensitive	Output names refused because the analysis tabs would read them as run steps.
CHANNEL_ORDER	('timeStampImps', 'timeStampDemods', 'tickStampImps', 'tickStampDemods', 'ImpedanceRe', 'ImpedanceIm', 'AbsZ', 'AuxInput1')	Column order in exports: time axes first. HDF5 lists datasets alphabetically, so without this the table would start with <code>AbsZ</code> . Datasets not in this tuple follow, in alphabetical order.

Functions

read_h5_file(path)

Read one step file into plain Python values.

Parameter	Type	Meaning
path	str	.h5 step file.

Returns (attrs, channels):

- `attrs`: dict of the root attributes, in the order `h5py` lists them (alphabetical in the files the app writes). Numpy scalars become Python scalars (`.item()`), bytes are decoded as UTF-8, and `run_params` is parsed from its JSON string into a dict (left as a string if it is not valid JSON).
- `channels`: dict of channel name to numpy array in the file's own dtype (`uint64` tick stamps stay `uint64`), ordered by `CHANNEL_ORDER` and then alphabetically.

Side effects Opens the file read-only.

Example

```
attrs, ch = h5txt.read_h5_file('p25p0.h5')
attrs['setpoint_C'], attrs['run_params']['Number of Reps'], ch['ImpedanceIm'][:3]
```

Unlike `impedanceAnalysis_Tools.read_h5_record()`, this function also returns the attributes and does not cast `uint64` to `int64`.

default_output_path(h5Path, fmt='txt')

The default export name.

Parameter	Type	Meaning
<code>h5Path</code>	<code>str</code>	Input path.
<code>fmt</code>	<code>str</code>	'txt' or 'json'. Not checked.

Returns `<h5Path without extension>_export.<fmt>`, for example `C:\run\p25p0_export.txt`.

Side effects None.

Example

```
h5txt.default_output_path(r'C:\run\p25p0.h5', 'json') # 'C:\\run\\p25p0_export.json'
```

format_from_path(path)

Returns 'json' if the extension of `path` is `.json` (any case), else 'txt'.

Parameter	Type	Meaning
<code>path</code>	<code>str</code>	Output path.

Side effects None.

Example

```
h5txt.format_from_path('out.JSON') # 'json'
h5txt.format_from_path('out.csv')  # 'txt'
```

write_txt(path, attrs, channels, source)

Write the tab-separated text table.

Parameter	Type	Meaning
<code>path</code>	<code>str</code>	File to write (overwritten).
<code>attrs</code>	<code>dict</code>	Attributes from <code>read_h5_file()</code> .
<code>channels</code>	<code>dict</code>	Channels from <code>read_h5_file()</code> .
<code>source</code>	<code>str</code>	Name written in the first header line.

Layout, UTF-8 with `\n` line ends:

```
1. # DLTS step exported from <source>
```

2. One `# <key>: <value>` line per attribute. A `dict` value (the run parameters) is written as one line of JSON, with `NaN` as `null`. Other values: floats as `repr()` (`nan` for `NaN`), everything else as `str()`.
3. `# <rows> rows`; columns are tab-separated, empty cells mean a shorter channel, where `rows` is the length of the longest channel.
4. The header row of channel names, tab-separated.
5. One row per sample. Floats as `repr()` (`nan` for `NaN`), integers as `str()`. A channel shorter than the longest one gets empty cells at the end.

Returns `None`.

Side effects Writes `path`.

Example

```
attrs, ch = h5txt.read_h5_file('p25p0.h5')
h5txt.write_txt('p25p0_export.txt', attrs, ch, 'p25p0.h5')
```

read_txt(path)

Read the channel table back from a `write_txt()` file, for verification.

Parameter	Type	Meaning
<code>path</code>	<code>str</code>	A file written by <code>write_txt()</code> .

Returns `dict` of column name to `list[str]` of the non-empty cells. Lines starting with `#` are ignored; the first remaining line is the header row.

Side effects `None`.

Example

```
cols = h5txt.read_txt('p25p0_export.txt')
float(cols['ImpedanceIm'][0])
```

write_json(path, attrs, channels, source)

Write the JSON export: `{"source": source, "attributes": attrs, "channels": {name: [values]}}`, indented by 2, UTF-8, non-ASCII characters kept. Non-finite floats (`NaN`, `±inf`) become `null`, both in attributes and in channels.

Parameter	Type	Meaning
<code>path</code>	<code>str</code>	File to write (overwritten).
<code>attrs</code>	<code>dict</code>	Attributes from <code>read_h5_file()</code> .
<code>channels</code>	<code>dict</code>	Channels from <code>read_h5_file()</code> .
<code>source</code>	<code>str</code>	Stored under <code>"source"</code> .

Returns `None`.

Side effects Writes `path`.

Example

```
h5txt.write_json('p25p0_export.json', attrs, ch, 'p25p0.h5')
```

A step without a measured stage temperature (for example a file made by `convert_json_to_h5.py`) therefore has `"stage_temperature_C": null` in JSON and `# stage_temperature_C: nan` in the text table.

`verify(path, fmt, channels)`

Check that an export holds exactly the `.h5` channel values.

Parameter	Type	Meaning
<code>path</code>	<code>str</code>	The written export.
<code>fmt</code>	<code>'txt'</code> or <code>'json'</code>	How to read it back. Anything other than <code>'json'</code> is read as text.
<code>channels</code>	<code>dict</code>	The channels from <code>read_h5_file()</code> .

Returns `None` when everything matches.

Side effects Reads `path`. Raises `ValueError` if the channel names or their order differ, if a channel has a different number of values, or if any value differs. Integer channels are parsed with `int()` into the original dtype. Float channels are parsed with `float()` (`null` becomes NaN) and compared with NaN equal to NaN.

Example

```
h5txt.verify('p25p0_export.txt', 'txt', ch)
```

`export_h5(h5Path, outputPath=None, fmt=None)`

Export one file. This is what the command line and the GUI button call.

Parameter	Type	Meaning
<code>h5Path</code>	<code>str</code>	<code>.h5</code> step file.
<code>outPath</code>	<code>str</code> or <code>None</code>	Output file. <code>None</code> : <code>default_output_path(h5Path, fmt)</code> .
<code>fmt</code>	<code>'txt'</code> , <code>'json'</code> or <code>None</code>	<code>None</code> : <code>format_from_path(outPath)</code> if <code>outPath</code> is given, else <code>'txt'</code> .

Checks, in order, before anything is read:

1. `fmt` must be in `FORMATS`, else `ValueError: unknown format ...`.
2. If both `fmt` and an output ending in `.txt/.json` are given, they must agree, else `ValueError: format 'json' doesn't match the output extension of 'x.txt'`.
3. The output must not be the input file (compared by absolute path).
4. The output must not end in `.h5`, else `ValueError: the output must be a text file (.txt or .json), not .h5: that would replace data`.

5. The output's file name must not look like a step file (`_STEP_NAME: p25p0.txt, n10p0.json, p30p0.csv, p120C.h5, p25p0_1.txt`, any letter case). Otherwise: `ValueError: 'p25p0.txt' is named like a run's step file, so the analysis tabs would read it as data; choose another name, e.g. p25p0_export.txt`.

The export is then written to `<outPath>.tmp`, verified, and renamed to `outPath` with `os.replace()` (an existing file is overwritten). The `.tmp` file is removed in every case.

Returns `dict`:

Key	Meaning
'output'	Output path.
'format'	'txt' or 'json'.
'channels'	Number of channels.
'rows'	Length of the longest channel.
'bytes'	Size of the output file.

Side effects Writes `outPath`. Raises on any failure; no partial output is left.

Example

```
r = h5txt.export_h5(r'C:\run\p25p0.h5', fmt='json')
print(r['output'], r['rows'], r['bytes'])
```

main(argv=None)

The command-line entry point.

Parameter	Type	Meaning
argv	<code>list[str]</code> or <code>None</code>	Arguments without the program name. <code>None</code> uses <code>sys.argv[1:]</code> .

Returns `int`: `1` if any file failed, else `0`.

Side effects Calls `export_h5()` for each file and prints one line per file.

Example

```
h5txt.main([r'C:\run\p25p0.h5', r'C:\run\p30p0.h5', '--format', 'json'])
```

Internal helpers

_plain(value)

Makes a value JSON-safe: a non-finite Python `float` becomes `None`; `dict` and `list` are converted recursively; anything else is returned unchanged.

_values(arr)

`arr.tolist()`: a numpy array as Python ints or floats. `repr()` of those floats round-trips exactly.

_cell(value)

Text for one table cell or header value: `'nan'` for a NaN float, `repr(value)` for other floats, `str(value)` for everything else.

Notes and limitations

- The step-name refusal covers `.txt`, `.json`, `.csv` and `.h5` in any letter case, and no output may end in `.h5`, so an export can neither be taken for run data nor replace a step file. (Earlier versions only refused `.txt/.json` step names; fixed, with tests in `tests/test_convert_h5_to_text.py`.)
- `--format` must agree with a `.txt/.json` output extension. With another extension (for example `-o out.dat`), `--format` decides the content.
- The output folder must exist; it is not created.
- The whole step is held in memory, twice (arrays and Python lists). A 2^{18} -point step exports in a few seconds.
- `read_h5_file()` calls `.item()` on array-valued attributes too. The app writes only scalar and string attributes, so this is not a problem for its own files. An array attribute with more than one element would raise `ValueError`.
- Tests: `tests/test_convert_h5_to_text.py` (see [other_scripts.md](#)).

Data formats

This page describes the files the application writes and reads: the run folder layout, the step file names, `runParams.txt`, JSON step files, HDF5 step files, and the Zurich Instruments MFIA CSV exports that the analysis tabs also accept. The last section shows how to open HDF5 step files in HDFView, MATLAB and Python.

All examples come from real files on the lab PC unless marked otherwise. The main example run is

`C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\`, a 25 to 50 °C scan in 5 °C steps, 2^{18} points per step, 100 repetitions, written as HDF5.

Run folder layout

`runDlts_Tools.dltsRun.init_experiment()` creates one folder per run:

```
<Data Root Folder>\
  MMDDYY\           date the run started, e.g. 092826 = 28 Sep 2026
    HHMMSS\         time the run started, 24 h, e.g. 114655 = 11:46:55
      p25p0.h5       one step file per temperature setpoint
      p30p0.h5
      ...
      runParams.txt
```

The real example folder:

```
C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\
p25p0.h5      8,658,758 bytes
p30p0.h5      8,679,853 bytes
p35p0.h5      8,677,551 bytes
p40p0.h5      8,671,607 bytes
p45p0.h5      8,676,121 bytes
p50p0.h5      8,694,667 bytes
runParams.txt  1,148 bytes
```

Details from the code:

- **Data Root Folder** comes from the Input Parameters tab. The run folder is `os.path.join(root, MMDDYY, HHMMSS)` (date and time from one `datetime.now()` call) and is created if missing.
- Data Root Folder is empty by default. A run with it empty, or set to a relative path, does not start: `init_experiment()` logs Error: Data Root Folder is not set and the GUI logs Error: Failed to initialize the experiment. Earlier versions wrote such a run to `\MMDDYY\HHMMSS\` at the root of the current drive, and older run folders have a doubled backslash in their recorded paths.
- The file extension follows **Data File Format**: HDF5 gives `.h5`, TXT gives `.txt`. The code also maps JSON (and any unknown value) to `.json`, but the GUI dropdown offers only TXT and HDF5. TXT and JSON both write JSON content.
- Step files are written one at a time as each temperature finishes. Redo overwrites a step's file; Remove & Retake deletes it first and then writes it again.

- `runParams.txt` is written by `finish_experiment()`, which runs only when the main sequence completes. A run that stops with an error, or is aborted by closing the GUI, has step files but no `runParams.txt`. HDF5 step files still carry the parameters in their `run_params` attribute.

Older data on the lab PC also has other layouts, for example step files directly in a date folder (`DLTS\062626\n10p0.txt`) or `.json` step files (`DLTS\091526\133851\p25p0.json`). Folders converted with `convert_json_to_h5.py` also hold a `json_originals\` subfolder with the original JSON files.

Step file names

`init_experiment()` builds each name from the setpoint `T` in `tempGrid`:

```
prefix = 'n' if '-' in str(T) else 'p'
name = prefix + str(np.abs(T)).replace('.', 'p') + ext
```

Setpoint (°C)	File name
25.0	p25p0.h5
-10.0	n10p0.h5
50.5	p50p5.h5
300.0	p300p0.txt (real, DLTS\092526\114016)

The name is built from `str()` of the number. `instecTempStage_Control.build_temp_grid()` returns a float numpy array (`Tinit + step * np.arange(n)`), so whole-degree setpoints give `p25p0`. Fractional steps can give floating-point artifacts in the name. Checked with the same expression: a 0 to 1 °C grid in 0.1 °C steps gives `p0p30000000000000000004`, `p0p60000000000000000001`, `p0p70000000000000000001`, and a grid starting at -1 °C gives `n0p3999999999999999` and similar. The readers still parse these names (the fraction is just a long digit string).

The readers accept a wider pattern. `LiveDataTab._LEGACY_FILENAME_PATTERN`, used by the Qualitative Analysis folder scan in the Live Tools tab and by the Detailed Analysis tab:

```
re.compile(r'^([npNP])(\d+)(?:[pP](\d+))?[cC]?(?:_\d+)?\.(txt|csv|h5)$')
```

Part	Meaning
<code>[npNP]</code>	<code>p</code> = positive, <code>n</code> = negative
<code>(\d+)</code>	integer part
<code>(?:[pP](\d+))?</code>	optional <code>p</code> + fraction digits (<code>p5</code> = .5)
<code>[cC]?</code>	optional <code>C</code>
<code>(?:_\d+)?</code>	optional <code><n></code> suffix, ignored
<code>(txt csv h5)</code>	extension

`convert_json_to_h5.STEP_FILE_PATTERN` is the same pattern with the extensions `txt|json` instead. Note that `.json` step files are not picked up by the Live Tools / Detailed Analysis folder scan.

`impedanceAnalysis_Tools.impdData.read_data()` (Live Tools, Offline Run) applies the same name rule (`_STEP_STEM_PATTERN`, which also accepts a name without the sign letter as positive) and keys each file by its exact setpoint in kelvin, `T_C + 273.15`: `p25p0` gives 298.15, `p50p5` 323.65, `n10p5` 262.65, `p25C` 298.15. Earlier versions truncated to whole degrees and added 273 (`p50p5` gave 323), and failed on names with a `C`.

runParams.txt

A JSON object with every run parameter: the MFIA parameters, then the temperature-controller parameters, then the output parameters (`dltsRun._current_run_params()`). It is written with `ziDevice.writeDataJson()` (`json.dump(..., indent=4)`), despite the `.txt` extension. The real file from `092826\114655`:

```
{
  "Oscillation Amplitude": 0.3,
  "Oscillation Frequency": 501000.0,
  "Oscillation ON/OFF": 1,
  "Max bandwidth": 10000.0,
  "Input Control": 0,
  "Current Range": 0.01,
  "Voltage Range": 3.0,
  "Omega Suppression": 80.0,
  "Filter Harmonic": 1,
  "Filter Bandwidth": 2,
  "Data Transfer Rate": 60000,
  "Equivalent Circuit Mode": 0,
  "Threshold Input Signal": 59,
  "State Enable Time": 0.006,
  "State Disable Time": 0.003,
  "Logic Unit Not": 1,
  "Aux Output Signal": 13,
  "Aux Output Scale": -1.0,
  "Aux Output Offset": -0.5,
  "Aux Output Lower Limit": -10.0,
  "Aux Output Upper Limit": 0.0,
  "Signal Output Add": 1,
  "Trigger Source Signal": 36,
  "Initial Temperature (C)": 25.0,
  "Final Temperature (C)": 50.0,
  "Temperature Step (C)": 5.0,
  "Temperature Ramp (C/min)": 5.0,
  "Stability Delay (s)": 0,
  "Room Temperature (C)": 30.0,
  "Room Ramp (C/min)": 10.0,
  "Number of Points (power of 2)": 18,
  "Number of Reps": 100,
  "Data File Format": "HDF5",
  "Data Root Folder": "C:\\Users\\spencer\\Desktop\\DATA\\DLTS"
}
```

The meaning and units of each parameter are described in [runParamsTab.md](#). Two are used by the readers:

- `State Enable Time` (s) is the reverse-bias duration and `State Disable Time` (s) the fill-pulse duration. `liveDataTab._legacy_run_timing()` reads them to preset the Qualitative Analysis timing fields (here 6 ms and 3 ms).
- `Number of Points (power of 2)` sets the samples per step: $2^{18} = 262144$ here.

`impdData.read_data()` looks for `runParams.txt` in the folder of the first selected file and prints a warning if it is missing.

The 8 channels

Every step file, JSON or HDF5, holds the same 8 channels. They come from `zurichInstruments_Control.ziDevice.pull_data()`. Runs always call it with `trigger=True`: a triggered acquisition with the MFIA DAQ module in grid mode, Number of Points columns and Number of Reps repetitions, subscribed to the averaged (`.avg`) sample nodes.

Channel	Source in a run (trigger=True)	Units
<code>timeStampImps</code>	Copy of <code>tickStampImps</code>	s
<code>timeStampDemods</code>	Copy of <code>tickStampDemods</code>	s
<code>tickStampImps</code>	The DAQ grid's time axis, taken from the <code>/dev32271/imps/0/sample.param1.avg</code> result	s (float), despite the name
<code>tickStampDemods</code>	The same <code>param1.avg</code> time axis as <code>tickStampImps</code>	s (float), despite the name
<code>ImpedanceRe</code>	<code>/dev32271/imps/0/sample.param0.avg</code> (impedance parameter 0)	see below
<code>ImpedanceIm</code>	<code>/dev32271/imps/0/sample.param1.avg</code> (impedance parameter 1)	see below
<code>AbsZ</code>	Computed as <code>sqrt(ImpedanceRe**2 + ImpedanceIm**2)</code>	see below
<code>AuxInput1</code>	<code>/dev32271/demods/0/sample.auxin0.avg</code> (Aux Input 1)	V

In a triggered run all four time channels are therefore identical. In the example file they run from `0.0` to `4.893336` s in steps of `1.8666666666666665e-05` s (about 53.6 kSa/s), 262144 samples.

The comments in `impedanceAnalysis_Tools.impdData.read_data()` give the intended meaning:

Channel	Meaning per <code>read_data()</code> comments
<code>tickStampImps</code> , <code>tickStampDemods</code>	Impedance / demodulator time stamps in hardware clock ticks
<code>timeStampImps</code> , <code>timeStampDemods</code>	Impedance / demodulator time stamps in seconds
<code>ImpedanceRe</code>	Real part of the impedance, Ω
<code>ImpedanceIm</code>	Imaginary part of the impedance, or capacitance, F
<code>AuxInput1</code>	Demodulation signal or excitation (the fill-pulse / reverse-bias waveform), V
<code>AbsZ</code>	Absolute value of the impedance, Ω

What `param0` and `param1` actually are depends on how the MFIA impedance module's display parameters are set on the instrument. This software does not set them. The analysis code treats `ImpedanceIm` as capacitance in farads (it multiplies by `1e12` to get pF). In older runs `ImpedanceIm` is about `2.4e-10` (240 pF, `DLTS\062626\n10p0.txt`); in the example HDF5 run it is about `5e-6`. Because `AbsZ` is computed from `param0` and `param1`, it is the magnitude of the impedance only if those two are the real and imaginary parts. With `param1` a capacitance, `AbsZ` is effectively `abs(ImpedanceRe)` (see the export excerpt in [convert_h5_to_text.md](#)).

With `trigger=False` (not used by runs), `pull_data()` polls the device instead: the tick stamps are then the integer 60 MHz clock ticks, the time stamps are `ticks / 60e6` minus the first value, `AbsZ` is the device's own `abs(z)`, and the Imps and Demods channels can differ in length.

JSON step files (.txt, .json)

Written by `ziDevice.writeDataJson(data, fName)`: one JSON object, `indent=4`, numpy arrays converted to lists. The parent folder is created if needed. The keys are the 8 channels in the order `pull_data()` created them:

```
{
  "tickStampImps": [
    0.0,
    1.8666666666666665e-05,
    3.733333333333333e-05,
    ...
  ],
  "tickStampDemods": [ ... ],
  "timeStampImps": [ ... ],
  "timeStampDemods": [ ... ],
  "ImpedanceRe": [ ... ],
  "ImpedanceIm": [ ... ],
  "AuxInput1": [ ... ],
  "AbsZ": [ ... ]
}
```

(Start of the real file `DLTS\091526\133851\p25p0.json`, a 16-point test run. All values are floats.)

Properties:

- One value per line, so the files are large: `DLTS\062626\n10p0.txt` (131072 points) is 28.9 MB. The same data in HDF5 is about 8 times smaller (see `benchmarks/bench_results.json`).
- There is no metadata in the file: no setpoint (only the file name), no measured stage temperature, no time stamp (only the file modification time). The run parameters are only in `runParams.txt`.
- The file is written in place, not through a temporary file.

`convert_json_to_h5.py` converts these files to HDF5 without loss.

HDF5 step files (.h5)

Written by `ziDevice.writeDataH5(data, fName, setpoint_C=None, stage_temperature_C=None, runParams=None, acquired_at=None, extraAttrs=None)`, one file per temperature step.

Datasets

One 1-D dataset per channel, at the root of the file, named like the JSON keys.

Channel	dtype
<code>tickStampImps</code> , <code>tickStampDemods</code>	<code>uint64</code> if the values are integers (clock ticks); <code>float64</code> if they are floats, as in every triggered run

Channel	dtype
all other channels	float64

Every non-empty dataset is chunked and compressed with gzip level 4 plus the shuffle filter. An empty channel is stored as an empty, uncompressed dataset (gzip needs a chunked layout, which an empty dataset cannot have). The chunk size is chosen by h5py; in the example file it is 2048 elements.

Root attributes

Attribute	Type	Meaning
format_version	int64	Layout version, <code>ziDevice.H5_FORMAT_VERSION</code> , currently 1. Bumped whenever the layout changes.
acquired_at	string	Local time the file was written (or <code>acquired_at</code> if given), ISO 8601 to the second, no time zone, e.g. 2026-09-28T11:48:02.
setpoint_C	float64	Temperature setpoint in °C, or NaN if not given.
stage_temperature_C	float64	Measured stage temperature in °C: the mean of one controller reading before and one after the acquisition (either one if the other failed), or NaN if neither could be read.
run_params	string	<code>json.dumps()</code> of the run parameters (the same dictionary as <code>runParams.txt</code> , on one line). Absent if no parameters were passed.
converted_from	string	Only in files made by <code>convert_json_to_h5.py</code> : the name of the source JSON file.

Writing and crash safety

The file is first written as `<name>.h5.tmp` and then renamed to `<name>.h5` with `os.replace()`. The live watcher waits for `<name>.h5` to appear, so it never sees a half-written file. If the target is locked (for example the watcher is reading the previous version during a Redo), the rename is retried 10 times, 0.5 s apart, before the `PermissionError` is raised. A crash can only cost the step being written.

Real example: 092826\114655\p25p0.h5

Inspected with h5py:

```
import h5py
with h5py.File(r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5", "r") as f:
    for k, v in f.attrs.items():
        print(k, repr(v)[:100])
    for k in f:
        d = f[k]
        print(k, d.shape, d.dtype, d.chunks, d.compression, d.compression_opts, d.shuffle)
```

Output (the `run_params` string is cut):

```

acquired_at '2026-09-28T11:48:02'
format_version np.int64(1)
run_params '{"Oscillation Amplitude": 0.3, "Oscillation Frequency": 501000.0, "Oscillation ON/OFF": 1, "Max ...
setpoint_C np.float64(25.0)
stage_temperature_C np.float64(25.06325)
AbsZ (262144,) float64 (2048,) gzip 4 True
AuxInput1 (262144,) float64 (2048,) gzip 4 True
ImpedanceIm (262144,) float64 (2048,) gzip 4 True
ImpedanceRe (262144,) float64 (2048,) gzip 4 True
tickStampDemods (262144,) float64 (2048,) gzip 4 True
tickStampImps (262144,) float64 (2048,) gzip 4 True
timeStampDemods (262144,) float64 (2048,) gzip 4 True
timeStampImps (262144,) float64 (2048,) gzip 4 True

```

Value ranges in this file:

Channel	First value	Min	Max
AbsZ	0.08808	0.000867	3.037
AuxInput1	-0.49956	-1.50072	-0.49919
ImpedanceIm	5.1616e-06	4.5715e-06	5.6927e-06
ImpedanceRe	0.08808	-1.345	3.037
all four time channels	0.0	0.0	4.893336

AuxInput1 switches between about -0.5 V and -1.5 V: the GUI's default Aux Output Offset (-0.5 V) and Scale (-1 V). The tick-stamp channels are `float64` because this was a triggered run.

To get a readable copy of a step file, use `convert_h5_to_text.py` or **Export HDF5 File...** on the Input Parameters tab.

Zurich Instruments MFIA CSV exports

The Qualitative Analysis section of the Live Tools tab and the Detailed Analysis tab also load data exported with the Zurich Instruments LabOne software instead of this application. When you pick a folder, both tabs (`liveDataTab._scan_manual_directory_async()` and `detailedAnalysisTab._load_detailed_data()`) try these formats in order and use the first that matches:

1. single combined ZI export,
2. subfolder-per-temperature ZI export,
3. legacy per-temperature files (JSON `.txt`, `.h5`, or `.csv`).

No ZI CSV export is present in the lab data folder, so the descriptions below come from the reader code only.

1. Single combined ZI export

The folder directly holds:

File (regex, case-insensitive)	Content
<code>...imps_0_sample_param1_avg_header... ending .csv</code>	Header table, ; -separated, one row per chunk. Columns used: <code>chunk_number</code> , <code>history_name</code> , and from the first row <code>grid_col_offset</code> (s), <code>grid_col_delta</code> (s), <code>chunk_size</code> .
<code>...imps_0_sample_param1_avg_<digits>.csv (without header)</code>	Data table, ; -separated. Columns used: <code>chunk</code> and <code>value</code> .

Each header row whose `history_name` starts with `<digits>C_` (for example `25C_...`) becomes one temperature, mapped to its `chunk_number`. Only positive whole-degree names are recognized; there is no `n` prefix in this format. The first data file found is used for all chunks.

For each temperature, the `value` rows of its chunk (up to `chunk_size`) form the averaged transient. The time axis is $(\text{grid_col_offset} + k * \text{grid_col_delta}) * 1000$ ms, so $t = 0$ is the start of the reverse bias when `grid_col_offset` is minus the fill-pulse length. Defaults when the header cannot be read: `grid_col_offset` = `-0.001`, `grid_col_delta` = `1.86667e-05`, `chunk_size` = `32768`. If the largest absolute value is below $1e-3$, the values are taken as farads and multiplied by $1e12$ (pF).

The folder name can preset the timing fields: `FP...<number>ms` sets the fill pulse and `RB...<number>ms` the reverse bias, for example `..._FP0V1ms_RB-5V500ms_...`.

2. Subfolder-per-temperature ZI export

The folder holds one subfolder per temperature, named like `0C`, `100C`, `n10C` (negative) or `120C_000`, matching `^(n?)(\d+)C(?:_\d+)?$`. Each subfolder holds its own `...imps_0_sample_param1_avg_<digits>.csv` data file and optionally its own `...header...csv`. The data file is read with columns `chunk;timestamp;value`, and only chunk `0` is used. Grid parameters come from that subfolder's header (same defaults as above). This is the convention of the standalone `DLTS_APP.py`, which the Detailed Analysis tab was ported from. `DLTS_APP.py` itself only looks for the exact name `dev32271imps_0_sample_param1_avg_00000.csv` and skips subfolders with fewer than 100 rows.

3. Legacy CSV files

In the legacy branch, each `.csv` file whose name matches `_LEGACY_FILENAME_PATTERN` is classified by its first line:

First line	Treatment
Contains <code>;</code> and <code>chunk</code>	Chunked ZI-style CSV. With more than one chunk (or <code>p120C-p160C</code> in the name) the chunks are mapped to fixed temperatures: <code>chunk 0..8</code> = 120, 125, ..., 160 °C. With one chunk, the temperature comes from the file name. The last column whose name contains <code>value</code> , <code>smoothed</code> , <code>cap</code> or <code>impedance</code> is the transient; F is converted to pF as above; the time axis is <code>k</code> times the sampling interval (<code>dltsc.manual_samplingRate</code> , default <code>1.8666666666666665e-05</code> s; Detailed Analysis always uses the default).
Contains <code>;</code> and <code>smoothed_value</code> or <code>timestamp</code>	Same, without chunk filtering.
Anything else	Read as a plain comma-separated table: column 0 = time in ms, column 1 = capacitance, used as is (no unit conversion).

The chunk-to-temperature map for multi-chunk files is fixed in the code (from `DrKayisScript.py`); it is correct only for the one data set it was written for.

CSV input to `impdData.read_data()`

`impedanceAnalysis_Tools.impdData.read_data()` (Live Tools, Offline Run) has a separate CSV reader for a set of LabOne export files selected together. Files are classified by substrings of their names: `_imps_0` = capacitance (header files contain `_header_`), `_auxin0` = excitation, `_r_avg` = demodulator R. Missing files are asked for in a file dialog. Temperatures come from column 16 of the capacitance header (names like `p25C_...` or `n10C_...`). Data files are `;-` separated with `chunk`, `timestamp` and `value` columns; `timestamp` is taken as 60 MHz clock ticks. `ImpedanceRe` is filled with zeros.

Opening HDF5 step files

HDFView

Open the `.h5` file with **File > Open**. The tree shows the 8 datasets under the root group `/`. Select `/` to see the root attributes (setpoint, stage temperature, acquisition time, `run_params`). Double-click a dataset to see its values in a table. `gzip` and `shuffle` are standard HDF5 filters, so HDFView reads the files without plugins.

Do not open a step file while a run is writing to the same folder, and never save changes from HDFView into a run's files.

MATLAB

```
f = 'C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5';

h5disp(f)                                % layout and attributes
cap = h5read(f, '/ImpedanceIm');          % column vector, double
t    = h5read(f, '/timeStampImps');       % seconds
aux  = h5read(f, '/AuxInput1');           % volts
Tset = h5readatt(f, '/', 'setpoint_C');
Tmea = h5readatt(f, '/', 'stage_temperature_C'); % NaN if not measured
p    = jsondecode(h5readatt(f, '/', 'run_params'));
```

`jsondecode` turns the parameter names into valid MATLAB field names (spaces and brackets are removed), so use `fieldnames(p)` to see them. If a file stores integer tick stamps, `h5read` returns them as `uint64`; convert with `double()` before arithmetic, or subtract the first value first to keep precision.

Python

With the application's own reader:

```
import os
os.environ.setdefault("LOKY_MAX_CPU_COUNT", "8")
import impedanceAnalysis_Tools as iaT

rec = iaT.read_h5_record(r"C:\Users\spencer\Desktop\DATA\DLTS\092826\114655\p25p0.h5")
rec["ImpedanceIm"][:5]
rec = iaT.read_h5_record(path, keys=("AuxInput1", "ImpedanceIm")) # only two channels
```

`read_h5_record(file_path, keys=None)` returns the same `{channel: values}` dictionary a JSON step file gives, with numpy arrays instead of lists. `uint64` tick stamps are returned as `int64`, the dtype numpy gives JSON integers, because `cleanup_data()` takes `np.diff()` of them. It does not return the attributes. Importing `impedanceAnalysis_Tools` also imports `zhinst`, `torch`, `sklearn`, `lmfit` and other heavy packages.

With h5py only, including the attributes:

```
import json, h5py

with h5py.File(path, "r") as f:
    cap = f["ImpedanceIm"][()]
    setpoint = f.attrs["setpoint_C"]
    params = json.loads(f.attrs["run_params"])
```

`convert_h5_to_text.read_h5_file(path)` returns both the attributes (with `run_params` decoded) and the channels.

Other scripts

This page covers the files in the repository that are not part of the GUI tabs or the instrument/analysis libraries:

File	What it is
<code>runDLTS.py</code>	Old command-line run script. Does not work with the current <code>runDlts_Tools</code> .
<code>DLTS_APP.py</code>	Standalone tkinter app "DLTS Multiwindow Analysis". Ported into the Detailed Analysis tab.
<code>DrKayisScript.py</code>	Standalone PyQt6 app "Offline DLTS Core Transient & Defect Signature Suite". Ported into the Qualitative Analysis section of the Live Tools tab and the Quick Analysis tab.
<code>scrapwork.py</code>	Scratch script for trying MFIA DAQ acquisitions. Talks to the instrument when run.
<code>postprocessingTab.py</code>	Empty file (0 bytes). Placeholder; imported by nothing.
<code>tests/</code>	pytest test suite (no hardware needed).
<code>benchmarks/benchmark_hdf5.py</code>	Synthetic HDF5 vs JSON benchmark (no hardware).
<code>benchmarks/hardware/</code>	Real-hardware HDF5 benchmark. Moves the stage and biases the sample.

The standard way to run a measurement is the GUI (`DLTSGUI_MainWindow.py`, see [DLTSGUI_MainWindow.md](#)).

runDLTS.py

What it's for

An early entry point for running a DLTS scan without the GUI. It imports the instrument and analysis modules, sets `LOKY_MAX_CPU_COUNT=4`, reloads `impedanceAnalysis_Tools`, and under `if __name__ == '__main__':` does:

```
run = rdT.dltsRun()
run.initSetup()
run.runExperiment()
run.finishExperiment()
```

The rest of the file is commented-out analysis calls with hard-coded paths on other PCs.

Command line

```
python runDLTS.py
```

This does not work. `runDlts_Tools.dltsRun` has no methods `initSetup`, `runExperiment` or `finishExperiment`; the current names are `init_experiment()`, `run_experiment()` and `finish_experiment()`. The script stops with `AttributeError: 'dltsRun' object has no attribute 'initSetup'` right after the imports. It does not reach any instrument. Even with the names fixed, `dltsRun` reads its parameters from the GUI state in `dltsConfig` (`dltsConfig.dltsImpDev`, `dltsConfig.dltsTempDev`, `dltsConfig.dltsZ_params_vars`, ...), which this script never sets up.

To run a scan without the GUI, follow `benchmarks/hardware/hw_common.py` and `hw_run.py`, which fill the same `dltsConfig` state headlessly.

Importing it also needs `pyserial` (`import serial`), `h5py`, `lmfit`, `pandas` and everything `impedanceAnalysis_Tools` imports.

DLTS_APP.py

What it's for

"DLTS Multiwindow Analysis - Interactive GUI App v1.3", a self-contained tkinter program. It loads a ZI MFIA temperature sweep exported as one subfolder per temperature, computes rate-window DLTS spectra for many windows, and fits an Arrhenius line to get the trap activation energy E_t , the apparent capture cross-section and an estimate of the trap density N_t . The banner reads "SiC PiN Diode Deep Level Transient Spectroscopy" and the physical-constant defaults are for 4H-SiC.

It needs only `numpy`, `pandas`, `scipy` and `matplotlib` (plus tkinter). It does not import any other module of this repository.

Relation to the GUI

The **Detailed Analysis** tab (`detailedAnalysisTab.py`) is a port of this app: the same palette, grid defaults, folder convention and multi-window Arrhenius pipeline, rewritten as module-level functions with `dlts.detailed_*` globals, background workers instead of blocking calls, and log lines instead of message boxes. The tab also reads the other data formats (single combined ZI export, JSON `.txt` and `.h5` step files) through `LiveDataTab`'s loaders, and adds peak-finding and fit options. See [data_formats.md](#).

How to launch

```
python DLTS_APP.py
```

(The docstring says `python DLTS_App.py`; on Windows the case does not matter.) The window opens maximized. Browse to a data folder, click **Load Data**, then **Run Analysis**.

Data it reads

`load_data()` scans the chosen folder for subfolders named like `25C_001`, `85C` or `n10C` (`_folder_re`, `n` = negative). In each it reads exactly `dev32271imps_0_sample_param1_avg_00000.csv` (; -separated, columns `chunk;timestamp;value`), keeps chunk `0`, and skips the subfolder if the file is missing or has fewer than 100 rows. Values are multiplied by $1e12$ (F to pF) unconditionally. The time axis is `grid_off + k * grid_dt` (s), converted to ms.

Module constants and globals

Name	Value / type	Meaning
<code>C0</code> , <code>C1</code> , <code>SURFACE</code> , <code>TEXT_PRI</code> , <code>TEXT_SEC</code> , <code>TEXT_MUT</code> , <code>GRIDLINE</code> , <code>BASELINE</code>	hex color strings	Plot palette.

Name	Value / type	Meaning
DEFAULT_GRID_OFF	-0.001	ZI grid column offset, s (time of the first sample relative to reverse-bias start).
DEFAULT_GRID_DT	1.86667e-5	ZI grid column spacing, s.
DEFAULT_CHUNK_SIZE	32768	Samples per temperature.
DEFAULT_RB_MS	500.0	Reverse-bias duration, ms.
_folder_re	re.compile(r'^(n?)(\d+)C(?:_\d+)?\$', re.IGNORECASE)	Temperature subfolder name.

Functions

folder_to_tempC(name)

Temperature in °C from a subfolder name ('n10C' gives -10.0), or None if the name does not match _folder_re.

cap_at(t_ms, cap, tv_ms)

Capacitance at time tv_ms, linearly interpolated with np.interp. Returns float.

find_peak_parabolic(T_arr, S_arr, hw=2)

Peak temperature of a DLTS spectrum: fits a parabola to the 2*hw+1 points around the maximum of S_arr and returns its vertex if it opens downward and lies inside that range; otherwise the temperature of the maximum sample.

load_data(base, grid_off, grid_dt, chunk_size, rb_ms, cinf_lo, cinf_hi)

Loads all temperature subfolders under base (see [Data it reads](#)). C_inf (the steady-state capacitance C0) is the mean over cinf_lo * rb_ms <= t <= cinf_hi * rb_ms, or the mean of the whole transient if that range is empty. Returns (data, temps): data[T_C] = (t_ms, cap_pF, c_inf) and the sorted list of temperatures.

compute_arrhenius(windows, temps, data, gamma, t_peak_lo, t_peak_hi, peak_min_frac=0.05)

For each rate window (t1, t2) in ms: the signal $S(T) = (C(t_2) - C(t_1)) / C_{inf}$, flipped in sign if its minimum is larger in magnitude than its maximum, restricted to $t_{peak_lo} \leq T \leq t_{peak_hi}$ (K). Windows whose peak is at most peak_min_frac of the largest |S| are skipped. The peak temperature comes from find_peak_parabolic() and the emission rate from $en = \ln(t_2/t_1) / ((t_2 - t_1) * 1e-3) (s^{-1})$. A linregress of $\ln(en/Tp^2)$ against $1000/Tp$ gives $Et = -slope * 1000 * kB$ (eV, $kB = 8.617333e-5$ eV/K), its standard error, and $\sigma = \exp(intercept) / \gamma$. Returns None if fewer than 2 windows give a peak, else a dict with en_arr, Tp_arr, Sp_arr, Et, Et_se, sigma, R2, N, slope, intercept, x, y, detail (rows (t1, t2, en, Tp_C, peak)).

Class DLTSApp(root)

The main window. root is a tk.Tk. Main methods:

Method Purpose	--- ---	__init__(self, root) Sets the title, maximizes the window, builds the UI.
_build_ui(self) Banner, scrollable control panel on the left, figure and results box on the right, status bar.		
_build_controls(self) All input sections (defaults in the table below) and the action buttons.		
_section(self,		

title), `_row(self, label, widget_factory, **kw)`, `_spinbox(self, parent, var, lo, hi, inc, fmt='%g', width=9)` | Layout helpers. | `_browse(self)` | Folder dialog for the data folder. | `_set_status(self, msg, color='#8fb4d8')` | Sets the status bar text. The `color` argument is ignored. | `_log(self, text)` | Replaces the results box text. | `_load_data(self)` | Calls `load_data()` with the current settings; enables **Run Analysis** on success. | `_run(self)` | Builds `N` windows log-spaced `t1` values from `t1 min` to `t1 max` with `t2 = ratio * t1`, runs `compute_arrhenius()` for them and for the 5 standard windows, estimates `Nt = 2 * max|S_ref| * Nd` with `S_ref = (C(RB duration) - C(2 ms)) / C_inf`, then plots and writes the results. | `_plot(self, res_mw, res_std, mw, std_wins, T_K, Nt)` | Draws the enabled panels (Arrhenius plot, DLTS spectra, 2D transient map, rate-window map). | `_install_drag(canvas, fig, artist, kind)` (static) | Mouse dragging for legends (`kind='legend'`) and the results annotation (`'annot'`). | `_plot_transient_map(self, ax, res_mw, T_K)` | Filled contour of $\Delta C/C_0(t, T)$ on a log time axis, optional $\tau(T) = 1/en(T)$ overlay. | `_plot_rate_window_map(self, ax, res_mw)` | `Nt/N_D` map in $(1/kT, T^2/en)$ space with the Arrhenius line and a `Z1/2` marker. | `_style_rwm_axes(self, ax)` | Black-background styling for that map. | `_open_label_editor(self)` | Dialog to edit titles, axis labels, legend text and font sizes. | `_write_results(self, res_mw, res_std, Nt)` | Text summary: `Et ± error`, `σn`, `R2`, `Nt`, the `Et-error` improvement of multi-window over the 5 standard windows, and the per-window table. | `_save_png(self)` | Saves the figure (180 dpi). | `_export_txt(self)` | Saves the results text. |

Input defaults:

Section	Setting	Default
ZI MFIA Grid	Grid offset (s), Grid dt (s), Chunk size, RB duration (ms)	-0.001, 1.86667e-5, 32768, 500
C0 Estimation Window	Start, End (fraction of RB)	0.40, 0.90
Physical Constants (4H-SiC)	γ (cm ⁻² s ⁻¹ K ⁻²), <code>N_D</code> (cm ⁻³)	1.66e21, 3.2e14
Peak Search Range	T min, T max (K)	250, 400
Multi-Window Parameters	N windows, t1 min (ms), t1 max (ms), ratio t2/t1	16, 5, 90, 5
Standard Windows	t1/t2 (ms)	5/25, 10/50, 20/100, 50/250, 100/490
Display	Spectra panel, spectra count, 2D transient map, τ overlay, rate-window map	on, 5, on, on, on

DrKayisScript.py

What it's for

"Offline DLTS Core Transient & Defect Signature Suite (v8 - ZI/SiC Format)", a PyQt6 program with three tabs that form a pipeline:

- Transient Extraction:** pick a folder, choose temperatures, extract and average the capacitance transients.
- Rate Window Analysis:** four double-boxcar rate windows, one DLTS spectrum each, peak found with a pseudo-Voigt fit.
- Arrhenius Defect Mapping:** straight-line fit of $\ln(en/T^2)$ against $1000/T$, giving `Et`, the apparent capture cross-section and `Nt`.

Relation to the GUI

DrKayisScript.py tab	Ported to
1. Transient Extraction	Qualitative Analysis in the Live Tools tab (liveDataTab.py: <code>_compute_zi_dataset</code> , <code>_compute_legacy_dataset</code> , <code>_compute_zi_transients</code> , <code>_compute_legacy_transients</code> , <code>_LEGACY_FILENAME_PATTERN</code>)
2. Rate Window Analysis	Quick Analysis tab (dataAnalysisTab.py)
3. Arrhenius Defect Mapping	Quick Analysis tab (dataAnalysisTab.py)

The ports changed some things, as noted in their source comments. For example, Qualitative Analysis finds fill pulses with a threshold midway between the file's two excitation levels instead of the fixed -2.5 V used here, also reads `.h5` step files and a subfolder-per-temperature ZI export, and runs off the GUI thread. Quick Analysis makes the capture-cross-section prefactor a user input (fixed here at the silicon value `3.256e21`) and uses `lmfit` instead of `scipy.optimize.curve_fit`.

How to launch

```
python DrKayisScript.py
```

It needs PyQt6, which is **not installed** in the lab's Python (`C:\Users\spencer\anaconda3\python.exe`: `ModuleNotFoundError: No module named 'PyQt6'`). Install it with `pip install PyQt6` first. At start-up the app tries to load a hard-coded folder on another PC (`C:\Users\ikayi\OneDrive\Desktop\DLTS_SiC_PiN_DevR6C7_FP0V1ms_RB-5V500ms_...`) and silently skips it if it does not exist.

Module constants and globals

Name	Value / type	Meaning
<code>K_BOLTZMAN</code> <code>N</code>	<code>8.617333262145e-5</code>	Boltzmann constant, eV/K.
<code>C_CONSTANT</code> <code>_SI</code>	<code>3.256e21</code>	Prefactor γ used for the capture cross-section: <code>sigma = exp(intercept) / C_CONSTANT_SI</code> . Labeled for Si in the code.

Class DLTSSuiteApp()

A `QMainWindow`. State kept on the instance: `dataset_registry` (temperature to data source), `processed_transients` (temperature to `{'time_ms', 'avg_cap_pf', 'C_infinity'}`), `rate_window_signals`, `extracted_peaks`, the ZI grid parameters (`zi_grid_col_offset = -0.001`, `zi_grid_col_delta = 1.86667e-05`, `zi_chunk_size = 32768`), and `sampling_rate_s = 1.8666666666666665e-05` for legacy files.

| Method | Purpose | |---|---| | `__init__(self)` | Sets up state and UI; loads the hard-coded default folder if it exists. | | `init_ui(self)` | Creates the three tabs. | | `setup_transient_tab(self)` | Folder button, temperature check list, timing fields (fill 1.0 ms, reverse bias 500 ms, slice 2 to 490 ms), **Extract & Average Transients** button, plot. | | `browse_folder(self)` | Folder dialog, then `load_directory_path()`. | | `load_directory_path(self, dir_path)` | Uses the ZI loader if a `*imps_0_sample_param1_avg_header*.csv` is in the folder, else the legacy loader; fills the temperature list. | |

`_load_zi_dataset(self, dir_path, header_filename)` | Single combined ZI export: grid parameters from the header's first row, temperature per chunk from `history_name` (`^(\\d+)C_`), timing fields from `FP...ms / RB...ms` in the folder name. | `_load_legacy_dataset(self, dir_path)` | Per-temperature `.txt` (JSON) and `.csv` files matching `^([npNP])(\\d+)(?:[pP](\\d+))?[cC](?:_\\d+)?\\. (txt\\|csv)$`; multi-chunk CSVs use the fixed map chunk $0.8 = 120..160$ °C. | `process_raw_transients(self)` | Runs the ZI or legacy extraction for the checked temperatures, with `C_inf` taken at $0.9 \times$ reverse bias, and plots them. | `_process_zi_transients(self, selected_temps, c_inf_target_ms, execution_errors)` | Reads the data CSV once and cuts out each chunk; F to pF if values are below $1e-3$. | `_process_legacy_transients(self, selected_temps, rb_duration_ms, c_inf_target_ms, execution_errors)` | JSON `.txt`: fill pulses where `AuxInput1` falls below -2.5 V, averages the following reverse-bias windows of `ImpedanceIm` $\times 1e12$. CSV: see [data_formats.md](#). | `setup_rate_window_tab(self)` | Four window pairs ($t1/t2 = 10/50, 20/100, 50/250, 100/490$ ms), **Compute Boxcar Spectrums** button, peak table, plot. | `_pseudo_voigt(self, x, amp, center, fwhm, eta, offset)` | Pseudo-Voigt peak shape for the spectrum fit. | `calculate_rate_windows(self)` | For each window: $en = \ln(t2/t1)/((t2 - t1) \cdot 1e-3)$, $S(T) = (C(t2) - C(t1))/C_{inf}$ at the nearest samples, pseudo-Voigt fit for the peak (falls back to the largest-magnitude sample if the fit fails). | `setup_arrhenius_tab(self)` | `N_D` input (default $1.5e15 \text{ cm}^{-3}$), solver button, result labels, plot. | `run_arrhenius_solver(self)` | Linear fit, $E_t = -\text{slope} \cdot 1000 \cdot k_B$, $\sigma = \exp(\text{intercept})/C_CONSTANT_SI$, $N_t = 2 \cdot \max|S_{peak}| \cdot N_D$. Needs at least 2 windows. |

scrapwork.py

A scratch file for trying triggered MFIA DAQ acquisitions by hand. It is not used by the application.

At import it creates a `ziDevice`, connects to the MFIA and loads parameters, then runs `runThis(daq_module, numReps, numPoints)` (1024 points, 100 repetitions), which sets up the DAQ module the way `ziDevice.pull_data(trigger=True)` does, busy-waits for completion, builds the 8 channels and plots them. The rest of the file (from about line 99 to the end, 910 lines in total) is commented-out analysis experiments.

Do not run it next to a measurement: it talks to the instrument. As written it also calls `impdDev.connectDevice()` and `impdDev.loadParams()`, which no longer exist (the methods are `connect_device()` and `load_params()`), so it stops with `AttributeError` before the connection.

postprocessingTab.py

An empty file (0 bytes). No module imports it. `DLTSGUI_MainWindow.py` creates a frame `dltsc.postprocessingTab` but never adds it to the notebook, and the call `ppT.construct_postprocessingTab()` there is commented out. `dltscConfig.postprocessingTab` starts as `None`. It is a placeholder for a future tab.

tests/

How to run

From the repository folder:

```
python -m pytest tests -q
```

On the lab PC (Python 3.13.9, pytest 8.3.4) the suite gave 163 passed in 46.53s on 28 Sep 2026. Set `LOKY_MAX_CPU_COUNT` (for example `$env:LOKY_MAX_CPU_COUNT = 8` in PowerShell) to avoid scikit-learn's CPU-count warning. No instrument is needed or touched: the tests use synthetic data and fake devices.

`conftest.py` puts the repository root on `sys.path`, selects matplotlib's `Agg` backend, and provides:

Name	Meaning
<code>SAMPLE_DT_S = 1.8666666666666665e-05</code>	Sample interval the app assumes (s).
<code>TICKS_PER_SAMPLE = 1120</code>	60 MHz clock ticks per sample.
<code>make_step(T, n=2**14, seed=0, fill_ms=1.0, rb_ms=500.0)</code>	Synthetic <code>pull_data()</code> output for one temperature: 0 V / -5 V fill-pulse train on <code>AuxInput1</code> , a capacitance transient on <code>ImpedanceIm</code> whose time constant depends on <code>T</code> , integer <code>uint64</code> tick stamps.
<code>step_file_name(T, ext)</code>	The file name <code>init_experiment()</code> gives setpoint <code>T</code> , e.g. <code>n10p0.h5</code> .
<code>zi_device</code> fixture	A <code>ziDevice</code> created without <code>__init__</code> and without a connection, enough for its file writers.
<code>write_run(dev, folder, temps, ext, n=2**14, rb_ms=500.0)</code>	Writes one step file per temperature plus <code>runParams.txt</code> , as a run does.

Test files

File	What it covers
<code>test_cleanup_time_axis.py</code>	<code>impdData.cleanup_data()</code> with both kinds of tick-stamp channel: integer 60 MHz clock ticks and the seconds axis of triggered runs. The seconds axis keeps its origin and units, a missing sample is interpolated, clock ticks are still filled and converted, and the axis kind is detected.
<code>test_convert_h5_to_text.py</code>	<code>convert_h5_to_text.py</code> : exact <code>.txt</code> and <code>.json</code> exports, NaN stage temperature as <code>null</code> , float tick stamps, unequal channel lengths, refusal of <code>p25p0.txt</code> / <code>p25p0.json</code> / <code>n10p0.txt</code> as output names, exports ignored by the folder scan, nothing left behind after a failed verification, the CLI.
<code>test_convert_json_to_h5.py</code>	<code>convert_json_to_h5.py</code> : <code>setpoint_from_name()</code> for accepted and rejected names, conversion with originals moved to <code>json_originals</code> , the root attributes of converted files, converted runs reading exactly like the JSON originals, triggered runs with float tick stamps, <code>--delete-json</code> , <code>--keep-json</code> , <code>--dry-run</code> , skipping existing <code>.h5</code> unless <code>--overwrite</code> , running twice, <code>--recursive</code> , unreadable files, failed verification, missing <code>runParams.txt</code> , a non-folder argument.
<code>test_hdf5_analysis.py</code>	The analysis code on HDF5 runs, each compared with the same run in JSON <code>.txt</code> : the Live Tools offline pipeline, live ingest, <code>read_data()</code> with mixed <code>.txt</code> and <code>.h5</code> , <code>append_data()</code> across formats, the folder scan (finds <code>.h5</code> , ignores partial <code>.tmp</code> files), Qualitative Extract & Average, Detailed Analysis loading (also a mixed folder), and the live watcher picking up <code>.h5</code> files only once renamed into place.
<code>test_hdf5_run.py</code>	<code>dltsRun</code> writing HDF5 through the real <code>_run_single_step()</code> / <code>run_experiment()</code> with fake devices: one file per step, stage temperature unreadable (step still succeeds) or read only once, Redo and Remove & Retake replacing only the selected steps, GUI closed during an acquisition or a retake (nothing written), a failed write marking the step failed and keeping the previous file.
<code>test_hdf5_writer.py</code>	<code>ziDevice.writeDataH5()</code> and <code>read_h5_record()</code> : lossless round trip with native dtypes, tick stamps beyond float precision, float tick stamps from triggered runs, a triggered step analysing like JSON, attributes, NaN for missing temperatures, empty channels, creating a missing parent folder, reader dtypes matching JSON, reading only some channels, overwrite, the file appearing only once complete, failure handling (no data file, previous file kept, temp file cleaned up), rename retries while the target is locked and giving up if it stays locked.

File	What it covers
<code>test_detailed_peaks.py</code>	Detailed Analysis peak errors: a peak on the edge of the search range (spline and parabolic) or with an error below <code>MIN_TP_ERR_K</code> gets no error, <code>_peak_on_edge</code> , and a synthetic 0.60 eV trap with the search range cut so slow windows peak on its edge: those windows are excluded and Et stays at 0.60 eV (the old behaviour gave 0.99 eV).
<code>test_quick_vs_detailed.py</code>	Quick Analysis and Detailed Analysis on the same synthetic 0.60 eV run, windows and search range give the same Et, its error, σ , R^2 , fitted/excluded counts and Nt, with and without edge peaks; Quick Analysis excludes edge-peak windows; a curve-fit peak on the range edge loses its error; both tabs have the same default windows, search range, γ , Nd and kB.
<code>test_temperatures_and_gaps.py</code>	<code>impdData</code> temperature keys from step file names and ZI history names (exact kelvin, decimals kept, <code>C/_N</code> suffixes, names without a temperature rejected) and Quick Analysis' °C conversion; <code>_gap_remove</code> : forward gaps, backward jumps from appended records, several gaps, seconds axes, unsigned ticks, uniform data untouched, small jitter kept.
<code>test_run_folder_and_serial.py</code>	<code>dltsRun.init_experiment()</code> refusing an empty or relative Data Root Folder and building <code>Root\MMDDYY\HHMMSS\</code> paths; <code>mK2000B</code> serial timeouts: timeouts set on connect, <code>read_temp()</code> retrying a missed reply, raising <code>TimeoutError</code> when the controller stays silent (lock released), and <code>go_to_temp()</code> failing instead of hanging.
<code>test_standalone_helpers.py</code>	No-GUI helpers: <code>ziDevice.configure()</code> (defaults with overrides, needs a connection to push, rejects unknown names), <code>mK2000B.configure()</code> (grid and motion attributes, rejects unknown names), and <code>dltsConfig.log_to_textbox()</code> printing when there is no GUI.
<code>test_transient_extraction.py</code>	Qualitative Extract & Average's pulse search (<code>liveDataTab._compute_legacy_transients / _find_reverse_bias_starts</code>) for the GUI default pulse levels and for a short fill pulse, the message for a constant excitation, a transient from a default-level run, the reasons given for skipped temperatures, <code>_legacy_run_timing()</code> from <code>runParams.txt</code> (and <code>None</code> when missing), and <code>_capacitance_axis_units()</code> .

benchmarks/bench_hdf5.py

What it's for

Re-runs the HDF5 vs JSON measurements on synthetic data (from `tests/conftest.make_step`), through the application's own writers and readers. No hardware is used.

Command line

```
python benchmarks/bench_hdf5.py
```

It takes a few minutes and writes about 650 MB of synthetic data to `benchmarks/benchdata/`, which is deleted at the end. The results are written to `benchmarks/bench_results.json` (overwriting the stored results) and printed as it goes.

What it measures

Section	Measurement
Write	File size and write time of one step with 2^{16} and 2^{20} points, JSON vs HDF5, and whether both read back losslessly.
Run folder	Total size of a 40-temperature run (-50 to 145 °C in 5 °C steps, 2^{16} points per step) in each format.
Offline load	Time of <code>read_data()</code> , <code>cleanup_data()</code> , <code>selected_emissions()</code> and <code>filter_emissions('pca')</code> for the whole run, and whether the results are identical.
Qualitative Extract & Average	<code>_compute_legacy_dataset()</code> + <code>_compute_legacy_transients()</code> time, and identical transients.
Detailed Analysis	<code>_load_detailed_data()</code> time, and identical data.
Live ingest	Time to append and process the 40th temperature.
Peak memory	<code>tracemalloc</code> peak while reading one temperature.

The stored `bench_results.json` shows, for example, 583.9 MB (JSON) against 71.3 MB (HDF5) for the 40-temperature run, 4.56 s against 0.59 s to read it, and identical analysis results in every section.

benchmarks/hardware/

A real-hardware benchmark: a short temperature scan on the MFIA and the Instec mK2000B, run through the application's own run code (not the GUI) with HDF5 output, then checked file by file and compared with the same data written as JSON. The run folder `DATA\DLTS\092826\114655` used as the example in [data_formats.md](#) matches its default settings.

This moves the stage and biases the mounted sample. Only run it when the sample and the temperature range are safe. The MFIA uses the GUI's default parameters.

Steps

```
python benchmarks/hardware/hw_preflight.py          # stage read-only, times  $2^{18}$ -point pulls
python benchmarks/hardware/hw_run.py                # about 65 min with the defaults
python benchmarks/hardware/hw_analyze.py OUTPUT_DIR # the folder hw_run.py printed
python benchmarks/hardware/build_report.py OUTPUT_DIR --pdf
benchmarks/hardware/reports/HDF5_Hardware_Benchmark_YYYY-MM-DD.pdf
```

File	Purpose
<code>hw_common.py</code>	Headless stand-in for the Input Parameters tab: fills the same <code>dltsConfig</code> state with the GUI defaults, redirects <code>log_to_textbox()</code> to a log file, and connects the instruments (<code>setup_gui_state()</code> , <code>connect_impedance()</code> , <code>connect_temperature()</code>). <code>DEFAULT_OUTPUT</code> is <code>benchmarks/hardware/output</code> .
<code>hw_preflight.py</code>	Reads the stage temperature once (no ramp; the controller is left as it was), connects the MFIA and times <code>pull_data(trigger=True)</code> at the requested size to check it finishes well inside its 60 s completion timeout. Options: <code>--points</code> (log2 of points, default 18), <code>--reps</code> (list, default 1 2 10), <code>--out</code> . The stage is only read, but the MFIA is connected, gets its parameters pushed (<code>reload_params()</code>) and runs real acquisitions.

File	Purpose
hw_run.py	The run, in the GUI's sequence: connect and push parameters, <code>init_experiment</code> , <code>run_experiment</code> , <code>finish_experiment</code> , return to room temperature, Redo one step, return, Remove & Retake another step, return, disconnect. Logs the stage temperature every 10 s, per-step timings, and repeats the live-ingest work on each file. A watchdog aborts and parks the stage at room temperature if a step does not stabilize within <code>--step-limit</code> minutes. Options: <code>--start 25</code> , <code>--stop 50</code> , <code>--step 5</code> , <code>--room 30</code> , <code>--points 18</code> , <code>--reps 100</code> , <code>--root C:\Users\spencer\Desktop\DATA\DLTS</code> , <code>--redo 45</code> , <code>--retake 40</code> , <code>--no-redo</code> , <code>--no-retake</code> , <code>--step-limit 40</code> , <code>--out</code> .
hw_analyze.py	Checks the run folder (one <code>.h5</code> per temperature plus <code>runParams.txt</code> , no leftover <code>.tmp</code> files, and more), writes a JSON copy of the run to <code>OUTPUT_DIR\json_copy\</code> and compares both formats through the analysis code. Writes <code>analysis.json</code> . Without an argument it uses the newest <code>output\run_*</code> folder. Nothing is written to the data folder.
build_report.py	Builds <code>report.html</code> from <code>analysis.json</code> and <code>report_template.html</code> . <code>--notes FILE.html</code> adds a "Found during the run" section; <code>--pdf OUT.pdf</code> prints it with headless Microsoft Edge (looked up under <code>C:\Program Files (x86)\Microsoft\Edge\...</code> and <code>C:\Program Files\Microsoft\Edge\...</code>).
report_template.html	The report page (interactive charts, print styles for the PDF).
notes/2026-09-28.html	Findings for the 28 Sep 2026 run's report.
reports/HDF5_Hardware_Benchmark_2026-09-28.pdf	The stored report of that run.

Everything the scripts write goes to `benchmarks/hardware/output/run_<timestamp>/` (git-ignored): the run log, stage-temperature log, timings, `analysis.json`, `report.html` and the JSON copy of the run (about 60 MB per 2^{18} -point step). The run's data files stay in the Data Root Folder. See also `benchmarks/hardware/README.md`.